Research Methods 1

Damian Clarke

Table of contents

I Introduction **3**

1 About This Course **5**
1.1 Background Information . . . 5
1.2 Proposed Schedule . . . 6
1.3 Evaluation . . . 7
1.4 Reading List . . . 11

2 Background on Computational Languages in Economics **12**
2.1 Computer Languages in Economics . . . 12
2.2 Considerations in Adopting a New Language . . . 14
2.2.1 1. What type of research will I be doing? . . . 15
2.2.2 2. Are there any relevant network effects? . . . 15
2.2.3 3. What is my current knowledge? . . . 16
2.2.4 4. What type of support do I look for? . . . 16
2.2.5 5. Do I have any deeper preferences to satisfy? . . . 16
2.3 How Should I Learn a New Language? . . . 18
2.3.1 Online Forums . . . 18
2.3.2 Large Language Models . . . 19
2.4 Hardware and Operating Systems . . . 19
2.4.1 Hardware . . . 19
2.4.2 Operating Systems . . . 20

3 Getting Set Up **21**
3.1 R . . . 21
3.1.1 Introduction . . . 21
3.2 Installing and Working with R . . . 21
3.3 Further Resources . . . 22
3.4 Python . . . 23
3.4.1 Introduction . . . 23
3.4.2 Installing and Working with Python . . . 23
3.4.3 Further Resources . . . 24
3.5 Stata . . . 24
3.5.1 Introduction . . . 24
3.5.2 Installing and Working with Stata . . . 25

3.5.3 Further Resources . . . 26
3.6 Other Tools . . . 26

II Programming Languages **27**
An Overview of the Languages We Will Use . . . 28

4 An Introduction to R **29**

5 Preliminaries **30**
5.0.1 Checking this out locally . . . 34

6 An introduction to the language and data types **35**
6.1 Data Types . . . 36
6.1.1 Text (String) Data . . . 36
6.1.2 Numeric Data . . . 37
6.1.3 Vectors and Arrays . . . 38
6.1.4 Data frames . . . 40
6.2 Subsetting Multidimensional Elements . . . 42
6.3 Basic Operations . . . 44
6.4 Functions . . . 47
6.5 Managing packages . . . 49
6.6 Loops and control-flow statements . . . 52
6.6.1 The `for` Loop . . . 52
6.6.2 The `while` Loop . . . 53
6.6.3 Control Flow Mechanisms . . . 54
6.6.4 `Apply` functions . . . 55

7 Working with Data **61**
7.1 Loading and viewing data . . . 61
7.2 Tidying data . . . 63
7.2.1 Aggregating Data . . . 64
7.2.2 Merging/Joining Data . . . 66
7.2.3 Filtering Data . . . 67
7.2.4 Generating variables . . . 68
7.2.5 Reshaping Data . . . 69

8 Basic Plots **71**

9 Writing Our Own Functions **78**
9.1 An Introduction to Functions in R . . . 78
9.1.1 Creating a Simple Function . . . 78
9.1.2 Understanding Function Arguments . . . 78
9.1.3 Variable Scope and Return Value . . . 79

10 An Introduction to Python **80**

11 Preliminaries **81**
11.0.1 Checking this out locally . . . 84

12 An introduction to language and data types **86**
12.1 Data Types . . . 87
12.1.1 Text (String) Data . . . 87
12.1.2 Numeric Data . . . 88
12.1.3 Tuples, Lists and Arrays . . . 88
12.1.4 Dictionaries . . . 90
12.1.5 Data frames . . . 91
12.2 Subsetting Multidimensional Elements . . . 91
12.3 Basic Operations . . . 97
12.4 Functions . . . 99
12.5 Managing modules . . . 100
12.6 Loops and Control Flow . . . 102
12.6.1 **1. The `for` Loop** . . . 103
12.6.2 **2. The `while` Loop** . . . 103
12.6.3 **3. Loop Control Mechanisms** . . . 104

13 Working with Data **105**
13.1 Loading and viewing data . . . 105
13.2 Tidying data . . . 107
13.2.1 Aggregating Data . . . 108
13.2.2 Merging/Joining Data . . . 109
13.2.3 Filtering Data . . . 110
13.2.4 Generating variables . . . 111
13.2.5 Reshaping Data . . . 112

14 Basic Plots **114**

15 Writing Our Own Functions **119**
15.1 An Introduction to Functions in Python . . . 119
15.1.1 Creating a Simple Function . . . 119
15.1.2 Understanding Function Arguments . . . 120
15.1.3 Documenting our Functions . . . 120

16 An Introduction to Stata **122**
16.1 An Introduction to Stata . . . 122

17 Preliminaries **123**
17.0.1 Checking this out locally . . . 127

18 An introduction to the language and data types **128**
18.1 Data Types . . . 129
18.1.1 Data Stored as Variables . . . 129
18.1.2 Working with frames (≥ Stata 16) . . . 131
18.1.3 Other Types of Data Stored by Stata . . . 132
18.2 Basic Operations . . . 134
18.3 **Loops in Stata** . . . 137
18.3.1 **1. The `foreach` Loop** . . . 137
18.3.2 **2. The `while` Loop** . . . 138
18.3.3 **3. Loop Control Mechanisms in Stata** . . . 139
18.4 Stata Commands . . . 140
18.5 User-Written Commands . . . 147

19 Working with Data **148**
19.1 Loading and viewing data . . . 148
19.2 Tidying data . . . 150
19.2.1 Aggregating Data . . . 151
19.2.2 Merging/Joining Data . . . 152
19.2.3 Filtering Data . . . 154
19.2.4 Reshaping Data . . . 156

20 Basic Plots **157**

21 Writing Our Own Programs **159**
21.1 An Introduction to Programs in Stata . . . 159
21.1.1 Creating a Simple Program . . . 159
21.1.2 Understanding Program Arguments . . . 160
21.1.3 Variable Scope and Return Values . . . 160

22 Data Wrangling and Data Management **163**
22.1 More on "Data Wrangling" . . . 163
22.1.1 Joining/Merging . . . 163
22.1.2 Appending . . . 167
22.1.3 Reshaping . . . 168
22.2 An Applied Example . . . 169

23 Project Management & Replicability **190**
23.1 Workflow, Project Structure and Documentation . . . 190
23.1.1 Workflow . . . 190
23.1.2 Project Structure . . . 192
23.1.3 Documentation . . . 192
23.2 Version Control . . . 193

24 Setting up a Project **199**

25 Adding code, staging, and committing **202**

26 Working with github **206**
26.1 1. Publishing our repositories on github . . . 206
26.2 2. Contributing to an existing repository . . . 211
26.3 Programming Style and Documentation . . . 220
26.3.1 Programming Style . . . 220
26.3.2 Automation and Abstraction . . . 222
26.3.3 Testing, Debugging, and Profiling . . . 223
26.3.4 Documentation . . . 225
26.4 Reproducibility . . . 225
26.5 An Example in R . . . 227

27 Data Visualisation **235**

28 The Importance of Data Visualisation **236**
28.1 A Motivation for Graphical Work . . . 236
28.2 Graphical Work in Economics Papers . . . 237

29 Basic Graph Examples **240**
29.1 Some Graph Types and Procedures . . . 240
29.2 A Small Aside on Printing . . . 250

30 A Worked Example of Something More Complex **252**

31 Statistical Tools **266**
31.1 Statistical Analysis . . . 266
31.1.1 This Class . . . 266
31.2 Weighting . . . 267
31.3 Exporting Models . . . 268
31.4 Examples . . . 270
31.4.1 Summary Statistics . . . 271
31.4.2 Linear regression models . . . 275
31.4.3 Non-linear regression models . . . 280

32 Simulations as Part of Research Workflow **289**
32.1 Introduction . . . 289
32.2 Monte Carlo Simulations to Examine Estimators . . . 289
32.3 Alternative Methods Based on Pseudo-Random Draws . . . 294
32.3.1 Bootstrap Inference . . . 295
32.3.2 Randomization Inference . . . 298

33 Other Key Research Tools **304**

34 References **305**

Part I

Introduction

This page provides the course materials for BEEM136, the Research Methods 1 course at the University of Exeter. This course is designed to accompany you as you conduct your own research in economics. This course has a particular focus on empirical projects, with a micro-economics flavour. Nonetheless, I hope many of the research tools discussed will be useful regardless of the precise nature of the research you conduct.

Throughout this course we will cover the following topics:

1. An introduction to Various Statistical Programming Languages
2. Further information on Data Wrangling and Data Management
3. Information Related to Research Project Management
4. Further Information on Data Visualisation and Graphics
5. A Discussion of Statistical Tools and the Generation of Tables
6. A Discussion of Simulation and Econometrics
7. Some Further Useful Tools such as git/gitub, LaTeX, websraping

Precise details about the nature of the course are found on the following page.

Welcome!

1 About This Course

1.1 Background Information

This course seeks to provide you with a base for your research in economics. In particular, the focus of this course will be on computational methods which are frequently used in economic research. We will be working with scientific programming languages, and discussing themes relevant to conducting research. The goal of this course is that by the end, you will have knowledge of the computation tools necessary to embark on a serious research project in economics, such as a paper or a thesis.

This course is in some sense language-neutral, and I will ask that each of you work with a language that you have never worked with before. In the early portions of the course, information will be provided on a number of languages, and then we will provide instruction in particular in R, Python and Stata. In later parts of the course, we will rotate, considering particular themes in one of these languages, where the focus will be on the substance of the theme, rather than the particular syntax of the language. In this portion of the course, if you have chosen to work in another language, the idea is that you can work on the same these, however in your chosen language. In Tutorials, at one point during the semester, I will request if each one of you could demonstrate to the class how to apply the week's methods in the language that you have chosen to work with. If, however, you would like to work in another language apart from R, Python or Stata, please let me know, to confirm that this will work.

The only restriction here is that I am requesting that each person start working in a language **that they do not have prior experience in**. I am doing this for two reasons. The first is that this way we will all start with the same level (none) of language specific knowledge on the particular language we are working in. The second is that in this way, at the end of the semester you will each have knowledge of an entirely new language that may open more doors for you later on. If you have no experience at all programming in the past this is entirely fine. Indeed, I will start this class with the assumption that you have no background knowledge, and so anything that you do know is only a bonus for you (and the class).

The principal resource of this course will be this website which will provide notes, code and code output. A pdf or epub version of these notes can be downloaded at top left-hand corner of the page, though generally presentation is optimised for the website. The site will be updated section-by-section, with each section of the notes going live on the day the class will be given. These resources will then be available on the website for the entire semester. I would encourage you to attend classes in person where ever possible!

1.2 Proposed Schedule

The approximate schedule I plan to work with is described below. It is highly likely that the precise dates of each class will vary as we move through the course, and decide to spend more or less time on each section of the course. Below, the column "Language" refers to the language that I will use in class when we are working with a specific example. It does not imply that we will not discuss other languages or that you are expected to work with this particular language. Ideally, you will work in the same language throughout the entire semester.

Table 1.1: Proposed Schedule

Week number	Tutorial	Lecture 1	Lecture 2	Evaluation	Language
1 (22/09)	—	Introduction 1	Introduction 2	—	All
2 (29/09)	Intro/tools	Programming 1	Programming 2	—	All
3 (06/10)	—	Programming 3	Data 1	—	All
4 (13/10)	—	Data 2	Data 3	—	Stata
5 (20/10)	Sign up [TB]	Projects 1	Projects 2	Presentation	R
6 (27/10)	—	—	—	—	—
7 (03/11)	—	Projects 3	Visualisation 1	—	R/Python
8 (10/11)	Sign up [SL,NJR,SH]	Visualisation 2	Visualisation 3	Presentation	Python
9 (17/11)	—	Statistics 1	Statistics 2	—	R
10 (24/11)	Sign up [YL,ZD,XY]	Statistics 3	Simulation	Presentation	R/Stata
11 (01/12)	—	Other Tools 1	Other Tools 2		Python
Mon Jan 12, 12:00	—	—	—	PROJECT	Select

We will meet on:

- Fridays 12:30-14:30 (2 classes) in Seminar Room B, XFI
- TUTORIALS: Mondays at 14:30 in Seminar Room B, XFI.

Currently, the only exception to this is during **weeks 1 and 2** when we will meet:

- Thursdays 9:30-11:30 (2 classes)

1.3 Evaluation

The evaluation of this course will consist of the following two elements:

1. Ongoing assessment (20%): Tutorial presentation (please sign up for your desired slot).
2. An Empirical Project (80%): Full details provided around week 3 of term. Hand in date is at the end of semester.

Full details on these evaluations will be made available below in line with the dates in the calendar.

Tutorial Presentation

For this evaluation, I will ask you to prepare an example of code for us to work through in the tutorial slot together. The idea of this presentation is that you will show us how to do something that we saw in class, however apply it to the language that you are working with. Thus, the date you present will depend on the language that you decide to work in. The only restriction here is that I will ask you not to present about a specific theme if we have already seen this theme in that language in classes with me. Please refer to the "Language" column in the proposed schedule to ensure that this does not occur.
For this presentation I will ask that you generate an example along the lines of the themes we have seen in class the week before. So, for example, if you wish to present on data management and cleaning and are not working in Stata, you may wish to show us an example of joining some data bases and cleaning particular variables in the language that you are working with. Or if you would like present about data visualisation and graphics and are not working in Python, you may wish to show us how to make some particularly nice graph using real data of interest to you in the language that you are working with.
I would ask that you prepare an example that will take about 10-15 minutes to explain. 10-15 minutes may sound like a long presentation, but when you are explaining your code and the goal of your analysis, it is probably less than you think. You may wish to simply work with code and output in the program that you are working with, or generate some set of slides along side of the code that you will work with in class. I would ask that you show us how the code runs on your computer or on the screen, so that we can look into particular lines of code together. If your presentation generates discussion or extensions and goes for more than 10-15 minutes this is perfectly fine. We will have the entire tutorial block to discuss these examples, and any questions which come up as a result.
Please confirm a date with me and I will include the date for your presentation in the calendar above. Please also feel free to discuss with me the precise themes that we will work with each week if you want to set-up your presentation a few weeks in advance.

Empirical Project

At the end of the semester, please hand in an empirical project. This empirical project is expected to cover the themes discussed in the course, and can be on a topic of your choosing. You should view this empirical project as a mini research project. However what is important in this project is not the results *per se* but rather the process you follow in bringing everything together.
Some potential ideas of projects could be things such as:

- Find a paper of interest to you and aim to replicate the question (in a limited way) in another setting
- Replicate and extend a paper using the data provided by the authors*
- Examine a number of papers on a similar topic and document how methods and codes vary across papers
- Find data about some topic of interest to you and run a mini-analysis. In this case, what is important is not empirical design, but rather the process you follow in setting up your project.
- Write and document some package which can extend the functionality of Python, R or Stata

  * *If you choose to replicate a paper, it is not sufficient to simply show that it replicates. The idea is that you should extend the analysis, ideally with further data.*

You should not feel like this is the limit of what you can do. If you would like to do alternative things that can be conceived to be an research project, this is great, and please just confirm your intentions with me.
If you wish to check with me about what you propose to do for your empirical project, feel free to ask me or write me at any time! I would encourage you to do so at some point during the term when you start to get an idea of what you would like to do for this project.
Please hand in all the materials you used for your project in a single organised zipped folder (.zip or .7z), or pdf pointing to a github repo or similar. Please hand this in via the submission point on the ELE page of this course no later than 11:59am on Monday 12th of January, 2025. What you hand in will depend a bit on the nature of your project, but at a minimum your materials should consist of:

- All data and code
- a README file explaining how to replicate everything you have done

- A compiled pdf containing sections including: Introduction, Data, Methods, Results, and Conclusion. This pdf should not exceed 15 pages. If you wish to include appendix results this is fine, and will not count towards the 15 pages.

The evaluation of the project will consider the elements below. The "proportion" refers to the approximate importance that these elements will be given in the final mark. The final mark will consist of an overall grade along with written comments from me, but will not provide an itemised break down, and as such the elements below are simply to help you ensure that your project touches on all relevant components. Marking will follow the typical system used in the MRes where 50 or above is a pass, 70 or above signifies excellent work (ie a level of first class honours), and 80 signifies something truly outstanding.

Table 1.2: Marking Scheme of Empirical Project

Element	Approximate Proportion
README provides sufficient information to replicate	5
All work replicates	10
Suitability/difficulty of theme	10
Good use of work flow within project	5
Code is correct and follows description in pdf	25
Code is documented clearly	5
All work methods described	10
All data clearly described	10
Adequate Summary statistics provided	10
Overall clarity of document	10

General Comment

There is no single "correct" way for this to be done, and it depends on exactly what your project is like. If, however, you are applying the tools of the class to some particular research question of interest (such as the fourth bullet point of the list at the top of this description), I would suggest that you should hand in a pdf structured as follows:

1. Introduction - Around 1-3 pages, noting the context you are looking at, explain what you do, citing some key literature and the motivation (somewhat similar to a short version of the introduction of a standard economic paper)
2. Data - Around 2-5 pages, explaining fully the data you use, any cleaning done, any other information useful for readers, and providing full summary statistics and relevant descriptives
3. Methods - Around 1-2 page, explain simply what you will do, laying out any estimating equations. Here you could also include information related to weighting of models (if relevant), or other key methodological points
4. Results - Around 2-6 pages, providing any results you produce which may be tables, graphs, and so forth. Explain to me what you find, and the implications of this, noting any key limitations
5. Conclusion - Around 1 page, simply summarising what you have found, any key take-aways, key limitations, and future work that may be motivated by this project
6. Bibliography

Please note, the above description may not correspond exactly to what you are doing, and so should be viewed as simply one particular way that this *could* be addressed, rather than a prescriptive list which *must* be followed. Please talk to me if you would like to discuss what you are doing, or get any interim feedback!

1.4 Reading List

There is no formal reading list for this course, as we will be working together through code. Nevertheless, through the semester I will refer to various resources which might be useful to further expand on the themes we see in class. For this, pleae note the References section of this site, which will be updated each week.

2 Background on Computational Languages in Economics

2.1 Computer Languages in Economics

A range of computer languages are currently in use in economics, and in your own reasearch career, if you are conducting empirical research, it is likely that you will use a number of languages. What we will spend most of our time working on in this course are languages with scientific programming capacities. These languages will generally be designed to – or have certain aspects designed to – work with data, conduct numerical calculations, and, ideally, have a set of inbuilt statistical routines which we frequently encounter in economics. Ideally, as well as allowing us to conduct our research, the languages we work with will allow us to generate the types of exhibits we would like to communicate our research with our peers: things like tables and figures.

In modern economic practice, I believe that the most frequently used statistical programming languages (in alphabeticl order) are Julia, Matlab, Python, R and Stata. Among these, Julia is newer and less adopted, but may take off in the years ahead. This is not to say that you will not encounter other languages in your career. Other languages are indeed used. For example, many old structural models are written in Fortran, and certain sub-fields in economics may use other languages such as SAS, OxMetrix, Mathematica, SPSS and other tools.

One may ask why so many languages exist? Languages are designed with different philosophies, and often new languages are developed to remedy perceived shortcomings of older languages. Originally, computer languages were **low level**. Low level languages are computational languages which require the programmer to think carefully about how the program they write interacts with a computer's memory, and are generally more complex to write, but faster to run. Currently, all of the languages frequently used in economic research are **high level**. High level languages are written to take care of things like memory and hardware use, garbage collection, assignment of variable types and so forth, and so the programmer can simply focus on the task at hand. High level languages are generally faster to program in, but codes run more slowly. This is an "abstraction penalty" with moving to higher level coding – more is taken care of, but some cost is *generally* paid in terms of computing time. To see below the example of what the difference can look like between more low level and high level languages, refer to the following two codes to print "Hello World!" on the computer screen. The left-hand side

is the code for Python, while the right-hand side is the code for C. Over time this movement from low to high level has implied a number of new languages have been developed.

Listing 2.1 `Hello World in a High Level Language`

```
print("Hello, World!")
```

Listing 2.2 `Hello World in a Low Level Language`

```
#include <stdio.h>
int main() {
  // printf() displays the
  // string inside quote
  printf("Hello, World!");
  return 0;
}
```

Another key reason why multiple languages exist is that some languages are written by companies to be sold, while other are written to be **open source** and increase access to scientific tools without payment. Commercial languages are produced with paid developers, and generally offer high quality documentation and warranties on performance. However, large open source languages have many active contributers (working on a voluntary basis), and so are often able to match or exceed the performance of **commercial** languages, especially when it comes to rapidly incorporating new advances into their code base.

Finally, different languages generally have different focuses. For example, among the languages we will work with in this course, Stata is most focused on rapidly being able to work with data and perform statistical analyses, R has a somewhat similar focus but is open source, Python provides more control over a computer's entire operating system, and certain other languages such as Matlab are designed specifically for matrix-based calculations (which are extremely frequent in economics). In general, languages will have specific design choices, and when you work with one or another they will have a different feel. A key distinction which we will see throughout the course is in how they deal with repeated calculations or processes. Certain languages are more structured to conduct repeated calculations in sequence, essentially "looping" around and doing one thing after another, while others are more structured to think in terms of matrices, applying a process in block to many cells of a matrix.

Some of the distinctions between languages which you may encounter are summarised in the Table below. Many of these categorisations are just my perception. For example, all computer languages are "fast" in that they can perform millions of calculations per second, but in my experience some or faster than others, and this may be relevant if you end up working on very computationally intensive projects. If you are interested in some more formal comparisons of

languages, you may wish to read the paper by Aruoba and Fernández-Villaverde (2015) .

Table 2.1: Characteristics of Some Computer Languages

Language	High/Low Level	Matrixy/Loopy	Active User Base	Free/Open Source	Fast?
Julia	High Level	Matrixy	Medium	Yes	Yes
Matlab	High Level	Matrixy	Yes	No	No
Python	High Level	Loopy*	Yes	Yes	No
R	High Level	Matrixy	Yes	No	No
Stata	High Level	Loopy*	Yes	No	No
Fortran	Low High Level	Matrixy	Not so much now	Yes	Yes
C/C++	Low Level (C)	Loopy	Not so much in econ	Yes	Yes

* Certain elements are optimized for matricial calculations (Numpy)
+ Certain elements are optimized for matricial calculations (Mata)

2.2 Considerations in Adopting a New Language

When you are learning a new programming language, many of the things you learn will be general knowledge that will also help you in other languages. For example, certain programming constructs such as the use of formal logic, control flow statements such as loops, the generation of functions and so forth will be something you encounter in nearly any language. However, there is also a non-trivial investment in language specific factors, for example the *syntax* of the language (the way the language is written and used), as well as the use of particular commands, will not cross over from language to language. For this reason, it is worth thinking about which is the best language for you prior to starting.

Generically, there is no right answer to this question as it depends on what *you* do and your own prefernces. There are, however, a number of considerations which I believe can help you decide which programming language is right for you. Among other things, relevant factors to consider include the following:

1. What type of research will I be doing?
2. Are there any relevant network effects (for example coauthors)?
3. What is my current knowledge?
4. What type of support do I look for?
5. Do I have any deeper preferences to satisfy?

I will discuss these points briefly below.

2.2.1 1. What type of research will I be doing?

There are certain types of research where specific languags may be especially well suited to what you are doing. For example, if you plan to be largely working with and cleaning data and performing econometric analyses, you could conceivably work with Julia, R, Python or Stata (among others). However, Stata or R might be the most logical choices, because often these are the first place where new econometric routines turn up. Often econometricians or other applied reseachers release new tools in Stata or in R first, and then only later do they appear in other languages.

If, however, you plan to be working with more modern machine learning (ML) tools, Python or R are probably leaders here. While ML routines are also certainly available in Stata and Julia (among others), at the moment my feeling is that they are most well developed in R and Python, as they are often contributed by non-economists.

You may have other preferences, for example your research may involve solving formal optimisation problems, for example in structural work. In this case, the optimisation routines in Matlab may be particularly relevant. Julia may catch up to Matlab with optimisation routines, though at the moment my feeling is that Matlab is still the leader here.

One useful way to have an idea about the types of codes that might be relevant in particular research projects is to look at the replication materials that are available with papers that you read. If you see that they all are using a certain language, this may be a useful insight for you. One thing to note with this is that historically, much of the code base in economics has been written in Stata, so it is likely that this will feature heavily in many fields.

2.2.2 2. Are there any relevant network effects?

A highly relevant consideration in many cases is the need to interact with coauthors, supervisors, and so forth. Often, our decision about the language we use may be largely driven by the people we work with. For example, if all of my coauthors use one language and want to be involved in programming in a paper, it makes sense for me to also work in that language. Or if part of a paper is already written by another coauthor in a particular language, it may make sense to continue working in this language to ease the work flow involved in the project.

As mentioned above, historically a lot of work in economics has been conducted in Stata, and the network effects from this have meant that Stata remains highly relevant. It seems to me that more recently there is some broadening of the scope of languages used, and you should of course feel free to *not* be guided by what others do to make your choice. The relevance of network effects will probably be most relevant if you plan to be frequently involved in collaborative projects.

2.2.3 3. What is my current knowledge?

You may already know a programming language and feel comfortable in it. In this course, I will ask that you all work with a programming language **that you have not worked with before**, and we will discuss a range of languages thoughout the course. The reason for this is that this way it is guaranteed that we will all start at a reasonably similar base (even if people have never programmed before), and at the end of the course you will at least feel comfortable in a new programming language. Often the spillovers from one language to the next will depend a little bit on their structure. For example, if you have worked before with a matrix-oriented language such as Matlab, you may wish to work in R which has something of a similar logic. However, if you have experience in Python and like the idea of looping, Stata may be a more logical choice in terms of a new syntax.

2.2.4 4. What type of support do I look for?

A fourth point to consider is related to the support you want in your work. In the section below we will discuss more fully the types of support available for different languages, and this definitely depends a lot on the language. Certain languages have dedicated and extremely active forums where you can ask questions, and get feedback. Languages also differ in terms of the degree of quality of the manuals, the activity level of people in online forums, and the quality of responses in online forums. Another relevant point in this dimension is how well the languages are supported in large language models like ChatGPT. This will be discussed at more length below.

At present, my subjective opinion is that if we consider the languages Stata, Python, R and Julia, the ordering of support depends highly upon what type of support one seeks. In terms of quality of manuals, the ordering is probably something like – from best to worst – (1) Stata (2) R (3) Python and Julia. However in terms of support from ChatGPT it is probably (1) Python (2) R (3) Julia (4) Stata. And in terms of forums, specifically for relatively new users, the support available is probably ordered as (1) Stata (2) Python and R (3) Julia.

2.2.5 5. Do I have any deeper preferences to satisfy?

Finally, you may have deeper preferences. For example, you may appreciate the value of Open Source software, and prefer to opt for open source tools. A surprising amount of the current technological world is built with or on top of the contributions of open source software (see the comic below!), and you may wish to be part of this. You may also have other preferences which are relevant for you at the moment you decide.

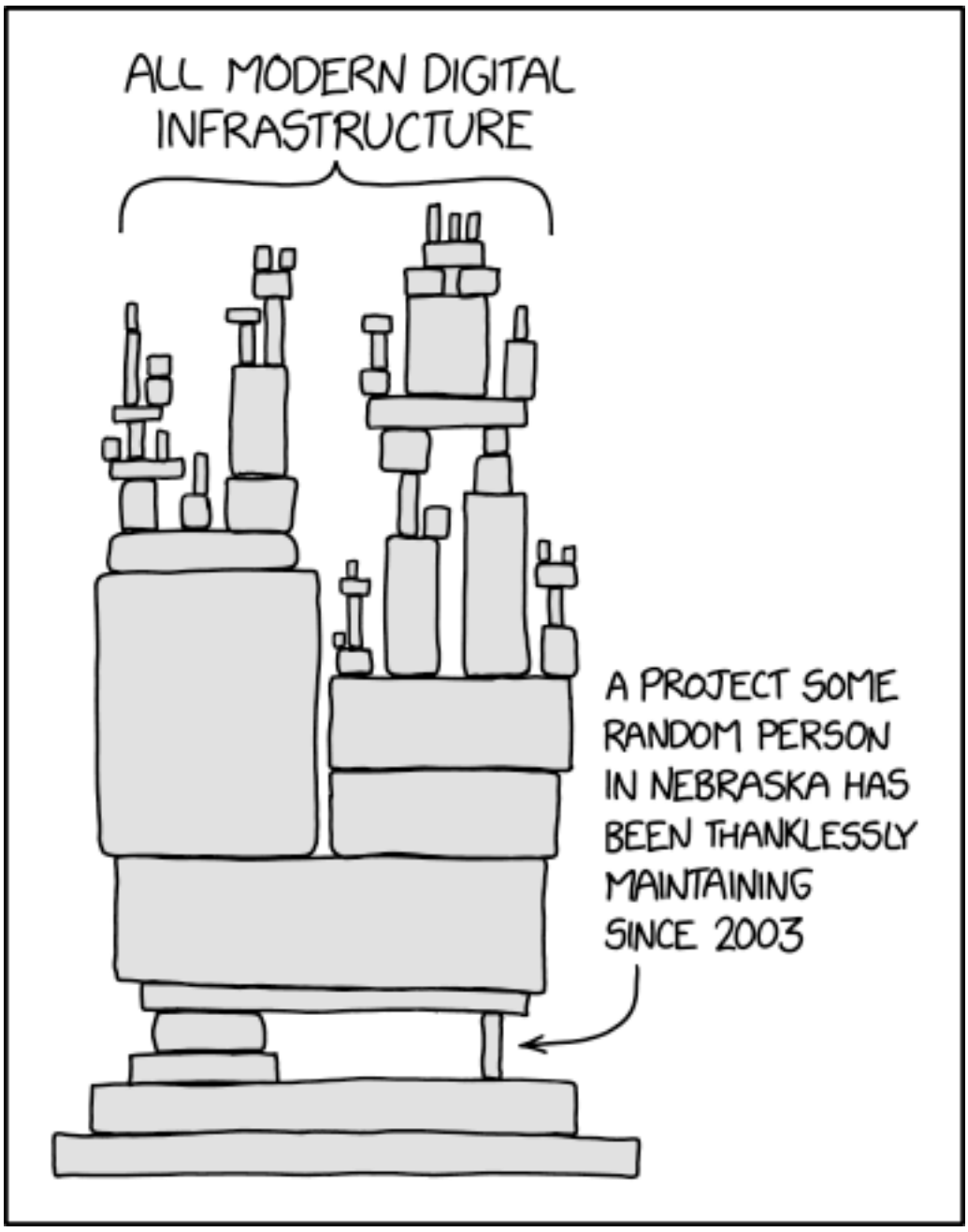


Figure 2.1: Dependencies (from xkcd.com)

2.3 How Should I Learn a New Language?

There are a number of tools which can help you in learning a new language, but by far, the most useful way to learning a programming language is by programming in it! Right at the beginning, it is useful to understand the minimum basic elements of the language, which is what we will focus on immediately after this section of the course. At all points, my suggestion would be to put in practice what you are learning, working with examples which are interesting to you. Personally, I would not endorse trying to perfect your knowledge of a language in theory before you put it into practice. Once you have a basic understanding of the language, in my opinion, the best thing you can do is to start working in it, and learn the particular things you need in your everyday usage by trying to do particular things of interest, for example generate graphs, examine data, and so forth.

There is an important caveats here, and this is that I would certainly *not* endorse using a language for a real paper before you are sure that you understand the key elements. There are certain elements of research projects which are critical, for example cleaning and joining data, and if done incorrectly can mean that your final results are entirely wrong. We will discuss these elements at more length later in the course, but for now it is worth keeping in mind that while learning by doing is an efficient way to figure out how a language works and how it doesn't work, when you are dealing with real programs, it is important to build in checks throughout your work to ensure that you aren't making mistakes. We will discuss this a little later in the course.

Beyond learnign by doing, there are a number of highly useful resources online that will help you get up to speed. We discuss these below.

2.3.1 Online Forums

A hugely useful source of information are online forums which let you ask questions, and suggest answers to questions if you know about a particular language. The most common forum is Stack Overflow where you can generate an account and ask questions about any language. These questions are often answered within minutes, and multiple responses can be provided, with individuals voting to highlight best answers. As well as a place to answer your own qustions, Stack Overflow is a hugely relevant resource to search for previous responses. Frequently, questions you may have are already answered on Stack Overflow, and can be found by searching Stack overflow or by using a search engine.

For certain languages, alternative forums exist. For example, the Statalist in the case of Stata has a much richer set of questions and answers than Stack overflow. This would be my first port of call for Stata-related queries, with the many questions already answered in the archives of the list. This can also be searched quite simply prior to asking a new question, and a well-established etiquette exists see here, for example.

2.3.2 Large Language Models

More and more, large language models (LLMs) like chatGPT and other Artificial Intelligence (AI) tools can help us with our programming tasks. Certain platforms also have AIs which can help you in developing your code, such as github's copilot. If you are unsure of certain language constructs, using an LLM can be very helpful, though it is also important to generate your own understanding of the language alongside inputs from LLMs. It is also relevant to note that the quality of responses from LLMs will depend slightly upon the amount of data they have to learn from, though in general for most languages we encounter LLMs are do a good job.

My personal view is that you should avoid using these LLMs as a crutch. For the large majority of types of programming we will be doing in economic research, we will not want to be relying on vibe coding, nor outsourcing the thinking to LLMs. Of course, LLMs can help increase productivity and offer suggestions for ways to optimise code. Things like line completion in VS code and other constructs can be very helpful to speed up programming, but we must be vigilant in ensuring that we're getting what we actually want. Actually learning how to programme the old fashioned way and knowing syntax off by heart offers considerable benefits in terms of understanding code, as well as more generally getting a feel for different programming constructs. It's also useful for when you want to programme when you're on the road with less access to wifi, or in settings without access to internet, such as data rooms maintained by governmental data providers.

2.4 Hardware and Operating Systems

While this course is focused on learning a scientific computing languages for research in economics, it is useful for you to have some understanding of the hardware you use, as well as make an informed choice about operating systems. While generally, current laptops have remarkable processing capacity, for certain types of projects you may run into bottlenecks either in terms of memory or processing power.

2.4.1 Hardware

Broadly speaking, and at a very high level of abstraction, there will be three particularly relevant elements for us when thinking about the hardware we use. The first is the processing capacity of the computer, which depends upon the Central Processing Unit, or CPU. The second is the virtual memory capacity the computer has to manipulate specific data bases, which depends upon the Random Access Memory, or RAM. And the last is the hard disk space and the type of hard disk used to store all of the data and programmes installed on the computer.

We will not discuss this in great depth, but it is useful to note that particular types of projects will imply demands on particular types of resources, and if you are deciding on what type of computer you need, this will be driven by the type of research you are doing. For example, if you are doing research which is particularly demanding in terms of numerical processing or in terms of calculations, the most important thing that you will want to consider will be the type and number of CPUs that you have. More recent generations of CPUs have greater processing capacities[1] In extremely demanding projects where processing capacity is a limiting constraint, processing can even be done on the Graphical Processing Unit, or GPU, of your computer, though this is likely not relevant for the majority of projects in economics, so we won't dig into it here. If, however, your research requires working with large databases, for example running regressions with many millions of operations, it is more likely that your bottleneck will be in terms of RAM. Given that you need enough RAM to hold databases in working memory, if you plan to work with databases of >16GB for example, you will require a machine with more than 16GB of RAM. Finally, though potentially less limiting, is the relevance of the hard disk. If you plan to store massive amounts of data on your computer (of the order of terrabytes), you may find that you run out of hard disk space. In general this is relatively easy to remedy by purchasing an external hard drive, but is also a choice which can be made when deciding on a computer to use.

2.4.2 Operating Systems

The operating system is the over-arching environment where we interact with all our programs, data management, and so forth. It is extremely likely that you will work with some flavour of Windows, OS, or Unix-based operating systems. Often, operating systems are bundled with hardware, and when you purchase your computer this choice will be made.

The simplicity with which you are able to program *does* vary by the operating system you choose. In my opinion, Unix based operating systems are by far the most flexible and most well-suited to scientific computing. However, there is a learning curve involved in getting up to speed with a new OS, and you may not wish to do this. Regardless of the choice of OS you make, all the tools we examine in the course will be available, and more generally, nearly all the computational tools you will require in economics can be somehow made to work on each type of OS.

[1] CPU capacity is measured in FLOPS, or floating point operations per second. The capacity of computers has grown more or less constantly over time, with an ever growing increase in the capacity in the number of FLOPS. This has been characterised as Moore's Law.

3 Getting Set Up

In the following section, we will provide a primer to number of languages, specifically R, Python and Stata. These introduction seek to provide the necessary tools to get up and running with the things that we will need throughout the rest of the course, but are not desiged to be comprehensive overviews of the entirety of each language. We will further develop our understanding of the languages as we progress with this course.

Below we provide information about installing necessary components for each software.

3.1 R

3.1.1 Introduction

R has a long history, dating the early 1980s, and also an ever-growing body of user-contributed add-on packages. It also has an active community providing online support on forums such as Stack Overflow, and a substantial corpus of code online, meaning that large language models will be well suited to support you in your programming in R. It provides you with the large majority of tools you will need throughout this course, as well as the programming environment necessary to extend available tools where required.

3.2 Installing and Working with R

Information on how to download R is available on the R website, and further support can be found searching the web. In general, I recommend that you install an Integrated Development Environment (IDE) such as RStudio. RStudio (image below) provides a user-friendly interface for R, allowing you to view any elements that you have generated in the memory in R, output like graphics, help documentation, and so forth. An example of what RStudio looks like is provided below, where you can see that code is visible, as well as graphical output, and information about what is in the your "environment". For further information on installing RStudio, see the RStudio website. You are of course welcome to use R however you prefer, and you may prefer to edit code in your favourite text editor and execute code from the command line. Throughout the rest of these resources, we will assume that you have R installed on your computer and have some way to interact with it, either via an IDE or some other work flow that suits you.

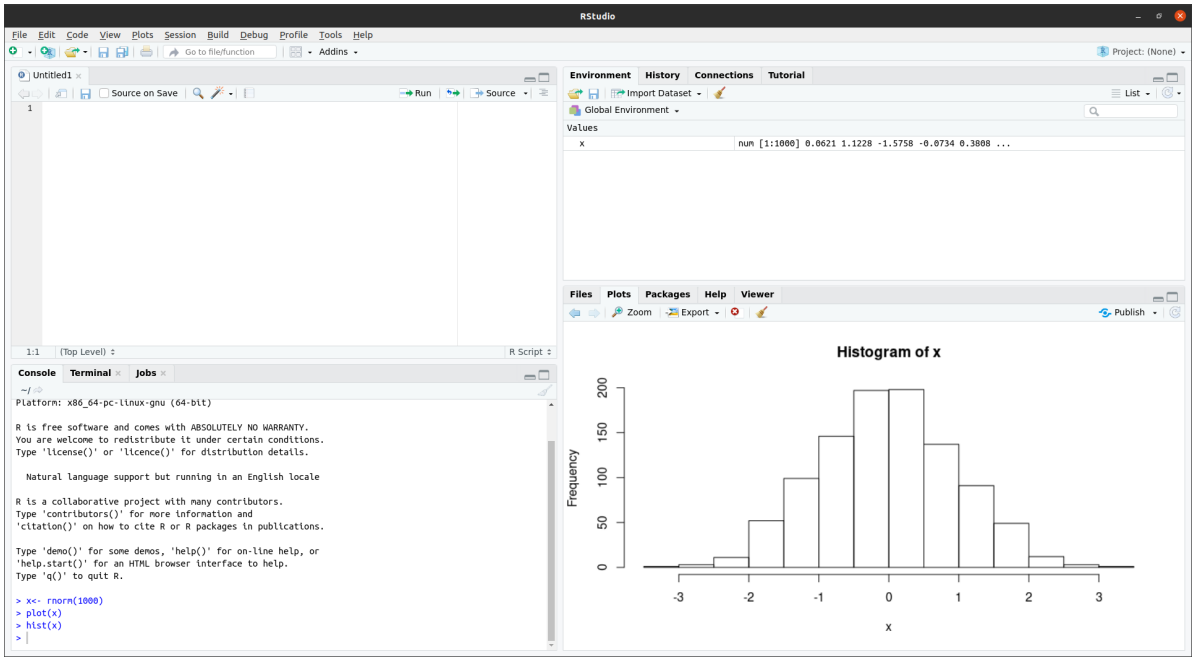


Figure 3.1: RStudio screenshot

3.3 Further Resources

From here we will move to a first class about R as a language, and an overview of a number of key elements. If you are interested in generating a more complete overview of R, a number of free resources can be consulted, such as the following books provided free online:

- Wickham, Çetinkaya-Rundel, and Grolemund (2023) R for Data Science
- Wickham (2016) ggplot2: Elegant Graphics for Data Analysis
- Wilke (2019) Fundamentals of Data Visualization: A Primer on Making Informative and Compelling Figures

A range of other textbooks are available that may help you with your work in R including:

- Matloff (2011) The Art of R Programming.

and, with a particular focus on economics and causal methods:

- Huber (2023) Causal Analysis: Impact Evaluation and Causal Machine Learning with Applications in R.

3.4 Python

3.4.1 Introduction

Python was developed in the late 1980s by Guido van Rossum, and is an open source language which has gained considerable traction with users and developers, meaning that a diverse array of packages now exist to expand the base language. Like R, it has an active community providing online support on forums such as Stack Overflow, and a lot of online code available. It provides you with the large majority of tools you will need throughout this course, as well as the programming environment necessary to extend available tools where required. Python has had a number of major version upgrades throughout its history, but now versions 3 and above of Python are widely adopted. I recommend working with some version greater than 3 when you install Python, as code written in version 2 will not necessarily work in version 3.

3.4.2 Installing and Working with Python

Information on how to install Python can be found at the Python website, which has a brief wiki discussing installation on various operating systems. Once you have Python installed in your operating system you could work directly with Python from the command line by typing "Python" and you may see output like the below.

```
python
```

```
Python 3.8.10 (default, May 26 2023, 14:05:08)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

I recommend that along with Python, you install in Integrated Development Environment (IDE), or simply work with a text editor which provides the capabilities of an IDE. There are many IDEs which can be used with Python, see for example the list here. I can recommend Spyder (see below for a screen shot of what this looks like), or alternatively, might recommend VS Code as a broader editor which offers these capabilities and more. The value of having this IDE is that you can easily view any elements that you have generated in the memory in Python, output like graphics, help documentation, and so forth. An example of what Spyder looks like is provided below, where you can see that code is visible on the left, as well as graphical output, and calls to the command line on the right. You can also display other pieces of information such as documentation and information about what is in the your "environment". For further information on installing Spyder, see the Spyder website. You are of course welcome to use Python however you prefer, and you may prefer to edit code in your favourite text editor and execute code from the command line. Throughout the rest of these resources, we will assume

that you have Python installed on your computer and have some way to interact with it, either via an IDE or some other work flow that suits you.

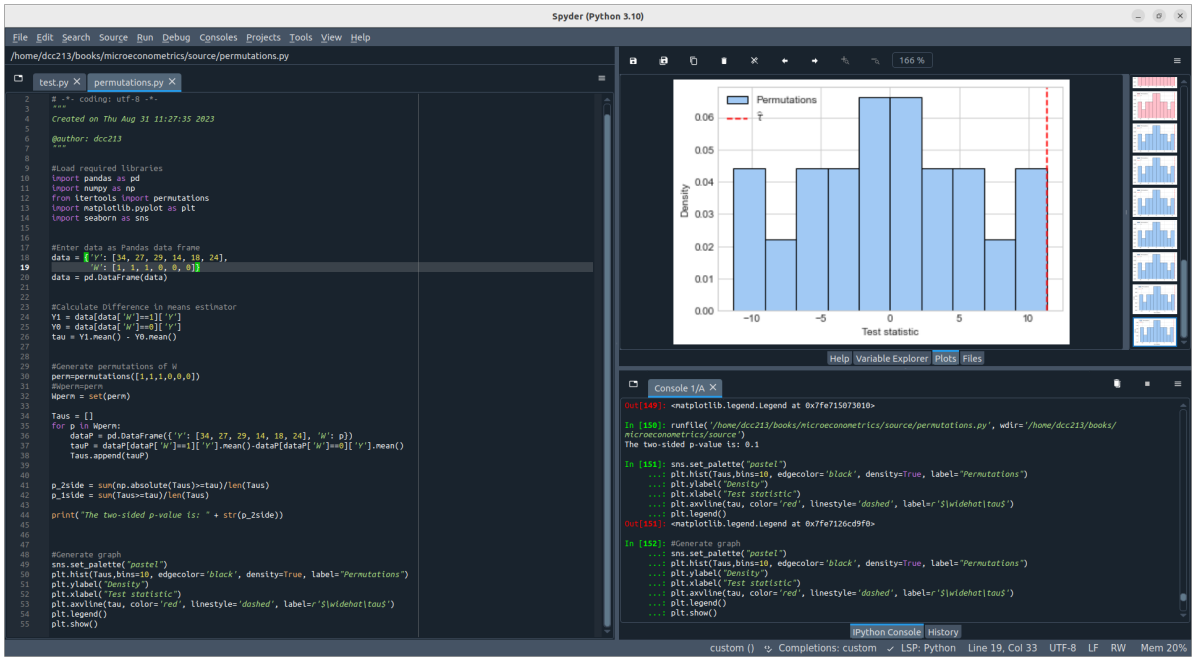


Figure 3.2: Spyder screenshot

3.4.3 Further Resources

A number of broader resources are available about Python. A very nice free online text book is:

- McKinney (2023) Python for Data Analysis.

3.5 Stata

3.5.1 Introduction

Stata was first developed in the mid 1980s, and is now up to release 18. Stata is particularly strong at data manipulation and statistical modelling, and along with their own developers, has an active community of researchers who contribute user-written packages. It also has an active community providing online support on forums such as Stata List. It provides you with the large majority of tools you will need throughout this book, as well as the programming environment necessary to extend available tools where required. Stata releases a new version

of the program every couple of years, and a strength of Stata is that it guarantees backwards compatability. Thus, code written in older versions of Stata can be run seamlessly in future versions. However, code written in newer versions of Stata will not necessarily work in older versions of Stata, because in each version of Stata new tools are added.

3.5.2 Installing and Working with Stata

Stata is a paid software, and installation is described when licences are purchased. The installation of Stata is relatively straight-forward provided access is available.

For students at Exeter, Stata is available for free fom the Home Software Hub. Details regarding the process of installing can be found at this page: https://www.exeter.ac.uk/departments/it/new/softwarecatalogue/stata/

When installing Stata, the language comes with its own Graphical User Interface (GUI) which is quite user friendly. This GUI provides a command line to interact with Stata and an output window, as well as a list of commands recently executed. It also has a number of emerging windows including a built in file editor, and a graphics window. An example of what Stata looks like is provided below, where you can see that a code editor is visible, as well as a screen below where one can interact directly with Stata.

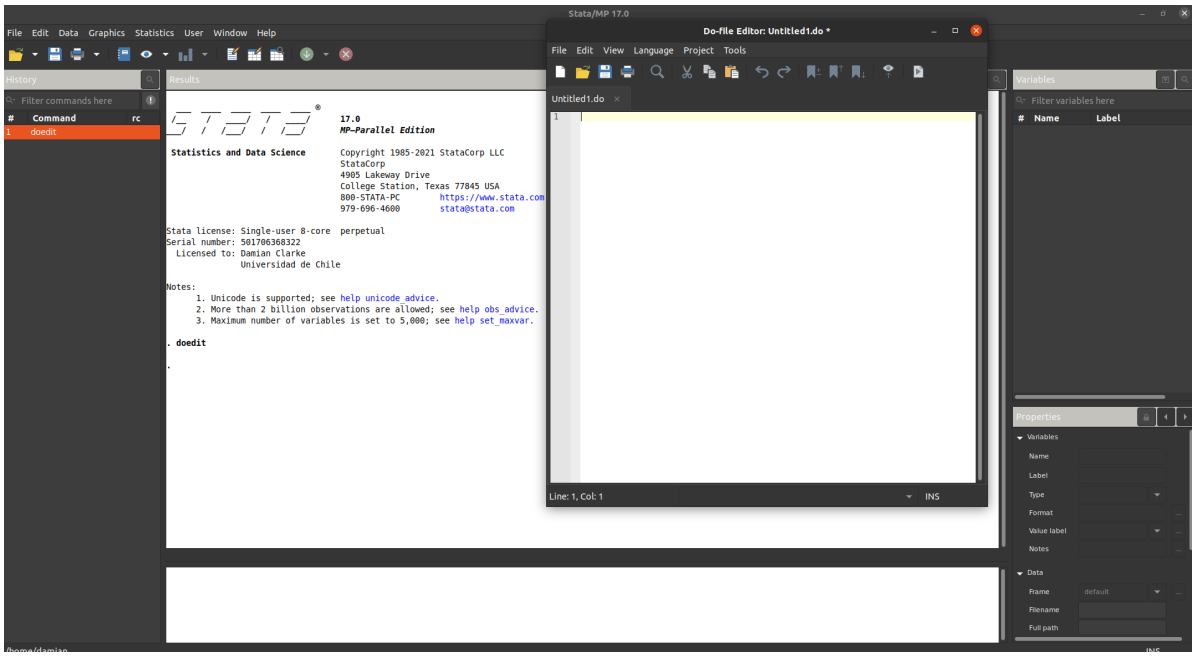


Figure 3.3: Stata

3.5.3 Further Resources

From here we will move to a first tutorial about Stata as a language, and an overview of a number of key elements. If you are interested in generating a more complete overview of Stata, I would highly recommend the list of resources provided on the Stata webpage. At this link they include things such as books and blogs on Stata, but also provide a number of video tutorials and online resources specifically related to learning Stata.

3.6 Other Tools

There are surely a range of other tools that you will want to have in your arsenal, and each person's optimal "research infrastructure" will be different. As a minimum, doing research well in a computationally demanding environment likely requires the following types of tools, and we will work with many of these throughout the semester:

- A code editor (note, this might be bundled with the language you use), or you may prefer something like emacs, VS Code, or so forth which offers broader tools.
- A way to write your paper. This may be some typesetting tool like LaTeX
- Some way to keep track of changes in versions of your code. Git is the most common tool for this.
- Some scientific programming language like the ones discussed above.

There are also a range of other tools which you may use which depending on your work flow. This may include things like bibliography management tools, alternative softwares for typesetting slides or online presentation of papers (including markdown-style options), programmes for optimising communication and workflow in teams (such as slack), some sort of note taking, to do list management, and calendar tool for your, and so forth. Of course the list goes on, and will depend on your own particular needs and wants!

Part II

Programming Languages

An Overview of the Languages We Will Use

An overview of each of the languages we will explore in this course is available in this section. These are set up in three separate pages; one for R, one for Python, and one for Stata. While it may be useful for you to have access to information on each of the three languages, you are not expected to follow each of these for the course, but rather simply go along with the language that you plan to work in for the rest of the course.

In general, these pages aim to be as "mirrored" as possible, covering the same information and tools. In certain cases languages have different logic and so two pages will not have the exact same ordering, examples, or section titles, but in general the pages are well aligned, and at the end will give you the same knowledge base in each language. Throughout the rest of this course we will also introduce some elements which are not introduced on these first pages, but which are useful later on as we delve into specific elements.

4 An Introduction to R

5 Preliminaries

We will assume that when you are reading this you have downloaded R and have been able to access R in some way. This may be via the command line, but most likely we imagine you will have some GUI which allows you to see a number of windows including a space for graphical output, a description of the contents of your R Environment (for example any data you have loaded), potentially a place to write your code in R, and, most importantly, a "shell", or R interface, which looks like the following:

```
R
```

```
R version 4.1.2 (2021-11-01) -- "Bird Hippie"
Copyright (C) 2021 The R Foundation for Statistical Computing
Platform: x86_64-pc-linux-gnu (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

  Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

>
```

Here, R code can be entered directly following a prompt which is the `>` symbol at the bottom of the window above. From the moment you open `R` you are in an `R` session until the moment you close it, from now on understand a session as this time between you open and close `R`. The most simple way to execute code in `R`is to simply input commands into your `R` console. So, for example, you could write the following at to the right of the `>` symbol, and R will evaluate this code:

```
4 + 4
```

```
[1] 8
```

As you can see, R has understood the instruction that you want to add 4 plus 4 and show the result, which is evaluated, and output directly below the code in the R console. Later in this chapter we will properly explain these mathematical operations and the different R data types. Beyond evaluating such simple mathematical operations, R has a host of in-built functions which can be invoked at the command line. While we will interact with this in more detail later, a simple example is the following, which you can type in your R console:

```
getwd()
```

```
[1] "/home/dcc213/trabajo/Teaching/Exeter/ResearchMethods-I/site"
```

This function which can be understood as "get working directory" will show you the path on your computer where R is currently working. Thus, any files saved will be exported there, and R will search for any thing to import from this directory. If you want to change this directory you must use the function `setwd("Path/to/the/directory/you/want/to/use")` with the path written between `" "` or `' '` in order to R understand that is a character, again a topic that will be seen deeper latter. It is important to note that paths should be separated with the slash character: `/` rather than the backlash `\`, and this will work on any operating system. Indeed, if you try to set directories with a backslash, you will see that R returns an error.

We will return to discuss functions in more detail a bit later on. For now you will note that we have simply been typing instructions to R directly into the console, and R has been providing us with output. Generally, we will not work this way in an empirical research project, as we will want to develop code over time, and be able to return to R and replicate code in the future. For this reason, generally we will work in R scripts, or files named as `something.R` (where `something` will be logical name for the file). We will use these scripts to store the commands which we require for particular processes or routines, and can then save them to the disk of our computer before running them in R. These scripts can be generated in any type of text editor, and then can be run in R using the `source()` command. For example, below, you can see a screenshot of a session on R Studio where we have a basic R script named `firstRscript.R`, and below this, in the R console we see that the script is run using `source` and output is provided.

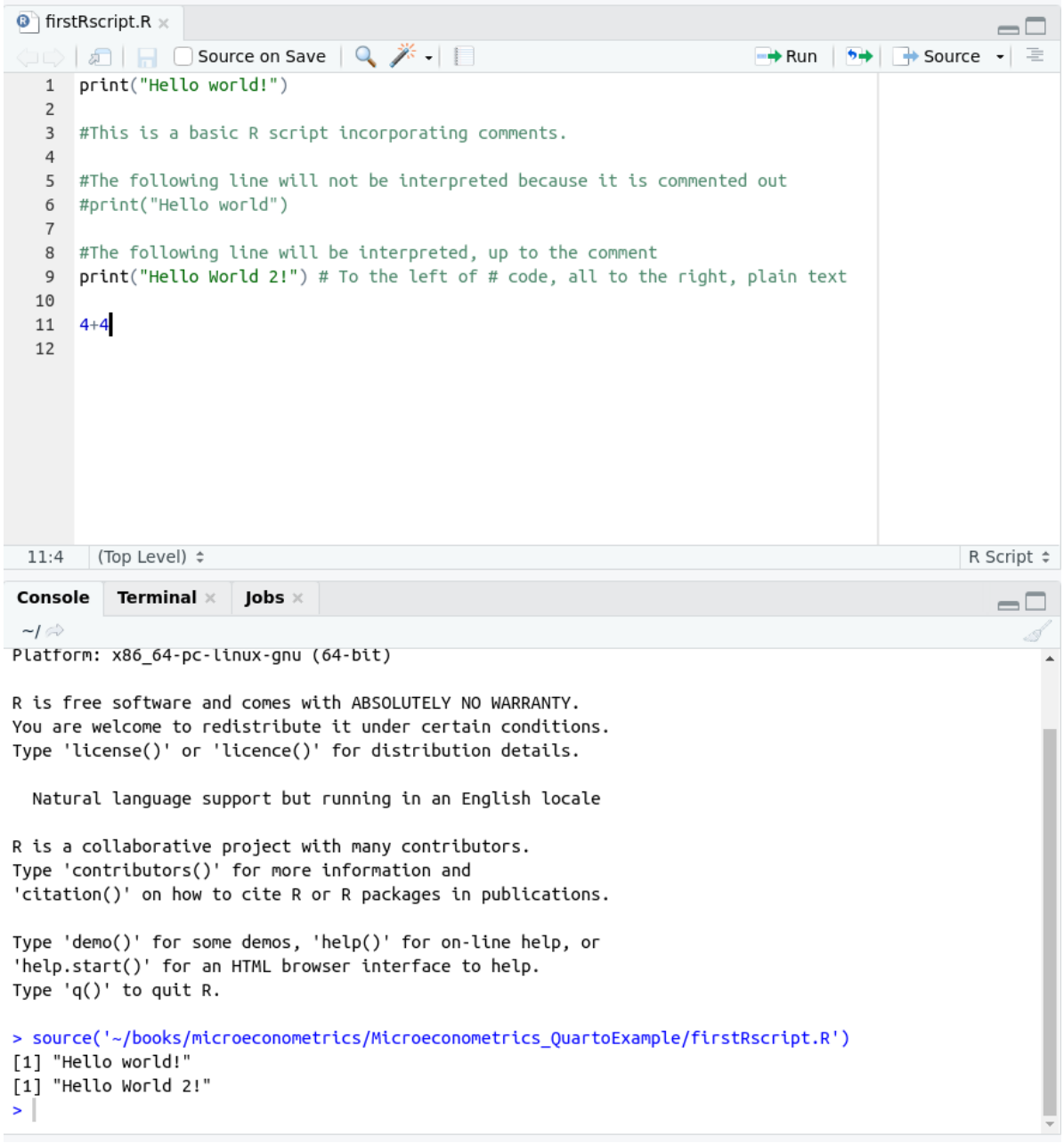


Figure 5.1: An R Script and Output in RStudio

You may notice something strange about the code and the output in the `R` console. The R script has 7 distinct lines of text, but the console only produces two lines of output. The reason for this is because we have included comments, or messages for human eyes, but not for the computer to interpret. Any time that the `#` character is included in code, `R` understands

that this is a comment, and hence is ignored in executing the code. So, for example, when we write:

```
print("Hello World!")
```

```
[1] "Hello World!"
```

in our code, the `R` console correctly echoes *Hello World!* back at us. However, if we enter precisely the same code, but begin the line with a comment symbol, `R` will not do anything given that it interprets everythin to the right of `#` as plain text which should be ignored:

```
# print("Hello World")
```

Similarly, comments can be used within lines:

```
print("Hello World") # To the left of # code, all to the right, plain text
```

```
[1] "Hello World"
```

As you can, this lines has executed the instruction to the left of `#` but ignores all those to the right. It is good practice to comment code extensively, both for sharing code with other users, but also for sharing code with yourself in the future, where you may not remember precisely what you were thinking when you originally wrote your code!

One final thing to note from this `R` script is that the output from the final line of code (`4+4`) was not printed in the console. In R, when you type `4+4` at the command line, you get immediate output because R is operating in an interactive mode. In interactive mode, R evaluates expressions as soon as you enter them and displays the result to the console. However, when you write `4+4` in a script and then run the script, there is no automatic output to the console. However, it is important to note that the calculation *is* conducted. You can confirm this yourself if you'd like by explicitly requesting that `R`provide this output when it runs the code, replacing `4+4` in the code with `print(4+4)`.

> **i** R and the Operating System
>
> While below we focus on the particulars of the R language, R interacts directly with your computer's file system, and you can move through your file system and see where you are with a number of R commands. In particular, some useful functions in this line are below:
>
> - `getwd()`: shows your **current working directory** (where R is "looking").

- `setwd("path/to/folder")`: changes the working directory.
- `list.files()` or `dir()`: lists the files in the current directory.
- Files containing R code (ie scripts which you may wish to "run" in R) usually end with `.R`.
- These files can be run using commands like `source("path/to/myscript.R")`
- You can save data which you are working with in R to the disk with functions like `saveRDS(object, "file.rds")` and later load it with `readRDS("file.rds")`. Below we will discuss this at more length.

5.0.1 Checking this out locally

1. Open R on your system. What is your current working directory (`getwd()`)?
2. Create a new folder on your operating system. Use `setwd()` in R to move into that folder.

3. Check the contents with `list.files()`. What do you see?

4. Create a simple object (e.g. `x <- 1`) and save it using `saveRDS()`. Now what do you see with `list.files()`

5. Then close R, reopen, and load the object you generated with `readRDS()`.
6. Generate a file called myFirstFile.R and save it in the directory you created. Confirm that you can run it using the `source()` function.

6 An introduction to the language and data types

Until now we have executed some code in `R` and seem some specific functions which print output, but we have never stored any results for later use within our code. Generally we *will* want to store the output of intermediate steps in our code for later use. This can be done in `R` using any of the following assignment symbols: `->`, `=` or `<-` (though `->` is generally preferred). For example, below we store a number of values as a variable which we define, arbitrarily named `x`:

```
x <- 1
x
```

```
[1] 1
```

```
x = 2
x
```

```
[1] 2
```

```
3 -> x
```

With the use of `->` or `<-` the arrows make clear that we are assigning some particular value (or values) to a named variable, where the named variable is indicated by the arrow head. However, with the use of `=` the assignment must respect `name = value`. If, instead, we try to assign some variable as `value=name` an error will occur:

```
4 = x
```

```
Error in 4 = x: invalid (do_set) left-hand side to assignment
```

This error implies that the code has not been executed, and if we inspect the contents of `x` we will see that its prior value remains.

```
x
```

```
[1] 3
```

In general, a command can produce a number of conditions which contain important messages for us. These are:

- `Error`: These kinds of messages tell you that something has gone wrong and `R` couldn't execute your code. Usually error messages include a brief explanation of what has gone wrong, to help you diagnose and fix your code. Any expected results from your command will not be assigned.
- `Warning`: These kinds of message indicate that somes has occurred that could potentially cause problems, but is not critical for code execution. In these cases, `R` will still execute the code and give the result, but warn you that not everything is in order and this may cause inaccurate results or future problems. At times this messages can be ignored without further issues, but warnings can also be a sign of peril, and so time should be taken to review warnings and understand whether they are problematic.
- `Message`: This kinds of messages simply provide information about the execution of your command, and `R` is providing you with some piece of information which is useful for you to know. This will not affect the output of your code, and does not generally indicate an error with your code.

6.1 Data Types

6.1.1 Text (String) Data

To get up and running with `R`, it is useful to understand the different ways which data can be stored. `R` can hold a number of different types of objects in its working memory, and it is important that `R` understands what type of information we are passing or storing. Among others, R allows for information to be stored as characters, numeric, arrays, data frames and lists. Different types of data allow different types of operations, and should be stored in different ways. Characters are perhaps the conceptually simplest type of data. Characters are simply strings of text, and should be specified between `" "`, or `' '`. We have seen the use of characters when printing "Hello world" previously. If characters are not indicated between strings, `R` will understand that they refer to variable names. For example, returning to *Hello World*, if this is entered without quotes, an error will appear:

```
Hello World
```

```
Error: <text>:1:7: unexpected symbol
1: Hello World
          ^
```

Indeed, if we look carefully at the error message we may see that the error has occurred when `R` arrives to *World*. This is because it assumes that *Hello* is a variable name, but this must be followed by some valid assignment. For example, had we written `Hello<-"world"` this would have been perfectly fine! Alternatively, if we do actually want to write this as characters, we can do so simply by enclosing the text in quotes:

```
"Hello World"
```

```
[1] "Hello World"
```

We can confirm that this is effectively understood to be character data by using the `class()` function. This function indicates the type of data of the argument it receives, as we can see below. Note also that if we generate some variable using the assignment operators discussed above, `class()` will return to us the type of data that the variable contains:

```
class("Hello World")
```

```
[1] "character"
```

```
my_var <- "Hello World"
class(my_var)
```

```
[1] "character"
```

6.1.2 Numeric Data

Numeric data is entered without any special behaviour, simply writing the number in the case of scalar values. Indeed, if numeric data is accidentally enclosed between quotes, it will be treated as character, and *not* numeric data. For example, below we assign two variables based on the number `2` and confirm that one is numeric while the other is viewed as a character.

```
num1<-2
num2<-"2"
class(num1)
```

```
[1] "numeric"
```

```
class(num2)
```

```
[1] "character"
```

If you try to perform numerical operations based on the above variables (more on this below), you will see that `num1` can be involved in such calculations, while `num2` cannot.

6.1.3 Vectors and Arrays

Generally, we will be dealing with more than a single number in our work, and for this reason need ways to collect groups of numbers. Vectors and arrays (or matrices) provide some ways to do this. An easy way to create vectors or arrays is via the `c()` function, which stands for "combine". This allows for numbers to be combined in a vector. For example,

```
c(1,2)
```

```
[1] 1 2
```

This is required to group numbers. If we simply write a series of numbers without wrapping them in the `c` function, error will occur:

```
1 2
```

```
Error: <text>:1:3: unexpected numeric constant
1: 1 2
      ^
```

```
1,2
```

```
Error: <text>:1:2: unexpected ','
1: 1,
     ^
```

A precaution to note here is related to mixing of data types in objects that combine different elements because different data types will influence the way this combination works. For example vectors will coerce all data to the most restrictive data type, if you mix characters with numbers in a vector, `R` will coerce the vector type to a character:

```r
class(c(1,2))
```

```
[1] "numeric"
```

```r
class(c(1,"2"))
```

```
[1] "character"
```

This behaviour can be overruled using `as.` which allows for specific data types to be indicated. For example, if we take the previous vector of a number and a character, we can ensure that the result is numerica as follows:

```r
as.numeric(c(1,"2"))
```

```
[1] 1 2
```

However, note that this does has limits if the underlying object cannot be converted to that data type!

```r
as.numeric(c(1,"A"))
```

```
Warning: NAs introduced by coercion
```

```
[1]  1 NA
```

In the above, the inclusion of a string which cannot be logically converted to a number implies that `R` has issued a warning and converted this element to `NA`. `NA` is `R`'s "Not available" or missing value indentifier. We will discuss a bit later on about how to deal with this in your programming.

You can also create arrays with the `matrix` function, this allows you to get objects in two dimensiones, rows and columns

```r
matrix(c(1,2,3,4), nrow = 2)
```

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

It is worth noting a couple of potentially important details with these matrices. A first point which we already have come across is related to coercion of types. For example, if passing a vector via the matrix command, R will coerce the matrix to the more restrictive data type:

```
matrix(c(1,2,3,"4"), nrow = 2)
```

```
     [,1] [,2]
[1,] "1"  "3"
[2,] "2"  "4"
```

A second, potentially more surprising behaviour is the "recycling" of values if matrices are not of the correct length. For example, as you can see below, if we add a 5th element to the vector, this can no longer correctly fit in two rows. So R's default behaviour is to begin "recycling" values from the beginning until the matrix is full, and so it returns to the first element of the vector and fills in the "missing" cell. If, however, you actually want this to be an NA value, you need to instruct R about this, as in the second case below.

```
matrix(c(1,2,3,4,5), nrow = 2)
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    1
```

```
matrix(c(1,2,3,4,5,NA), nrow = 2)
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4   NA
```

6.1.4 Data frames

Likely the type of object which we will be working with the most in our analysis in R are data frames. Data frames allow us to mix mutliple other types of data, viewing the collection of variables and observations as a database with multiple rows and columns. Within data frames, we can easily subset to certain rows (observations), columns (variables), and conduct univariate and multivariate analysis. It will likely be the starting point for much of your work when you import a standard dataset into R. As an example we create a data frame named df that will use later in other examples:

```
df <- data.frame(A = c(1, 2, 3), B = c(4, 5, 6),
                 C = c("XXX", "YYY", "ZZZ"))
df
```

```
  A B   C
1 1 4 XXX
2 2 5 YYY
3 3 6 ZZZ
```

As we have entered the data frame, we have three columns named "A", "B" and "C". Each column consists of three observations, and when the data frame is printed, we see that R has helpfully assigned observation numbers in the left-most side of the data frame. Shortly we will see more about how to manipulate and work with data frames.

The last data type we will briefly introduce in this section are lists. Lists allow us a very flexible way to group things together, allowing us to store any data type we need. For example, as we see below in a single object we are combining characters, numbers and a data frame, all in a single "container" called a list. The flexibility of lists to contain many data types is why many functions that we will work with later on return their results as lists or as objects created from lists. As in data frames, in lists, each element is given its own name, and when printed are referred to by this name:

```
l <- list(Character = c("Hello", "World"),
          Number = c(1, 2, 3, 4),
          DataFrame = df)
l
```

```
$Character
[1] "Hello" "World"

$Number
[1] 1 2 3 4

$DataFrame
  A B   C
1 1 4 XXX
2 2 5 YYY
3 3 6 ZZZ
```

6.2 Subsetting Multidimensional Elements

As we saw in data frames and lists, often we will work with objects that store multiple elements. Sometimes we may want to work with a subset of this multidimensional data instead the entire object. In data frames, or lists, you can easily access an entire column, or element, by using `$` followed by the column, or object, name:

```
df$A
```

```
[1] 1 2 3
```

```
l$Character
```

```
[1] "Hello" "World"
```

But sometimes you may want to access more, or less, than one entire column or object, so the more general way to access to the elements are with the use of `[ ]` to subset. For one dimensional objects like vectors or a column of a data frame, we can access elements by using the number of the element position between `[ ]`. For example, for a column "A" in our data frame:

```
df$A[1]
```

```
[1] 1
```

Identical nomenclature is used for vectors.

This syntax work to access more than a single object by combining it with a vector, for example as follows to access the first and third element of `df$A`:

```
df$A[c(1, 3)]
```

```
[1] 1 3
```

In the case of multidimensional elements such as an entire data frame or array you should use the `[row,column]` syntax to control subsetting. If you would like to access an entire row, this can be requested by just leaving the row position blank:

```
df[,1]
```

```
[1] 1 2 3
```

with the same logic holding to access an entire column:

```
df[1,]
```

```
  A B   C
1 1 4 XXX
```

Similarly, all of the above can be mixed to grab multiple rows or columns.

```
df[2,2]
```

```
[1] 5
```

```
df[c(1,3), 2]
```

```
[1] 4 6
```

In lists the syntax is slightly different given the way that lists are internally stored. In this case, first you must access the object of interst with a double `[ ]` and then once you access the object it behaves in the same way that we saw above. For example, if we wanted to access the second row and second column of our data frame `df` stored in the list `l`, we can do so as follows, noting that the data frame is stored as the third item in the list:

```
l[[3]][2,2]
```

```
[1] 5
```

Alternatively, between the first brackets you can also use the element's name to select it for later processing. The following code exactly replicates the previous example, however arguably is less error prone, because it is clear precisely which elements we wish to select ("DataFrame" from the list, and column "B" from the data frame):

```
l[["DataFrame"]][2,"B"]
```

```
[1] 5
```

6.3 Basic Operations

The standard mathematical operations in R likely work how you would expect. Basic mathematical operations are as follows, with an example of their use below:

Symbol	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
** or ^	Power
%%	Modulus
%/%	Integer Division

A few examples

```
10 + 3
```

```
[1] 13
```

```
10 - 3
```

```
[1] 7
```

```
10 * 3
```

```
[1] 30
```

```
10 / 3
```

```
[1] 3.333333
```

```
10 ^ 3
```

```
[1] 1000
```

```
10 ** 3
```

```
[1] 1000
```

```
10 %% 3
```

```
[1] 1
```

```
10 %/% 3
```

```
[1] 3
```

Also when you combine different operators it respects the PEMDAS order, first resolving Parentheses, second solve Exponents, third Multiplication and Division, fourth Addition and Substraction. Both multiplication and division, and addition and substraction are evaluated left to right, rather than sequentially by operation, so it is important to indicate to `R` explicitly what you need using parentheses in certain cases. For example in order to solve

$$-\frac{5+3^{5-3}}{5\times 3}$$

You should use the code

```
-(5+3^(5-3))/(5*3)
```

```
[1] -0.9333333
```

Specifically in the case of matrices, mathematical operations are assumed to refer to cell by cell calculations. If, instead, you would like to undertake matrix multiplication, this must be requested with **`%*%`**. For example, consider the two operations below and why they differ, also noting the `t()` is the matrix transpose operation.

```
a=matrix(c(1,2,3,4,5,6), nrow=2)
a*a
```

```
     [,1] [,2] [,3]
[1,]    1    9   25
[2,]    4   16   36
```

```
a%*%t(a)
```

```
     [,1] [,2]
[1,]   35   44
[2,]   44   56
```

R also has logical operatos that returns a boolean value (TRUE or FALSE, also abbreviated as T or F) pointing if the statement is true or false. This operators are:

Symbol	Comparison
==	Equals
!=	Difference
>	Greater
>=	Greater or Equal
<	Less than
<=	Less than or Equal
%in%	If it is in
|	Or
&	And

A few examples

```
4 == 3
```

```
[1] FALSE
```

```
4 != 3
```

```
[1] TRUE
```

```
4 > 3
```

```
[1] TRUE
```

```
4 >= 3
```

```
[1] TRUE
```

```
4 < 3
```

```
[1] FALSE
```

```
4 <= 3
```

```
[1] FALSE
```

```
4 %in% c(1,2,3,4)
```

```
[1] TRUE
```

```
TRUE | FALSE
```

```
[1] TRUE
```

```
TRUE & FALSE
```

```
[1] FALSE
```

This boolean values are interpreted by `R` as the numbers 0 for `F` and 1 for `T`, characteristic that allows you work with them numerically and is useful when you want to check a series of conditions.

6.4 Functions

Beyond these logical and mathematical operators, `R` comes with a large suite of built in functions which perform specific tasks, as well as a large number of user-contributed functions (discussed below) which can extend the basic functionality. A basic installation of `R` comes with a range of packages. The functions included in `R`'s base package can be consulted by typing the following at your `R` console:

```
library(help="base")
```

Functions are designed to perform specific tasks, potentially accepting input (known as "arguments"), and returning output which you can interact with. We have already seen a number of functions above such as `getwd()` which we used without arguments, and `print()` which we have used with specific arguments such as "Hello World!". Arguments are simply the contents we provide the function inside the parentheses. Understanding how packages work, including the arguments they accept and the objects they return is key. This can be found by consulting the help file. If you know the function you wish to use, all these details can be found by typing `help(function_name)` or simply `?function_name`. For example, if we type `help(getwd)` we will see a section entitled "Arguments" which explains to us the arguments the function accepts, as well as the elements returned under the heading "Value". In certain cases if we are not sure exactly what function we need we can type `??search_term` and `R` will advise us of any functions which may do what we're after. For example, if we type `?? sum`, we see a range of functions, including one called `sum`. We can then consult the help file to see if this is precisely what we are after. Reading the help file we see that the function "returns the sum of all the values present in its arguments", and accepts numerical vectors. We can confirm that this is the case by applying this function to our data:

```
sum(df$A)
```

```
[1] 6
```

If you consult the help file to `sum` you will note quite prominently the option for "na.rm", with the explanation *Should missing values (including 'NaN') be removed?*. You will also note that in the "Usage" section of the help file, "na.rm = FALSE" is explicitly indicated. Here we have encountered two particularly important elements: firstly, the concept of `NA` which we briefly mentioned above, and secondly the use of default options in functions. We will discuss these in turn. `NA` formally refers to Not available, or missing data. Similar concepts exist in other languages, such as NaN in Matlab, "." in Stata, and so forth. Frequently we may encounter data where certain individuals have no information for certain variables. For example, we may have a data with some individuals who participate in the labour market and some who do not, where the hourly wage is a variable. For individuals who do not participate in the labour market, it probably makes most sense for their hourly wage to be recorded as `NA`. In `R`, these missing values can be recorded as `NA`. Functions must know how to deal with values which are recorded as `NA`. This is what `sum` does with its `na.rm` option. If `na.rm = TRUE`, values of NA will be removed prior to processing. Otherwise, vales of NA will remain in the data when processing. What's more, the fact that in its "Usage" section the help file tells us `na.rm = FALSE` implies that `FALSE` is the default behaviour. Should we not say anything, `R` understands that as us accepting this default. To see how this works, we will define a vector `x` below with a *missing value* `NA`

```
x <- c(1, 2, NA)
```

Now, if we use the sum function this will return to us an `NA` value, given that we can't logically sum over a vector containing `NA`, because NA has no mathematical value:

```
sum(x)
```

```
[1] NA
```

However, if we explicitly tell `R` that we wish to remove the `NA`s prior to processing, it will, returning us the sum of the remaining two elements:

```
sum(x, na.rm = TRUE)
```

```
[1] 3
```

This behaviour ensures that we will know if our vectors contain `NA`, and in effect `R` makes us confirm to it that we are happy with this to return numerical values.

Along with the `sum` function, `R` comes with a considerable range of functions in the base environment, including functions to generate descriptive statistics, work with dates, conduct statistical modelling, amond many other things. These functions are sufficient to do nearly anything we will need to do, but are not necesarily optimal for all our tasks. As we will see now, there are a large number of useful functions and data types that don't come with base `R`, but are avaialble as packages which be installed externally.

6.5 Managing packages

What are packages? Packages are a collection of functions, help files, and potentially data, which allow for the implementation of some group of procedures. These are add-on elements to `R`, which can significantly ease certain routines which are not a part of base R. In order to install a package you have several alternatives depending on how the package is distributed, though installing a package is a process which you only need to do once, and from then on this package will be avaialble to you should you wish to load it and use it. The majority of stable `R` packages are stored on the CRAN (the Comprehensive R Archive Network), which is a repository from which packages can be installed directly into your version of `R`. By far the most frequent way to install `R` packages is to install them from CRAN. This can be done quite simply, using the `install.packages()` function. This function accepts as an argument the name of the package you wish to install. For example, to install `dplyr`, a package we will work with beloe for data manipulation, you simply execute the following code:

```
install.packages("dplyr")
```

Now that we have installed this package, it is available to you in the future and you don't need to install it again unless you wish to update the package. However, that a package is installed doesn't mean that `R` automatically loads the package (along with its functions and data types). You must instruct `R` in every session that you wish to use load the package. For example, if you want to use the function `mutate` which is a function available as part of the `dplyr` package that allows you generate new columns as functions of old columns in data, you cannot unless you first tell `R` that you wish to load `dplyr`. For example, below we try to conduct a valid operation with `mutate` without having loaded `dplyr`:

```
mutate(df, C = A + B)
```

```
Error in mutate(df, C = A + B): could not find function "mutate"
```

To avoid such errors, we must simply load the package with `R`'s `library()` function:

```
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

Now if you want to use `dplyr`'s function `mutate` it will work

```
mutate(df, C = A + B)
```

```
  A B C
1 1 4 5
2 2 5 7
3 3 6 9
```

To avoid unexpected errors, it is important to ensure that packages are correctly lodad each time you use `R`. Generally, the best practice is to simply load the packages that you require in the R script that you are working on, as then each time you run the script you will be sure that you have the correct packages loaded.

At first glance, it may seem like failing to load a package is not so serious, because `R` will not find the package and simply produce an error, which you can then remedy by loading the package. But this is not necessarily the case! At times, different packages may have functions named *in exactly the same way*, and if you fail to tell `R` you are using a particular package, it may find another function with the same name, that does not necessarily do the same thing. This can lead to considerable confusion, hence the importance of being careful with your loading of packages. As a matter of fact, this is something that we have seen when we loaded the `dplyr` package! If you note above, a Message was printed when we loaded `dplyr` to draw our attention to some functions which exist in `dplyr`, and also in base `R`. When we load a package, these pre-existing functions will be masked, implying that `R` will use the newly loaded functions, and not the masked versions. To see why this matters, consider the first message above: "*The following objects are masked from 'package:stats':*". And note what happens when we use `dplyr`'s `lag` function:

```
df$A
```

```
[1] 1 2 3
```

```
lag(df$A)
```

```
[1] NA  1  2
```

In this case, we see it produces a vector which takes, for each observation, the observation one unit prior in the vector. However, we can also see what the function does with the `stats` package by using the syntax `package::function`, without first loading the package:

```
stats::lag(df$A)
```

```
[1] 1 2 3
attr(,"tsp")
[1] 0 2 1
```

Here, the same function produces a different result: namely for each observation it takes the mathematical value one below it. In many cases such clashes in function names will not occur, but we note the importance of this so that you can be cautious to always load the packages you need, and if in some strange case you do *actually* need two functions with the same name in different packages, that you have the knowledge that you can "unmask" functions by pre-empting the function name with the package name.

6.6 Loops and control-flow statements

Loops and control-flow statements allow us to control our code in a more precise way, and iterate through repetitive tasks. Looping constructs which are present in the majority of programming languages provide a mechanism to execute a set of instructions multiple times, contingent upon certain conditions. In `R`, there are a number of ways which loops can be set up.

6.6.1 The `for` Loop

In `R`, the `for` loop is commonly used to iterate over vectors, lists, or other objects which contain multiple elements which we can iterate over. the generic set-up of for loops is something akin to:

```
for(i in elements) {
  do something to i
}
```

Here `elements` is a list of items, and we use `i` (without loss of generality) to refer to a particular element. Within the brackets `{ }` we indicate the potentially multiple lines of code we wish to run which interact with each `i`. Some real examples are provided below.

6.6.1.1 Iteration Over Data Structures

If we have data structures such as a list, we may wish to iterate over one element, and access another. For example, consider a ficticious dataset of student grades:

```
grades <- list(Alice = 85, Bob = 90, Charlie = 78, Diana = 92)
for (student in names(grades)) {
    cat(student, "received a grade of", grades[[student]], ".\n")
}
```

```
Alice received a grade of 85 .
Bob received a grade of 90 .
Charlie received a grade of 78 .
Diana received a grade of 92 .
```

Here, we have simply iterated over each name in grades, and accessed the record within the loop. We could have also done this in a position-based way, and for this, sequences of numbers may be useful, as below.

6.6.1.2 The `seq()` Function

For tasks requiring iteration over a series of numbers, R's **`seq()`** function is invaluable:

```
for (i in seq(3, 8)) {
    cat("Processing data point:", i, "\n")
}
```

```
Processing data point: 3
Processing data point: 4
Processing data point: 5
Processing data point: 6
Processing data point: 7
Processing data point: 8
```

In the above we have simply generated a sequence from 3 to 8, though depending on our particular needs, we could generate any other type of sequence, and subsequently interact with elements of the sequence within our loop.

6.6.2 The `while` Loop

An alternative type of loop is the **`while`** loop, which keeps executing statements so long as a particular condition remains true. Consider a scenario where we're approximating the square root of a number using the Babylonian method. In this case, we can iterate onto a square root for some number p by taking an initial guess x, and taking steps so that $x_{new} = \frac{1}{2}(x + \frac{p}{x})$. When x and x_{new} converge, we have found our solution. We can do this with a **`while`** loop, asking our code to keep iterating so long as the difference between x_{new} and x is not very small:

```
number <- 81
guess <- number / 2  # Initial guess
difference <- 1
tolerance <- 0.01

while (difference > tolerance) {
    new_guess <- 0.5 * (guess + number / guess)
    difference <- abs(new_guess - guess)
    guess <- new_guess
}

cat("Approximated square root:", round(guess, 2), "\n")
```

```
Approximated square root: 9
```

6.6.3 Control Flow Mechanisms

There are a number of additional ways which we will encounter to control the flow of our code. A simple example is the `if`, else if`, and`else‘ statements. These can be used independently, for example simply conducting a process if some condition is met:

```
condition <- TRUE
if (condition) {
   print("the condition is true")
}
```

```
[1] "the condition is true"
```

where here, the fact that `condition` is true means that the statement in the `if` function will be evaluated. You can confirm to yourself that if `condition` is set to FALSE, the statment will not be evaluated. This can be extended in a number of ways. A first is by using other types of conditions apart from `TRUE` and `FALSE`. A second is that we multiple conditions can be strung together to be evaluated in turn using `if`, `else if` and `else`. In case such as thus, a single `if` and `else` must be used, but as many `else if`s as desired can be incorporated. When evaluating an `if`, `else` string, the conditions will only be evaluated up to the point that a condition is met, and then the respective code will be run. To see a simple example of such a case, refer to the below which has three conditions:

```
x <- 10
if (x > 15) {
   print("x is greater than 15")
} else if (x > 5) {
   print("x is greater than 5 but less than or equal to 15")
} else {
   print("x is less than or equal to 5")
}
```

```
[1] "x is greater than 5 but less than or equal to 15"
```

Finally, note that there are also control statements which can be used *within* loop execution. This includes elements such as `break` which causes a loop to be existed prematurely, skipping to the next element of a code, and `next` which skips the current iteration of a loop, moving to the next one.

To see an example of `break` in action, let's consider the following simple code, which determines whether a number is a prime. First, we will define a particular number (in this case 29), and then iterate through all candidates with which the number may be divisible. If we find the number is perfectly divisible by any of these candidates, we can thus confirm that the number is not a prime, and we do not need to keep iterating, in which case we use `break` to break out of our loop. Otherwise, the loop will keep iterating until the end, and we will conclude that the number considered is indeed a prime.

```
number <- 29
is_prime <- TRUE

for (i in 2:sqrt(number)) {
    if (number %% i == 0) {
        is_prime <- FALSE
        break
    }
}

cat("Is", number, "a prime number?", is_prime, "\n")
```

```
Is 29 a prime number? TRUE
```

We will encounter loops and control flow throughout the codes we examine in the rest of this book, given that they can help us facilitate repetitive tasks, and break down complex computations.

6.6.4 `Apply` functions

A somewhat distinct way to consider the repeated application of procedures, and the repeated application of functions in particular is `R`'s `apply` family of functions. These are a powerful suite of tools for performing operations iteratively over the elements on various types of data structures. They are particularly useful for applying a specific function to elements of a data structure, such as matrices, lists, or data frames, without the need for explicit loops. Here we present an introduction to the most commonly used functions: `apply`, `lapply`, `sapply`, `tapply` and `mapply`.

1. The `apply` function is used to apply a function to the rows or columns of a matrix or to the margins of an array. As a simple example consider a 3×3 matrix and calculate the mean of each row in three ways: (i) Manually indexing each row, (ii) With a loop iterating over each row number and (iii) using the `apply` function:

```r
# Create a 3x3 matrix for the example
x <- matrix(1:9, nrow = 3)

# Get the mean of each row manually
mean(x[1,])
```

```
[1] 4
```

```r
mean(x[2,])
```

```
[1] 5
```

```r
mean(x[3,])
```

```
[1] 6
```

```r
# Iterate over each row with a loop
for(i in 1:nrow(x)){print(mean(x[i,]))}
```

```
[1] 4
[1] 5
[1] 6
```

```r
# Use the apply function
apply(X = x, MARGIN = 1, FUN = mean)
```

```
[1] 4 5 6
```

Note the use of the arguments in the function: `X` argument refer to the object over we want to apply the function, `MARGIN` points to the dimension along which we want to apply the function and `FUN` is the function to apply. If you see the help documentation of the `apply` function there is a `...` in the arguments that is for additional arguments for the functions. As an example below we replace one of the elements of the matrix with a missing value and, as you could know until now, if you use the mean function with a missing value in the vector it returns a missing value unless you specify the `na.rm = TRUE` argument.

```r
# Assign the missing value
x[2,2] <- NA

# Apply mean to the rows
apply(X = x, MARGIN = 1, FUN = mean)
```

```
[1]  4 NA  6
```

```r
# Specify the additional argument
apply(X = x, MARGIN = 1, FUN = mean, na.rm = T)
```

```
[1] 4 5 6
```

2. The `lapply` function applies a function to each element of a list and returns a list. Below we create a list and try to use the `apply` function seeing how an error is returned as a list object doesn't have a dimension

```r
# Create a list
x <- list(Nums = rep(1:3, 3), Letters = rep(c("A", "B", "C"), 3))

# Apply function as error
apply(X = x, FUN = unique)
```

```
Error in apply(X = x, FUN = unique): dim(X) must have a positive length
```

```r
# Inspect the dimension
dim(x)
```

```
NULL
```

Now instead the `lapply` function operate without problems

```r
lapply(X = x, FUN = unique)
```

```
$Nums
[1] 1 2 3

$Letters
[1] "A" "B" "C"
```

3. The `sapply` function is a variant of lapply that simplifies the result to a vector or matrix if possible with the `simplify` argument as `TRUE`, instead if we specify `simplify = FALSE` we get the same result as in `lapply` function.

```
# Sapply as the same result of lapply
sapply(X = x, FUN = unique, simplify = F)
```

```
$Nums
[1] 1 2 3

$Letters
[1] "A" "B" "C"
```

```
# Sapply as the default functionaility
sapply(X = x, FUN = unique)
```

```
     Nums Letters
[1,] "1"  "A"
[2,] "2"  "B"
[3,] "3"  "C"
```

4. The `tapply` function applies a function over subsets of a vector, defined by some other vector that interprets like a factor. Below we make a simple example using first groups defined by some variable and then by a factor variable, showing that this produces the same result

```
# Create vector of values
values <- 1:9

# Create vector of groups
groups <- rep(c("A", "B", "C"), 3)

# Use of tapply without factor
tapply(X = values, INDEX = groups, FUN = mean)
```

```
A B C
4 5 6
```

```r
# Groups as factor
groups <- as.factor(groups)

# Use of tapply with factor
tapply(X = values, INDEX = groups, FUN = mean)
```

```
A B C
4 5 6
```

Another remark related to the `tapply` function is that it works as `sapply` and tries to return a dataframe or matrix, but if you specify `simplify = FALSE` it will return a list.

5. The `mapply` function is a multivariate version of sapply. It applies a function to the first elements of each argument, the second elements, and so on. As an example consider the following where we have two elements: `v1` and `v2`

```r
# Create values
v1 <- c(1,2)
v2 <- c(3,4)

# Get sum of the first elements of each object
v1[1] + v2[1]
```

```
[1] 4
```

```r
# Get sum of the second elements of each object
v1[2] + v2[2]
```

```
[1] 6
```

```r
# Use of mapply function
mapply(FUN = sum, v1, v2)
```

```
[1] 4 6
```

One remark is that this function iterates over the elements of the different objects we impute so the different inputs must have same length or lengths such that each length be a multiple of the shorters lengths, if we don't accomplish this requirement we will get an error

```r
# Create one element of length 3
v2 <- 3:5

# Get error as the lengths aren't multiple
mapply(FUN = sum, v1, v2)
```

```
[1] 4 6 6
```

Finally we show how with lengths that are multiple it works fine by just repeat the shorter length element in the same order

```r
# Create one element of length 4
v2 <- 3:6

# Get error as the lengths aren't multiple
mapply(FUN = sum, v1, v2)
```

```
[1] 4 6 6 8
```

7 Working with Data

7.1 Loading and viewing data

In most – if not all – of our research projects, we will not actually type in data to `R` by hand which is a cumbersome and highly error-prone activity, but rather we will wish to read in data which we have stored on our disk in one or a number of files. Such data may be stored in many formats; among others this may be free formats such as `csv` or `txt`, or even proprietry formats such as `excel` or `dta`. The first such formats do not require specific packages to be loaded into R, but the latter do.

For data stored as `csv` (comma separated values) or `txt` formats `R` comes with the functions `read.csv`, `read.csv2`, `read.delim` and `read.delim2`. If you consult the help documentation, which is something I would recommend anytime you come across a new function, you will see that the four of these functions belongs to the family of the `read.table` function and have a very similar usage, along the lines of the following:

```
read.csv(file="Path/To/Your/File.csv")
```

Note that you could do this all directly with the `read.table` function, but would just need to make clear any file-specific details, such as the characters which indicate column breaks, whether a header is included, and so forth. So, for a file where columns are separated by commas and which variable names are included, we could mimic the above behaviour by typing:

```
read.table(file="Path/To/Your/File.csv", header=TRUE, sep=",")
```

Similar such procedures can be followed for other data stored in flat formats such as .txt or others, as laid out in the help file.

In the case of files which are stored in other formats, more specialised packages are likely required. For example, for `excel` files you could consider one of a number of packages available from CRAN, such as `readxl`, `openxlsx` and `xlsx`. Each package provides functions to deal with data stored in xlsx format, along with different options such as reading or writing files depending on the particular package. Once again, in general the basic syntax is simple, and further control can be achieved as desired following instructions in the help files. For example,

if we have installed and loaded the `readxl` library, we can use functions such as `read_xls`, `read_xlsx` and `read_excel` as follows.

```
read_xlsx(path = "Path/To/Your/File.xlsx")
```

In practice the syntax can be more complex as you may wish to import only some specific sheet, or some specific range of rows or columns.

Finally if you wish to import other proprietry data formats such as Stata's `dta` files, you could install the `haven` package, and use the functions `read_stata` or `read_dta`, both with the basic usage as follows

```
read_dta(file = "Path/To/Your/File.dta")
```

The examples so far have been to illustrate basic syntax, though imported data has not been stored in any particular object for later manipulation. In general we will of course want to go on to *use* the data we load, which is why we loaded it in the first place. This can be done in the same way as any other assignment in `R`. Below we will import a particular dataset which is available internally in `R` (ie you will not have to download anything), to have some data to work with below. In particular, we will load a dataset from the package `wooldridge`, that contains 115 data sets from Wooldridge's undergraduate Econometrics Textbook ("Introductory Econometrics: A Modern Approach", 7th Edition by Jeffrey Wooldridge):

```
mroz <- wooldridge::mroz
```

This dataset contains the data from T. A. Mroz (1987), "The Sensitivity of and Empirical Model of Married Women´s Hours of Work to Economic and Statistical Assumptions", Econometrica 55, 765-799, and we have stored it as an object called `mroz`. We can inspect this element to see how the data has been imported:

```
class(mroz)
```

```
[1] "data.frame"
```

seeing that, conveniently, it has been stored as a data frame. We can also look at a quick snapshot of the data by using the `head()` command, which will print just the first few rows of our data:

```
head(mroz)
```

```
  inlf hours kidslt6 kidsge6 age educ   wage repwage hushrs husage huseduc
1    1  1610       1       0  32   12 3.3540    2.65   2708     34       12
2    1  1656       0       2  30   12 1.3889    2.65   2310     30        9
3    1  1980       1       3  35   12 4.5455    4.04   3072     40       12
4    1   456       0       3  34   12 1.0965    3.25   1920     53       10
5    1  1568       1       2  31   14 4.5918    3.60   2000     32       12
6    1  2032       0       0  54   12 4.7421    4.70   1040     57       11
  huswage faminc    mtr motheduc fatheduc unem city exper  nwifeinc      lwage
1  4.0288  16310 0.7215       12        7  5.0    0    14 10.910060 1.21015370
2  8.4416  21800 0.6615        7        7 11.0    1     5 19.499981 0.32851210
3  3.5807  21040 0.6915       12        7  5.0    0    15 12.039910 1.51413774
4  3.5417   7300 0.7815        7        7  5.0    0     6  6.799996 0.09212332
5 10.0000  27300 0.6215       12       14  9.5    1     7 20.100058 1.52427220
6  6.7106  19495 0.6915       14        7  7.5    1    33  9.859054 1.55648005
  expersq
1     196
2      25
3     225
4      36
5      49
6    1089
```

Depending on the way that we are working with `R`, we can also explore the data more interactively using the `View()` function.

7.2 Tidying data

Once we have loaded data in `R`, we will likely want to manipulate the data in a number of ways. This can include relatively simple things like sub-setting to certain columns or rows (which we have already encountered above), but also more complicated things like aggregating data to a higher level, calculating statistics for one or mutliple variable(s) over other variables, joining together a number of datasets, or "reshaping" data in certain ways. Here we will provide a very brief overview of these procedures, and introduce other things as we need them thoughout the book.

While these things can all be done in Base `R`, an increasingly common way to do such tasks is via a group of functions known as the *tidyverse*. These packages seek to provide a common logic for data manipulation (and other) tasks, and includes packages such as `dplyr` (data manipulation), `tibble` (a simplified version of data frames), `tidyr` (tidying or otherwise "wrangling" data) and others like `ggplot2` for plots, `lubridate` for dates and `stringr` for strings. We will discuss some of these libraries below. As with other packages, the `tidyverse` can be loaded

from the CRAN, though note that this will install a range of packages and so may a quite memory-heavy operation. Provided that we have installed the `tidyverse`, we can load it as standard:

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v forcats   1.0.0     v readr     2.1.4
v ggplot2   3.5.2     v stringr   1.5.0
v lubridate 1.9.3     v tibble    3.2.1
v purrr     1.0.2     v tidyr     1.3.0
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

7.2.1 Aggregating Data

To start with some basic data manipulations, we can return to the `dplyr` package discussed previously. By typing `help(package="dplyr")` we can see a full index of the range of functions included. For example, imagine that we wished to turn our individual level data frame we imported as `mroz` above into some group-level averages. We can do this using `dplyr`'s `summarise` function. Below, we will calculate the mean number of children aged less than six:

```
summarise(mroz, mean(kidslt6))
```

```
  mean(kidslt6)
1     0.2377158
```

Indeed, here with summarise we have requested that it provide us with the mean of this variable, but we could just as easily request some other statistic such as `sd`, `min`, `max`, `median`, `quantile` etc. It is quite simple to concatenate operations to conduct a range of operations. For example, imagine that if instead of wishing to simply know the mean number of children less than 6 in our data, we wanted to know this for women in the labour market and those out of the labour market. We could easily add this distinction using `dplyr`'s `group_by` function. We will also name the resulting average "meankidslt6":

```
summarise(group_by(mroz, inlf), meankidslt6 = mean(kidslt6))
```

```
# A tibble: 2 x 2
   inlf meankidslt6
  <int>       <dbl>
1     0       0.366
2     1       0.140
```

As we will see below, these operations can become somewhat complex and a bit tricky to follow, and so `dplyr` allows for an alternative way to write these commands, essentially breaking out the functions into a series of steps, where the output of each step is "piped" into the next step. This is by using the "pipe operator", written as %>%. To see what this looks like, consider the previous command, and note that we can alternatively write it as follows:

```
mroz %>% group_by(inlf) %>% summarise(meankidslt6 = mean(kidslt6))
```

```
# A tibble: 2 x 2
   inlf meankidslt6
  <int>       <dbl>
1     0       0.366
2     1       0.140
```

You will see that the output is identical, though potentially the input is easier to follow. In some examples further below, we will see that when the number of functions in a pipe grows, it is likely easier to follow in pipe notation than as a single nested function.

You may have noticed that the output of these commands was a type of data which we have not come across before. This is a `tidyverse`-specific construct known as a "tibble", essentially a modified data frame including features including enhanced printing (refer to `help(tibble)` for full information). However, if you actually *did* prefer to generate a standard data frame, we could certainly do this, either via piping or just enclosing everything in a `data.frame()` function. For example, in the below line we will save the output from the above pipe in a data frame named *kids*:

```
kids<- mroz %>% group_by(inlf) %>% summarise(meankidslt6 = mean(kidslt6)) %>%
  data.frame()
```

Note that the only addition to the pipe is the last step converting output to a data frame.

7.2.2 Merging/Joining Data

We will also, at times, wish to join two (or more) pieces of data together, for example "merging" data frames or tibbles by a column or columns. For example, it may be that we have a survey where each individuals surveyed belongs to a household, where all of the individual-level questions in the data are stored as an individual-level file (along with the individual's household ID), while the household level measures are stored in another file at the household level. If we wish to work with information on the individual *and* their household characteristics, we will need to "join", or "merge" the file using the household ID.

We can see an example of this using a series of joining functions included as part of `dplyr`. A simple example is if we would like to take the grouped-level statistics generated previously, and join them back into our original `mroz` data. What we want to do then, is for each individual who is in the labour force (`inlf==1`), join the information on the mean number of children under 6 in this group to their row of data, and similarly, for individuals *not* in the labour force (`inlf==0`), incorporate the group-level value to their row.

```
mroz %>% group_by(inlf) %>% summarise(MeanKidslt6 = mean(kidslt6)) %>%
   left_join(mroz, by = "inlf")
```

```
# A tibble: 753 x 23
    inlf MeanKidslt6 hours kidslt6 kidsge6   age  educ  wage repwage hushrs
   <int>       <dbl> <int>   <int>   <int> <int> <int> <dbl>   <dbl>  <int>
 1     0       0.366     0       0       1    49    12    NA       0   2550
 2     0       0.366     0       2       0    30    16    NA       0   1928
 3     0       0.366     0       1       0    30    12    NA       0   1100
 4     0       0.366     0       0       4    41    12    NA       0   3193
 5     0       0.366     0       0       1    45    12    NA       0   2250
 6     0       0.366     0       0       5    43    12    NA       0   2012
 7     0       0.366     0       0       1    42    13    NA       0   3856
 8     0       0.366     0       0       0    60    12    NA       0   1645
 9     0       0.366     0       0       0    57    12    NA       0   1554
10     0       0.366     0       0       2    38    10    NA       0   2352
# i 743 more rows
# i 13 more variables: husage <int>, huseduc <int>, huswage <dbl>,
#   faminc <dbl>, mtr <dbl>, motheduc <int>, fatheduc <int>, unem <dbl>,
#   city <int>, exper <int>, nwifeinc <dbl>, lwage <dbl>, expersq <int>
```

You will note that the only new part of the pipe is the addition of `left_join()` where we state that we wish to join the output of our pipe with the original `mroz` data, joining by the variable `inlf`. It would be exceedingly easy to generalise this, for example if we wished to insted have the aggregates by labour market status *and* the individual's age:

```
mroz %>% group_by(inlf,age) %>% summarise(MeanKidslt6 = mean(kidslt6)) %>%
   left_join(mroz, by = c("inlf", "age"))
```

```
`summarise()` has grouped output by 'inlf'. You can override using the
`.groups` argument.
```

```
# A tibble: 753 x 23
# Groups:   inlf [2]
    inlf   age MeanKidslt6 hours kidslt6 kidsge6  educ  wage repwage hushrs
   <int> <int>       <dbl> <int>   <int>   <int> <int> <dbl>   <dbl>  <int>
 1     0    30        1.26     0       2       0    16    NA       0   1928
 2     0    30        1.26     0       1       0    12    NA       0   1100
 3     0    30        1.26     0       2       0    17    NA       0   3570
 4     0    30        1.26     0       2       2    11    NA       0   2352
 5     0    30        1.26     0       2       0    17    NA       0   2240
 6     0    30        1.26     0       1       2    12    NA       3   2940
 7     0    30        1.26     0       2       2    12    NA       0   2100
 8     0    30        1.26     0       1       0    12    NA       0   2315
 9     0    30        1.26     0       1       1    12    NA       0   2496
10     0    30        1.26     0       1       1    12    NA       0   3119
# i 743 more rows
# i 13 more variables: husage <int>, huseduc <int>, huswage <dbl>,
#   faminc <dbl>, mtr <dbl>, motheduc <int>, fatheduc <int>, unem <dbl>,
#   city <int>, exper <int>, nwifeinc <dbl>, lwage <dbl>, expersq <int>
```

Here we have used the function `left_join()` to conduct our merge. In this **particular** setting, we could have used any if `inner_join`, `left_join`, `right_join` or `full_join`. This is because in this case, all of the levels of "inlf" or "inlf" and "age" which are in one set of data are also in the other set of data. In many cases this may not be the case, and we may have observations in one dataset that are not in the other. In this case, we must be careful to indicate whether we wish to keep observations that appear in both sets of data (`inner_join`), keep those that only appear in the left-hand side of the pipe (`left_join`), those that only appear in the right-hand side of the pipe (`right_join`), or those appearing in either data set (`full_join`).

7.2.3 Filtering Data

Previously we have discussed how to select particular columns or rows of data, however we can also filter on observation values. This is straightforward, and can be incorporated in pipes using the `filter` function. Additionally, if we wish to use the standardised syntax of `tidyverse` we can use the `select` function to choose certain variables or groups of variables.

For example, below we will subset our data first to a pair of variables by using `select`, and then keep only individuals with at least 2 children under 6 years of age using the `filter` function:

```
mroz %>% select(kidslt6, hours) %>% filter(kidslt6 > 2)
```

```
  kidslt6 hours
1       3     0
2       3     0
3       3     0
```

7.2.4 Generating variables

Above we have briefly introduced the idea of generating variables using the `mutate` function. In tibbles as well as in data frames the use of `$` operator also creates new variables:

```
mroz$exper2 <- mroz$exper^2
head(mroz)
```

```
  inlf hours kidslt6 kidsge6 age educ   wage repwage hushrs husage huseduc
1    1  1610       1       0  32   12 3.3540    2.65   2708     34       12
2    1  1656       0       2  30   12 1.3889    2.65   2310     30        9
3    1  1980       1       3  35   12 4.5455    4.04   3072     40       12
4    1   456       0       3  34   12 1.0965    3.25   1920     53       10
5    1  1568       1       2  31   14 4.5918    3.60   2000     32       12
6    1  2032       0       0  54   12 4.7421    4.70   1040     57       11
  huswage faminc    mtr motheduc fatheduc unem city exper  nwifeinc      lwage
1  4.0288  16310 0.7215       12        7  5.0    0    14 10.910060 1.21015370
2  8.4416  21800 0.6615        7        7 11.0    1     5 19.499981 0.32851210
3  3.5807  21040 0.6915       12        7  5.0    0    15 12.039910 1.51413774
4  3.5417   7300 0.7815        7        7  5.0    0     6  6.799996 0.09212332
5 10.0000  27300 0.6215       12       14  9.5    1     7 20.100058 1.52427220
6  6.7106  19495 0.6915       14        7  7.5    1    33  9.859054 1.55648005
  expersq exper2
1     196    196
2      25     25
3     225    225
4      36     36
5      49     49
6    1089   1089
```

This is the standard way that we will generate variables in R, and it can work with any of the mathematical or logical operations, as well as many numerical outputs from functions discussed above.

7.2.5 Reshaping Data

Finally the last tool that we introduce for data manipulation is the idea of "reshaping" or transforming data from a wide format to a long format (and *vice versa*). We refer to data in a **long** format if for each observation we have multiple rows of data, covering different levels of some other variable. For example, a classic example of this would be a panel of data over countries and years. On the other hand, we refer to data as being in a **wide** format if for each observation we have multiple columns (or variables) which cover different levels of some other variable. In certain settings we will wish to work with data in one format, and in other settings we may find it more convenient to work with data in another format. To make this a bit clearer, consider the below data frame:

```
Data <- data.frame(Zone = c("A", "A", "A", "B", "B", "B"),
                   Year = c(2020, 2021, 2022, 2020, 2021, 2022),
                   Sales = c(90, 100, 115, 82, 98, 106))
Data
```

```
  Zone Year Sales
1    A 2020    90
2    A 2021   100
3    A 2022   115
4    B 2020    82
5    B 2021    98
6    B 2022   106
```

Here we can see that data is in the long format, with multiple observations for one unit (zone in this example) over a number of years. To convert this data to wide format we can use the `pivot_wider` function, as follows:

```
Data = pivot_wider(data = Data, id_cols = Zone, names_from = Year,
                   values_from = Sales)
Data
```

```
# A tibble: 2 x 4
  Zone  `2020` `2021` `2022`
  <chr>  <dbl>  <dbl>  <dbl>
1 A         90    100    115
2 B         82     98    106
```

As you can see now there is now one observation per unit, where each observation has three resulting columns (one for each year). Now, if we wish to return to the long format we can do so with the similar `pivot_longer` function:

```
Data = pivot_longer(data = Data, names_to = "Year", values_to = "Sales",
                    cols = -Zone)
Data
```

```
# A tibble: 6 x 3
  Zone  Year  Sales
  <chr> <chr> <dbl>
1 A     2020     90
2 A     2021    100
3 A     2022    115
4 B     2020     82
5 B     2021     98
6 B     2022    106
```

In practice, we will generally use these commands with a considerably larger number of units, but the basic principles are the same regardless.

8 Basic Plots

To end this initial introduction, we will briefly explore the generation of basic plots in `R`. The basic function to make a plot in `R` is, perhaps unsurprisingly, the `plot` function. This function maps pairs of points in the (x, y) coordinate axis and places them on a graph. For example, let's consider a case where x is a sequence of integers between -5 and 5. This can be generated as below:

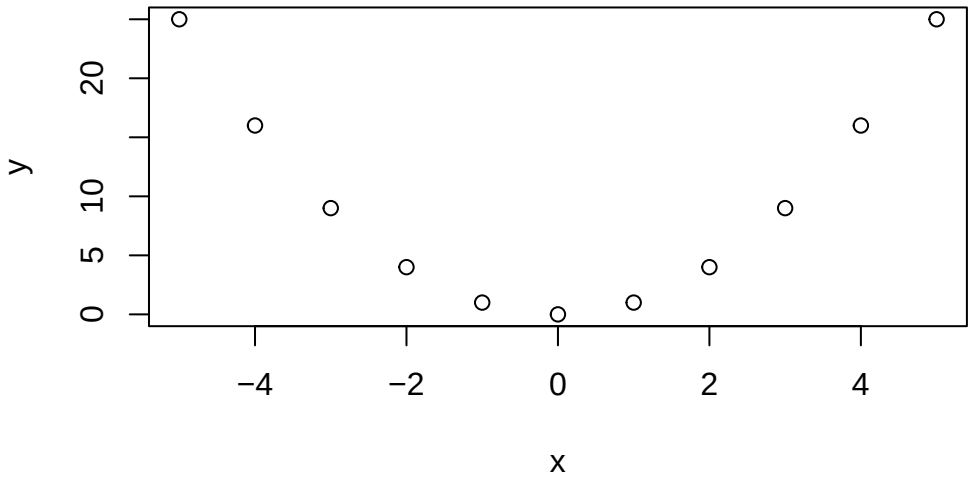
```
x <- -5:5
```

Now consider the variable $y = x^2$

```
y = x^2
```

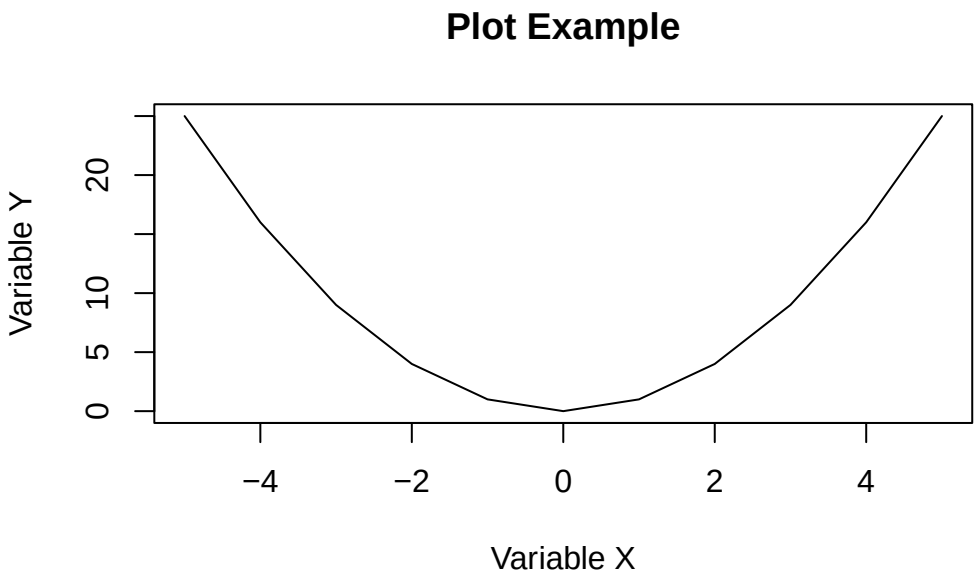
`plot`, by default generates a scatter plot of points, making reasonable decisions for axes and labelling:

```
plot(x,y)
```



If you access help documentation for `plot` you can see how to add further elements to the graph. For example, below we will request a line plot instead of a scatter plot (`type="l"`), and add a title and some axis labels:

```
plot(x,y, type = "l", main = "Plot Example", xlab = "Variable X",
     ylab = "Variable Y")
```



There are many ways to further customise graphics in `R`, and indeed, we will see a range of other plots throughout the remainder of these pages. One widely-used package to allow for the customisation of graphis in `R` is the `ggplot2` package, incorporated as part of the `tidyverse`. Given that we have previously loaded the `tidyverse` we do not need to load `ggplot2` again, but if you haven't loaded it, remember that with the below code you can do so:

```
library(ggplot2)
```

The `ggplot2` package consists of a graphs which can be built up in a step by step fashion. To see this, we will first define an employ plot region:

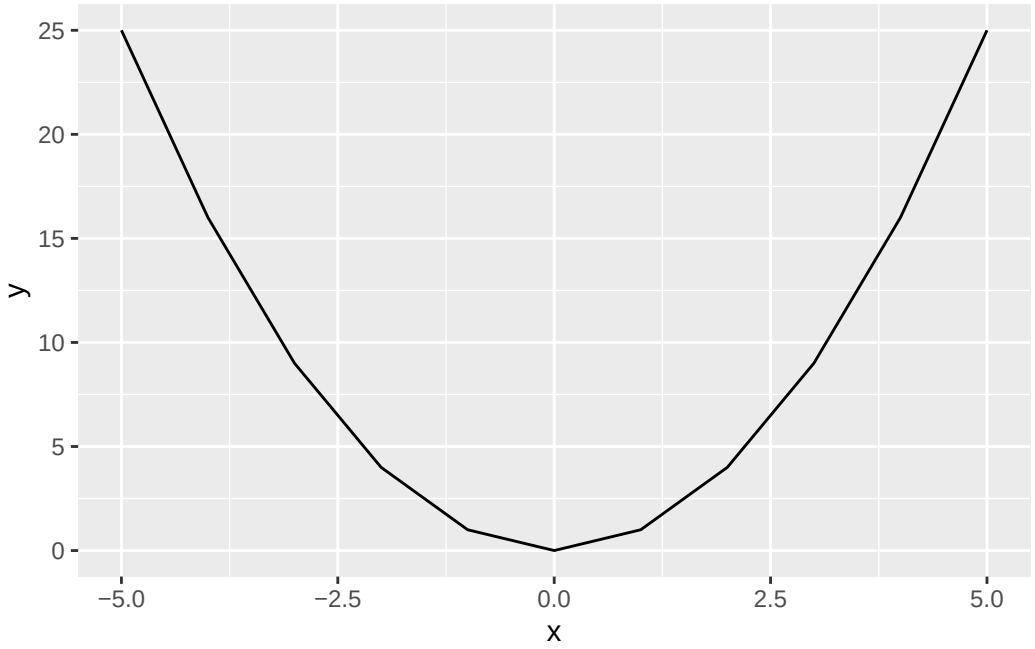
```
ggplot()
```

This provices us with a blank canvas where we can add elements, as an example, below we add a line object using the *geom_line()* function:

```
ggplot() + geom_line()
```

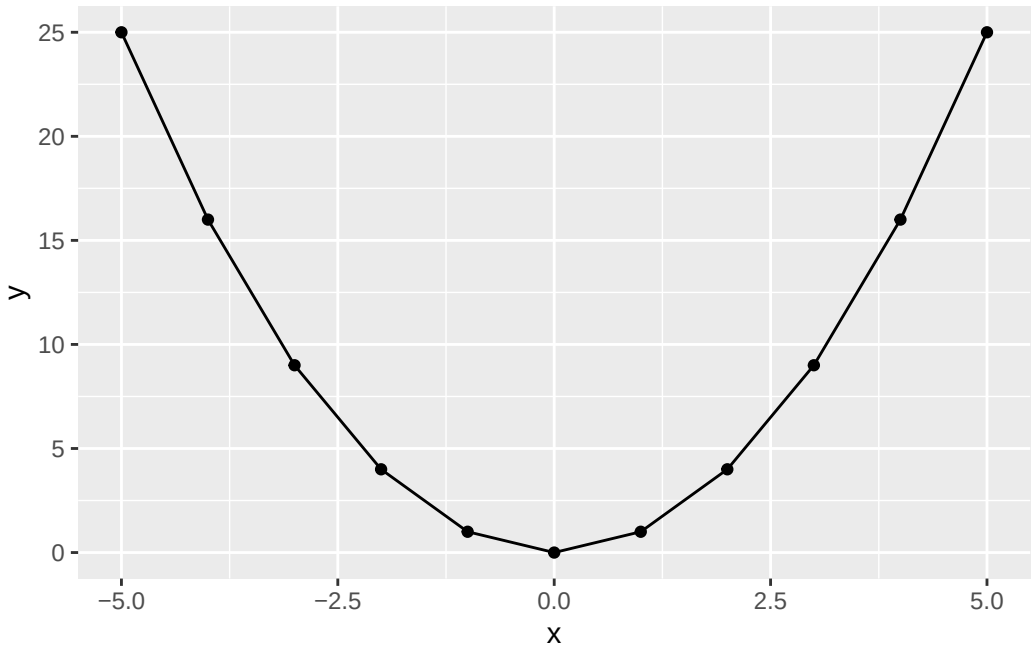
However, so far we have not added any data, so `ggplot2` is still generating a blank canvas. Now we can add a pair of points, which is done inside the `aes` mapping, which refers to "aesthetic mappings" we will add to our graphs.

```
ggplot() + geom_line(aes(x = x, y = y))
```



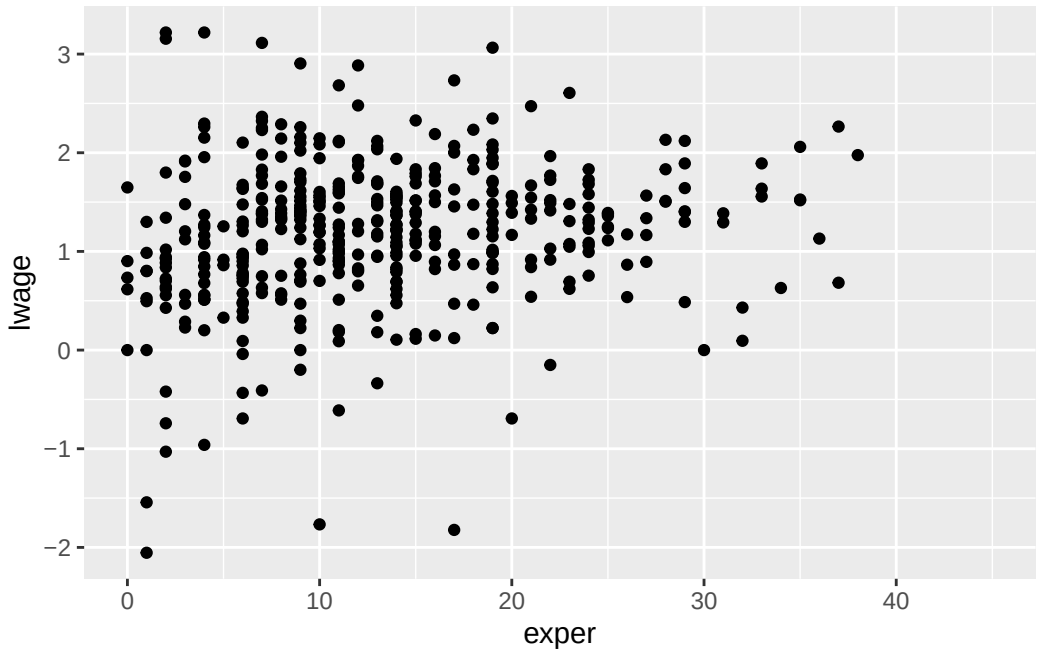
We can continue to add elements if we would like, concatenating additional portions of the plot using a + symbol.

```
ggplot() + geom_line(aes(x , y)) + geom_point(aes(x, y))
```



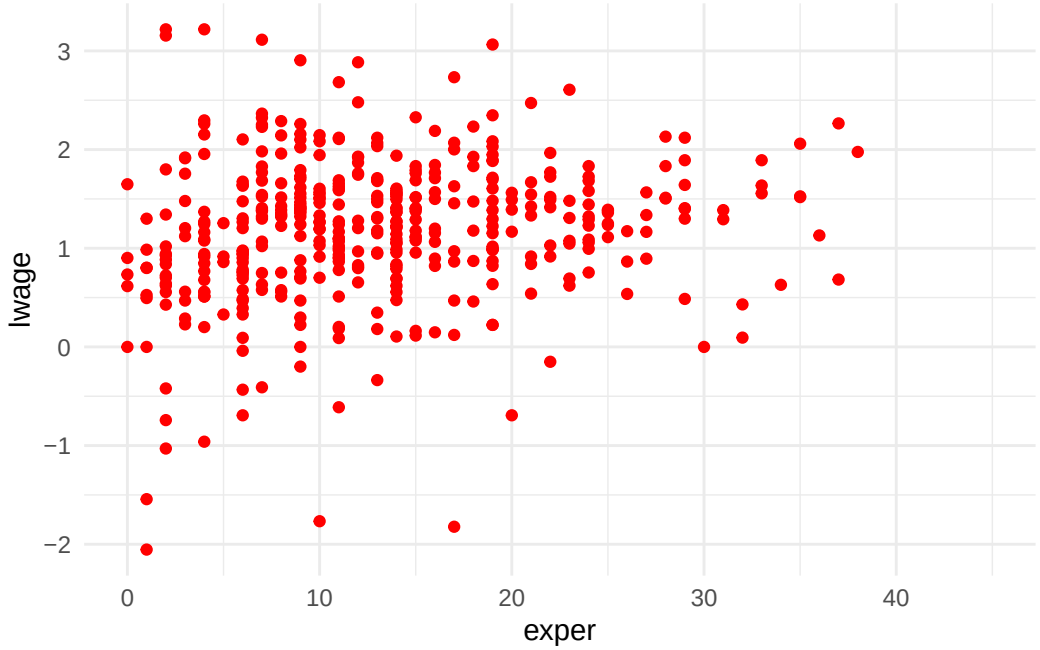
If using a data frame, which will often be the case when graphing, you can include this is an argument to `ggplot`, and then directly define `aes` inside `ggplot` function, as follows:

```
ggplot(mroz, aes(x = exper, y = lwage)) + geom_point()
```



Further editing graphs is quite simple, allowing alternative themes, colours, and so forth:

```
ggplot(mroz, aes(x = exper, y = lwage)) + geom_point(color = "red") +
  theme_minimal()
```



This results in plots which can be built up piece by piece, for example the below plot will be extended to include an OLS regression line and 95% confidence intervals using the `geom_smooth()` function, and additionally adding axis labels:

```
ggplot(mroz, aes(x = exper, y = lwage)) + geom_point(color = "red") +
  theme_minimal() + geom_smooth(method = "lm", color = "blue") +
  xlab("Years of Experience") + ylab("ln(Wage)")
```

```
`geom_smooth()` using formula = 'y ~ x'
```


ln(Wage)
3
2
1
0
−1
−2
0
10
20
30
40
Years of Experience

9 Writing Our Own Functions

9.1 An Introduction to Functions in R

Functions in R are essential for efficient and organized programming. They allow you to encapsulate code for specific tasks, making your scripts more modular, reusable, and easier to understand and debug.

9.1.1 Creating a Simple Function

A simple function in R typically includes a name, arguments (inputs), a body (code that performs the operation), and a return value (the output). Here's an example of a basic function that adds two numbers:

```
addNumbers <- function(a, b) {
  sum <- (a + b)
  return(sum)
}
addNumbers(10,5)
```

```
[1] 15
```

9.1.2 Understanding Function Arguments

Arguments are placeholders for the data you pass to the function. They can be mandatory or optional, with optional arguments having default values:

```
multiplyNumbers <- function(a, b = 2) {
  return(a * b)
}
multiplyNumbers(10)
```

```
[1] 20
```

9.1.3 Variable Scope and Return Value

Variables defined inside a function are local to that function. The `return()` function specifies the output of your function. If `return()` is not explicitly called, R will return the last evaluated expression.

10 An Introduction to Python

11 Preliminaries

We will assume that when you are reading this you have downloaded Python and have been able to access Python in some way. This may be via the command line, but most likely we imagine you will have some type of IDE which allows you to see a number of windows including a space for graphical output, a description of the contents of your Python Environment (for example any data you have loaded), potentially a place to write your code in Python, and, most importantly, a "shell", or Python interface, which looks like the following:

```
Python 3.8.10 (default, May 26 2023, 14:05:08)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

It may be preferable to install `ipython` (details here) which provides a slightly more interactive version of Python. This is available by default in certain IDEs such as Spyder. The value of working in an `ipython` shell over a standard `python` shell is the availability of a number of "magic commands" such as simple ways to get help, run files, and so forth. We will delve into this slightly more below, but under the hood, both Python and IPython work with the Python language, and have a very similar feel; for example, see below for an example of the IPython shell compared to Python above.

```
Python 3.10.12 (main, Jun 11 2023, 05:26:28) [GCC 11.4.0]
Type 'copyright', 'credits' or 'license' for more information
IPython 8.14.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]:
```

Here, Python code can be entered directly following a prompt which is the `>>>` symbol at the bottom of the python window above, or `In [1]:` in the case of IPython. From the moment you open Python you are in a Python session until the moment you close it, and from now on we will understand a session as this time between when we open and close Python. The most simple way to execute code in Python is to simply input commands into your Python prompt. So, for example, you could write the following at to the right of the `>>>` symbol, and Python will evaluate this code:

```
4 + 4
```

```
8
```

As you can see, Python has understood the instruction that you want to add 4 plus 4 and show the result, which is evaluated, and output directly below the code in the Python prompt. Later in this chapter we will properly explain these mathematical operations and the different Python data types. Beyond evaluating such simple mathematical operations, Python has a host of in-built functions which can be invoked at the command line. While we will interact with this in more detail later, a simple example is the following, which you can type in your Python console:

```
sum([1,3,2,4])
```

```
10
```

This function simply sums the list of values provided within it, printing the output (10) below. We will return to discuss functions in more detail a bit later on.

For now you will note that we have simply been typing instructions to Python directly into the command prompt, and Python has been providing us with output. Generally, we will not work this way in an empirical research project, as we will want to develop code over time, and be able to return to Python and replicate code in the future. For this reason, generally we will work in Python scripts, or files named as `something.py` (where `something` will be logical name for the file). We will use these scripts to store the commands which we require for particular processes or routines, and can then save them to the disk of our computer before running them in Python. These scripts can be generated in any type of text editor, and then can be run in Python. If you are using IPython, running a script can be done simply using the %run command, by typing `%run something.py`. This is however a magic command, and so is available in IPython, but not in Python. Similarly, in certain other IDEs there may be other environment-specific ways to run `.py` files, for example the `runfile` command which is used in Spyder below:

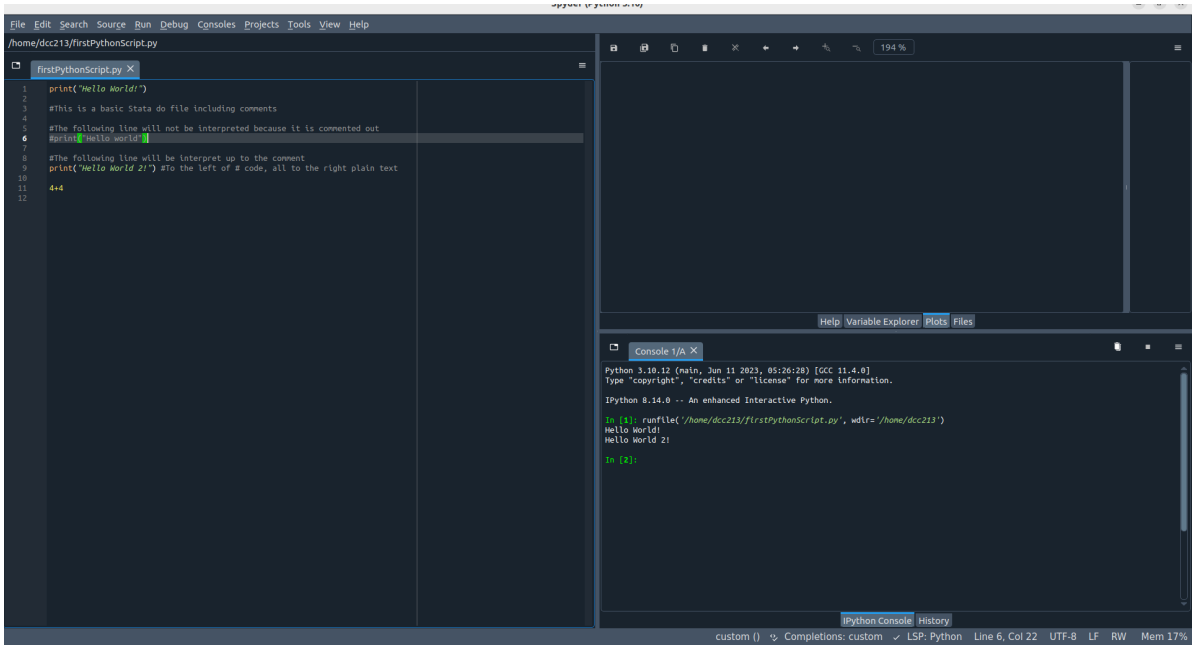


Figure 11.1: A Python Script and Output in Spyder

You may notice something strange about the code and the output in the Python console in the image above. The Python script has 7 distinct lines of text, but the console only produces two lines of output. The reason for this is because we have included comments, or messages for human eyes, but not for the computer to interpret. Any time that the **#** character is included in code, Python understands that this is a comment, and hence is ignored in executing the code. So, for example, when we write:

```
print("Hello World!")
```

```
Hello World!
```

in our code, the Python console correctly echoes *Hello World!* back at us. However, if we enter precis ely the same code, but begin the line with a comment symbol, Python will not do anything given that it interprets everything to the right of **#** as plain text which should be ignored:

```
# print("Hello World")
```

Similarly, comments can be used within lines:

```
print("Hello World") # To the left of # code, all to the right, plain text
```

```
Hello World
```

As you can, this lines has executed the instruction to the left of `#` but ignores all instructions to the right. It is good practice to comment code extensively, both for sharing code with other users, but also for sharing code with yourself in the future, where you may not remember precisely what you were thinking when you originally wrote your code! If you wish to include multiple line comments, you can also do this by enclosing various lines in three inverted commas. For example, any lines contained between an opening " " " and a closing " " " will be assumed to be one long comment. You can confirm this for yourself in a Python script.

One final thing to note from this Python script is that the output from the final line of code (`4+4`) was not printed in the console. In Python, when you type `4+4` at the command line, you get immediate output because Python automatically sends output to the console. However, when you write `4+4` in a script and then run the script, there is no automatic output to the console. However, it is important to note that the calculation *is* conducted. You can confirm this yourself if you'd like by explicitly requesting that Python provide this output when it runs the code, replacing `4+4` in the code with `print(4+4)`.

i Python and the Operating System

While below we focus on the particulars of Python as a language, Python interacts directly with your computer's file system, and you can move through your file system and see where you are with a number of Python commands. In particular, some useful functions in this line are below:

- Before dealing with the operating system or data we need to "import" some libraries from our machine (we will discuss this at more length below). Specifically: `import os` and `import pickle`.
- `os.getcwd()`: shows your **current working directory**.
- `os.chdir("path/to/folder")`: changes the working directory.
- `os.listdir()`: lists the files in the current directory.
- Python scripts usually end with `.py`.
- You can run a script from within Python using `python exec(open("myscript.py").read())`‘
- You can save data (called `obj` here) using the pickle module as `pickle.dump(obj, open("file.pkl","wb"))` and reload it as `obj = pickle.load(open("file.pkl","rb"))`.

11.0.1 Checking this out locally

1. Open Python. What is your current working directory (`os.getcwd()`)?

2. Create a new folder on your operating system. Use `os.chdir()` in Python to move into that folder.

3. List the contents with `os.listdir()`. What do you see?

4. Create a simple object (e.g. `x = 1`) and save it using `pickle.dump()`. What do you see in the directory now?

5. Close Python, reopen, and load the object with `pickle.load()`.

6. Create a script called `myFirstFile.py` in the folder. Confirm that you can run it from Python with `exec(open("myFirstFile.py").read())` or from the command line with `python myFirstFile.py`.

12 An introduction to language and data types

Until now we have executed some code in Python and seen some specific functions which print output, but we have never stored any results for later use within our code. Generally we *will* want to store the output of intermediate steps in our code for later use. This can be done in Python using the assignment symbol `=`. For example, below we store a value as a variable which we define, arbitrarily named `x`:

```
x = 2
x
```

```
2
```

The use of `=` in the assignment must respect `name = value`. If, instead, we try to assign some variable as `value=name` an error will occur:

```
4 = x
```

```
SyntaxError: cannot assign to literal here. Maybe you meant '==' instead of '='?
```

This error implies that the code has not been executed, and if we inspect the contents of `x` we will see that its prior value remains.

```
x
```

```
2
```

These errors tell you that something has gone wrong and Python couldn't execute your code. Error messages include a brief explanation of what has gone wrong, to help you diagnose and fix your code. Any expected results from your command will not be assigned. At certain times, you may also see warnings which are printed when certain processes are run. These "warnings" that something has occurred that could *potentially* cause problems, but is not critical for code execution. In these cases, Python will still execute the code and give the result, but warn you that not everything is necessarily in order and this may cause inaccurate results or future problems. At times these messages can be ignored without further issues, but warnings can also be a sign of peril, and so time should be taken to review warnings and understand whether they are problematic.

12.1 Data Types

12.1.1 Text (String) Data

To get up and running with Python, it is useful to understand the different ways which data can be stored. Python can hold a number of different types of objects in its working memory, and it is important that Python understands what type of information we are passing or storing. Among other things, Python allows for information to be stored as characters, numeric values, arrays, data frames and lists. Different types of data allow different types of operations, and should be stored in different ways. Characters are perhaps the conceptually simplest type of data, and Python has excellent text processing libraries. Characters are simply strings of text, and should be specified between `" "`, or `' '`. We have seen the use of characters when printing "Hello world" previously. If characters are not indicated between quotes, Python will understand that they refer to variable names. For example, returning to *Hello World*, if this is entered without quotes, an error will appear:

```
Hello World
```

```
SyntaxError: invalid syntax
```

In practice, what has actually occurred here is that, because we did not tell it otherwise, Python assumes that Hello is a variable name, but this must be followed by some valid assignment. For example, had we written `Hello = "world"` this would have been perfectly fine (though probably not what we were trying to do)! Alternatively, if we do actually want to write this as characters, we can do so simply by enclosing the text in double or single quotes:

```
"Hello World"
```

```
'Hello World'
```

We can confirm that this is effectively understood to be character data by using the `type()` function. This function indicates the type of data of the argument it receives, as we can see below. Note also that if we generate some variable using the assignment operators discussed above, `type()` will return to us the type of data that the variable contains:

```
type("Hello World")
```

```
str
```

```
my_string = "Hello World"
type(my_string)
```

```
str
```

In both cases `Python` correctly informs us that we are working with string (text) data, printing `str`.

12.1.2 Numeric Data

Numeric data is entered without any special behaviour, simply entering the number in the case of scalar values. Indeed, if numeric data is accidentally enclosed between quotes, it will be treated as character, and *not* numeric data. For example, below we assign two variables based on the number `2` and confirm that one is numeric while the other is viewed as a character.

```
num1 = 2
num2 = "2"
type(num1)
```

```
int
```

```
type(num2)
```

```
str
```

If you try to perform numerical operations based on the above variables (more on this below), you will see that `num1` can be involved in such calculations, while `num2` generally cannot.

12.1.3 Tuples, Lists and Arrays

Typically, we will be dealing with more than a single number in our work, and for this reason need ways to collect groups of numbers. Tuples and lists (or arrays) provide some ways to do this.

To create a list of numbers, we simply need to enclose a group of values within square parentheses, separating the numbers by commas. For example, to create a list with two values:

```
my_list = [1,2]
type(my_list)
```

```
list
```

This is required to group numbers. If we simply write a series of numbers without separating them by commas, an error will occur:

```
[1 2]
```

```
SyntaxError: invalid syntax. Perhaps you forgot a comma?
```

though the error message has helpfully pointed us to the source of our error. Similarly, if we forget the comma *and* fail to wrap our numbers if `[ ]`, we will once again generate an error.

However, we *can* generate an alternative way to store numbers without the square brackets if we separate numbers by commas. This will generate a tuple, as we can see below:

```
my_tuple = 1,2,3
type(my_tuple)
```

```
tuple
```

As this data types just combine elements they allow to combine different data types. For example:

```
type([1,"Hello"])
my_tuple = 1, 'Hello'
type(my_tuple)
```

```
tuple
```

In order to create arrays with a matrix-like form, we can list lists, which allows us to get objects in two dimensiones: rows and columns. To see this, we will generate a matrix below, and then print it out to the left of the string "Matrix =":

```
matrix = [[1,2],[3,4]]
print("Matrix =", matrix)
```

```
Matrix = [[1, 2], [3, 4]]
```

While this allows us to view data in a matrix-like form, we would likely not want to start here if we actually want to do matrix-based calculations. Indeed, there are a range of extended functionalities to Python which we can access by installing external packages and libraries. We will hold off our discussion of the installation of packages until a little bit later in this section, but in the interests of completion would just note that there is a package known as `NumPy` which allows for more convenient matrix processing and numerical operations. *Provided* that you have this package installed, you can generate a numpy array as follows, noting that we will discuss the installation of packages and the idea behind `import` slightly later in this document.

```
import numpy
numpy.array([[1,2],[3,4]])
```

```
array([[1, 2],
       [3, 4]])
```

Unlike in lists, tuples, and so forth, numpy's interest in systematic processing of elements means that it *will* force the data into a uniform type, in particular coercing the array to the most restrictive data type, as you can see below, where the list `[1,2]` has been coerced to `['1', '2']`:

```
numpy.array([[1,2],["Hello", "World"]])
```

```
array([['1', '2'],
       ['Hello', 'World']], dtype='<U21')
```

We will return to discuss numpy at more length below.

12.1.4 Dictionaries

Dictionaries are similar to lists, however values can be assigned to a particular key. As we will see shortly, in certain circumstances this may be preferable in terms of providing control over elements within the dictionary.

```
my_dictionary = {'First': [1,2], 'Second':[3,4]}
my_dictionary
```

```
{'First': [1, 2], 'Second': [3, 4]}
```

12.1.5 Data frames

Likely the type of object which we will be working with the most in our analysis in `Python` are data frames. Data frames allow us to mix mutliple other types of data, viewing the collection of variables and observations as a database with multiple rows and columns. Within data frames, we can easily subset to certain rows (observations), columns (variables), and conduct univariate and multivariate analysis. It will likely be the starting point for much of your work when you import a standard dataset into `Python`. Similarly to the case of numpy arrays, data frames in Python are handled by an external library known as Pandas, and again, we will discuss the installation of libraries properly below.

However, to see way that a data frame can be used in Python, *if* you have Pandas installed on your computed you can generate a data frame as follows:

```
import pandas
df = pandas.DataFrame({'Col1': [1, 2], 'Col2': ["Hello", "World"],'Col3': [3, "String"]})
df
```

	Col1	Col2	Col3
0	1	Hello	3
1	2	World	String

You may notice that all we have done to generate this data frame has been to enclose a dictionary inside of the command `pandas.DataFrame()`. As is the case with numpy arrays, given that Pandas seeks consistency within columns of data, all types of data in a single column will be coerced to the most restrictive type. In the case of `df` above, the third column (`Col3`) will be stored as a string. If you would like to work with Pandas immediately, you may wish to jump ahead to the section where we discuss the Installation of libraries, and then return here.

12.2 Subsetting Multidimensional Elements

As we saw in data frames and lists, often we will work with objects that store multiple elements. Sometimes we may want to work with a subset of this multidimensional data instead the entire object. In the case of lists you can easily gain access to any specific value by using the list's name followed by square parentheses and the location of the item(s) you wish to extract:

```
my_list[0]
```

```
1
```

A particularity of Python is that it uses zero-based indexing, so element [0] refers to the first item in a list, while the last item in a list will be element N-1. However, items can also be accessed by referring to their position with regards to the end of the list. For example, specifying [-1] will return the final item in the list:

```
my_list[-1]
```

```
2
```

If you wish to access more than one object you can use the slice operator which consists of a colon `:` between `[ ]` in the format of `[StartIndex : EndIndex]`. As an example we will replace `my_list` with a longer list and access from the first to the second element.

```
my_list = [1,2,3,4,5,6]
my_list[0:2]
```

```
[1, 2]
```

You will likely have notede here that the `EndIndex` was 2, which in Python points to the third element of the list. But as we can see, the third element wasn't included in the output, and this is because Python returns elements *up until* `EndIndex` but not including `EndIndex`. Similarly, with the slide operator we can also use positions relative to the end of the list, provided that `EndIndex` is greater than `StartIndex`.

```
my_list[2:-1]
```

```
[3, 4, 5]
```

In the case of the `Pandas` data frames that we have discussed, there are a number of ways that we can access data: either referring to column names, or referring to the order of columns, along with the row numbers that we wish to access. If we wish to access columns based on their names, we can use the `loc` method, while if we wish to access columns based on their column number, we can use the `iloc` method. Previously we have generated a data frame which we named `df`:

```
df
```

	Col1	Col2	Col3
0	1	Hello	3
1	2	World	String

If we wish to access data through column names (also known as column index) we use the syntax: `DataFrameName.loc[RowIndex, ColIndex]`. In the data frame `df`, our column index is: *Col1, Col2* and *Col3* while the row index is *0* and *1*. This, in order to access the "World" element we can simply write:

```
df.loc[1, 'Col2']
```

```
'World'
```

Alternatively, the `iloc` method works similarly, however accesses rows through their numeric index, which as standard begins at *0*, and goes up from their. So, to do the equivalent of the above using `iloc` we can simply write the following.

```
df.iloc[1,1]
```

```
'World'
```

In this case, because the rows are named numerically, the `RowIndex` value is 1 in both examples. Frequently our data frames will have rows named numerically, and so this is quite standard. If however we want to name our rows differently, we can do this by using `df.index` as below:

```
df.index=['a','b']
```

Now, if we attempt to use the previous `loc` call where *1* was a valid row index, we will see that this now results in an error, and we instead need to use the correct row index.

```
df.loc[1, "Col2"]
```

```
KeyError: 1
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
~/.local/lib/python3.10/site-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3811         try:
-> 3812             return self._engine.get_loc(casted_key)
   3813         except KeyError as err:
```

```
pandas/_libs/index.pyx in pandas._libs.index.IndexEngine.get_loc()
pandas/_libs/index.pyx in pandas._libs.index.IndexEngine.get_loc()
pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item
pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item
KeyError: 1
The above exception was the direct cause of the following exception:
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_429376/3558747443.py in <module>
----> 1 df.loc[1, "Col2"]

~/.local/lib/python3.10/site-packages/pandas/core/indexing.py in __getitem__(self, key)
   1182             key = tuple(com.apply_if_callable(x, self.obj) for x in key)
   1183             if self._is_scalar_access(key):
-> 1184                 return self.obj._get_value(*key, takeable=self._takeable)
   1185             return self._getitem_tuple(key)
   1186         else:
~/.local/lib/python3.10/site-packages/pandas/core/frame.py in _get_value(self, index, col, ta
   4230             #  results if our categories are integers that dont match our codes
   4231             # IntervalIndex: IntervalTree has no get_loc
-> 4232             row = self.index.get_loc(index)
   4233             return series._values[row]
   4234
~/.local/lib/python3.10/site-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3817             ):
   3818                 raise InvalidIndexError(key)
-> 3819             raise KeyError(key) from err
   3820         except TypeError:
   3821             # If we have a listlike key, _check_indexing_error will raise
KeyError: 1
```

```
SyntaxError: invalid syntax
```

```
df.loc["b", "Col2"]
```

```
'World'
```

Similarly, if we accidentally mix up our notation and try to use column naming in `iloc`, things will not work as we hope:

```
df.iloc[1, "Col2"]
```

```
ValueError: Location based indexing can only have [integer, integer slice (START point is INC
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
~/.local/lib/python3.10/site-packages/pandas/core/indexing.py in _validate_tuple_indexer(self
    966             try:
--> 967                 self._validate_key(k, i)
    968             except ValueError as err:
~/.local/lib/python3.10/site-packages/pandas/core/indexing.py in _validate_key(self, key, axis
   1614         else:
-> 1615             raise ValueError(f"Can only index by location with a [{self._valid_types}]
   1616
ValueError: Can only index by location with a [integer, integer slice (START point is INCLUDED
The above exception was the direct cause of the following exception:
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_429376/1009555997.py in <module>
----> 1 df.iloc[1, "Col2"]

~/.local/lib/python3.10/site-packages/pandas/core/indexing.py in __getitem__(self, key)
   1183             if self._is_scalar_access(key):
   1184                 return self.obj._get_value(*key, takeable=self._takeable)
-> 1185             return self._getitem_tuple(key)
   1186         else:
   1187             # we by definition only have the 0th axis
~/.local/lib/python3.10/site-packages/pandas/core/indexing.py in _getitem_tuple(self, tup)
   1689
   1690     def _getitem_tuple(self, tup: tuple):
-> 1691         tup = self._validate_tuple_indexer(tup)
   1692         with suppress(IndexingError):
   1693             return self._getitem_lowerdim(tup)
~/.local/lib/python3.10/site-packages/pandas/core/indexing.py in _validate_tuple_indexer(self,
    967                 self._validate_key(k, i)
    968             except ValueError as err:
--> 969                 raise ValueError(
    970                     "Location based indexing can only have "
    971                     f"[{self._valid_types}] types"
ValueError: Location based indexing can only have [integer, integer slice (START point is INC

ValueError: Location based indexing can only have [integer, integer slice (START point is INC
```

```
df.loc[1, 1]
```

```
KeyError: 1
---------------------------------------------------------------------------
```

```
KeyError                                  Traceback (most recent call last)
~/.local/lib/python3.10/site-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3811         try:
-> 3812             return self._engine.get_loc(casted_key)
   3813         except KeyError as err:
pandas/_libs/index.pyx in pandas._libs.index.IndexEngine.get_loc()
pandas/_libs/index.pyx in pandas._libs.index.IndexEngine.get_loc()
pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item
pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item
KeyError: 1
The above exception was the direct cause of the following exception:
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_429376/3985365408.py in <module>
----> 1 df.loc[1, 1]

~/.local/lib/python3.10/site-packages/pandas/core/indexing.py in __getitem__(self, key)
   1182             key = tuple(com.apply_if_callable(x, self.obj) for x in key)
   1183             if self._is_scalar_access(key):
-> 1184                 return self.obj._get_value(*key, takeable=self._takeable)
   1185             return self._getitem_tuple(key)
   1186         else:
~/.local/lib/python3.10/site-packages/pandas/core/frame.py in _get_value(self, index, col, ta
   4223             return series._values[index]
   4224
-> 4225         series = self._get_item_cache(col)
   4226         engine = self.index._engine
   4227
~/.local/lib/python3.10/site-packages/pandas/core/frame.py in _get_item_cache(self, item)
   4647             #  pending resolution of GH#33047
   4648
-> 4649             loc = self.columns.get_loc(item)
   4650             res = self._ixs(loc, axis=1)
   4651
~/.local/lib/python3.10/site-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3817             ):
   3818                 raise InvalidIndexError(key)
-> 3819             raise KeyError(key) from err
   3820         except TypeError:
   3821             # If we have a listlike key, _check_indexing_error will raise
KeyError: 1


KeyError: 1
```

As we saw previously with lists, we can use the slice operator to access ranges of cells:

```
df.iloc[0:2, 2]
```

```
a         3
b    String
Name: Col3, dtype: object
```

In case we wish to acess an entire row, we can do this by using the slice operator with no values, which defaults to accessing the entire row:

```
df.iloc[1,:]
```

```
Col1         2
Col2     World
Col3    String
Name: b, dtype: object
```

This can similarly be used with `loc`:

```
df.loc["a",:]
```

```
Col1        1
Col2    Hello
Col3        3
Name: a, dtype: object
```

12.3 Basic Operations

The standard mathematical operations in `Python` likely work how you would expect. Basic mathematical operations are as follows, with an example of their use below:

Symbol	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Power

Symbol	Operation
`%`	Modulus
`//`	Integer Division

A few examples

```
10 + 3
10 - 3
10 * 3
10 / 3
10 ** 3
10 % 3
10 // 3
```

```
3
```

Two important things to note are that the power symbol is *not* ^, but rather `**`, and that integer division simply returns the integer, discarding any remainder.
Also note that when you combine different operators Python follows the PEMDAS order, first resolving Parentheses, second solve Exponents, third Multiplication and Division, fourth Addition and Substraction. Both multiplication and division, and addition and substraction are evaluated left to right, rather than sequentially by operation, so it is important to indicate to `Python` explicitly what you need using parentheses in certain cases. For example in order to solve

$$-\frac{5+3^{5-3}}{5\times 3}$$

You should use the code

```
-(5+3**(5-3))/(5*3)
```

```
-0.9333333333333333
```

Python also has logical operatos that returns a boolean value (`True` or `False`) pointing if a statement is true or false. These operators are:

Symbol	Comparison
`==`	Equals
`!=`	Difference
`>`	Greater

Symbol	Comparison
`>=`	Greater or Equal
`<`	Less than
`<=`	Less than or Equal
`|` or `or`	Or
`&` or `and`	And
`not`	Negation

A few examples

```
4 == 3
4 != 3
4 > 3
4 >= 3
4 < 3
4 <= 3
4 in [1,2,3,4]
True | False
True or False
True & False
True and False
not(True)
```

```
False
```

These boolean values are interpreted by Python as the numbers 0 for `False` and 1 for `True`, a characteristic which allows you to work with them numerically and is useful when you want to check a series of conditions.

12.4 Functions

Beyond these logical and mathematical operators, `Python` comes with a basic suite of built in functions which perform specific tasks, as well as a large number of other functions (discussed below) which can be imported, or installed and imported, to extend this basic functionality. Every time you open Python, a range of built in functions are always available (listed here).

Functions are designed to perform specific tasks, potentially accepting input (known as "arguments"), and returning output which you can interact with. We have already seen a number of functions above such as `print()`, `sum()` and `type()`, all of which we have used with arguments. Arguments are simply the contents we provide the function inside the parentheses.

Understanding how packages work, including the arguments they accept and the objects they return is key. This can be found by consulting the help file by typing `help(function_name)`. For example, if we type `help(abs)` we will see that the function accepts an argument "x", and "Return the absolute value of the argument". We can explore this to confirm that this is what happens with a simple example:

```
abs(-1)
```

```
1
```

12.5 Managing modules

While not available "out of the box", an enormous range of functions can be imported into Python by loading Modules or libraries. Modules are a set of predefined objects and functions that can be imported and used in Python to extend functionality in many directions. This modules can themselves contain various levels of submodules which contain individual functions, and in general the syntax to access functions within a module is `module.function`, or if there are submodules `module.submodule.function`.

There are a large number of modules which are available with a standard installation of Python. If you would like to see a list of *all* the modules available – which will likely be large – you can see this is in Python by typing:

```
help("modules")
```

To access the functions contained in a particular module we must import this into Python every time that we run Python. This process of importing a function uses the `import` command, and should generally be done at the top of any script we write. So, for example, if we wish to access functions from within the `os` module, we can do this by typing:

```
import os
```

And similarly, if we'd like to be able to access functions from the math module we can do this as:

```
import math
```

We can then use function which are available within these modules provided that we prepend the module name. Thus, for example, from `math`:

```
math.log(5)
```

```
1.6094379124341003
```

and

```
math.pi
```

```
3.141592653589793
```

are quite self-explanatory, while from `os` we can find a large arrange of tools to interact with our operating system, for example the `getcwd` function if we would like to see the directory where we are currently working or the `chdir` function to change to a new working directory:

```
os.getcwd()
```

```
'/home/dcc213/trabajo/Teaching/Exeter/ResearchMethods-I/site'
```

However, at times we may wish to install modules which are not available in our standard installation. A range of additional packages are available via `pip`, which is the package installer for Python (hence pip). Two packages which we referred to previously were `NumPy` and `Pandas`. Both of these packages will be fundamental for our work, and can be installed from pip using the command `pip install packagename`. For example, to install Pandas we can type:

```
pip install pandas
```

Now that the module is installed you will not need to install it again unless you want to update the module. However, as above we will need to import the package every time we wish to use it using import. While this can be done as previously:

```
import pandas
```

In practice we can assign the package an alias which makes it easier to use later. It is standard usage to instead load pandas as follows, where by writing `as pd` means that we now refer to the package as `pd` rather than Python:

```
import pandas as pd
df = pd.DataFrame(df)
df
```

	Col1	Col2	Col3
a	1	Hello	3
b	2	World	String

Similarly, it is standard to load numpy as:

```
import numpy as np
```

Implying that we can refer to any numpy functions as simply `np.function_name()`. If there are specific functions that will be particulary commonly used, we can import these directly, and then they can be used without first typing the package name. For example, if we want to simplify access to `numpy`'s `sqrt` function to take square roots, we can do this as below:

```
from numpy import sqrt
sqrt(4)
```

```
np.float64(2.0)
```

What's more, if for some reason we wish to provide alternative names for specific functions, we can do this once again with `as`:

```
from numpy import sqrt as squareroot
squareroot(4)
```

```
np.float64(2.0)
```

12.6 Loops and Control Flow

Loops in Python

Looping constructs in programming languages provide a mechanism to execute a set of instructions multiple times, contingent upon certain conditions. Python, renowned for its clarity and efficiency, offers several looping paradigms. Among these, the `for` and `while` loops are paramount.

12.6.1 1. The `for` Loop

In Python, the **`for`** loop is adept at iterating over sequences, be they lists, tuples, dictionaries, sets, strings, or other iterable objects.

1.1. Iteration Over Data Structures

Consider a dataset of student grades:

```
grades = {"David": 85, "Victoria": 90, "Daniel": 78, "Megan": 92}
for student, grade in grades.items():
    print(f"{student} received a grade of {grade}.")
```

```
David received a grade of 85.
Victoria received a grade of 90.
Daniel received a grade of 78.
Megan received a grade of 92.
```

1.2. The `range()` Function

For tasks requiring iteration over a series of numbers, Python's **`range()`** function is invaluable:

```
for i in range(3, 8):
    print(f"Processing data point {i}.")
```

```
Processing data point 3.
Processing data point 4.
Processing data point 5.
Processing data point 6.
Processing data point 7.
```

12.6.2 2. The `while` Loop

The **`while`** loop offers a means to execute statements as long as a particular condition remains true.

Consider a scenario where we're approximating the square root of a number using the Babylonian method:

```
number = 49
guess = number / 2  # Initial guess
difference = 1
tolerance = 0.01

while difference > tolerance:
    new_guess = 0.5 * (guess + number / guess)
    difference = abs(new_guess - guess)
    guess = new_guess

print(f"Approximated square root: {guess:.2f}")
```

```
Approximated square root: 7.00
```

12.6.3 3. Loop Control Mechanisms

Python introduces several control statements to influence loop execution:

- **break**: Exits the loop prematurely, proceeding to the next statement.
- **continue**: Skips the current iteration, moving to the next one.
- **pass**: A no-op statement, serving as a syntactical placeholder.

3.1. Utilizing `break` in Prime Number Detection

```
number = 29
is_prime = True

for i in range(2, int(number ** 0.5) + 1):
    if number % i == 0:
        is_prime = False
        break

print(f"Is {number} a prime number? {is_prime}")
```

```
Is 29 a prime number? True
```

13 Working with Data

13.1 Loading and viewing data

In most – if not all – of our research projects, we will not actually type in data to `Python` by hand which is a cumbersome and highly error-prone activity, but rather we will wish to read in data which we have stored on our disk in one or a number of files. Such data may be stored in many formats; among others this may be free formats such as `csv` or `txt`, or even proprietry formats such as `excel` or `dta`. We will generally do this using functions which are a part of `Python` which has the added benefit of importing our data as an easy to use data frame.

For data stored as `csv` (comma seperated values) or `txt` formats `Panda` comes with the functions `read_csv`, and `read_table`. If you consult the help documentation, which is something I would recommend anytime you come across a new function, you will see that these functions provide a large number of options to control how data is read into `Python`. If we would like to import a standard comma separated value (csv) file, the basic usage is

```
pd.read_csv("Path/To/Your/File.csv")
```

For `txt` data, or in fact any type of delimited data, a useful function is `read_table`:

```
pd.read_table("Path/To/Your/File.csv")
```

If you explore the help documentation you will notice that you can specify a variety of arguments such as indicating which character is used to indicate column breaks, whether a header is included, and so forth.

While the options vary, a standardised syntax is also available for `excel` files such as those ending in .xls or .xlsx:

```
pd.read_excel("Path/To/Your/File.xlsx")
```

If you refer to the help documentation you will see that there are a variety of arguments which allows you to point to a specific sheet you wish to import, a specific range of columns or rows, whether or not to use the first row as column names, etc.

Finally in order to import `dta` the function is `read_stata`

```
pd.read_stata("Path/To/Your/File.dta")
```

The examples so far have been to illustrate basic syntax, though imported data has not been stored in any particular object for later manipulation. In general we will of course want to go on to *use* the data we load, which is why we loaded it in the first place. This can be done in the same way as any other assignment in `Python`. Below we will import a particular dataset which is available as part of the wooldridge library, to have some data to work with below. This library is available for download from pip if you do not have access on your system. And contains 115 data sets from Wooldridge's undergraduate Econometrics Textbook ("Introductory Econometrics: A Modern Approach", 7th Edition by Jeffrey Wooldridge):

```
import wooldridge
mroz = wooldridge.data('mroz')
mroz
```

	inlf	hours	kidslt6	kidsge6	age	educ	wage	repwage	hushrs	husage	...	faminc	mtr
0	1	1610	1	0	32	12	3.3540	2.65	2708	34	...	16310.0	0.7215
1	1	1656	0	2	30	12	1.3889	2.65	2310	30	...	21800.0	0.6615
2	1	1980	1	3	35	12	4.5455	4.04	3072	40	...	21040.0	0.6915
3	1	456	0	3	34	12	1.0965	3.25	1920	53	...	7300.0	0.7815
4	1	1568	1	2	31	14	4.5918	3.60	2000	32	...	27300.0	0.6215
...	...	...	...	...	...	...	...	...	...	...	...	...	...
748	0	0	0	2	40	13	NaN	0.00	3020	43	...	28200.0	0.6215
749	0	0	2	3	31	12	NaN	0.00	2056	33	...	10000.0	0.7715
750	0	0	0	0	43	12	NaN	0.00	2383	43	...	9952.0	0.7515
751	0	0	0	0	60	12	NaN	0.00	1705	55	...	24984.0	0.6215
752	0	0	0	3	39	9	NaN	0.00	3120	48	...	28363.0	0.6915

This dataset contains the data from T. A. Mroz (1987), "The Sensitivity of and Empirical Model of Married Women's Hours of Work to Economic and Statistical Assumptions", Econometrica 55, 765-799, and we have stored it as an object called `mroz`. We can inspect this element to see how the data has been imported:

```
type(mroz)
```

```
pandas.core.frame.DataFrame
```

seeing that, conveniently, it has been stored as a data frame. We can also look at a quick snapshot of the data by using the `head()` method, which will print just the first few rows of our data:

```
mroz.head()
```

	inlf	hours	kidslt6	kidsge6	age	educ	wage	repwage	hushrs	husage	...	faminc	mtr	r
0	1	1610	1	0	32	12	3.3540	2.65	2708	34	...	16310.0	0.7215	1
1	1	1656	0	2	30	12	1.3889	2.65	2310	30	...	21800.0	0.6615	7
2	1	1980	1	3	35	12	4.5455	4.04	3072	40	...	21040.0	0.6915	1
3	1	456	0	3	34	12	1.0965	3.25	1920	53	...	7300.0	0.7815	7
4	1	1568	1	2	31	14	4.5918	3.60	2000	32	...	27300.0	0.6215	1

Depending on the way that we are working with `Python`, we may also be able to explore the data more interactively if we have some data viewer available.

13.2 Tidying data

Once we have loaded data in `Python`, we will likely want to manipulate the data in a number of ways. This can include relatively simple things like sub-setting to certain columns or rows (which we have already encountered above), but also more complicated things like aggregating data to a higher level, calculating statistics for one or mutliple variable(s) over other variables, joining together a number of datasets, or "reshaping" data in certain ways. Here we will provide a very brief overview of these procedures, and introduce other things as we need them thoughout the book.

Data manipulation in data frames will largely depend upon `Panda`'s functions and methods. We have seen examples of methods above, namely `loc()` and `iloc()`, and `head()`. **Methods** are functions which are associated with an object and which can be called from the object, which will work in a specific way depending on the type of object this is. We have seen above that data frames have methods, but methods can also be applied to columns within data frames. As an example, we can consider `describe` which can be applied to a column of a data frame to generate descriptive statistics:

```
mroz.wage.describe()
```

```
count    428.000000
mean       4.177682
std        3.310282
min        0.128200
25%        2.262600
50%        3.481900
75%        4.970750
```

```
max       25.000000
Name: wage, dtype: float64
```

You will note here that we have used a new way to refer to an entire column of a data frame, namely using its column name. In order to see the range of methods which can be applied to an object you can use the `dir()` function. For example if you want to know the methods which can be applied to a data frame you can type:

```
dir(mroz)
```

which will provide you a long list of methods.

13.2.1 Aggregating Data

To start with some basic data manipulations, we will consider data aggregation. For example, imagine that we wished to turn our individual level data frame we imported as `mroz` above into some group-level averages. We can do this using as we have seen previously, we have been able to generate summary stats, and if we are interested in particular in the mean, example, we can use the `mean()` method. Below, we will calculate the mean number of children aged less than six in our data:

```
mroz.kidslt6.mean()
```

```
np.float64(0.23771580345285526)
```

Indeed, here with summarise we have requested that it provide us with the mean of this variable, but we could just as easily request some other statistic such as `std`, `min`, `max`, `median`, `quantile` etc. It is quite simple to concatenate operations to conduct a range of operations. For example, imagine that if instead of wishing to simply know the mean number of children less than 6 in our data, we wanted to know this for women in the labour market and those out of the labour market. We could easily add this distinction using the `groupby` method. We will also name the resulting data frame `meankidslt6`:

```
meankidslt6 = mroz.groupby(['inlf']).kidslt6.mean()
meankidslt6
```

```
inlf
0    0.366154
1    0.140187
Name: kidslt6, dtype: float64
```

This process of data aggregation can be considerably extended depending on what we require, for example working with multiple levels over which to group, generating a larger range of statistics, and so forth.

13.2.2 Merging/Joining Data

We will also, at times, wish to join two (or more) pieces of data together, for example "merging" data frames or tibbles by a column or columns. For example, it may be that we have a survey where each individuals surveyed belongs to a household, where all of the individual-level questions in the data are stored as an individual-level file (along with the individual's household ID), while the household level measures are stored in another file at the household level. If we wish to work with information on the individual *and* their household characteristics, we will need to "join", or "merge" the file using the household ID.

We can see an example of this using the `join()` method. A simple example is if we would like to take the grouped-level statistics generated previously, and join them back into our original `mroz` data. What we want to do then, is for each individual who is in the labour force (`inlf==1`), join the information on the mean number of children under 6 in this group to their row of data, and similarly, for individuals *not* in the labour force (`inlf==0`), incorporate the group-level value to their row.

```
mroz_kids_mean = mroz.groupby(['inlf']).kidslt6.mean()
mroz_mod = mroz.join(mroz_kids_mean, on = ['inlf'], how = 'left', rsuffix = '_Mean')
mroz_mod.loc[:,['inlf', 'kidslt6', 'kidslt6_Mean']].head()
```

	inlf	kidslt6	kidslt6_Mean
0	1	1	0.140187
1	1	0	0.140187
2	1	1	0.140187
3	1	0	0.140187
4	1	1	0.140187

```
mroz_mod.loc[:,['inlf', 'kidslt6', 'kidslt6_Mean']].tail()
```

	inlf	kidslt6	kidslt6_Mean
748	0	0	0.366154
749	0	2	0.366154
750	0	0	0.366154
751	0	0	0.366154

	inlf	kidslt6	kidslt6_Mean
752	0	0	0.366154

Here we have used the `join()` method to conduct our merge, and specified `how='left'`. In this **particular** setting, we could have used any if the various `join` modes which can be `left`, `right`, `inner` or `outer`. This is because in this case, all of the levels of "inlf" which are in one set of data are also in the other set of data. In many cases this may not be the case, and we may have observations in one dataset that are not in the other. In this case, we must be careful to indicate whether we wish to keep observations that appear in both sets of data (`inner`), keep those that only appear in the left-hand side of the command (`left`), those that only appear in the right-hand side of the command (`right`), or those appearing in either data set (`outer`). Note that here when we refer to the left-hand side and the right-hand side, we refer to the data frame indicated before `.join(` as the left-hand side, and that which appears after as the right-hand side.

While there is some mechanics in setting this up, once we have understood the logic of group by and join it would be exceedingly easy to generalise this, for example if we wished to instead have the aggregates produced by labour market status *and* the individual's age (or any other less aggregated level). It would be worth exploring such generalisations in the code above to ensure that you are happy with the way things are working.

13.2.3 Filtering Data

Previously we have discussed how to select particular columns or rows of data, however there are other ways which we can do this. For example, the `filter` method allows us to filter columns or rows that meet certain condition in their labels. For example selecting columns:

```
mroz.filter(items = ["kidslt6", "age"])
```

	kidslt6	age
0	1	32
1	0	30
2	1	35
3	0	34
4	1	31
...	...	...
748	0	40
749	2	31
750	0	43
751	0	60

	kidslt6	age
752	0	39

Perhaps more useful, we can also filter on observations values. This is done using logical indexing

```
mroz[mroz.kidslt6 == 3]
```

	inlf	hours	kidslt6	kidsge6	age	educ	wage	repwage	hushrs	husage	...	faminc	mtr
483	0	0	3	1	31	13	NaN	0.0	3600	36	...	73600.0	0.4415
604	0	0	3	3	37	13	NaN	0.0	2419	39	...	13440.0	0.7715
714	0	0	3	0	31	15	NaN	0.0	3000	34	...	51200.0	0.4415

13.2.4 Generating variables

In order to create new columns, Pandas automatically understands that when we include a name that is *not* a part of the data frame, that we wish to generated a new column of data. For example, in the below we will generate a new variable, which is experience squared:

```
mroz["exper2"] = mroz.exper**2
mroz.head()
```

	inlf	hours	kidslt6	kidsge6	age	educ	wage	repwage	hushrs	husage	...	mtr	motheduc
0	1	1610	1	0	32	12	3.3540	2.65	2708	34	...	0.7215	12
1	1	1656	0	2	30	12	1.3889	2.65	2310	30	...	0.6615	7
2	1	1980	1	3	35	12	4.5455	4.04	3072	40	...	0.6915	12
3	1	456	0	3	34	12	1.0965	3.25	1920	53	...	0.7815	7
4	1	1568	1	2	31	14	4.5918	3.60	2000	32	...	0.6215	12

If we instead include a variable name which is already present, any existing information will be replaced with the new values we assign.

13.2.5 Reshaping Data

Finally the last tool that we introduce for data manipulation is the idea of "reshaping" or transforming data from a wide format to a long format (and *vice versa*). We refer to data in a **long** format if for each observation we have multiple rows of data, covering different levels of some other variable. For example, a classic example of this would be a panel of data over countries and years. On the other hand, we refer to data as being in a **wide** format if for each observation we have multiple columns (or variables) which cover different levels of some other variable. In certain settings we will wish to work with data in one format, and in other settings we may find it more convenient to work with data in another format. To make this a bit clearer, consider the below data frame:

```
Data = pd.DataFrame({'Zone': ["A", "A", "A", "B", "B", "B"], 'Year': [2020, 2021, 2022, 2020
Data
```

	Zone	Year	Sales
0	A	2020	90
1	A	2021	100
2	A	2022	115
3	B	2020	82
4	B	2021	98
5	B	2022	106

Here we can see that data is in the long format, with multiple observations for one unit (zone in this example) over a number of years. To convert this data to wide format we can use the `pivot` method, as follows:

```
Data = Data.pivot(index = 'Zone', columns = 'Year', values = 'Sales')
Data
```

Year Zone	2020	2021	2022
A	90	100	115
B	82	98	106

As you can see now there is now one observation per unit, where each observation has three resulting columns (one for each year). Now, if we wish to return to the long format we can do so with the converse `melt` function:

```
Data = Data.melt(value_vars=[2020,2021,2022], value_name = 'Sales')
Data
```

	Year	Sales
0	2020	90
1	2020	82
2	2021	100
3	2021	98
4	2022	115
5	2022	106

In practice, we will generally use these commands with a considerably larger number of units, but the basic principles are the same regardless.

14 Basic Plots

To end this initial introduction, we will briefly explore the generation of basic plots in `Python`. In Python one of the most used modules to create plots is `pyplot` from the `matplotlib` module. If you have not previously installed this module, you can do so using `pip install matplotlib`, and then we will import the pyplot submodule as `plt`.

```
from matplotlib import pyplot as plt
```

`pyplot` maps pairs of points in the (x, y) coordinate axis and places them on a graph. For example, let's consider a case where x is a sequence of integers between -5 and 5. This can be generated as below:

```
x = np.array(range(-5, 6))
x
```

```
array([-5, -4, -3, -2, -1,  0,  1,  2,  3,  4,  5])
```

Now consider the variable $y = x^2$:

```
y = x**2
y
```

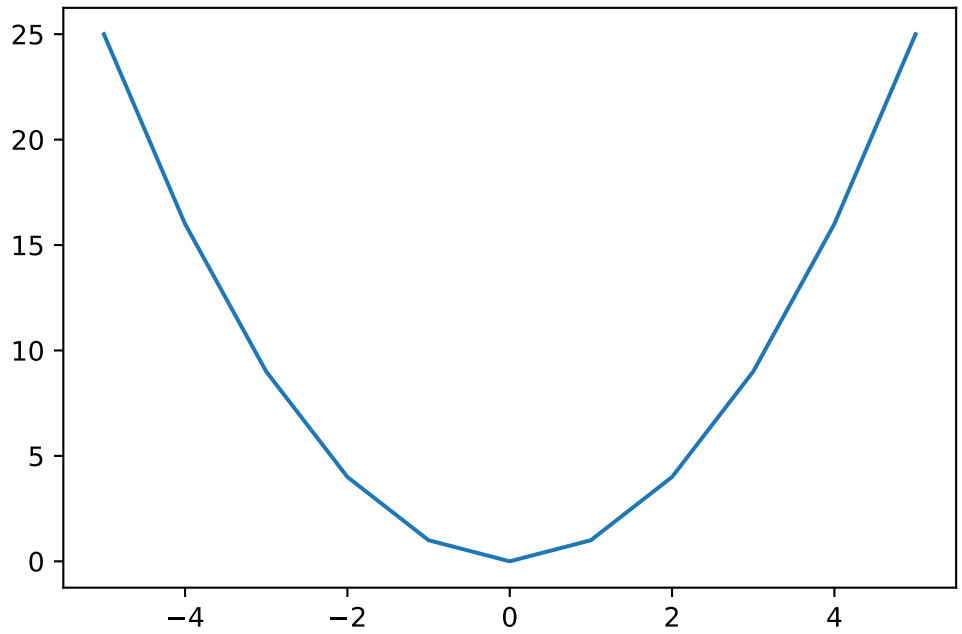
```
array([25, 16,  9,  4,  1,  0,  1,  4,  9, 16, 25])
```

We will store these two objects as a data frame, by creating a dictionary containing `x` and `y` (though we could proceed to plot without first storing our data as a data frame):

```
Data = pd.DataFrame({'x': x, 'y': y})
```

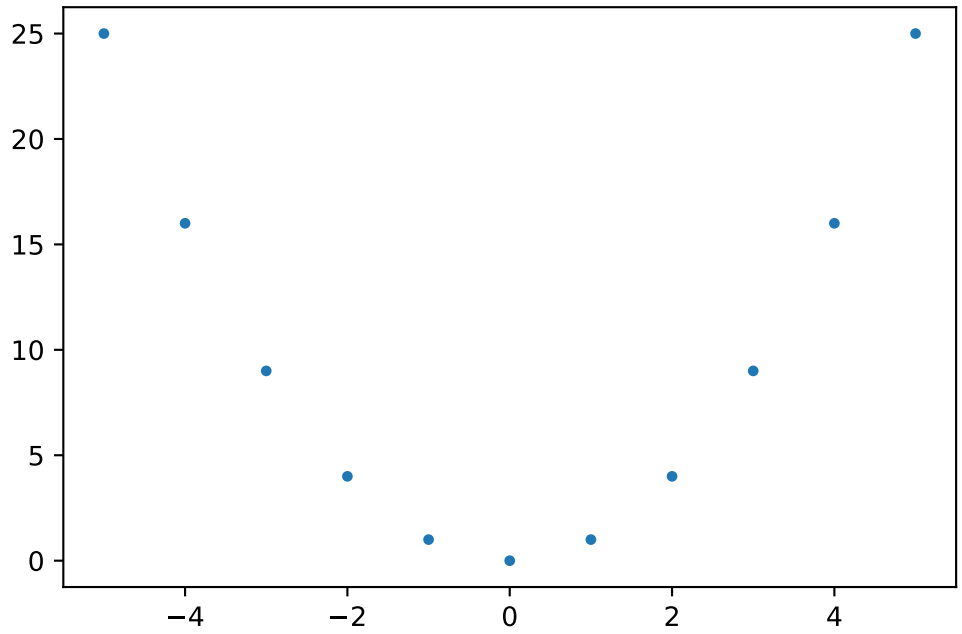
The relevant function in `pyplot` is `plot`, which by default generates a line plot, making reasonable decisions for axes and labelling. Unless you have an IDE which automatically displays your plot in a plot panel, you should use the `show` method from `pyplot` to display your plot:

```
plt.plot(Data['x'], Data['y'])
plt.show()
```



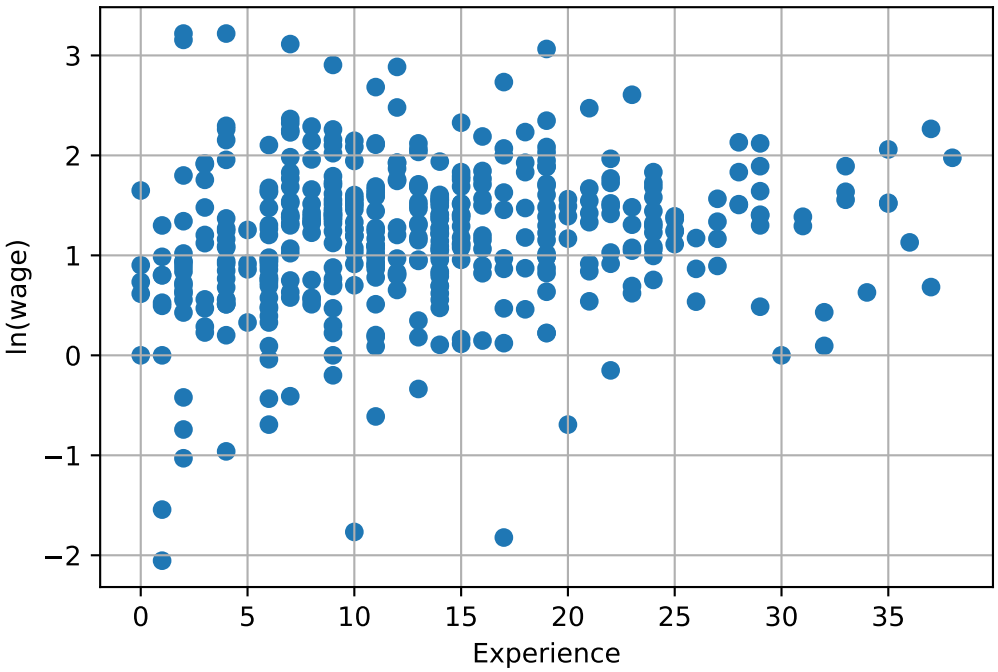
If you wish to change the default behaviour of `plot` you can do so with a range of options. For example, in order to change the plot to a scatter plot, we can set the linestyle to `""` and the markers to points:

```
plt.plot(Data['x'], Data['y'], linestyle = "", marker = ".")
plt.show()
```



There are many ways to further customise graphics in `Python`, and indeed, we will see a range of other plots throughout the remainder of these pages. In general, editing basic plots can be done relatively straight-forwardly, allowing us to build up plots bit by bit. For example, we could use the `mroz` data we have worked with previously to plot the log(wage) against years of experince:

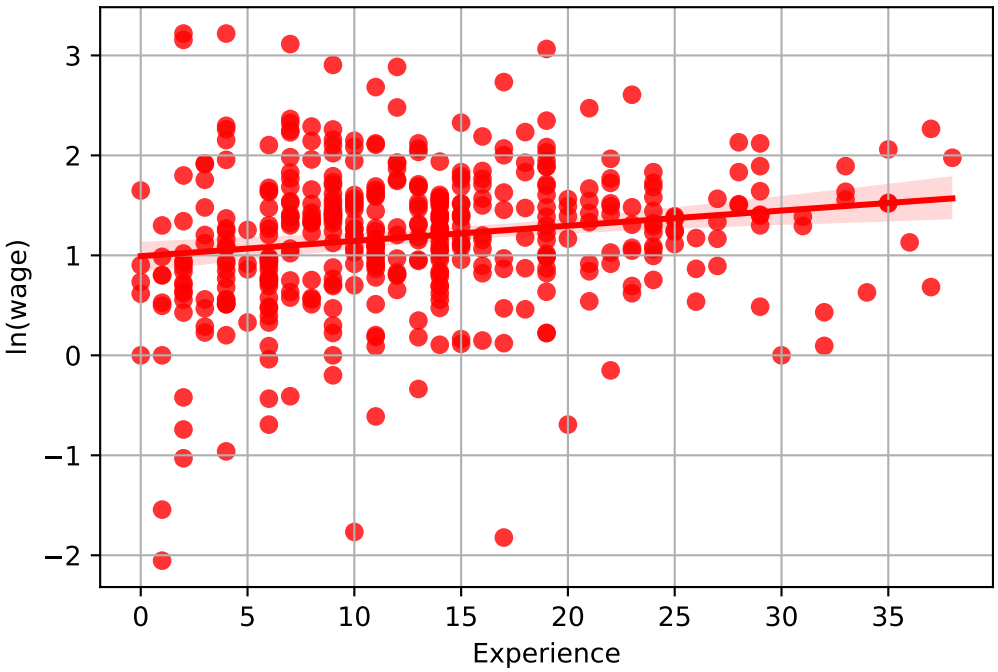
```
plt.scatter(mroz['exper'], mroz['lwage'])
plt.grid(True)
plt.ylabel("ln(wage)")
plt.xlabel("Experience")
plt.show()
```



Another widely-used package to allow for further customisation of graphs in `Python` is the `seaborn` library, which can be installed from pip. For example, if we wish to overlay a regression line on the scatter plot above, along with its 95% confidence intervals, we can do so quite easily with `seaborns regplot` function. To see this, we will first import seaborn, and then finally generate the plot:

```
import seaborn as sns
```

```
plt.clf()
sns.regplot(data = mroz, x = 'exper', y = 'lwage', color='red')
plt.grid(True)
plt.ylabel("ln(wage)")
plt.xlabel("Experience")
plt.show()
```


ln(wage)
Experience
3
2
1
0
−1
−2
0
5
10
15
20
25
30
35

15 Writing Our Own Functions

15.1 An Introduction to Functions in Python

Functions in Python are key to efficient and organized programming. They allow you to encapsulate specific tasks into reusable blocks of code, enhancing modularity and readability. Functions are fundamental in Python, whether for simple operations or complex data analysis tasks.

15.1.1 Creating a Simple Function

In Python, functions are defined using the `def` keyword. A basic function typically includes a name, arguments, and a return statement. Names must begin with a letter or underscore, and should be followed by letters, underscore or number, and are case-sensitive. Here's an example of a simple function that adds two numbers:

```
def add_values(a, b):
    return a + b

result = add_values(12, 3)
print(result)
```

```
15
```

This function, `add_values`, takes two arguments, `a` and `b`, adds them together, and returns the sum. Functions need not have arguments. If no arguments are required, a function can be simply defined as `function_name()`. Similarly, functions need not return a result. If a function is issued with a `return` statement, the function will simply execute the lines with any arguments provided, and return nothing.

15.1.2 Understanding Function Arguments

Function arguments are the values passed to the function when it is called. They can be mandatory or optional. Optional arguments have default values, allowing the function to be called with fewer arguments. For example, below we define a function with an optional second argument. We will call it once with only one argument, in which case the second argument will revert to default. We will then call it a second time with two arguments, and in these cases, the default behaviour will be over-ridden:

```
def multiply_values(a, b=2):
    return a * b

print(multiply_values(10))
print(multiply_values(10,3))
```

```
20
30
```

15.1.3 Documenting our Functions

Finally note that we can document our function using what are known as *docstrings*. Docstrings are strings enclosed in triple quotes that appear as the first statement in a function body. They provide documentation about the purpose and usage of the function. We can then consult these docstrings using Python's `help` function. For example, see below the edit we have made to the above `multiply_values` function, which now makes it much easier to quickly remember what the function does when we use `help` below:

```
def multiply_values(a, b=2):
    """
    This function multiplies two values which are provided as arguments.
    The function returns the product of the two arguments provided.
    Two or optionally one argument can be provided.  If only one argument is
    provided, the second argument defaults to two.
    """
    return a * b

help(multiply_values)
```

```
Help on function multiply_values in module __main__:

multiply_values(a, b=2)
```

```
This function multiplies two values which are provided as arguments.
The function returns the product of the two arguments provided.
Two or optionally one argument can be provided.  If only one argument is
provided, the second argument defaults to two.
```

16 An Introduction to Stata

16.1 An Introduction to Stata

17 Preliminaries

We will assume that when you are reading this you have access to some version of Stata. Likely, the way you will interact with Stata will be with Stata's native GUI which provides you with a number of windows including a panel showing the history of commands, an area displaying any variables you have loaded, a place to type commands, a number of emerging windows which we will discuss below, and most importantly, a "shell", or Stata interface, which looks like the following:

```
  ___  ____  ____  ____  ____ ®
 /__    /   ____/   /   ____/      18.0
___/   /   /___/   /   /___/       MP-Parallel Edition

 Statistics and Data Science       Copyright 1985-2023 StataCorp LLC
                                   StataCorp
                                   4905 Lakeway Drive
                                   College Station, Texas 77845 USA
                                   800-STATA-PC        https://www.stata.com
                                   979-696-4600        stata@stata.com

Stata license: Single-user 8-core  perpetual
Serial number: SERIAL HERE
  Licensed to: NAME HERE
               INSTUTION HERE

Notes:
      1. Unicode is supported; see help unicode_advice.
      2. More than 2 billion observations are allowed; see help obs_advice.
      3. Maximum number of variables is set to 5,000 but can be increased; see help set_maxva

.
```

Stata comes in versions ranging from 1 to 19, with later versions being assigned higher numbers. Later versions of Stata can run any code written in earlier versions of Stata, though at times new commands are introduced, which may be available in more recent versions of Stata, but not in earlier versions. In the below we will generally work in a way which will work with

any recent versions of Stata (for example from version 12 onwards, given that versions prior to Stata 12 are not common), though we will note in a few cases where commands may be specific to only certain versions of Stata and later.

If you are in the Stata GUI, `Stata` code can be entered directly in the command line which is generally located at the bottom of the window. The most simple way to execute code in `Stata` is to simply input commands at the command like. So, for example, you could write the following in the command line and press enter, and `Stata` will evaluate this code:

```
display 4 + 4
```

```
<IPython.core.display.HTML object>
```

```
8
```

As you can see, `Stata` has understood the instruction that you want to add 4 plus 4 and show the result, which is evaluated, and output directly in the `Stata` console above. Later in this chapter we will properly explain these mathematical operations and the different `Stata` data types. You will note that unlike in certain other languages, here we have not simply input a mathematical operation to be evaluated. This is because `Stata` requires every interaction to be based on a valid command, which in this case happens to be the `display` command. So if, for example, we were instead to type:

```
4+4
```

```
4 is not a valid command name
r(199);
```

we see that the answer is not returned to us, but rather some message suggesting that we have done something invalid (we will discuss such messages at more length below). `Stata` has a large range of commands which can be invoked at the command line. We will become familiar with a considerable number of these throughout this introduction. As another very simple example we can type `pwd` for "print working directory":

```
pwd
```

```
/home/dcc213/trabajo/Teaching/Exeter/ResearchMethods-I/site
```

This command show us the path on our computer where `Stata` is currently working. Thus, any files saved will be exported here, and `Stata` will search for any thing to import from this directory. If you wish to change this directory you must use the function `cd("Path/to/the/directory/you/want/to/use")` with the path written between `" "` in order for `Stata` understand that is a character, again a topic that will be introduced more deeply later. It is important to note that paths should be separated with the slash character: `/` rather than the backlash `\`, as this will work on any operating system. Indeed, if you try to set directories with a backslash, on certain operating systems you will see that `Stata` returns an error. This may not happen on your operating system, but it is good practice to separate paths by "/" as it ensures code can be run seamlessly on other systems.

We will return to discuss the precise nature of commands in more detail a bit later on. For now you will note that we have simply been typing instructions to `Stata` directly into the command line, and `Stata` has been providing us with output. Generally, we will not work this way in an empirical research project, as we will want to develop code over time, and be able to return to `Stata` and replicate code in the future. For this reason, generally we will work in do files, or files named as `something.do` (where `something` will be logical name for the file). We will use these do files to store the commands which we require for particular processes or routines, and can then save them to the disk of our computer before running them in `Stata`. These scripts can be generated in any type of text editor, and then can be run in `Stata` using the `do` command. For example, below, you can see a screenshot of a session on `Stata` where we have a basic `Stata` do file named `firstStataScript.do` which we are editing in `Stata`'s native do file editor (which can be accessed by typing `doedit`, and behind this, in the `Stata` console we see that the script is run using `do` and output is provided.

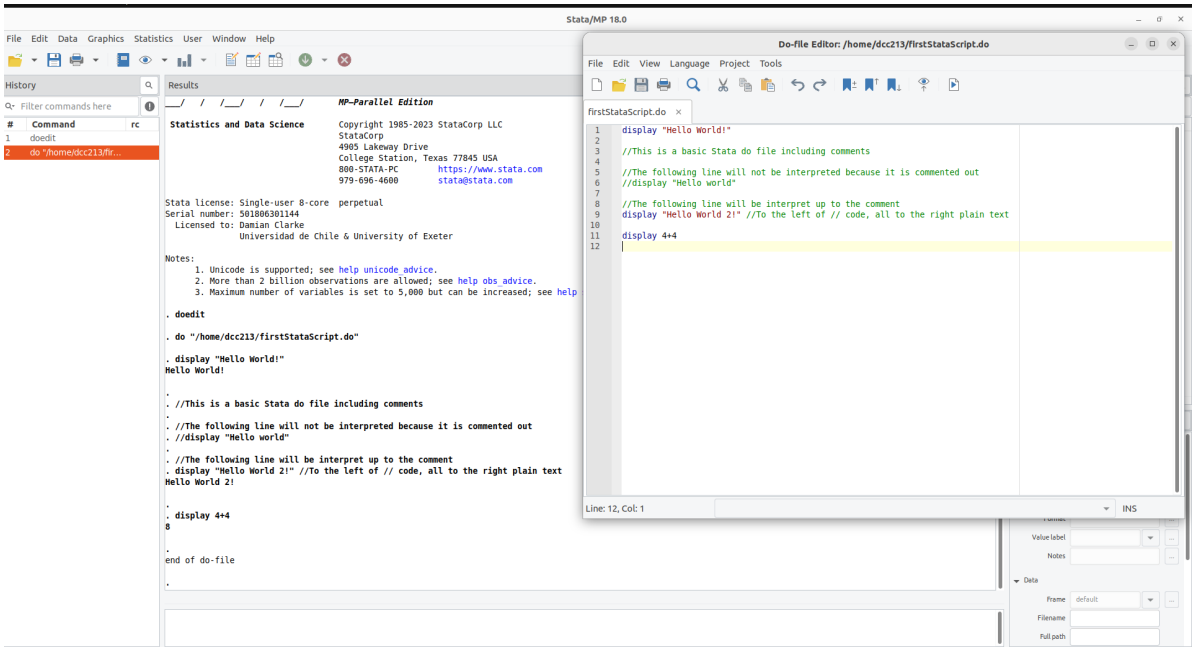


Figure 17.1: A Stata do file and Output in Stata

You may notice something strange about the code and the output in the `Stata` console. The Stata do file has 7 distinct lines of text, but the console only executes three lines of code (the lines in Stata which do not begin with a "."). The reason for this is because we have included comments, or messages for human eyes, but not for the computer to interpret. Any time that `//` is included in code, `Stata` understands that this is a comment, and hence is ignored in executing the code. So, for example, when we write:

```
display "Hello World!"
```

```
Hello World!
```

in our code, the `Stata` console correctly echoes *Hello World!* back at us. However, if we enter precisely the same code, but begin the line with a comment symbol, `Stata` will not do anything given that it interprets everything to the right of `//` as plain text which should be ignored. Similarly, the "*" symbol used at the beginning of a line also serves to act as a comment character. You will note that on line 9 of the code displayed in the do file editor, we are also perfectly able to use comments within a line in do files, and the line will only be executed up to the point which the comment is reached. It is good practice to comment code extensively, both for sharing code with other users, but also for sharing code with yourself in the future, where you may not remember precisely what you were thinking when you originally wrote your code! As a final note on comments, you can also write comments which span multiple lines

if these comments are contained within the characters: `/*` which opens the commend and `*/` which closes the comment. You may wish to confirm this yourself by opening a do file and checking that you can write a multi-line comment in this way.

i Stata and the Operating System

While below we focus on the particulars of Stata as a language, Stata interacts directly with your computer's file system, and you can move through your file system and see where you are with a number of Stata commands. In particular, some useful commands in this line are below:

- `pwd`: shows your **current working directory**.
- `cd "path/to/folder"`: changes the working directory.
- `dir`: lists the files in the current directory.
- Files containing Stata code usually end with `.do` (do-files) or `.ado` (programs).
- Do-files can be run using the command `do myfile.do`.
- You can save datasets in Stata format with `save mydata.dta` and later open them with `use mydata.dta`.

17.0.1 Checking this out locally

1. Open Stata. What is your current working directory (`pwd`)?
2. Create a new folder on your operating system. Use `cd` in Stata to move into that folder.
3. List the contents with `dir`. What do you see?
4. Create a small dataset in memory (e.g. `clear`, then `set obs 5`, `gen x = _n`) and save it with `save mydata.dta`.
5. Confirm that `dir` now shows `mydata.dta`.
6. Close Stata, reopen, and load the dataset using `use mydata.dta`.
7. Create a do-file called `myFirstFile.do` in the folder. Confirm that you can run it with `do myFirstFile.do`.

18 An introduction to the language and data types

Until now we have executed some code in `Stata` and seem some specific commands which print output, but we have never stored any results for later use within our code. Generally we *will* want to store the output of intermediate steps in our code for later use. We will show a number of ways to do this below.

In general, when we work with `Stata` we will work with variables, which are stored by `Stata` as a dataset. We can assign the values or characters to variables, provided that we first ensure that `Stata` has a defined number of observations. For example:

```
set obs 10
generate x1=1
```

```
Number of observations (_N) was 0, now 10.
```

If we already have data in memory, the first line of code `set obs 10` will not be necessary, as we will already have a defined number of observations. However in the second line where we generate a variable named `x1` and define its value as 1 for each observation, this will be generated for the number of observations in our entire dataset. We will discuss this structure shortly, as well as alternative ways to store information, when we lay out `Stata`'s data types below.

In the above code we have seen that Stata has used the command `generate` and then uses the equals symbol to assign a value to the variable generated. If we fail to remember to type generate, or make some other error in our assignation, Stata will inform us about our error by printing out an error message:

```
x1 = 1
```

```
command x1 is unrecognized
r(199);
```

These error messages include a brief explanation of what has gone wrong, to help you diagnose and fix your code, as well as an error code (in this case r(199)), which may allow you to find further information related to the error. All error codes can be found in `Stata`'s manuals. When an error occurs, anything which you have asked of your code will not be performed, and so in this particular case you will not see any variable named *x1* in your Stata memory.

18.1 Data Types

18.1.1 Data Stored as Variables

The logic of `Stata` is generally based on the idea that we will have a principal dataset which will allow us to store related variables. This is similar to the idea of data frames in other languages, and makes sense for a lot of our econometric work where we will often wish to store information on N individuals in rows, and K variables in columns. By default, when we generate variables these will be stored in a single dataset which we can browse in `Stata`. As we have seen above, we can generate this dataset "by hand" by first indicating the number of observations (rows) available, and then simply provide names for any newly added variables. As we will see below, in practice we will generally actually import an existing dataset, and then generate any variables to add to this data. Traditionally, Stata obliges us to work with a single dataset in memory, and so at times we need to think carefully about how to combine datasets if we are working with multiple pieces of information at once. However, as we will see shortly, frmo version 16 of `Stata` onwards, we are now able to hold multiple datasets in memory simultaneously, potentially making it more simple to work with varying types of information.

18.1.1.1 Text (String) Data

To get up and running with `Stata`, it is useful to understand the different ways which data can be stored. `Stata` can hold a number of different types of information in the variables we generate, and it is important that `Stata` understands what type of information we are passing or storing. Among others, `Stata` allows for variables to hold text (strings) or numeric data types. Text is perhaps the conceptually simplest type of data. Text is simply saved as strings of characters, and should be specified between `" "`. We have seen the use of characters when displaying "Hello world" previously. If characters are not indicated between strings, `Stata` will understand that they refer to variable names. For example, returning to *Hello World*, if we seek to generate a variable which stores this text, if this is entered without quotes, an error will appear:

```
generate mivar = Hello World
```

```
Hello not found
r(111);
```

Indeed, if we look carefully at the error message we may see that the error has occurred when `Stata` tried to evaulate `Hello`. As we did not enclose `Hello` in double quote, Stata seeks to find some internal information such as a variable or command called `Hello`. When it is unable to find tis, an error is produced. However, if we had correctly enclosed "Hello World" in double quotes, we would see that this would be stored as a variable, which we are naming `mivar`:

```
generate mivar = "Hello World"
```

We can confirm that this variable exists and is effectively understood to be string data by using the `describe` command. This command describes to us the variable which we request to describe, or if entered with no names, simply describes all the data held in the memory. Below we see that among other things, the storage type of this variable is denoted as `str11`, a string variable with 11 characters.

```
describe mivar
```

```
Variable      Storage   Display    Value
    name         type    format    label      Variable label
------------------------------------------------------------------------------
mivar           str11   %11s
```

Numeric data is entered without any special behaviour, simply writing the number in the case of numerica values. Indeed, if numeric data is accidentally enclosed between quotes, it will be treated as character, and *not* numeric data. For example, below we assign two variables based on the number `2` and confirm that one is numeric while the other is viewed as a character.

```
qui set obs 10
generate num1=2
generate num2 = "2"
describe num1 num2
```

```
Variable      Storage   Display    Value
    name         type    format    label      Variable label
------------------------------------------------------------------------------
num1            float   %9.0g
num2            str1    %9s
```

If you try to perform numerical operations based on the above variables (more on this below), you will see that `num1` can be involved in such calculations, while `num2` cannot.

18.1.2 Working with frames (≥ Stata 16)

One of the few points we will see here which is Stata version specific is the use of frames. From Stata version 16 and beyond, the ability to work with multiple datasets simultaneously in a single session was introduced. For example, given that we have generated a number of variables in our session, we can see that there is a current data frame named default.

```
frame
```

```
  (current frame is default)
```

If we would like to start working with multiple frames we may wish to rename this frame for simplicity using `frame rename oldname newname`

```
frame rename default my10obs
```

Now we will create a new data frame with `frame create name` and change to that data frame with `cwf name`, where `cwf` refers to "Change working frame":

```
frame create secondframe
cwf secondframe
describe
```

```
Contains data
 Observations:             0
    Variables:             0
Sorted by:
```

As you can see when we use the `describe` command, our new frame contains none of the information which we have previously generated in our session. However, if we request that Stata , but if you use

```
frames dir
```

```
* my10obs      10 x 4
  secondframe  0 x 0

Note: Frames marked with * contain unsaved data.
```

You will see how there is two data frames and the one named `my10obs` contains our previously generated data. We can now simply use `cwf` to toggle between data frames should we wish to work in two separate data environments.

18.1.3 Other Types of Data Stored by Stata

Although the most common way to work in `Stata` will be within a dataset or data frame, `Stata` provides a number of additional ways to store data including as scalars and matrices. Scalars are objects that store a single value, which can be either a string or a number. For example, we can define scalars as below, noting the use of the `scalar` command.

```
scalar x1 = "Hello World"
display x1
scalar x2 = 4 + 4
display x2
```

```
Hello World
8
```

This code succesfully store the string *Hello World* in the scalar named `x1` and 8 (as the result of 4 plus 4) in the scalar `x2`. This scalars can also be used in future operations

```
scalar x2 = 4 + 4
scalar x3 = x2 + 4
display x3
```

```
12
```

You may note that here we have generated a scalar named as `x1`, and previously we had generated a variable named `x1` in our Stata dataset. There is no formal reason why you cannot name scalars in the same way as you name variables, as Stata will view them as different objects. Of course, you likely will wish to avoid the confusion of having variables and scalars named in the same way, and it is good practice to use more descriptive names when generating variables and scalars. However, it is worth noting one difference in working with scalars and variables. If we wish to generate a variable which is already generate, `Stata` will

not allow us to, resulting in an error indicating a variable with this name is already defined. However, if we wish to replace a scalar with a new value, `Stata` will issue no error, simply destroying the old scalar, and creating a new one. For example, below (noting that the variable `x1` and the scalar `x1` have previously been defined), we can see this differing behaviour:

```
qui set obs 10
gen x1=1
gen x1=2
scalar x1=2
```

```
variable x1 already defined
r(110);
```

If we do *actually* wish to save over our variable, we need to explicitly indicate this by using the `replace` command instead of the `generate` command.

```
qui set obs 10
gen x1=1
replace x1=2
```

```
(10 real changes made)
```

Finally, we can also work with numerical matrices which can only store numbers and which work in the fashion of standard algebraic matrices, by using the group of `matrix` commands. To see this, consider the following matrices, which are input using `matrix input`, and which we name as `A` and `B`:

```
matrix input A = (1, 2 \ 3, 4)
matrix input B = (5, 6 \ 7, 8)
```

Stata's matrix notation is to separate columns by commas, and rows with the backslash. We can see the result of this `matrix input` using the `matrix list` command

```
matrix input A = (1, 2 \ 3, 4)
matrix input B = (5, 6 \ 7, 8)
matrix list A
matrix list B
```

```
A[2,2]
    c1  c2
r1   1   2
r2   3   4

B[2,2]
    c1  c2
r1   5   6
r2   7   8
```

This matrices are stored and can then be operated with to conduct permitted matrix operations, as seen below.

```
matrix input A = (1, 2 \ 3, 4)
matrix input B = (5, 6 \ 7, 8)
matrix define C = A * B
matrix list C
```

```
C[2,2]
    c1  c2
r1  19  22
r2  43  50
```

A list of matrix operations can be found in Stata's manuals.

18.2 Basic Operations

The standard mathematical operations in `Stata` likely work how you would expect. Basic mathematical operations are as follows, with an example of their use below:

Symbol	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Power

A few examples

```
display 10 + 3
display 10 - 3
display 10 * 3
display 10 / 3
display 10 ^ 3
```

```
13
7
30
3.3333333
1000
```

Also when you combine different operators it respects the PEMDAS order, first resolving Parentheses, second solve Exponents, third Multiplication and Division, fourth Addition and Substraction. Both multiplication and division, and addition and substraction are evaluated left to right, rather than sequentially by operation, so it is important to indicate to `Stata` explicitly what you need using parentheses in certain cases. For example in order to solve

$$-\frac{5+3^{5-3}}{5\times 3}$$

You should use the code

```
display -(5+3^(5-3))/(5*3)
```

```
-.93333333
```

In general, these operations are assumed to refer to variables or scalars. In the case of matrices, addition, substraction multiplication and division are also possible with the same symbols, where multiplication will result in matrix multiplication (and not element-wise multiplication), and division assumes the division of each element of the matrix by a single scalar. Matrix transpose can also be performed simply using `'`. If, however, you wish to perform more complicated matrix calculations such as element-wise operations, it may be worth using `Stata`'s own matrix processing language, which is known as `Mata`. In this introduction we will not explore `Mata`, however it is worth noting its existence, particularly if you wish to work with demanding numerical tasks.

Certain mathematical operators can also work with strings, specifically `+` and `*`. Stata automatically identifies when you use `+` if you are adding numerical values or concatenating strings, in this case you must use the same data type. When you use `*` to operate with strings you must specify a string and a number, because Stata will repeat the string as many times as the number indicates:

```
scalar x1 = "Hello " + "World"
scalar x2 = 2 + 2
scalar x3 = "Hello" * 2
display x1
display x2
display x3
```

```
2
4
HelloHello
```

Stata also has relational and logical operators that returns a numerical value associated with the truth value of an expression (0 for False and 1 for True) pointing if the statement is true or false. This operators are:

Symbol	Comparison
==	Equals
!=	Difference
>	Greater
>=	Greater or Equal
<	Less
<=	Less or Equal
|	Or
&	And
! or ~	Negation

A few examples

```
display 4 == 3
display 4 != 3
display 4 > 3
display 4 >= 3
display 4 < 3
display 4 <= 3
display (4 == 3) | (4 != 3)
display (4 == 3) & (4 != 3)
display ~(4 == 3)
```

```
0
```

```
1
1
1
0
0
1
0
1
```

We will return to these logical operators below when we discuss the use of conditions in manipulating variables.

18.3 Loops in Stata

Looping constructs in programming languages provide a mechanism to execute a set of instructions multiple times, contingent upon certain conditions. Stata offers several looping constructs. Among these, the **`foreach`** and **`while`** loops are most commonly encountered.

18.3.1 1. The `foreach` Loop

In Stata, the **`foreach`** loop is commonly used to iterate over lists or variables.

1.1. Iteration Over Variables

Consider a dataset of student grades:

```
clear
input str10 student grade
Alice 85
Bob 90
Charlie 78
Diana 92
end

foreach var of varlist grade {
    summarize `var', detail
}
```

grade

```
-------------------------------------------------------------
      Percentiles      Smallest
 1%           78            78
 5%           78            85
10%           78            90       Obs                   4
25%         81.5            92       Sum of wgt.           4

50%         87.5                     Mean                86.25
                        Largest      Std. dev.           6.238322
75%           91            78
90%           92            85       Variance            38.91667
95%           92            90       Skewness           -.5083257
99%           92            92       Kurtosis            1.738983
```

1.2. Iteration Over Lists

For tasks requiring iteration over a series of numbers:

```
foreach i of numlist 3/8 {
    display "Processing data point `i'."
}
```

```
Processing data point 3.
Processing data point 4.
Processing data point 5.
Processing data point 6.
Processing data point 7.
Processing data point 8.
```

18.3.2 2. The `while` Loop

The **`while`** loop in Stata executes statements as long as a particular condition remains true.

Consider a scenario where we're approximating the square root of a number using the Babylonian method:

```
clear
scalar number = 49
scalar guess = number / 2  // Initial guess
scalar difference = 1
scalar tolerance = 0.01
```

```
while difference > tolerance {
    scalar new_guess = 0.5 * (guess + number / guess)
    scalar difference = abs(new_guess - guess)
    scalar guess = new_guess
}

display "Approximated square root: " round(guess, .01)
```

```
Approximated square root: 7
```

18.3.3 3. Loop Control Mechanisms in Stata

The continue command in Stata's foreach, forvalues, or while loops interrupts the current iteration and omits the rest of the loop's commands, restarting from the loop's beginning. However, when paired with the break option, it exits the loop completely and proceeds with the next command outside the loop. This feature allows for flexible control within looping constructs, either to skip an iteration or to terminate the loop based on specific conditions.

3.1. Utilizing `continue, break` in Odd Numbers and Least Common Multiple (LCM)

```
clear

// Loop through numbers 1 to 10
forvalues i = 1/10 {
    // Display odd numbers and break the loop when the number reaches 7
    if mod(`i', 2) == 1 {
        display "Odd number: `i'"
        if `i' == 7 {
        continue, break // Break out of the loop when the number reaches 7
        }
    }
}

forvalues x = 1/10000 {
    if mod(`x',7)==0 & mod(`x',8)==0 & mod(`x',9)==0 {
        display "The least common multiple of 7, 8, and 9 is `x'"
        continue, break
    }
}
```

```
Odd number: 1
Odd number: 3
Odd number: 5
Odd number: 7
The least common multiple of 7, 8, and 9 is 504
```

In conclusion, loops are indispensable in Stata, facilitating repetitive tasks and complex computations. Mastery of these constructs is essential for academic research, statistical analysis, and data visualization.

18.4 Stata Commands

Beyond these logical and mathematical operators, `Stata` comes with a large suite of built in commands which perform specific tasks, as well as additional user-contributed functions (discussed below) which can perform a number of more specialised tasks. "Out of the box" `Stata` comes with many commands which are all comprehensively documented. A full list can be found in the Stata manuals.

Commands are designed to perform specific tasks, potentially accepting input (known as "arguments"), and returning output which you can interact with. We have already seen a number of functions above such as `display`, `generate`, and `describe` which at times we have used with arguments, and at times we have used without arguments. In order to use commands the highly-standardised Stata syntax is to provide the command's name, followed by its arguments (variable names, conditionals for a subset of observations and other requirements) and its specific options, which are separated by a comment from the main arguments. These options allow us to further specialise commands by requesting particular optional changes to the default implementation. To understand how a command works, we should consult the help documentation which is available for each command by typing `help` and the command name. For example, if we wish to understand the `describe` function, typing `help describe` will provide us with the following syntax, along with a list of options, further explanation, and so forth.

describe [varlist] [, memory_options]

The syntax is always laid out in this standardised way, where square brackets imply that something is optional. Here if we do not type a specific list of variables (varlist) Stata will describe to us all the variables in memory. We are also optionally able to include options, such as simple, and short, which are laid out below in the help file. You will also note one other thing in the syntax diagram, and that is that the `d` describe is underlined. Each Stata command will underline the shortest possible way a command can be written by providing this underline, and in this case this implies that even if we just write `d` Stata will understand that this refers to the describe command. Indeed, we can write command names as this shortest

version possible, or any longer version up to and including the full name, so `d`, `de`, `des` and so forth are all understood as `describe`.

In certain cases if we are not sure exactly what function we need we can type `search search_term` and `Stata` will advise us of any functions which may do what we're after. For example, if we are seeking to summarise our variables, we may type `search summary`, and we see a range of functions, including one called `summary`. We can then consult the help file to see if this is precisely what we are after. Reading the help file we see that the command "calculates and displays a variety of univariate summary statistics."

As in general we will work with data in using our commands, and to see how this `summarize` command works in practice, we will use an internal dataset which Stata provides for us. This is the `auto` dataset, a simple dataset often used to explore commands. In the following section we will further discuss ways to load data into Stata, but for now just note that we can load this particular dataset using the `sysuse` (for "system use") command, along with the name of the auto dataset:

```
sysuse auto, clear
des
summarize mpg
```

```
(1978 automobile data)

Contains data from /usr/local/stata19/ado/base/a/auto.dta
 Observations:            74                  1978 automobile data
    Variables:            12                  13 Apr 2024 17:45
                                              (_dta has notes)
------------------------------------------------------------------------------
Variable      Storage   Display    Value
    name         type    format    label      Variable label
------------------------------------------------------------------------------
make            str18   %-18s                 Make and model
price           int     %8.0gc                Price
mpg             int     %8.0g                 Mileage (mpg)
rep78           int     %8.0g                 Repair record 1978
headroom        float   %6.1f                 Headroom (in.)
trunk           int     %8.0g                 Trunk space (cu. ft.)
weight          int     %8.0gc                Weight (lbs.)
length          int     %8.0g                 Length (in.)
turn            int     %8.0g                 Turn circle (ft.)
displacement    int     %8.0g                 Displacement (cu. in.)
gear_ratio      float   %6.2f                 Gear ratio
foreign         byte    %8.0g      origin     Car origin
```

```
-------------------------------------------------------------------------------
Sorted by: foreign

    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
         mpg |         74     21.2973    5.785503         12         41
```

Here we can see that when loading this dataset and describing variables, there are number of variables and types, and when we type `symmarize mpg` (one of the variables) we are provided with summary statistics such as the mean, standard deviation, and so forth.

If we again consult the help file of `summarize` we may see that there are actually some further details we did not encounter in the help file for describe. In particular, this time the syntax diagram is as follows:

summarize [varlist] [if] [in] [weight] [, options]

As before, we see that the varlist is optional, and so we *could* type the command without any arguments, in which case `Stata` will assume that we want descriptive statistics for all variables:

```
su
```

```
    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
        make |          0
       price |         74    6165.257    2949.496       3291      15906
         mpg |         74     21.2973    5.785503         12         41
       rep78 |         69    3.405797    .9899323          1          5
    headroom |         74    2.993243    .8459948        1.5          5
-------------+---------------------------------------------------------
       trunk |         74    13.75676    4.277404          5         23
      weight |         74    3019.459    777.1936       1760       4840
      length |         74    187.9324    22.26634        142        233
        turn |         74    39.64865    4.399354         31         51
displacement |         74    197.2973    91.83722         79        425
-------------+---------------------------------------------------------
  gear_ratio |         74    3.014865    .4562871       2.19       3.89
     foreign |         74    .2972973    .4601885          0          1
```

What is new here are the *if*, *in* and *weight* options, which allow us to apply the command to a particular subset of observations, in the case of *if* you must specify a condition. For example we will summarize the price of those those observations with a `mpg` greater than its mean

```
su price if mpg > 21.2973
```

```
    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
       price |         31    4879.968    1344.659       3299       9735
```

As you see the number of observations and other statistics have changed. The use of *in* is similar, but instead of specifying a condition you must specify the position as row numbers. For example, if for some reason we wished to summarise the price for the first 20 observations we could do so as follows:

```
su price in 1/20
```

```
    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
       price |         20     6359.15    3710.311       3299      15906
```

Also you can see the *weight* observations if you wish to perform weighted calculations. We will not delve into this point here, but I direct you to the documentation for *weight* which can be consulted by typing: `help weights`.

We may also wish to explore certain *[, options]*, which are laid out in the help file below the syntax diagram. As an example in the summarize help documentation you can see that there is a *detail* option and its description which says "display additional statistics". So, noting the underlined *d* in the help file, we can provide more specialised output to the command by invoking this option:

```
su price, d
```

```
                            Price
-------------------------------------------------------------
      Percentiles      Smallest
 1%         3291            3291
 5%         3748            3299
10%         3895            3667       Obs                  74
25%         4195            3748       Sum of wgt.          74
```

```
50%       5006.5                      Mean           6165.257
                        Largest       Std. dev.      2949.496
75%         6342          13466
90%        11385          13594       Variance       8699526
95%        13466          14500       Skewness       1.653434
99%        15906          15906       Kurtosis       4.819188
```

A final point of note is something which you may have realised when we summarised the entire dataset. While our dataset contains 74 observations (something which we can confirm using the `count` command), but one variable (`rep78`) does not provide a summary for 74 observations:

```
count
su price rep78
```

```
  74

    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
       price |         74    6165.257    2949.496       3291      15906
       rep78 |         69    3.405797    .9899323          1          5
```

The reason for this is because the variable has certain missing observations.

```
count if mpg==.
count if rep78==.
count if rep78<1000
count if rep78>-1000
count if rep78>-1000 & rep78!=.
```

```
  0
  5
  69
  74
  69
```

In Stata, missing values are recorded as `.`, and are, by definition, not considered when conducting numerical analyses. In the code above we first confirm that the variable `mpg` has no missing observations, using the `count` command, and the `if` construct discussed above. We then confirm that the missing observations in `mpg` are indeed recorded as `.`. In general, `Stata`

makes it relatively easy to work with missing observations, simply omitting these observations from any analyses. However, it is important to note this when conducting analysis. There is one hugely important caveat here, and that is to realise that by construction, `Stata` views `.` as an infinitely large value. So, if you consider the examples above, you will note that when we count the observations with $mpg < 1000$, we are correctly told it is 69. However, when we could the number of observations with $mpg > -1000$, we are told that it is 74, precisely because `.`is seen as an infinitely large value. For this reason, it will be very careful for us to ensure that we deal carefully with missing values, for example, explicitly accounting for missings, as we do in the last line above.

When we first introduced commands, we noted both that they accept arguments, *and* that they potentially return objects. Conveniently, the objects which Stata returns are generally provided to use for further processing if desired. Anything return from commands can be found by inspecting the `return list`, or `ereturn list` for estimation commands, as follows:

```
su mpg
return list
```

```
    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
         mpg |         74     21.2973    5.785503         12         41

scalars:
                  r(N) =  74
              r(sum_w) =  74
               r(mean) =  21.2972972972973
                r(Var) =  33.47204738985561
                 r(sd) =  5.785503209735141
                r(min) =  12
                r(max) =  41
                r(sum) =  1576
```

Here you can see that there is a group of scalars that are automatically generated by Stata once the summarize command is executed, and which correspond to the output of the command. You can freely operate with these scalars

```
su mpg
display "Pearson's Variation Coefficient is: " r(sd)/abs(r(mean))
```

```
    Variable |        Obs        Mean    Std. dev.       Min        Max
```

```
-------------+---------------------------------------------------------
         mpg |         74     21.2973    5.785503         12          41
Pearson's Variation Coefficient is: .27165434
```

Similarly, we can incorporate these scalars directly into any variable generation we may wish to proceed with, for example the variable `mpg_meandev`

$$mpg_i - \overline{mpg}$$

Where $\overline{mpg}$ is the mean of `mpg` variable

```
su mpg
generate mpg_meandev = mpg - r(mean)
list mpg mpg_meandev in 1/5
```

```
    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
         mpg |         74     21.2973    5.785503         12          41

     +-----------------+
     | mpg   mpg_mea~v |
     |-----------------|
  1. |  22    .7027027 |
  2. |  17   -4.297297 |
  3. |  22    .7027027 |
  4. |  20   -1.297297 |
  5. |  15   -6.297297 |
     +-----------------+
```

As you can see in the list output there is a new variable named `mpg_meandev` which reports the deviations of each observation from the mean of the `mpg` variable. Note that is not necessary to use the `summarize` command each time we want to access the `return list` values, provide that no other command is used which populates the return list with other values. For example, if we now decide that we would like to generate a standardised version of this variable named `mpg_zscore` as

$$\frac{mpg - \overline{mpg}}{\hat{\sigma}_{mpg}}$$

Where $\hat{\sigma}_{mpg}$ is the sample standard deviation of `mpg` variable, we can do as follows:

```
qui su mpg
generate mpg_zscore = (mpg - r(mean)) / r(sd)
list mpg mpg_zscore in 1/5
```

```
     +-----------------+
     | mpg   mpg_zsc~e |
     |-----------------|
  1. |  22    .1214592 |
  2. |  17   -.7427698 |
  3. |  22    .1214592 |
  4. |  20   -.2242324 |
  5. |  15   -1.088462 |
     +-----------------+
```

18.5 User-Written Commands

While `Stata` provides a wide array of inbuilt commands, at times we may wish to install extensions written by users which are expected to provide both some specific functionality, as well documentation which can be read as a help file. In `Stata`, most user-written commands are stored on a central repository known as the Statistical Software Components (SSC). This can be done quite simply using the command `ssc install packagename`. As an example, in order to install `outreg2`, a library for formatting regression tables, you can do so with the following code:

```
ssc install outreg2
```

```
checking outreg2 consistency and verifying not already installed...
all files already exist and are up to date.
```

Now that the package is installed you can access the help file by typing `help outreg2`, and in the future will not need to re-install the package unless you wish to update the package. Once you have installed a user-written command, it will be permanently available to you.

There are other sources such as the Stata Journal or simply a URL, in both cases the command `net` will provide the tools for search and install packages.

19 Working with Data

19.1 Loading and viewing data

In most – if not all – of our research projects, we will not actually type in data to `Stata` by hand which is a cumbersome and highly error-prone activity, but rather we will wish to read in data which we have stored on our disk in one or a number of files. Such data may be stored in many formats; among others this may be free formats such as `csv` or `txt`, or proprietry formats such as `excel` or Stata's own `dta` files. We have already seen one example of reading in data, which is Stata's `sysuse` command, but this is a special case for data that is provided by Stata at installation.

For data which we have stored somewhere on our hard drive in formats such as `csv` (comma seperated values) or `txt` formats `Stata` comes with the `import delimited`, command. If you consult the help documentation, which is something I would recommend anytime you come across a new function, you will see that the basic usage is relatively standard, as follows:

```
*! eval=FALSE
import delimited "Path/To/Your/File.csv"
```

This will work as is, provided that your csv file is indeed delimited by commas and has variable names on the first line. If, however, your csv does not have variable names, or is delimited by some other character, you may refer to the options in the help file to see how to proceed. Similar such procedures can be followed for other data stored in flat formats such as .txt or others, as laid out in the help file.

This syntax crosses over to excel files too, via the `import excel` command:

```
*! eval=FALSE
import excel "Path/To/Your/File.xls"
```

Note that the example is based on a file assumed to end with `xls` but the same command can be used replacing `xls` with `xlsx` if a file is saved as .xlsx. If you further explore the help documentation you will notice that you can specify a variety of arguments that indicates options such as if the row should be used as variable names, the range of cells you wish to import, the sheet name and other such details.

Finally in order to import `dta` files, which is Stata's native data format you can simply specify the `use` command:

```
*! eval=FALSE
use "Path/To/Your/File.dta"
```

If you are working entirely in Stata the value of saving your data in a .dta format is that you will maintain any internal labelling you may be working with, but this may be more complex if you wish to share your data with others who are working in other languages, given that it is a Stata-specific data format.

Once a dataset is loaded in `Stata` it is immediately avaiable for further processing and use. For a number of examples below we will use another dataset which is provided by `Stata` upon installation (all such datasets can be seen by typing `sysuse dir`). This is an abstract of the 1988 US NLSW (National Longitudinal Survey of Mature and Young Women).

```
sysuse nlsw88.dta, clear
```

```
(NLSW, 1988 extract)
```

Once we have loaded a dataset if we wish we can see a quick snapshot of the data with the `list` command, for example using `list in 1/3` to observe the first three lines:

```
list in 1/3
```

```
     +-------------------------------------------------------------------+
  1. | idcode | age  |  race  | married  |    never_married  | grade  |
     |      1 |  37  | Black  |  Single  | Has been married  |    12  |
     |-------------------------------------------------------------------|
     |          collgrad  |      south  |  smsa  |            c_city  |
     |  Not college grad  |  Not south  |  SMSA  |  Not central city  |
     |-------------------------------------------------------------------|
     |               industry | occupation | union |     wage | hours |
     | Transport/Comm/Utility | Operatives | Union | 11.73913 |    48 |
     |-------------------------------------------------------------------|
     |            ttl_exp            |              tenure             |
     |           10.33333            |            5.333333             |
     +-------------------------------------------------------------------+

     +-------------------------------------------------------------------+
```

```
2. | idcode | age  |  race  | married  |    never_married  | grade  |
   |      2 |  37  | Black  |  Single  | Has been married  |    12  |
   |-----------------------------------------------------------------|
   |          collgrad  |      south  |  smsa  |            c_city  |
   |  Not college grad  |  Not south  |  SMSA  |      Central city  |
   |-----------------------------------------------------------------|
   |               industry | occupation | union |     wage | hours |
   |          Manufacturing |  Craftsmen | Union | 6.400963 |    40 |
   |-----------------------------------------------------------------|
   |            ttl_exp            |              tenure             |
   |            13.62179           |                5.25             |
   +-----------------------------------------------------------------+

   +-----------------------------------------------------------------+
3. | idcode | age  |  race  | married  |    never_married  | grade  |
   |      3 |  42  | Black  |  Single  |    Never married  |    12  |
   |-----------------------------------------------------------------|
   |          collgrad  |      south  |  smsa  |            c_city  |
   |  Not college grad  |  Not south  |  SMSA  |      Central city  |
   |-----------------------------------------------------------------|
   |               industry | occupation | union |     wage | hours |
   |          Manufacturing |      Sales |     . | 5.016723 |    40 |
   |-----------------------------------------------------------------|
   |            ttl_exp            |              tenure             |
   |            17.73077           |                1.25             |
   +-----------------------------------------------------------------+
```

Depending on the way that we are working with `Stata`, we may also be able to explore the data more interactively using the `browse` command.

19.2 Tidying data

Once we have loaded data in `Stata`, we will likely want to manipulate the data in a number of ways. This can include relatively simple things like sub-setting to certain columns or rows (which we have already encountered above), but also more complicated things like aggregating data to a higher level, calculating statistics for one or mutliple variable(s) over other variables, joining together a number of datasets, or "reshaping" data in certain ways. Here we will provide a very brief overview of these procedures, and introduce other things as we need them thoughout the book.

19.2.1 Aggregating Data

To start with some basic data manipulations, we can consider the case where we may wish to convert the data which we have imported into some group-level averages. We can do many such manipulations of this type using Stata's `collapse` command. At its most simple level, this command allows us to take some disaggregated data, and "collapse" it to some more aggregated data. For example, imagine we wished convert our data into a single line containing the mean wage over the entire data. We can do this simply with the collapse command:

```
collapse wage
list
```

```
     +----------+
     |     wage |
     |----------|
  1. | 7.766949 |
     +----------+
```

This aggregation can occur in a much richer way depending on how we wish to aggregate data (ie which type of statistics we desire), and over what variables we would like this aggregation to occur. For example, imagine instead of the mean wage we wished to measure mean wages *and* the standard deviation of wages over whether an individual graduated from college, and whether they are a member of a union. We can do so as follows, noting that statistics are indicated in parentheses along with other arguments, while the levels which we wish to group over are indicated with the `by()` option. Once we have generated this collapsed version of the data we will save the data file on disk using the `save` command for some processes below:

```
sysuse nlsw88.dta, clear
collapse (mean) mean_wage = wage (sd) sd_wage = wage, by(collgrad union)
list
save groupedNLSW.dta, replace
```

```
(NLSW, 1988 extract)

     +-------------------------------------------------+
     |         collgrad      union   mean_w~e    sd_wage |
     |-------------------------------------------------|
  1. | Not college grad   Nonunion    6.36869   3.351124 |
  2. | Not college grad      Union   7.823589   3.849597 |
  3. | Not college grad          .   7.946638   10.00492 |
```

```
  4. |     College grad   Nonunion   10.11737   5.052896 |
  5. |     College grad      Union   10.47342   4.275456 |
     |-------------------------------------------------|
  6. |     College grad          .   12.53987   12.33248 |
     +-------------------------------------------------+
file groupedNLSW.dta saved
```

Generically, the syntax of `collapse` is as follows,

```
collapse (stat) newname = varname (stat) newname = varname ..., by(groupvariables)
```

Where the use of `...` is to denote that we could add more stats to this collapse.

19.2.2 Merging/Joining Data

We will also, at times, wish to join two (or more) pieces of data together, for example "merging" data to match up observations by some particular column or columns. For example, it may be that we have a survey where each individuals surveyed belongs to a household, where all of the individual-level questions in the data are stored as an individual-level file (along with the individual's household ID), while the household level measures are stored in another file at the household level. If we wish to work with information on the individual *and* their household characteristics, we will need to "join", or "merge" the file using the household ID.

Such processes are conducted in Stata using the `merge` command. As a simple example of this, we could imagine that we would like to take the grouped-level statistics we generated previously and join them back into our original NLSW data.. What we want to do then, is for each individual who has a particular college and union status, join the information on the mean and standard deviation of the wage in this group to their row of data.

```
sysuse nlsw88.dta, clear
merge m:1 collgrad union using groupedNLSW
d
```

```
(NLSW, 1988 extract)
(label unionlbl already defined)
(label gradlbl already defined)

    Result                      Number of obs
    -----------------------------------------
    Not matched                             0
    Matched                             2,246  (_merge==3)
    -----------------------------------------
```

```
Contains data from /usr/local/stata19/ado/base/n/nlsw88.dta
 Observations:         2,246                  NLSW, 1988 extract
    Variables:            20                  1 May 2024 22:52
                                              (_dta has notes)
------------------------------------------------------------------------------
Variable      Storage   Display    Value
    name         type    format    label      Variable label
------------------------------------------------------------------------------
idcode          int     %8.0g                 NLS ID
age             byte    %8.0g                 Age in current year
race            byte    %8.0g      racelbl    Race
married         byte    %8.0g      marlbl     Married
never_married   byte    %16.0g     nev_mar    Never married
grade           byte    %8.0g                 Current grade completed
collgrad        byte    %16.0g     gradlbl    College graduate
south           byte    %9.0g      southlbl   Lives in the south
smsa            byte    %9.0g      smsalbl    Lives in SMSA
c_city          byte    %16.0g     ccitylbl   Lives in a central city
industry        byte    %23.0g     indlbl     Industry
occupation      byte    %22.0g     occlbl     Occupation
union           byte    %8.0g      unionlbl   Union worker
wage            float   %9.0g                 Hourly wage
hours           byte    %8.0g                 Usual hours worked
ttl_exp         float   %9.0g                 Total work experience (years)
tenure          float   %9.0g                 Job tenure (years)
mean_wage       float   %9.0g                 (mean) wage
sd_wage         float   %9.0g                 (sd) wage
_merge          byte    %23.0g     _merge     Matching result from merge
------------------------------------------------------------------------------
Sorted by:
     Note: Dataset has changed since last saved.
```

As we can see here, after completing the merge, we have our original NLSW data, but have additionally added the new columns `mean_wage`, `sd_wage` which are our generated variables, along with `_merge` which summarises the number of units which correctly merge (in this case, all of them). There are three types of merge which we may need to use. These are **m:1** (as above), **1:1**, or **1:m**. These are indicated immediately after invoking the `merge` command and one of these must be indicated. These values refer to the quantity of observations in the data in Stata's memory (in this case nlsw88.dta) and the quantity of observations in the data indicated after `using` (in this case groupedNLSW.dta), over which the merge should be conducted. When an **m** is indicated this implies that there are many observations in each level, and when a **1** is indicated it implies that there is a single observation in each level.

In the above example, given that in the data on the disk there is a single observation for each of the variables `collgrad` and `union`, this is indicated as **1**, while in the original data, there are multiple women in each group made up of `collgrad` and `union` (hence **m:1**). It is fundamental in merges to ensure that we confirm that either all observations are correctly merged to a partner in the using data, or if not, understand why this is not the case.

19.2.3 Filtering Data

For the remaining examples, we will clear all the information from Stata's memory to manually input a simplified data set to explore a number of tasks. We will do this using the `input` command, as follows:

```
clear all
input str2 Zone Year Sales
"A" 2021 100
"A" 2022 115
"B" 2021 98
"B" 2022 106
end
list
```

```
     +---------------------+
     | Zone   Year   Sales |
     |---------------------|
  1. |    A   2021     100 |
  2. |    A   2022     115 |
  3. |    B   2021      98 |
  4. |    B   2022     106 |
     +---------------------+
```

The `input` command is quite a unique command in that it tells Stata that we will manually write in variable values, and will be assumed to keep going until we type `end`. Here we have told Stata that we will input three variables manually, and these are named as `Zone`, `Year` and `Sales`. You may also note the addition of `str2` before the Zone variable name, which informs Stata that this variable Zone will be inputed as a string, as otherwise Stata assumes that variables will be input manually.

One way to filter variables is to use the `keep` and `drop` commands. These commands can either indicate variable names, in which case all indicated variables will either be kept or dropped, or can indicate conditions, in which case all observations complying with the conditions will be

kept or dropped. We will illustrate this below, additionally illustrating the use of `preserve` and `restore` which allow us to "store" a dataset in a particular moment, and then return to this dataset at a later moment.

As an example first we are going to delete the variable `Year` with the drop command and next restore to its previous status

```
preserve
drop Year
dis "Prior to restore"
list
restore
dis "After restore"
list
```

```
Prior to restore

     +--------------+
     | Zone   Sales |
     |--------------|
  1. |    A     100 |
  2. |    A     115 |
  3. |    B      98 |
  4. |    B     106 |
     +--------------+

After restore

    +---------------------+
    | Zone   Year   Sales |
    |---------------------|
 1. |    A   2021     100 |
 2. |    A   2022     115 |
 3. |    B   2021      98 |
 4. |    B   2022     106 |
    +---------------------+
```

As you can see the combination of `preserve` and `restore` allows to modify the dataset and go back, while `drop` remove what we specify, in the example was a variable but also can be observations in specific positions specifying *in* or depending on a condition specifying *if*. `keep` command works opposite to `drop`, as `drop` remove only what is specified, `keep` mantains only the specified and remove everything else. You may wish to confirm that this also works with particular conditions, such as `if Sales>100`.

19.2.4 Reshaping Data

Finally the last tool that we introduce for data manipulation is the idea of "reshaping" or transforming data from a wide format to a long format (and *vice versa*). We refer to data in a **long** format if for each observation we have multiple rows of data, covering different levels of some other variable. For example, a classic example of this would be a panel of data over countries and years. On the other hand, we refer to data as being in a **wide** format if for each observation we have multiple columns (or variables) which cover different levels of some other variable. In certain settings we will wish to work with data in one format, and in other settings we may find it more convenient to work with data in another format. Our simplified dataset above is precisely the example of a **long** format, given that for each Zone we have data over multiple years. In order to transform this dataset to a *wide* format the code is

```
reshape wide Sales, i(Zone) j(Year)
list
```

```
Data                               Long   ->   Wide
-----------------------------------------------------------------------------
Number of observations                4   ->   2
Number of variables                   3   ->   3
j variable (2 values)              Year   ->   (dropped)
xij variables:
Sales   ->   Sales2021 Sales2022
-----------------------------------------------------------------------------

   +---------------------------+
   | Zone   Sal~2021   Sal~2022 |
   |---------------------------|
1. |    A        100        115 |
2. |    B         98        106 |
   +---------------------------+
```

As you can see now the dataset has one observation for each zone and no repeated values for a variable. If we wish to return to the long format we can use the reverse "reshape long" function, where in this case we must provide a name for the variable which will record the levels over which A and B are observed. Namely:

```
reshape long Sales, i(Zone) j(Year)
```

In practice, we will generally use these commands with a considerably larger number of units, but the basic principles are the same regardless.

20 Basic Plots

To end this initial introduction, we will briefly explore the generation of basic plots in `Stata`. There are a variety of plots that you can create in Stata, and each plot has its own precise syntax. The range of plots which can describe relationships between two (or more) variables can be seen by consulting `help twoway` and I would encourage to scan the plot types, or explore online to see a graphical overview. In this subsection we are going to show you two basic plots: (i) the line plot and (ii) the scatter plot.

In Stata the command to do generate a line plot is, perhaps unsurprisingly, the `line` command. This function maps pairs of points in the (x, y) coordinate axis and places them on a graph. For example, let's consider a case where x is a sequence of integers between -5 and 5. This can be generated as below:

```
clear all
set obs 11
generate x = -6 + _n
```

```
Number of observations (_N) was 0, now 11.
```

In this code first we clear the Stata memory, then we set the observations in the dataset to 11, so that Stata understands that there is a dataset, empty for the moment, which has 11 observations (rows). Finally we create a variable named `x` which will take the value of -6 and the row number (`_n`) in the dataset. In this way, in row 1, where `_n=1`, x will equal -5, and will successively increase by one unit down rows:

```
list x in 1/4
```

```
     +----+
     |  x |
     |----|
  1. | -5 |
  2. | -4 |
  3. | -3 |
  4. | -2 |
     +----+
```

Next we create $y = x^2$ as follows:

```
generate y = x^2
```

Now with `line` function we create the line plot. `line`, by default generates a line plot of points, making reasonable decisions for axes and labelling:

```
*| fig-cap: "Line plot example"
twoway line y x
```

In the help documentation you can see a series of option that allows to modify the graph aspect.

Now we show you how to graph a scatter plot, for that we are going to simulate with `rnormal()` 100 pseudo-random numbers from a standard normal distribution in a variable `error`, then in a variable `x` we simulate another 100 pseudo-random numbers from a standard normal distribution and finally we create

$$y_i = 10 + 3 * x_i + error_i$$

```
*| fig-cap: "Scatter plot example"
clear all
set obs 100
gen error = rnormal()
gen x = rnormal()
gen y = 10 + 3*x + error
scatter y x
```

If you access help documentation for `scatter` you can see how add to elements to the graph and further edit the graphs appearance, for example by adding axis titles, adding lines, and so forth.

It is also relatively easy to include multiple graphs, laying these on a single graph window. Graphs can essentially be concatenated together, separating with "||" as follows, where we add a second graph, `lfitci` which fits a regression line along with 95% confidence intervals:

```
*| fig-cap: "Lfit Plot Example"
scatter y x || lfitci y x`
```

21 Writing Our Own Programs

21.1 An Introduction to Programs in Stata

User-written programs in Stata let you package steps into a single command. Programs can parse inputs, perform calculations, and **return results** (scalars, macros, or matrices) that you can reuse.

21.1.1 Creating a Simple Program

A minimal Stata program has a name and a body. By indicating a class of program (here `rclass`), elements are returned for post-program access in "return lists" accessible in `r()`. Below we write a simple program which adds to numbers:

```
cap program drop addnumbers
program define addnumbers, rclass
    // Parse two required numeric options a() and b()
    syntax , a(real) b(real)

    // Do the work using locals (local = temporary variable)
    local sum = `a' + `b'

    // Return a scalar so the result can be reused
    return scalar sum = `sum'

    // Also display for convenience
    dis as txt "sum = " as res %9.2f `sum'
end

// Call it:
addnumbers , a(10) b(5)

// Use the returned value:
return list
```

```
sum =     15.00

scalars:
                r(sum) =  15
```

21.1.2 Understanding Program Arguments

Stata typically uses **options** to pass inputs. Optional arguments can be handled by setting default values which will be applied unless they are overruled by users. Such optional arguments are indicated by using square brackets, as below:

```
cap program drop multiplynumbers
program define multiplynumbers, rclass
    // a() required, b() optional
    syntax , a(real) [b(real 2)]

    // conduct calculation of interest
    local product = `a'*`b'

    // return for post-estimation
    return scalar product = `product'

    di as txt "product = " as res %9.2f `product'
end

// With default b = 2
multiplynumbers , a(10)

// With explicit b
multiplynumbers , a(10) b(3)
```

```
product =     20.00
product =     30.00
```

21.1.3 Variable Scope and Return Values

It is important to understand what programs are doing, what occurs only inside programs, and what may affect that Stata session outside of a particular program. Relevant things to know in this regard are that locals (`local x = ...`) live only inside the program. Similary,

we can use temporary variables or other temporary objects to generate things locally within our programs which will no longer exist once the project finishes execution.

Of course as we see above, this does not mean that we cannot pass objects (even temporary objects) back to our session. We can use use **`return`** to expose results (for `rclass`), then access them with `r()`. If you need estimation-style results, write an `eclass` program and use `ereturn`. However note that if we make changes to data itself we hold in memory, these will be permanently made to the data, even once the program has finished. We can explore these ideas below.

```
clear all
set obs 10
gen x = 2
sum x

cap program drop demo_scope
program define demo_scope, rclass
    tempname hold
    scalar `hold' = 42     // temporary scalar; name auto-unique
    local msg "hello"

    // edit variable in data
    replace x = 3

    // Return both a scalar and a local as text
    return scalar answer = `hold'
    return local greeting "`msg'"
end

demo_scope
return list
di "greeting was: " r(greeting)
di "answer was: " %9.0f r(answer)

dis `hold'
dis "`msg'"
sum x
```

```
Number of observations (_N) was 0, now 10.

    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
           x |         10           2           0          2          2
```

```
(10 real changes made)

scalars:
             r(answer) =  42

macros:
           r(greeting) : "hello"
greeting was: hello
answer was:        42
```

```
    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
           x |         10           3           0          3          3
```

As a final tip: If your program changes the dataset and you want to restore the original state, we can wrap changes with `preserve` and `restore`, and this way revert to original data when programs finish running.

22 Data Wrangling and Data Management

22.1 More on "Data Wrangling"

We have briefly seen the idea behind merging, reshaping and other data preparation techniques, but will review further details here, before working with a more complete example where we will see such techniques in use in a way commonly found in research projects below.

22.1.1 Joining/Merging

Depending on the language used, the process of combining datasets in a row-wise fashion (ie matching within rows) is referred to as merging, or joining. In these procedures, it is fundamental for us to understand the identifiers which will be used to merge or join upon. These identifiers must allow us to uniquely identify observations, and be consistently used in both sets of data we wish to join. Identifiers may be based on a single variable (for example an individual's national identity number), or groups of variables (for example state names, county names and years). Prior to making any merge/join, it is important to have in mind what you expect from the output, and ensure that your expected identifiers do indeed match up across data sets. It is important when conducting merges/joins to spend time to ensure that you understand the structure of your databases, and, ideally, know the exact structure you expect from the merge so you can chase down any errors that may occur when conducting the merge.

Generically there are two types of considerations when dealing with joins/merges. The first is the type of join (inner, outer, left, or right), and the second is whether you will be merging each observation in one dataset to a single observation in another data set, or whether each observation may merge to multiple observations. We will discuss these in principle below, and then see a real example where we encounter a range of situations.

22.1.1.1 Inner, Outer, Left and Right Join

In languages like `Python` and `R`, when conducting a join we will need to indicate whether the join will be one of either: Inner, Outer, Left, or Right join. An inner join refers to the case where only observations which occur in both datasets will be kept. An outer join refers to the case where any observations, whether in both data sets, or in either of the datasets to

be joined, will be kept. In the language-specific syntax we will list a first file name on the left, and then a second file name on the right. Thus, of we indicate "left join", we are telling the language that it should keep all observations that are in the data listed first, whether or not they match with an observation in the right hand dataset. Similarly, if we indicate "right join", we are telling the language that it should keep all observations that are in the right hand data set, but *not* observations that are only in the left-hand data set. To see a very simple schematic example of this, refer to the four diagrams below, showing a case where the type of join will result in different outputs. Note that in each of these cases, the unique identifying key to join on is "ID".

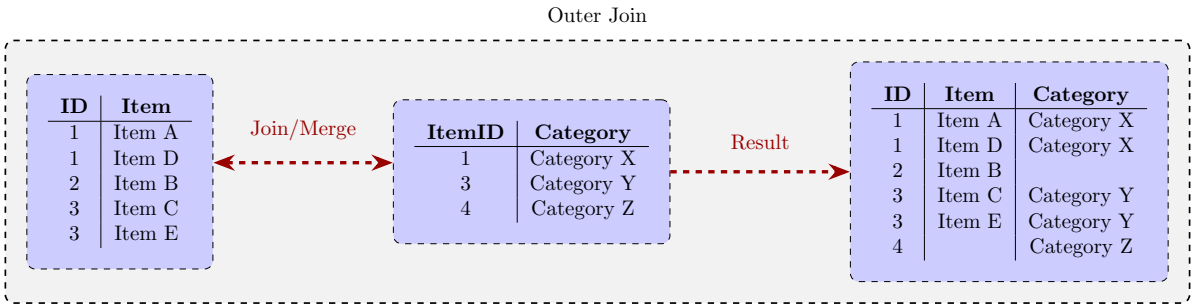


Figure 22.1: An Outer Join

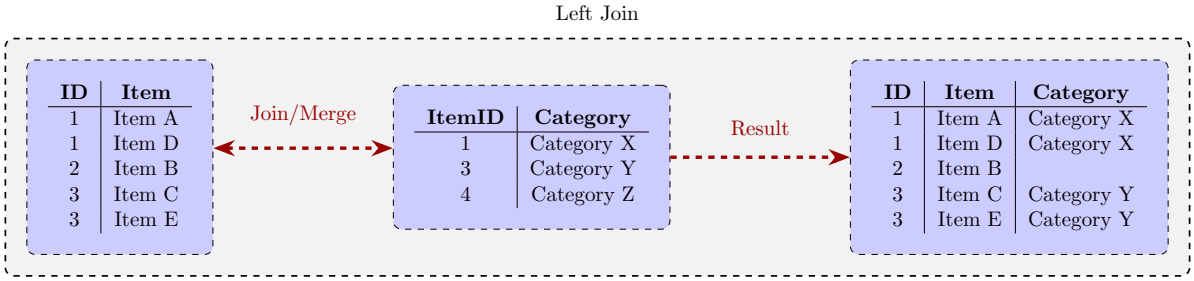


Figure 22.2: A Left Join

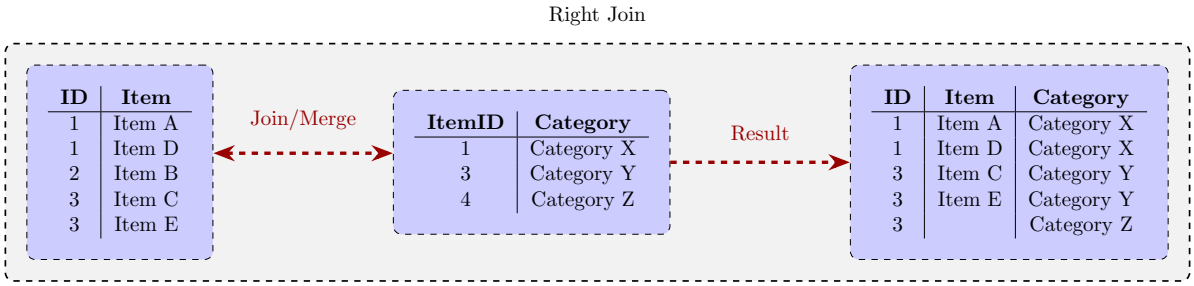


Figure 22.3: A Right Join

Unlike in `Python` or `R`, in `Stata` we will automatically perform an outer join. However, along

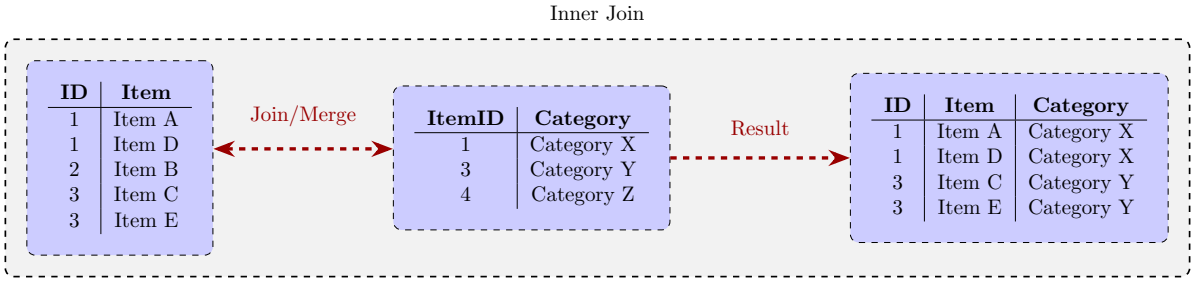


Figure 22.4: An Inner Join

with the merge, `Stata` will return to us a variable taking the value of 1 if the observation only exists in the left-hand ("master") dataset, a value of 2 if the observation only exists in the right-hand ("using") dataset, or a value of 3 if the observation exists in both datasets. Thus, if we keep observations with value of 1, 2 and 3, we are essentially conducting an outer join, if we keep observations with values of 3 we are conducting an inner join, if we keep observations with 1 and 3 we are conducting a left join, and if we keep observations with values of 2 and 3 we are conducting a right join.

22.1.1.2 1:1, m:1, or 1:m Merges

The second element we need to consider is the correspondence of observations across datasets. In `Stata`, we must explicitly indicate whether our merge will one of the types indicated below in the table. Note that this explanation is based on Stata's logic where a merge is conducted between one data set ("master") which is open in Stata, and another dataset ("using") which is housed on the disk.

Table 22.1: Types of Merge in Stata

Merge	Explanation
1:1	Each observation in the data set in memory matches to one observation on the disk
m:1	Multiple observations in the data set in memory match to each observation on the disk
1:m	Each observation in the data set in memory matches to multiple observations on the disk

If you return to the four schematic diagrams above (Figure 22.1 to Figure 22.4) you will note that this is a "m:1" merge, because in the left-hand data set there are potentially multiple lines with each ID, and these are joined to a single line in the right-hand side data.

Whether a merge will be 1:1, m:1, or 1:m depends entirely on the setting. For example, if we are matching two datasets with one pieces of information on the same individuals (for example their ages and their educational levels), we will be in a 1:1 situation. However, if we are matching the same observations with potentially multiple pieces of information (for exaple the ages of each of their children), we would be in a 1:m situation. If we mistakenly believe we are in a 1:1 merge but Stata finds multiple lines for each identifier, it will report an error message to us, and not allow us to continue.

Unlike in `Stata`, `R` and `Python` will automatically match each line with a given ID from one dataset to each line with the same ID in the other dataset. Thus, we will not need to explicitly indicate whether a join is m:1, 1:1, etc., but it is of course good to know this for our own clarity of thinking!

In theory there is one more type of merge which Stata permits, which is the m:m merge. However, an m:m merge should never be performed, because it is not replicable, and doesn't make sense. If we have multiple units with the same ID in one dataset, and multiple units with the same ID in the other dataset, Stata will simply join them based on the order the are sorted, which is likely arbitrary. The best explanation of this cames from the `Stata` manual itself (see Figure 22.5)!

m:m merges

> `m:m` specifies a many-to-many merge and is a bad idea. In an `m:m` merge, observations are matched within equal values of the key variable(s), with the first observation being matched to the first; the second, to the second; and so on. If the master and using have an unequal number of observations within the group, then the last observation of the shorter group is used repeatedly to match with subsequent observations of the longer group. Thus `m:m` merges are dependent on the current sort order—something which should never happen.
>
> Because `m:m` merges are such a bad idea, we are not going to show you an example. If you think that you need an `m:m` merge, then you probably need to work with your data so that you can use a `1:m` or `m:1` merge. Tips for this are given in *Troubleshooting m:m merges* below.

Figure 22.5: The Wisdom of the Stata Manual

Both `R` and `Python` have a different behaviour in this case. If we try to join datasets with multiple units with the same identified in each data file, the result will be a line for *every possible* join. A simple example of this can be seen below. If this is *actually* what we wish to do, we can do this in `Stata` with the `joinby` command. The "m:m" merge will not do this, and so should be avoided, as noted in the Stata manual above.

```
# Load the dplyr package
library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```r
# Create two sample data frames with duplicate IDs
data1 <- data.frame(ID = c(1, 2, 2, 3, 4, 4, 5),
                    Name = c("Alice", "Bob", "Carl", "David", "Eve", "Frank", "Grace"))
data2 <- data.frame(ID = c(2, 2, 3, 4, 5, 5, 6),
                    Age = c(25, 30, 35, 40, 45, 50, 55))

# Perform an inner join on the 'ID' column
result <- inner_join(data1, data2, by = "ID")

# Print the result
print(result)
```

```
  ID  Name Age
1  2   Bob  25
2  2   Bob  30
3  2  Carl  25
4  2  Carl  30
5  3 David  35
6  4   Eve  40
7  4 Frank  40
8  5 Grace  45
9  5 Grace  50
```

22.1.2 Appending

Unlike merging/joining, in the case where we simply wish to join two datasets in a column-wise fashion is the append. Appending is conceptually much simpler, as we simply will stack two (or more) datasets one on top of the other, and any columns with the same variables will be stacked, whereas any variables which exist in one and not the other will simply be replaced with missing values when they don't appear. A graphical representation of this process is provided below.

Given the relative simplicity of this task, commands to append are generally straightforward, consisting simply of "append", or "concat" or "bind" or similar, along with a list of the datasets

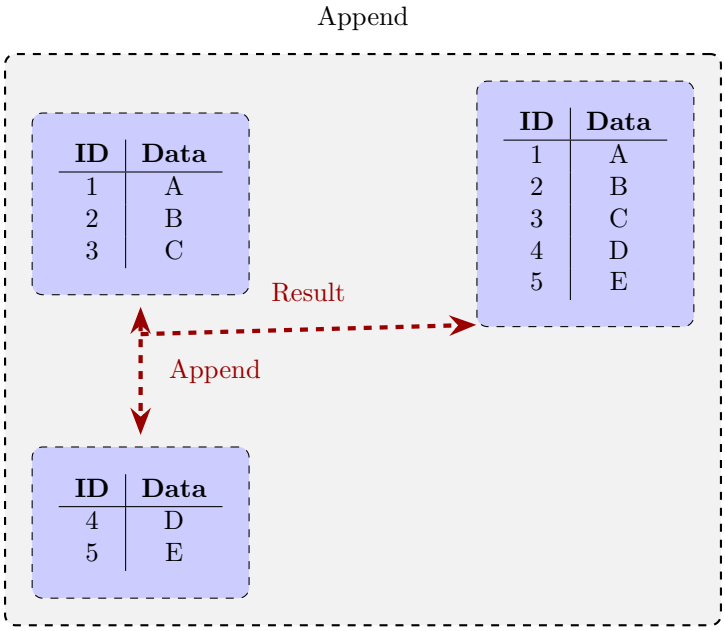


Figure 22.6: An Append

to stack. This is frequently used when we work with similar data over years and wish to join our data, as we will see in an applied example shortly.

22.1.3 Reshaping

Finally, in the interests of comprehensiveness, the figure below formalises the idea of reshaping data from wide format to long, or vice versa. Reshapes can work in either way, and whether we wish to work in long or wide format will depend entirely upon our processes of interest. Most cases where we wish to capture time dependence in data such as panel data methods will require data to be stored in long format (and organised by time). Again, in our applied example below we will see a real case where data is provided in wide format, and we will reshape to long format for use.

> **i** Checking your understanding
>
> To explore a merge/join and data manipulation in practice, we will work with an example from the Demographic and Health Surveys (DHS). The DHS is a widely used household survey with a number of modules with different unique keys and relationships among themselves. The DHS Program provide a complete description of how to conduct merges. To check your understanding of the above ideas in practice, I would suggest working with a single country (for example, you can find here the full data downloadable for the

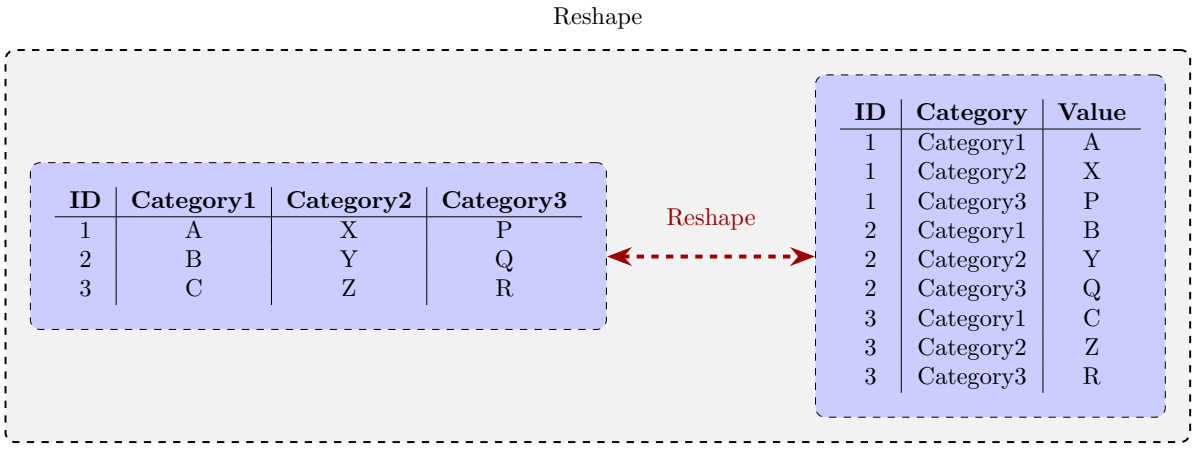


Figure 22.7: An Wide to Long Reshape

Tanzania, 2015-16 DHS). This is provided in Stata format, but if you are working in R or Python, can be read in directly using libraries we have discussed previously. If you would like to work with data from other countries, you can explore the DHS website here and make an account to download all the countries and waves that you like.
With this data, and the description of merging provided at the DHS website, explore various datasets and see if you can work out how to join them together, confirm that the data joins as you would expect, and so forth.

22.2 An Applied Example

While understanding these ideas in an abstract way is important, the best way to get comfortable with these processes, and see where they can be complicated, is by dealing with them in practice. Here we will work through an applied example to see the sorts of issues we should be aware of when setting up a research project. This applied example will use data at various levels to generate a data set by municipality and time. This data is, in a very rough way, following the application in Bentancor and Clarke (2017). In particular we will be interested in looking at rates of teenage pregnancy in Chile between the years of 2008 and 2009, and how they evolve in two types of municipalities: those where the emergency contraceptive (EC) pill became available in 2009, and those where the EC pill was not available.

While this is a simplified example, only working with two years of data, it illustrates all of the considerations which one has while working with real data collected from diverse sources. When we begin work on a "project" like this it is important for us to have clear what our goal is in terms of the final data structure, the final observation numbers we will expect, and the

variables we need in our "clean" data. Then we will consider the inputs we need to arrive to this target dataset, and the type of "data wrangling" operations we will need.

In particular in this setting we wish to generate a dataset of with a single line per municipality and year, where each line contains the number of pregnancies in this municipality, the population of adolescent women, and information on whether the municipality disbursed the EC pill. Theoretically, before we start we know that our dataset should look something like the following:

Table 22.2: Theoretical data set

Observation	Municipality	Year	TeenPreg	ECPill	Population
1	Algarrobo	2008	xx	xx	xxxx
2	Algarrobo	2009	xx	xx	xxxx
3	Alhué	2008	xx	xx	xxxx
4	Alhué	2009	xx	xx	xxxx
...	...	...	...	...	...
$(N_m \times 2) - 1$	Ñuñoa	2008	xx	xx	xxxx
$(N_m \times 2)$	Ñuñoa	2009	xx	xx	xxxx

While it does not matter exactly what the municipalities are called, what we can see here is that we want to generate a dataset where each municipality is listed two times: once for 2008 and once for 2009, and hence we know that there will be the number of municipalities (N_m) observations times 2 (one municipality for each year). It is very important for us to have this theoretical structure in mind, and this is something I would highly encourage you to do when you are conducting such analyses, as *before we start* we should know exactly what our observations are, and have an idea of the final dataset which we will be satisfied with. This way, if we eventually turn out to *not* have the observations that we think we should, we can go back and debug our code to find out where errors may have occurred. In this particular setting, if we look online, we could see that Chile has $N_m = 346$ municipalities, which tells us that eventually, when we have completed our work we should have $346 \times 2 = 692$ observations.

After having this precise data output in mind, we should work out what data sources we have (or can collect) which let us build this, as well as the structure of the data that we have available. In this particular case, we need to generate three variables:

- **Teenage Pregnancy**: This can be generated from publicly available microdata files on each birth occurring in the country where we observe the age of the mother at birth.
- **EC pill**: This can be generated from a survey conducted by legal scholars indicating whether the EC pill was available in each municipality in 2009.
- **Population**: The National Statistics Institute of the country provides an excel file where each municipality has a line for each age and gender with the population in each year from 2002-2020.

All of these datasets can be downloaded from the following link, and with this you can follow along with our example here.

We will explore this example in `Stata`. I would encourage you to ensure that you can "translate" this process into the language that you are working with. To start we will write a `Stata` do file clearing memory, indicating the version of Stata's commands which we are using (to ensure backwards compatibility), and setting "global variables" which list the directories we will be working in. Finally, we will open a log file which stores all the output of all our commands.

```
vers 12
clear all
set more off
cap log close

*------------------------------------------------------------------------------
*--- (1) Set any global/local variables
*------------------------------------------------------------------------------
global ROOT "/media/damian/DCC-0003/"
global DAT "$ROOT/trabajo/Teaching/Exeter/ResearchMethods-I/site"
global OUT "$ROOT/trabajo/Teaching/Exeter/ResearchMethods-I/site"

*------------------------------------------------------------------------------
*--- (2) Log output
*------------------------------------------------------------------------------
log using "$OUT/analisisTeenPreg.txt", text replace
```

```
(file
    /media/damian/DCC-0003//trabajo/Teaching/Exeter/ResearchMethods-I/site/an
    > alisisTeenPreg.txt not found)
file
    /media/damian/DCC-0003//trabajo/Teaching/Exeter/ResearchMethods-I/site/an
    > alisisTeenPreg.txt could not be opened
r(603);

end of do-file
r(603);
```

Above, the "global variables" we define in lines 9-10 can be referred to later in the code using a $ sign, followed by the name we have given them (ie $DAT or $OUT). Defining global variables at the top of the file like this is good practice, as if we share our code with others, they can just change these lines and nothing else.

We will now begin the work of setting up our data. When doing this, we begin with the population data, because this way we are sure to observe a single line for each municipality and year. If, on the other hand, we start with birth data, we may encounter difficulties if there are certain small municipalities where no births occur.

```
*-------------------------------------------------------------------------------
*--- (2) Create the population structure for the data
*---       Goal: 1 line for each municipalit in 08 and 20 with totals 15-19 F
*-------------------------------------------------------------------------------
import excel poblacionINE, firstrow clear
dis "What does 1 line of the raw population data look like?"
list in 1
//Keep data for female teenagers only
keep if Sexo==2 & edad>=15&edad<=19

//Keep relevant variables (population for 2008-2009 only)
keep Comuna nombre_comuna a2008 a2009 edad

//We must reshape so that we have a line for population in each year
reshape long a, i(Comuna nombre_comuna edad) j(ano_nac)

dis "What does our reshaped clean data look like?"
list in 1/5


//Now collapse to sum populations across each age group within a municipality*yr
collapse (sum) a, by(Comuna nombre_comuna ano_nac)
rename a population
rename Comuna comuna

dis "What does our collapsed data look like?"
count
list in 1/4
list in -2/l
dis ":-)"

//We save this as a "tempfile" for use below
tempfile population
save `population'
```

```
(27 vars, 56,052 obs)
```

What does 1 line of the raw population data look like?

```
     +-------------------------------------------------------------------------+
  1. | Region  | nombre_re~n  | provin~a  | Nombre~a  | Comuna  |  nombre~a  |
     |      1  | I  Tarapacá  |       11  |  Iquique  |  1,101  |   Iquique  |
     |-------------------------------------------------------------------------|
     | Sexo1h~s | edad | a2002  | a2003  | a2004  | a2005  | a2006  | a2007  |
     |        1 |    0 | 1,468  | 1,422  | 1,427  | 1,455  | 1,475  | 1,484  |
     |-------------------------------------------------------------------------|
     | a2008 | a2009 | a2010 | a2011 | a2012 | a2013 | a2014 | a2015 | a2016 |
     | 1,524 | 1,594 | 1,595 | 1,566 | 1,561 | 1,545 | 1,538 | 1,529 | 1,511 |
     |-------------------------------------------------------------------------|
     |      a2017      |      a2018      |      a2019      |      a2020      |
     |      1,476      |      1,442      |      1,405      |      1,367      |
     +-------------------------------------------------------------------------+
```

(54,322 observations deleted)

(j = 2008 2009)

```
Data                                Wide   ->   Long
-----------------------------------------------------------------------------
Number of observations             1,730   ->   3,460
Number of variables                    5   ->   5
j variable (2 values)                      ->   ano_nac
xij variables:
                             a2008 a2009   ->   a
-----------------------------------------------------------------------------
```

What does our reshaped clean data look like?

```
     +-------------------------------------------+
     | Comuna   nombre~a   edad   ano_nac       a |
     |-------------------------------------------|
  1. |  1,101    Iquique     15      2008   1,550 |
  2. |  1,101    Iquique     15      2009   1,540 |
  3. |  1,101    Iquique     16      2008   1,492 |
  4. |  1,101    Iquique     16      2009   1,477 |
  5. |  1,101    Iquique     17      2008   1,497 |
     +-------------------------------------------+
```

```
What does our collapsed data look like?

  692

     +-----------------------------------------------+
     | comuna   nombre_comuna   ano_nac   popula~n |
     |-----------------------------------------------|
  1. |  1,101         Iquique      2008      7,657 |
  2. |  1,101         Iquique      2009      7,631 |
  3. |  1,107   Alto Hospicio      2008      3,155 |
  4. |  1,107   Alto Hospicio      2009      3,321 |
     +-----------------------------------------------+

     +-----------------------------------------------+
     | comuna   nombre_comuna   ano_nac   popula~n |
     |-----------------------------------------------|
691. | 15,202   General Lagos      2008         19 |
692. | 15,202   General Lagos      2009         18 |
     +-----------------------------------------------+

:-)


file /tmp/St430580.000001 saved as .dta format
```

We have done quite a lot here, and it is worth spending time looking into each line carefully. Processes such as reshaping and collapsing early in projects are key, and it is important to understand that each line does what we think it should. We will work through this line by line in class! A key thing to note is that here we have started with data that had the informatin we wanted (population), but not in the format we wanted, and by doing some data wrangling, have perfectly converted it into a format which is the format we need (692 lines covering 346 municipalities over 2 years).

Now, we will work with our birth micro-data, to generate a single line for each municipality and year:

```
*-------------------------------------------------------------------------------
*--- (2a) Import births data and collapse to the municipal level
*-------------------------------------------------------------------------------
// Read in 2008 microdata of births
import delimited NAC2008.csv, delim(",")
```

```
count

// Save as dta to append to 2009 (could also use a tempfile)
lab dat "All births 2008 from DEIS"
save NAC2008.dta, replace

// Read in 2009 microdata of births
import delimited NAC2009.csv, delim(",") clear
count
// Append 2008 data so we now have all births
append using NAC2008

// Generate an indicator if a birth occurs to a teen mother
gen teenPreg = 1 if edad_m>=15 & edad_m<=19

// Collapse by municipality and year to generate counts
collapse (sum) teenPreg, by(comuna ano_nac)
sum teenPreg
dis "What has happened here?  Should we be worried?"


list in 1/4
list in -2/l
```

```
(encoding automatically selected: ISO-8859-1)
(31 vars, 246,581 obs)

  246,581


file NAC2008.dta saved

(encoding automatically selected: ISO-8859-1)
(31 vars, 252,240 obs)

  252,240


(419,292 missing values generated)

    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
```

```
    teenPreg |        688    115.5945    178.1329          0       1509

What has happened here?  Should we be worried?

     +-----------------------------+
     | ano_nac   comuna   teenPreg |
     |-----------------------------|
  1. |    2008     1101        406 |
  2. |    2009     1101        459 |
  3. |    2008     1107        345 |
  4. |    2009     1107        373 |
     +-----------------------------+

     +-----------------------------+
     | ano_nac   comuna   teenPreg |
     |-----------------------------|
687. |    2008    15202          0 |
688. |    2009    15202          0 |
     +-----------------------------+
```

Again, using a few data wrangling processes, we now have data in the format we wish. Again, we will talk through line by line in class to ensure that we are happy with how we convert our 500,000 or so births into a data set with a line for each municipality and year. It is also important for us to think why we *do not* have the full 692 lines we may have expected.

While this above process is perfectly appropriate, at times we may prefer not to store large amounts of data in memory. Below we conduct exactly the same process, but do it in a slightly more memory-efficient way. While concerns about memory should come *after* concerns about code being correct, in data-heavy tasks it is worth thinking if we can break up our work into pieces which let us read less data into memory at a single time.

```
*-------------------------------------------------------------------------------
*--- (2b) Import births data and collapse to the municip level (more efficient)
*-------------------------------------------------------------------------------
use NAC2008.dta, clear
gen teenPreg = 1 if edad_m>=15 & edad_m<=19
collapse (sum) teenPreg, by(comuna ano_nac)

lab dat "Births to mothers aged 15-19, 2008 (from DEIS microdata)"
tempfile teenPreg2008
save `teenPreg2008'

import delimited NAC2009.csv, delim(",") clear
```

```
gen teenPreg = 1 if edad_m>=15 & edad_m<=19
collapse (sum) teenPreg, by(comuna ano_nac)

append using `teenPreg2008'
sum teenPreg

lab data "Births to Mothers Aged 2008-2009 (from DEIS microdata)"
save embarazoAdolescente_2008_2009.dta, replace
```

```
(All births 2008 from DEIS)

(206,679 missing values generated)




file /tmp/St430685.000001 saved as .dta format

(encoding automatically selected: ISO-8859-1)
(31 vars, 252,240 obs)

(212,613 missing values generated)

    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
    teenPreg |        688    115.5945    178.1329          0       1509


file embarazoAdolescente_2008_2009.dta saved
```

Now, finally, we can do our first merge. Below we will use the `comuna` variable which is a unique municipality code in Chile. We know that each of our datasets has at most a single line per comuna and year, and so in this way we can confidently use a 1:1 merge, matching on `comuna` and `ano_nac` (year).

```
*-------------------------------------------------------------------------------
*--- (3) Merge in population data
*-------------------------------------------------------------------------------
merge 1:1 comuna ano_nac using `population'
```

```
    Result                      Number of obs
```

```
    -----------------------------------------
    Not matched                            20
        from master                         8  (_merge==1)
        from using                         12  (_merge==2)

    Matched                               680  (_merge==3)
    -----------------------------------------
```

	ano_nac	comuna	teenPreg	nombre~a	popula~n
90.	2008	5106	232		.
91.	2009	5106	230		.
94.	2008	5108	230		.
95.	2009	5108	204		.
126.	2008	5505	112		.
127.	2009	5505	111		.
130.	2008	5507	40		.
131.	2009	5507	43		.

	ano_nac	comuna	teenPreg	nombre_comuna	popula~n
689.	2009	2202	.	Ollagüe	6
690.	2008	5801	.	Quilpué	5,986
691.	2009	5801	.	Quilpué	5,961
692.	2008	5802	.	Limache	1,719
693.	2009	5802	.	Limache	1,699
694.	2008	5803	.	Olmué	616
695.	2009	5803	.	Olmué	607
696.	2008	5804	.	Villa Alemana	4,815
697.	2009	5804	.	Villa Alemana	4,841
698.	2008	12103	.	Río Verde	1
699.	2009	12103	.	Río Verde	1
700.	2008	12303	.	Timaukel	0

Unfortunately, our first merge has not worked entirely as we'd hoped. Specifically, there were 20 observations which *did not* merge. 8 of these were from our master database (births) and

12 of these were from our using database (populations). If we look into this data carefully, we can see that in the "using" data 4 of these observations are municipalities with very small populations (0, 1 and 6 teens), where it is conceivable that no teen births occurred. However, 8 of the observations are larger municipalities, and are missing in both years. It turns out that if we read online, we will see that Chile has changed its municipality codes a number of times, and these particular municipality codes in the birth data are the old codes, with the new codes listed in the population data. Such processes occur with some frequency in administrative datasets, pointing to the importance of understanding the contexts we are working in. The correspondence between old and new municipality codes is below.

Table 22.3: Changed municipality codes

Municipality	Old Code	New Code
Quilpué	5106	5801
Villa Alemana	5108	5804
Limache	5505	5802
Olmué	5507	5803

Now we can update these codes correctly in our data, and confirm that the merge occurs correctly:

```
*-------------------------------------------------------------------------------
*--- (2b) Import births data and collapse to the municip level (more efficient)
*-------------------------------------------------------------------------------
use NAC2008.dta, clear
gen teenPreg = 1 if edad_m>=15 & edad_m<=19
collapse (sum) teenPreg, by(comuna ano_nac)
tempfile teenPreg2008
save `teenPreg2008'

import delimited NAC2009.csv, delim(",") clear
gen teenPreg = 1 if edad_m>=15 & edad_m<=19
collapse (sum) teenPreg, by(comuna ano_nac)
append using `teenPreg2008'

replace comuna=5801 if comuna==5106
replace comuna=5802 if comuna==5505
replace comuna=5803 if comuna==5507
replace comuna=5804 if comuna==5108

lab data "Births to Mothers Aged 2008-2009 (from DEIS microdata)"
save embarazoAdolescente_2008_2009.dta, replace
```

```
*-------------------------------------------------------------------------------
*--- (3) Merge in population data
*-------------------------------------------------------------------------------
merge 1:1 comuna ano_nac using `population'
list ano_nac comuna teenPreg nombre_comuna population if _merge==1
drop _merge

lab data "Births to Mothers Aged 2008-2009 and Teen Populations"
```

```
(All births 2008 from DEIS)

(206,679 missing values generated)



file /tmp/St430771.000002 saved as .dta format

(encoding automatically selected: ISO-8859-1)
(31 vars, 252,240 obs)

(212,613 missing values generated)



(2 real changes made)

(2 real changes made)

(2 real changes made)

(2 real changes made)


file embarazoAdolescente_2008_2009.dta saved

    Result                      Number of obs
    -----------------------------------------
    Not matched                             4
        from master                         0  (_merge==1)
        from using                          4  (_merge==2)
```

```
    Matched                                688  (_merge==3)
    ---------------------------------------
```

```
file pregnancyPopulation.dta saved
```

This time we see that we have correctly conducted our merge, and effectively want to keep the observations where _merge==3 (municipalities with population and birth data), and those where _merge==2 (municipalities without birth data, likely because no births occurred this year). Later we must ensure that we replace these observations as having 0 teen births.

Finally, we will merge in data on the EC pill. In this case we are now in the presence of a m:1 merge. This is because we have two lines for each municipality (2008 and 2009), and our EC pill data has only one line (whether the municipality eventually provided the medication in 2009).

```
*------------------------------------------------------------------------------
*--- (4) Merge in information on the EC pill with information on teen pregnancy
*------------------------------------------------------------------------------
merge m:1 comuna using PAE

list if _merge==1
list if _merge==2
```

```
    Result                      Number of obs
    ---------------------------------------
    Not matched                            16
        from master                        12  (_merge==1)
        from using                          4  (_merge==2)

    Matched                               680  (_merge==3)
    ---------------------------------------
```

```
     +-------------------------------------------------------------------+
149. | ano_nac | comuna | teenPreg |           nombre_comuna | popula~n |
     |    2009 |   5801 |      230 |                 Quilpué |    5,961 |
     |-------------------------------------------------------------------|
     | comuna~e | year | pill | codcom18  | comuna~l  |          _merge  |
     |          |    . |    . |        .  |           | Master only (1)  |
     +-------------------------------------------------------------------+
```

```
     +-------------------------------------------------------------------+
150. | ano_nac | comuna | teenPreg |           nombre_comuna | popula~n |
     |    2008 |   5801 |      232 |                 Quilpué |    5,986 |
     |-------------------------------------------------------------------|
     | comuna~e | year | pill | codcom18  | comuna~l  |          _merge  |
     |          |    . |    . |        .  |           | Master only (1)  |
     +-------------------------------------------------------------------+

     +-------------------------------------------------------------------+
151. | ano_nac | comuna | teenPreg |           nombre_comuna | popula~n |
     |    2009 |   5802 |      111 |                 Limache |    1,699 |
     |-------------------------------------------------------------------|
     | comuna~e | year | pill | codcom18  | comuna~l  |          _merge  |
     |          |    . |    . |        .  |           | Master only (1)  |
     +-------------------------------------------------------------------+

     +-------------------------------------------------------------------+
152. | ano_nac | comuna | teenPreg |           nombre_comuna | popula~n |
     |    2008 |   5802 |      112 |                 Limache |    1,719 |
     |-------------------------------------------------------------------|
     | comuna~e | year | pill | codcom18  | comuna~l  |          _merge  |
     |          |    . |    . |        .  |           | Master only (1)  |
     +-------------------------------------------------------------------+

     +-------------------------------------------------------------------+
153. | ano_nac | comuna | teenPreg |           nombre_comuna | popula~n |
     |    2009 |   5803 |       43 |                   Olmué |      607 |
     |-------------------------------------------------------------------|
     | comuna~e | year | pill | codcom18  | comuna~l  |          _merge  |
     |          |    . |    . |        .  |           | Master only (1)  |
     +-------------------------------------------------------------------+

     +-------------------------------------------------------------------+
154. | ano_nac | comuna | teenPreg |           nombre_comuna | popula~n |
     |    2008 |   5803 |       40 |                   Olmué |      616 |
     |-------------------------------------------------------------------|
     | comuna~e | year | pill | codcom18  | comuna~l  |          _merge  |
     |          |    . |    . |        .  |           | Master only (1)  |
     +-------------------------------------------------------------------+

     +-------------------------------------------------------------------+
155. | ano_nac | comuna | teenPreg |           nombre_comuna | popula~n |
```

```
|    2008 |   5804 |      230 |            Villa Alemana |    4,815 |
|-------------------------------------------------------------------|
| comuna~e | year | pill | codcom18  | comuna~l  |          _merge  |
|          |    . |    . |        .  |           | Master only (1)  |
+-------------------------------------------------------------------+

+-------------------------------------------------------------------+
156. | ano_nac | comuna | teenPreg |            nombre_comuna | popula~n |
|    2009 |   5804 |      204 |            Villa Alemana |    4,841 |
|-------------------------------------------------------------------|
| comuna~e | year | pill | codcom18  | comuna~l  |          _merge  |
|          |    . |    . |        .  |           | Master only (1)  |
+-------------------------------------------------------------------+

+-------------------------------------------------------------------+
545. | ano_nac | comuna | teenPreg |            nombre_comuna | popula~n |
|    2009 |  12202 |        0 | Antártica                |        0 |
|-------------------------------------------------------------------|
| comuna~e | year | pill | codcom18  | comuna~l  |          _merge  |
|          |    . |    . |        .  |           | Master only (1)  |
+-------------------------------------------------------------------+

+-------------------------------------------------------------------+
546. | ano_nac | comuna | teenPreg |            nombre_comuna | popula~n |
|    2008 |  12202 |        3 | Antártica                |        0 |
|-------------------------------------------------------------------|
| comuna~e | year | pill | codcom18  | comuna~l  |          _merge  |
|          |    . |    . |        .  |           | Master only (1)  |
+-------------------------------------------------------------------+

+-------------------------------------------------------------------+
567. | ano_nac | comuna | teenPreg |            nombre_comuna | popula~n |
|    2009 |  13106 |      236 |         Estación Central |    5,090 |
|-------------------------------------------------------------------|
| comuna~e | year | pill | codcom18  | comuna~l  |          _merge  |
|          |    . |    . |        .  |           | Master only (1)  |
+-------------------------------------------------------------------+

+-------------------------------------------------------------------+
568. | ano_nac | comuna | teenPreg |            nombre_comuna | popula~n |
|    2008 |  13106 |      279 |         Estación Central |    5,191 |
|-------------------------------------------------------------------|
| comuna~e | year | pill | codcom18  | comuna~l  |          _merge  |
```

```
     |          |    . |    . |        .  |           | Master only (1)  |
     +-------------------------------------------------------------------+

     +-------------------------------------------------------------------+
693. | ano_nac | comuna | teenPreg | nombre~a | popula~n |    comunaname |
     |       . |   5106 |        . |          |        . |      QuilpuÃ© |
     |-------------------------------------------------------------------|
     |  year  |  pill  |  codcom18  |  comuna_minsal  |          _merge  |
     |  2009  |     0  |      5801  |        Quilpué  |  Using only (2)  |
     +-------------------------------------------------------------------+

     +-------------------------------------------------------------------+
694. | ano_nac | comuna | teenPreg | nombre~a | popula~n |    comunaname |
     |       . |   5108 |        . |          |        . | Villa Alemana |
     |-------------------------------------------------------------------|
     |  year  |  pill  |  codcom18  |  comuna_minsal  |          _merge  |
     |  2009  |     0  |      5804  |  Villa Alemana  |  Using only (2)  |
     +-------------------------------------------------------------------+

     +-------------------------------------------------------------------+
695. | ano_nac | comuna | teenPreg | nombre~a | popula~n |    comunaname |
     |       . |   5505 |        . |          |        . |       Limache |
     |-------------------------------------------------------------------|
     |  year  |  pill  |  codcom18  |  comuna_minsal  |          _merge  |
     |  2009  |     0  |      5802  |        Limache  |  Using only (2)  |
     +-------------------------------------------------------------------+

     +-------------------------------------------------------------------+
696. | ano_nac | comuna | teenPreg | nombre~a | popula~n |    comunaname |
     |       . |   5507 |        . |          |        . |        OlmuÃ© |
     |-------------------------------------------------------------------|
     |  year  |  pill  |  codcom18  |  comuna_minsal  |          _merge  |
     |  2009  |     0  |      5803  |          Olmué  |  Using only (2)  |
     +-------------------------------------------------------------------+
```

Once again here we have seen that our merge has not worked *exactly* like we thought, but we now have an idea why this is the case. Inspecting our problematic observations which did not merge, we once again see that we have encountered the changing municipality numbers which is now simple to correct.

```
*-------------------------------------------------------------------------------
*--- (4) Merge in information on the EC pill with information on teen pregnancy
```

```
*-------------------------------------------------------------------------------
use PAE
replace comuna=5801 if comuna==5106
replace comuna=5802 if comuna==5505
replace comuna=5803 if comuna==5507
replace comuna=5804 if comuna==5108


merge 1:m comuna using pregnancyPopulation

list if _merge==2
drop _merge
save pillPregPop.dta, replace
```

```
(Datos de la PAE en Chile, a partir de encuestas de Dides et al.)

(1 real change made)

(1 real change made)

(1 real change made)

(1 real change made)

    Result                      Number of obs
    -----------------------------------------
    Not matched                             4
        from master                         0  (_merge==1)
        from using                          4  (_merge==2)

    Matched                               688  (_merge==3)
    -----------------------------------------

     +-------------------------------------------------------------------+
617. | comuna~e | year | pill | codcom18 | comuna~l | comuna | ano_nac |
     |          |    . |    . |        . |          |  12202 |    2009 |
     |----------+---------------------------------------------------------|
     | teenPreg |            nombre_comuna | popula~n |         _merge |
     |        0 | Antártica                |        0 | Using only (2) |
     +-------------------------------------------------------------------+

     +-------------------------------------------------------------------+
```

```
618. | comuna~e | year | pill | codcom18 | comuna~l | comuna | ano_nac |
     |          |    . |    . |        . |          |  12202 |    2008 |
     |----------+-------------------------------------------------------|
     | teenPreg |            nombre_comuna | popula~n |         _merge |
     |        3 | Antártica                |        0 | Using only (2) |
     +-----------------------------------------------------------------+

     +-----------------------------------------------------------------+
629. | comuna~e | year | pill | codcom18 | comuna~l | comuna | ano_nac |
     |          |    . |    . |        . |          |  13106 |    2008 |
     |----------+-------------------------------------------------------|
     | teenPreg |            nombre_comuna | popula~n |         _merge |
     |      279 |         Estación Central |    5,191 | Using only (2) |
     +-----------------------------------------------------------------+

     +-----------------------------------------------------------------+
630. | comuna~e | year | pill | codcom18 | comuna~l | comuna | ano_nac |
     |          |    . |    . |        . |          |  13106 |    2009 |
     |----------+-------------------------------------------------------|
     | teenPreg |            nombre_comuna | popula~n |         _merge |
     |      236 |         Estación Central |    5,090 | Using only (2) |
     +-----------------------------------------------------------------+


file pillPregPop.dta saved
```

We can now see that our merge has occurred correctly. In practice, there are a few lines with missing data in the registers of the availability of the EC pill (Antarctica is one of them), and so this may point to the need of some follow up work to track down information, or simply the necessity of working with a sub-set of observations. Finally, we will conduct preliminary inspections and tests of our data to see if things make sense. In practice, we would want to be sure to inspect all output at considerably more length before we embark on the empirical steps of our research project!

```
*-------------------------------------------------------------------------------
*--- (5) Final cleaning
*-------------------------------------------------------------------------------
//We now know that areas with missing teen pregnancies are 0s
sum
replace teenPreg = 0 if teenPreg==.

sort nombre_comuna ano_nac
```

```
list ano_nac nombre_comuna teenPreg population in 1/4
list ano_nac nombre_comuna teenPreg population in -2/l
```

(Datos de la PAE en Chile, a partir de encuestas de Dides et al.)

```
    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
  comunaname |          0
        year |        688        2009           0       2009       2009
        pill |        496    .2741935    .4465574          0          1
    codcom18 |        688    9013.956    3816.284       1101      16305
comuna_min~l |          0
-------------+---------------------------------------------------------
      comuna |        692    8563.156    3354.111       1101      15202
     ano_nac |        692      2008.5    .5003617       2008       2009
    teenPreg |        688    115.5945    178.1329          0       1509
nombre_com~a |          0
  population |        692     2084.88    3267.602          0      24906

(4 real changes made)

     +-----------------------------------------+
     | ano_nac   nombre_~a   teenPreg   popula~n |
     |-----------------------------------------|
  1. |    2008   Algarrobo         27        335 |
  2. |    2009   Algarrobo         19        332 |
  3. |    2008       Alhué         23        209 |
  4. |    2009       Alhué         21        208 |
     +-----------------------------------------+

     +-----------------------------------------+
     | ano_nac   nombre~a   teenPreg   popula~n |
     |-----------------------------------------|
691. |    2008      Ñuñoa        130      6,591 |
692. |    2009      Ñuñoa        111      6,544 |
     +-----------------------------------------+
```

```
*-------------------------------------------------------------------------------
*--- (6) Data inspection
*-------------------------------------------------------------------------------
```

```
twoway scatter teenPreg population, ytitle("Pregnancies") xtitle("Population")
graph export pregnancyPop.png, replace


gen pregRate = teenPreg/population*1000
```

(Datos de la PAE en Chile, a partir de encuestas de Dides et al.)

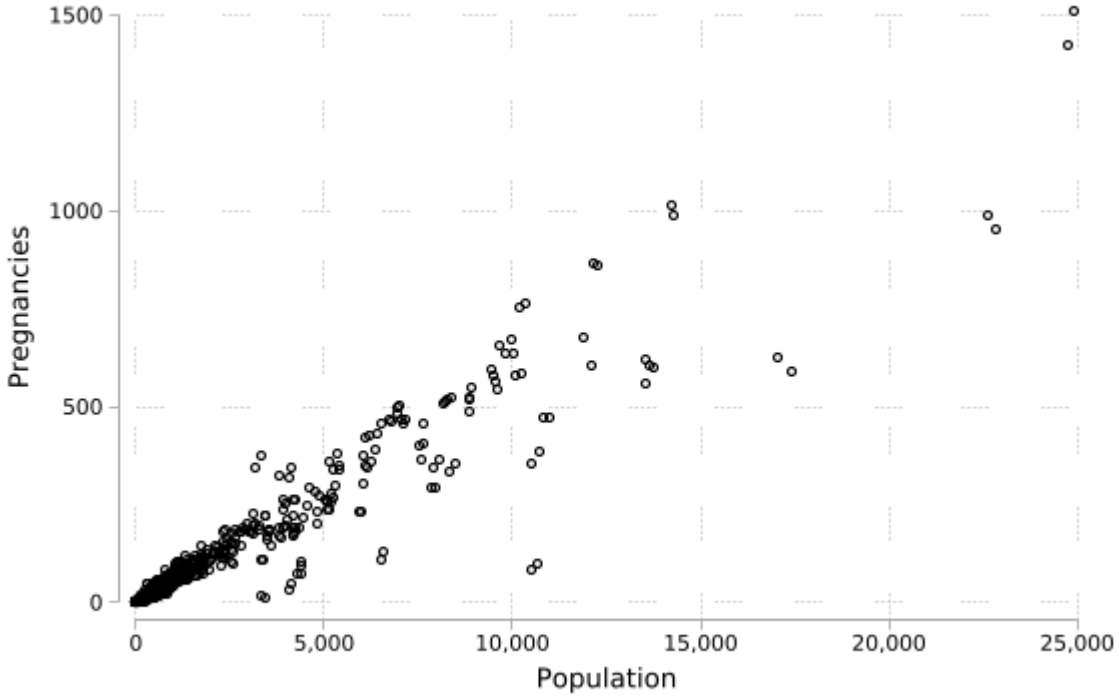
file pregnancyPop.png written in PNG format

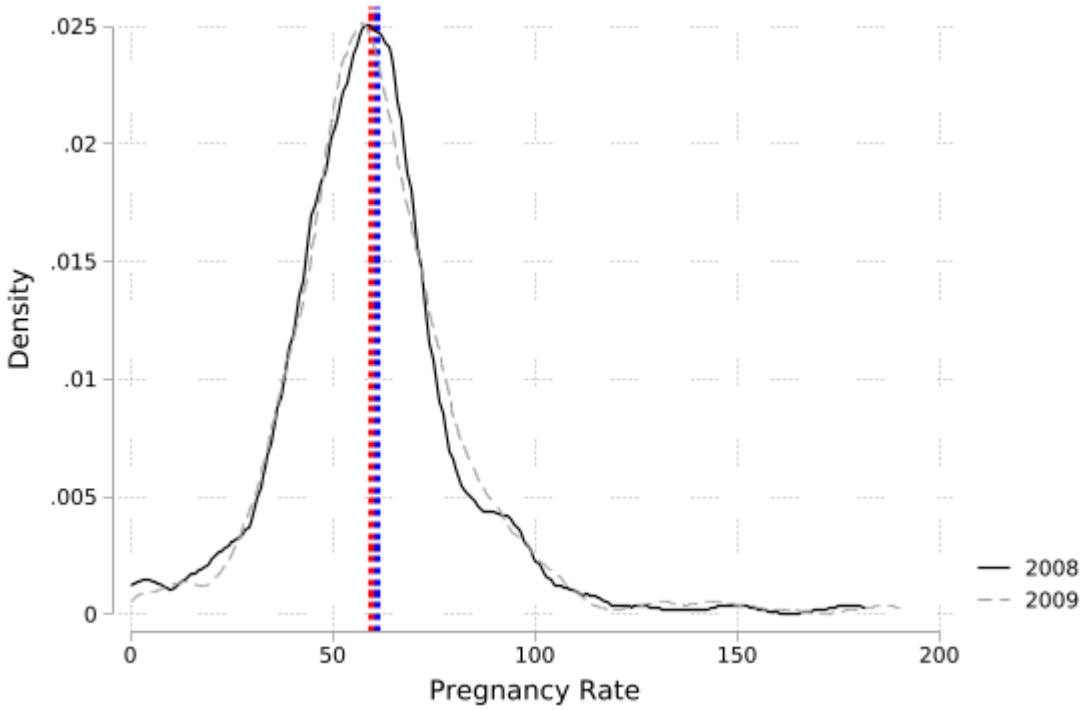
(4 missing values generated)

Variable	Obs	Mean	Std. dev.	Min	Max
pregRate	336	59.40411	22.72261	0	181.8182

Variable	Obs	Mean	Std. dev.	Min	Max
pregRate	335	61.16677	23.29519	0	190.4762

file pregRates.png written in PNG format


1500
1000
500
0
Pregnancies
0
5,000
10,000
15,000
20,000
25,000
Population


.025
.02
.015
.01
.005
0
Density
0
50
100
150
200
Pregnancy Rate
2008
2009

23 Project Management & Replicability

In economic research, it is common for our projects to last a number of years, include a number of quite separate analyses, and potentially include multiple people working on code and writing. For this reason, it is important to us that from the outset we think quite carefully about how we should best organise our work to ensure that we work as efficiently as possible, and to facilitate work and replication of results within teams. A well designed project and work flow will also make it considerably easier for other researchers to replicate and extend our work.

In this class we will talk about good coding and project management practices in an applied way. Often I will simply provide an overview of key considerations, before exploring together the structure of a project (in `R`). While these notes will provide an overview of many of the considerations, we could spend considerably more time on this theme, and will return to these issues throughout the rest of this course. There are a range of other resources here which are more complete in their coverages of these themes, and I direct you to them if you would like to read further. These include:

- Turrell (2023): **"Coding for Economists"** The sections on coding and workflow are particularly relevant for this class.
- Gentzkow and Shapiro (2014): **"Code and Data for the Social Sciences: A Practitioner's Guide"** These notes provide a very nice overview of the key elements in empirical projects, particularly in thinking about working within teams.
- Ristovska (2019): **"Coding for Economists: A Language-Agnostic Guide to Programming for Economists"** These slides focus in particular on coding style and best practices.

23.1 Workflow, Project Structure and Documentation

23.1.1 Workflow

A (stylised) economics research project may consist of the following steps:

1. Data collection
2. Data cleaning

3. Analysis and generation of results
4. Writing a paper and slides
5. Generation of replication materials

It is unlikely that one could (or would) want to use a single tool to complete all of these tasks. A somewhat complicated project may consist of the following type of tools for each step.

1. Data collection: Webscraped with Python
2. Data cleaning: Some scripting language, eg Stata, R, Julia, Python, MATLAB
3. Analysis and generation of results: Scripting language, as above
4. Writing a paper and slides: LaTeX and pdflatex compiler
5. Generation of replication materials: All housed in a Makefile, or very clearly documented in your README file.

You may note that the above steps *could* largely be done in a single language, but this list is used to provide the point that often in practice this is not what turns out to happen. Given this, when we are planning our projects, our solutions should be able to jump across languages if necessary.

Likely what we would like to do here is:

1. Automate this process completely or as much as possible
    - What happens if next month we want to update at step 1, eg grabbing new data?
    - And what if later a referee wants us to use an alternative method of data cleaning at step 2?
2. "Remember" what was being done in previous versions of code when changes are made to revert to old results if necessary

The first of these is more about designing an efficient replication process (for example using a Makefile or overview file) which we will discuss here, while the second is more about "version control", which we will discuss below.

As much as possible, we should seek to avoid any manual procedures in our workflow. This includes things like manually copying regression outputs to tables (we will discuss how to automate in a few weeks), manually updating figures in documents, and also the need to remember which order to run scripts in to generate final output. It is good practice to have some way to run a single script which does everything we need from start to finish.

One particular way to do this is by using **Make** a Unix tool which specifies the full path a project follows, implying that rather than having to go through a workflow piece by piece, one

can simply run the make file with a single command. The value added of make is that it only runs steps which are necessary If nothing prior to step

has been altered, make will see this and only run steps x, $x+1$, ... A very complete discussion of Make with examples is provided by Jesús Fernández-Villaverde. We won't discuss this extensively here because it is specific to a single operating system, but you may wish to check it out. Similarly, for windows, in their chapter 2 Gentzkow and Shapiro have a suggestion for Windows systems of using a shell script to automate workflow. Below in an empirical example we will see a simplified version of this in `R`

23.1.1.1 Backing up your work

At some point during your research it is exceedingly likely that you will lose access to your hard drive and any work saved on it. Whether this be because some component on your computer breaks or because you lose your computer or a hard drive, it is extremely likely that you will lose access to your hard drive at some point (or multiple points) during your research career. For this reason, it is **absolutely fundamental** that you view backing up your work as an integral part of your research workflow. You may have some syncing tool which does this automatically, and if this is the case I would encourage you to check with great frequency that it is indeed backing up as you believe. Alternatively, you may wish to manually back up your discs on external drives. Tools such as rsync provide efficient ways to do this. You should also consider the importance of having redundant back-ups, so that if one back up does not work as you believe, you still have another back-up in place. At every specific moment in time I would encourage you to be sure that if your computer stopped working at that moment that you would not lose more than a few hours work, and hopefully no work at all.

23.1.2 Project Structure

In line with the point on workflow above, we should aim to structure our projects in a logical fashion. At a minimum, projects should be located in their own directory, and should separate raw data, source codes, any intermediate data, and results. In the example at the end of this class we will see a structure which does these things adequately.

23.1.3 Documentation

A fundamental part of any research project is (global) documentation. This documentation should provide us, or other users of our project, an immediate idea of the structure of our project, how to get up and running with the project, and how to replicate our work.

For this reason, we should always accompany our projects with a **README** file, which should be located in the top directory of our projects. README files provide giving global instructions relating to a project, explain elements such as the procedures which should be

followed to replicate a project from start to finish, any external libraries needed, directory structure, etc. These should be the first thing you look for when you download a file to understand key elements of a project. If you cannot find a README, this may be evidence that you should doubt the code in a project. In general, while these files are known as README files, they may be named things like README, README.TXT, READ.ME (all likely readable with a text editor), or some other file format, like README.pdf, README.doc, or, in the case of markdown README.md. A template for the type of things which we might expect to be in the README is available here.

23.2 Version Control

When our projects become slightly more complex, we will wish to use some sort of version control software which will allow us to keep track of changes in projects, as well as seamelessly collaborate in teams. Arguably the most common way to do this now is via git, often using the online platform github.

One argument for using git is that it avoids issues like those laid out in the comic below! Using git is great to provide a full track of all the code we have written in all iterations, allowing for us to generate branches to try new things and remove things that we no longer want, while still maintaining a full record of the history of our work. More generally however, git also allows us to collaborate with others, working on code together in a decentralised way, and then bringing things together, for example on places like github. We will explore some of the practical elements of git in our next tutorial together.



Comic from Piled Higher and Deeper
https://phdcomics.com/comics/archive.php?comicid=1531

CC-BY hutchdatascience.org

Figure 23.1: The stereotypical argument for git

! Getting Up and Running with git

23.2.0.1 Introduction

To see some ideas about version control we will explore git and github, and go through a number of the steps here.
Information on installing git can be found here. I would ask that you would follow these instructions to get a working implementation of git on your machine. While there are GUIs available, by far the most flexible way to deal with git is via the command line, which is why we will introduce it this way. You can access the command line directly through VScode if you do not have simple access to the terminal on your machine, or else on Linux and Mac operating systems it is reasonably trivial to access the command line. If you are working on Windows, you may wish to install git bash as a way to do this. The terminal acts as the conduit through which our git commands can be issued, and so provided that you have some way to do this, you should be fine!
After we have some way to access git, let's first just confirm that we effectively have git installed, which we can do with the command below:

```
git --version
```

```
git version 2.34.1
```

Provided that we get some output printing a version number (which will depend upon when you installed it), this implies that git is installed. If you get some other message such as `'git' is not recognized as an internal or external command, operable program or batch file.` (or some other message suggesting git is not installed), you will need to return to the resources on git, and ensure that the installation has been completed.
If this is the first time you are working with git, you may want to complete some first time set-up procedures. While not strictly necessary, they will make interacting with github later on much easier, and so it is potentially worth at least setting up the basics now. In particular, we will want to tell git our username and email address. For the email address, we should use the same email which we will use to generate a github account, which ideally should be an email we plan to keep for an extended period of time. This can be done as follows:

```
git config --global user.name "Your name"
git config --global user.email youremail@example.com
```

where "Your name" and the email should be filled in appropriately. We can confirm that this has been updated as follows:

```
git config --list
```

```
user.email=damian.clarke@protonmail.com
user.name=damiancclarke
init.defaultbranch=main
credential.helper=cache
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
```

Depending on your installation, you may see slightly different configurations, for example if some particular other settings are configured as default.
To have a quick look under the hood and where we will be going with git, you may wish to briefly consult the top-level manual which you can do with `git --help`.

```
git --help
```

```
usage: git [--version] [--help] [-C <path>] [-c <name>=<value>]
           [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
           [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
           [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
```

```
           [--super-prefix=<path>] [--config-env=<name>=<envvar>]
           <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
   clone     Clone a repository into a new directory
   init      Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
   add       Add file contents to the index
   mv        Move or rename a file, a directory, or a symlink
   restore   Restore working tree files
   rm        Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
   bisect    Use binary search to find the commit that introduced a bug
   diff      Show changes between commits, commit and working tree, etc
   grep      Print lines matching a pattern
   log       Show commit logs
   show      Show various types of objects
   status    Show the working tree status

grow, mark and tweak your common history
   branch    List, create, or delete branches
   commit    Record changes to the repository
   merge     Join two or more development histories together
   rebase    Reapply commits on top of another base tip
   reset     Reset current HEAD to the specified state
   switch    Switch branches
   tag       Create, list, delete or verify a tag object signed with GPG

collaborate (see also: git help workflows)
   fetch     Download objects and refs from another repository
   pull      Fetch from and integrate with another repository or a local branch
   push      Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.
```

We can see from this that git is relatively light in terms of the number of different commands we may need, and indeed, probably much of the time we are interacting with git it will be with just a small sub-set of these commands. We will start to get into this now…

24 Setting up a Project

Let's imagine that we are starting a big software project, and know that we will need version control. This may involve multiple people, but for the time being, we may only just be working by ourselves on our local machine. To see a very contrived example, let's generate a new file called `myproject`, and then within this project begin our first git repository.

```
mkdir myproject
cd myproject
git init
```

```
Initialized empty Git repository in /home/dcc213/trabajo/Teaching/Exeter/ResearchMethods-I/
```

At times we may wish to begin a git project before we generate any files at all, but it is also not a problem to start a project in a directory which already contains files. We have seen that starting our git respository with `git init` provides us a message that an empty repository has been generated. This has been initialised in a new directory called `.git`. You may note that if you are in the directory where the repository is located and type `ls` we will not actually see anything:

```
ls
```

and this is because directories starting with `.` are hidden, both when typing `ls` as well as when looking at them in folders. However, if we ask `ls` to show us everything, we will see that it is there:

```
ls -a
```

```
.
..
.git
```

We do not need to know precisely what goes on inside the `.git` folder, just that this will keep track of our project and its changes if instructed. We can see in a broad sense what git is seeing with `git log` and `git status`. When we have not yet done anything `git log` will return an error as below:

```
git log
```

```
fatal: your current branch 'main' does not have any commits yet
```

while `git status` will, as yet, suggest relatively little is happening:

```
git status
```

```
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Because we have not yet added any code, there is not much that git needs to keep track of!

25 Adding code, staging, and committing

Let's now add a very simple first code, and see what happens. Below we will just generate a simple plain text file, consisting simply of the words "Hello world!".

```
echo "Hello world!" > file1.txt
```

We have stored this in a file called `file1.txt` (using notation we know from our work in BASH). Now, let's see what the `git status` tells us:

```
git status
```

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    file1.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Now we see that git tells us there is a file there, but it's not being tracked. If this is part of the code of our project, we will want to track it so we can have a history of how it evolves, and roll back to prior versions if necessary. We will now "stage" this file, which means it could be committed to our git repository, and enter the version control history.

```
git add file1.txt
git status
```

```
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   file1.txt
```

The key line above with `git add filename` which tells git to put file1.txt into the "staging area". This allows us to do a number of things such as see any differences between staged and unstaged versions of the file, and, fundamentally, commit our files to the git repository. We do this below with `git commit`.

```
git commit -m "Adds hello world file"
```

```
[main (root-commit) b51ee99] Adds hello world file
 1 file changed, 1 insertion(+)
 create mode 100644 file1.txt
```

The syntax of `git commit` is to first write the command, and then `-m` followed by a commit message. This provides us a brief snapshot of what we've done, and something that will hopefully jog our memory later on. If we type `git commit` without specifying a message inline with is fine, but our command line will open an editor and we will need to type one. What we have done with `git commit` is add our code to the version control history of the project. If we now type `git status` and `git log` we will see a few changes, with the most important being the change in `git log`.

```
git status
git log
```

```
On branch main
nothing to commit, working tree clean
commit b51ee99640c96d10cb5e6a958b21904334064338
Author: damiancclarke <damian.clarke@protonmail.com>
Date:   Thu Nov 27 16:58:08 2025 +0000

    Adds hello world file
```

We can see at the end that we have a new commit which is indicated by a long string of letters and numbers, as well as information on the author and date of the commit.
Essentially, we should commit code to our repository every time we have made a concrete change. A very nice discussion of commits, along with commit messages is available here.
This commit will be available forever after, and even if we later make many changes, we will always have the option to see the code as it existed at this moment in time, and roll back to it if we wish.
Let's imagine that we now need to add another piece of code to our repository, which depends on some new output of our old `file1.txt`. Below we will add a new line to `file1.txt`, and then stage the changed version of `file1.txt`:

```
echo "Some addition outputting something for file2.txt" >> file1.txt
git add file1.txt
git status
```

```
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   file1.txt
```

We will not commit this yet, because we want to also write `file2.txt` and then commit the two changes together as a single new function in the code.

```
echo "A new file taking input from file1.txt" >> file2.txt
git add file2.txt
git commit -m "Adds new function file2 which interacts with file1"
git status
```

```
[main 166240c] Adds new function file2 which interacts with file1
 2 files changed, 2 insertions(+)
 create mode 100644 file2.txt
On branch main
nothing to commit, working tree clean
```

We have now added this new commit to our repository, and if we look at the `git log` below, we will see that both versions of the files are tracked. However, if we look in our main folder (with `ls`), we will see that we do not have any bloat or have to worry about anything in particular about prior versions of the files. We can get at them if we want or roll-back to any previous versions, but if we don't ask for anything, we will just see the most recent versions of all our files.

```
git log
ls
```

```
commit 166240ccd88d1c84ea74701956351450d7766223
Author: damiancclarke <damian.clarke@protonmail.com>
Date:   Thu Nov 27 16:58:08 2025 +0000

    Adds new function file2 which interacts with file1

commit b51ee99640c96d10cb5e6a958b21904334064338
Author: damiancclarke <damian.clarke@protonmail.com>
Date:   Thu Nov 27 16:58:08 2025 +0000

    Adds hello world file
file1.txt
file2.txt
```

26 Working with github

While this is all well and good for keeping track of our own work, it does not really show case much of the true power of git, which is for working in a dencentralised way among potentially many different individuals. Let's now see how git can interact with github. I will assume that you have now generated a free github account based on the email indicated above. We will now talk through a few things:

1. Adding our code to a github repository allowing for public access and collaboration
2. Contributing to an existing repository on github

Probably much of our work will be on point 1, especially early on in our usage of git, but as you start to work on more collaborative projects, you may see that gradually you are spending more on point 2 above.

26.1 1. Publishing our repositories on github

Let's now imagine that the project we have created above is something that we want to share publicly, either so that people can see and use the code, or so that we can collaborate with others who may also contribute to the project. We can do this from within github with the + symbol pictured below, and then indicating "New repository".

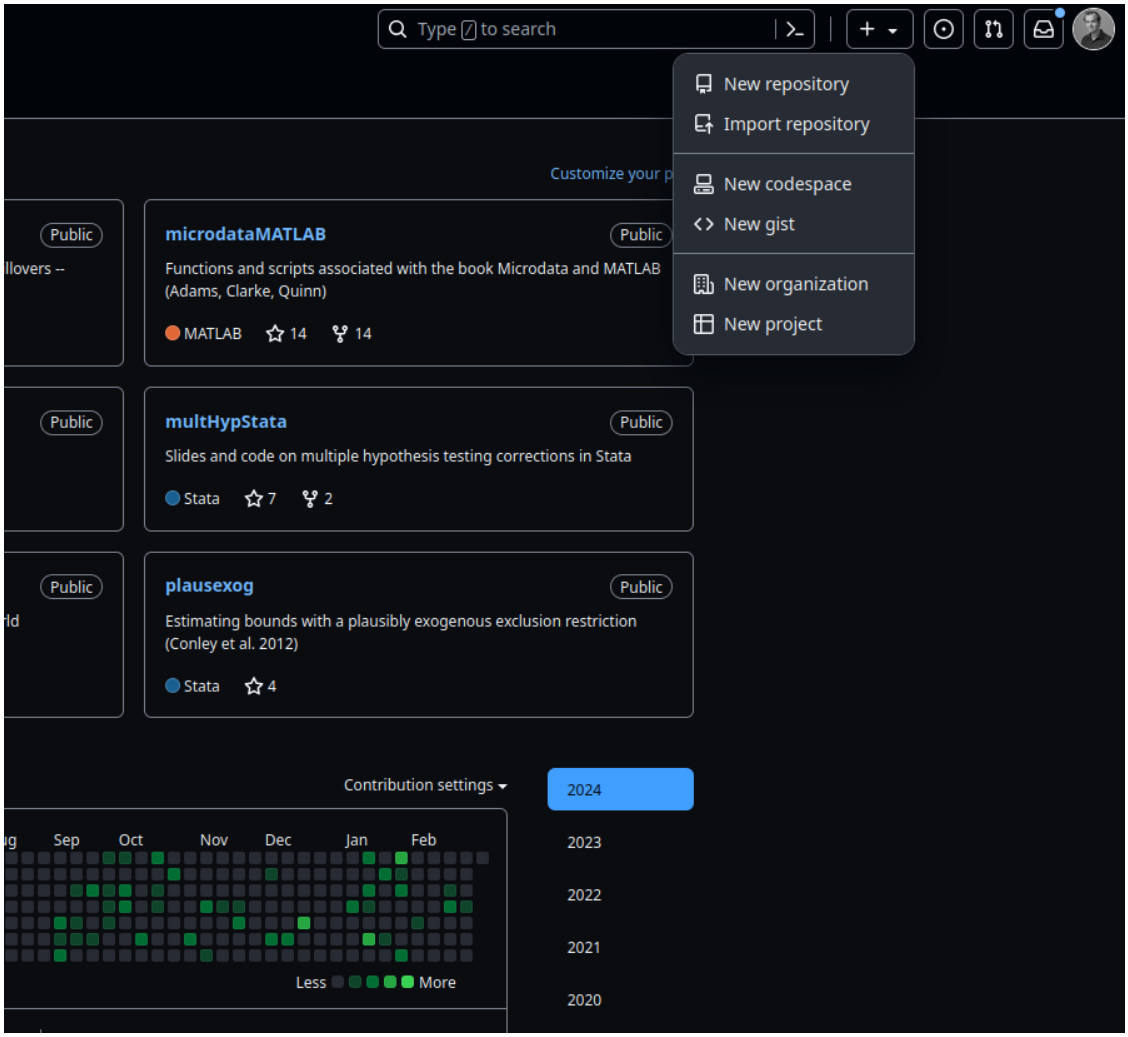


Figure 26.1: Creating a new github repository

This will then provide us with a place to indicate a repository name, as well as whether we wish to have this be public or private, as well as optionally whether we would like to include a License file to start. If we are starting a fresh repository without any code to push from our machine we might want to add a license file, otherwise we will probably prefer to begin with an empty repository to push everything we have locally on our machine into. Once we are satisfied with the details below, we can click "Create repository".

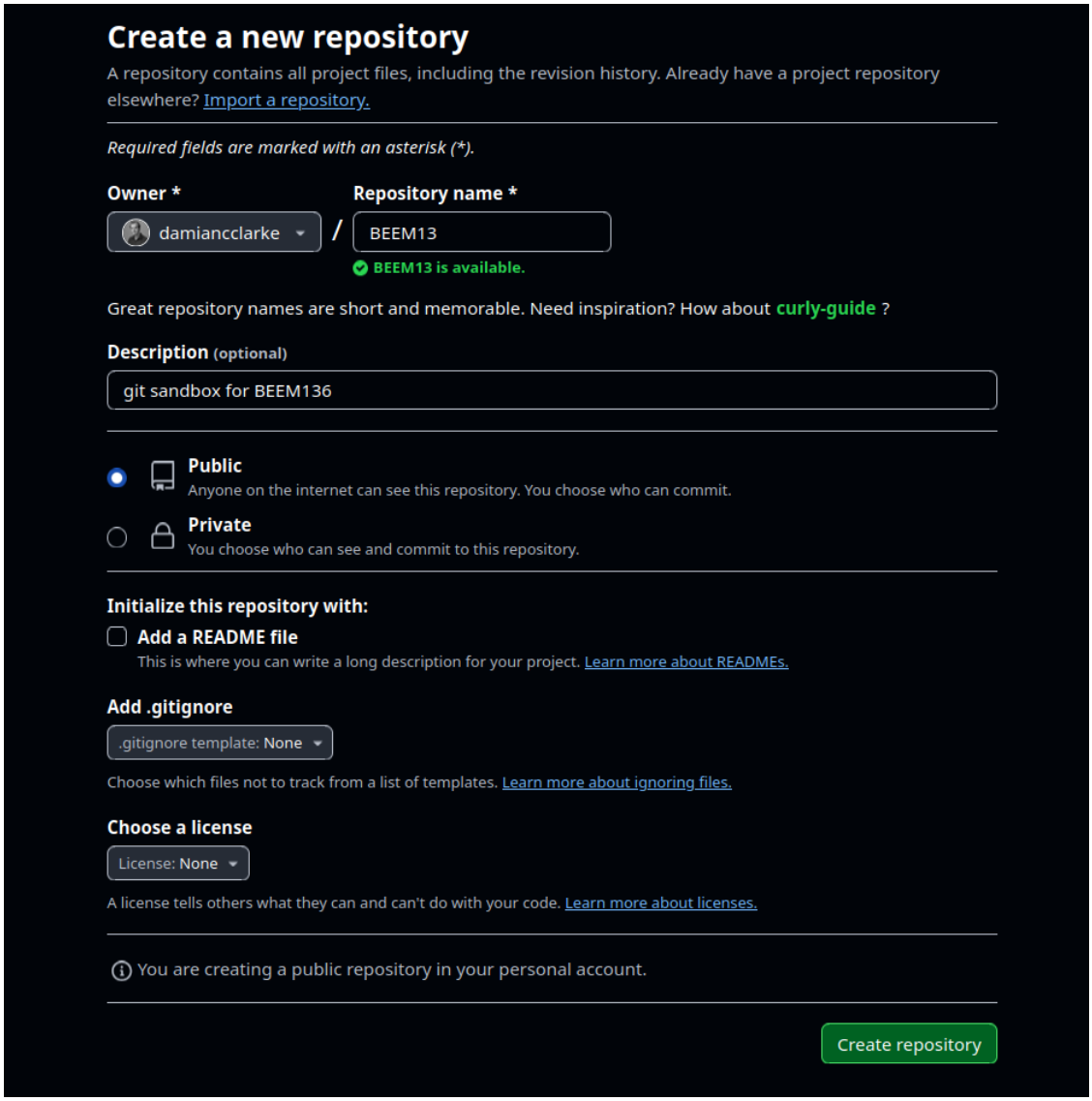


Figure 26.2: Details of a github repository

Now we need to go about including the code which we have stored locally on our machine into this repository. If we have a git repository already located on our machine that we wish to incorporate into this github repository, we need to take a few steps. Let's see this with the repository we have already set up previously. Within this repository on our machine, we will need indicate that we wish to contribute to a remote repository. We do this below, where the remote location will depend on our username and github repository name:

```
git remote add origin https://github.com/damiancclarke/BEEM136
```

If things work as planned, there should be no message here, but if we want to confirm that something has occured, we can check this as below.

```
git remote -v
```

```
origin  https://github.com/damiancclarke/BEEM136 (fetch)
origin  https://github.com/damiancclarke/BEEM136 (push)
```

Once we have done this, we are close to ready to "push" our code locally to the place we have indicated as "origin". To do this, we will first ensure that we have a branch called "main" on our machine. Because we only have one branch, it may be that by default this is called main, but the below code ensures that we know where we are pushing any code from. After we have done this, we will "push" the code that we have locally on "main" to the repository that we have on github which we have called "origin" above.

```
git branch -M main
git push -u origin main
```

```
Username for 'https://github.com': damiancclarke
Password for 'https://damiancclarke@github.com':
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 882 bytes | 882.00 KiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/damiancclarke/BEEM136
   28fbef7..23334ef  main -> main
   Branch 'main' set up to track remote branch 'main' from 'origin'.
```

When we do the `push` above, we need to be able to access our repository on github. As this is of course sensitive (we would not want anybody to be able to push to our repsitories), git will require a password. Some information related to possible alternative routes to do this from github are available here, and note that the password that github requires is not actually the password we use to login, but rather a personal access token which can be generated as discussed here. While it is a bit of hassle to have to do this additional step to push code, it is safer, and once it has been done once, we can use the personal access token for a long period of time, until we set it to expire.

Once we have done this, and given that we will have this repository available publicly, we may wish to add a README file which will provide some information on this repository to people who look at our repo on github. We can do this by adding a file called README.md. If we write this file using github's markdown, it will be visualised well online. Let's imagine we have edited some README.md file, we can then stage and commit it below.

```
git add README.md
git commit -m "Incorporates README to our repo"
```

```
Username for 'https://github.com': damiancclarke
Password for 'https://damiancclarke@github.com':
[main 00d2f26] Incorporates README to our repo
 1 file changed, 1 insertion(+)
```

Now when we look on github, everything we have done so far will be there! This should look something like the below:

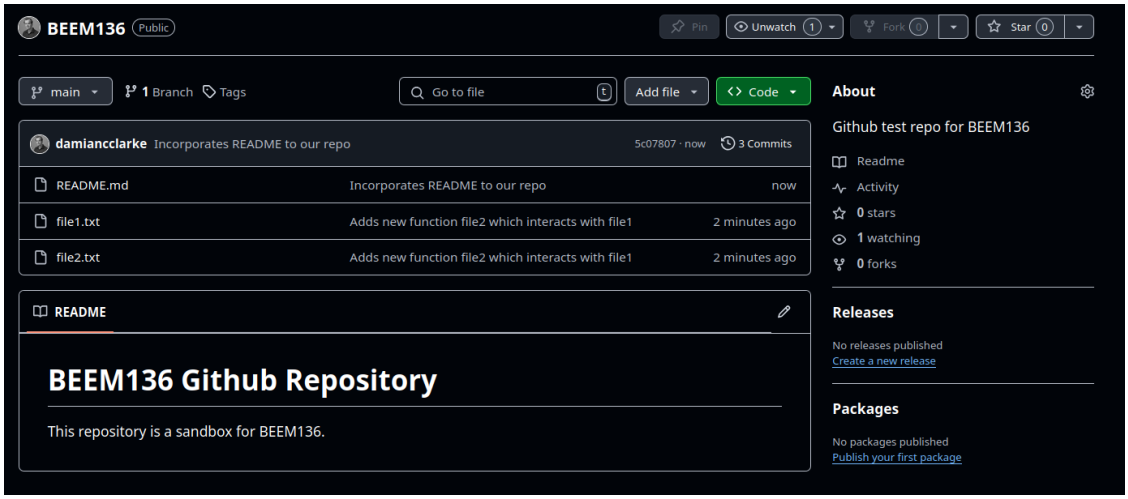


Figure 26.3: Our first github repo

If we look at individual codes, we can also see the full history, with an easy-to-use interface showing exact changes between any versions. In the below screen shot for example we see exactly what change was made when we moved from our first to second commit at the beginning of this project. In practice, when commits involve more lines of code, this feature can come in very handy.

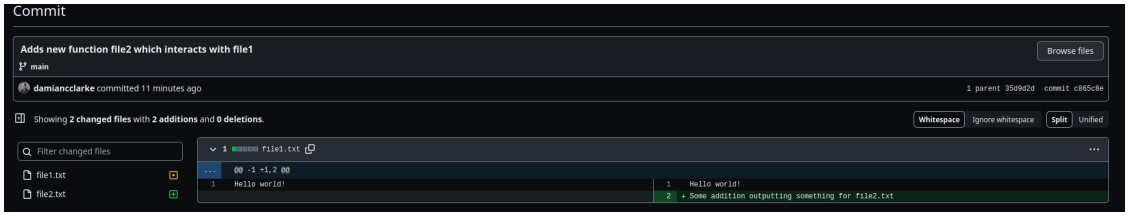


Figure 26.4: Visualising changes across code commits

At this stage our local git repository and github repository are in sync. At the beginning and at the end of any working session, we should always ensure that this is the case. If you are collaborating on a project with others, it is possible that by the time you start a new working session, your collaborators may have made some changes and pushed it to the github repository. In this case, we should always first ensure that we are up to date with all the recent updates from github before we make any other changes ourself. The nice thing about it is that you can use git pull even when in doubt. In this particular case, no change will appear because we are already in sync, and no one else has edited this repository.

```
git pull
```

```
Already up to date.
```

26.2 2. Contributing to an existing repository

As a final point, let's see how we could contribute to an existing repository on github. This may occur for a number of reasons. It may be that there is some piece of software that we frequently use and which we haved edited slightly to improve performance, and want to suggest this as a general feature to the code's maintainer. Or it might be that we are working in a team, and want to incorporate work into the repository of the project the team is working on. Regardless of why, if we wish to do work on the code, the first thing we will do is "fork" the repository on github. In order to give an idea of how this looks on both sides (from the maintainer of the repository, and the person who forks it), below I am working with another github account. This new account has no copy of the repository to begin with. Any screen shots below where github is coloured white are based on this new account which is copying the repository, and which wants to contribute. Any screen shots which are coloured black are showing things from the point of view of my original account (the account which maintains the repository).
The first thing the "contributor" must do is simply take a fork of the repository. This requires no permission as anyone can work with public code on github, and places a fresh copy of the respository on the contributor's github repository. At this point, any changes

made in that repository will be limited only to the contributor's repository. Below we see two screen shots showing how one makes a fork:

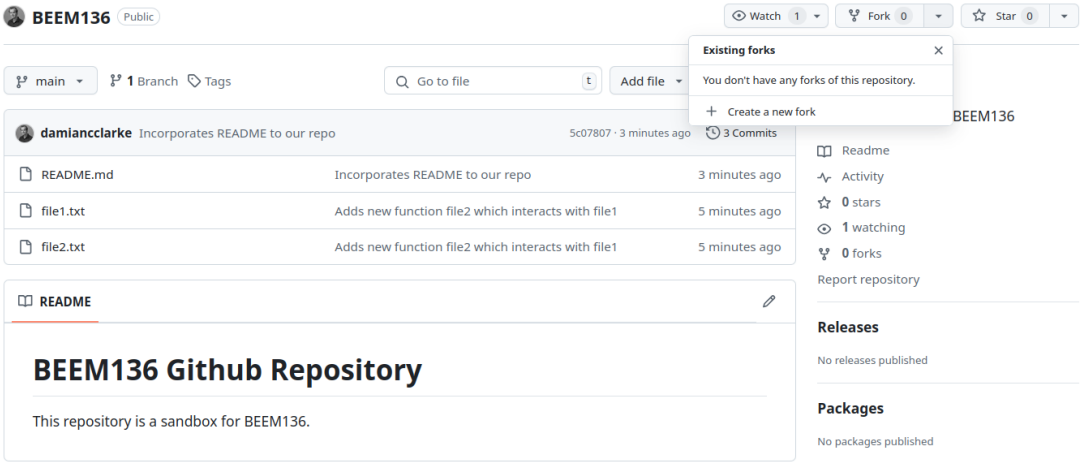


Figure 26.5: Forking Step 1: Click on "Fork"

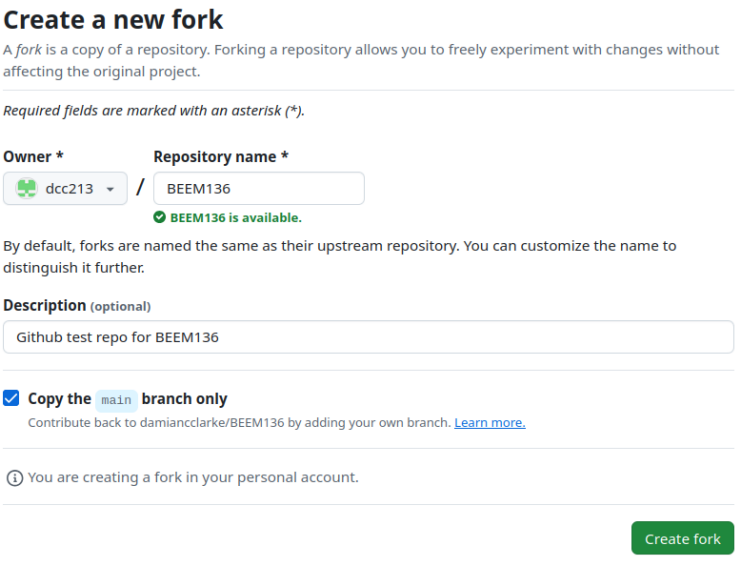


Figure 26.6: Forking Step 2: Provide any required details

Once we have forked this, we will see that a copy of this repository now exists on our github profile. To actually work with this repository, we probably want to copy it locally

to our machine. We actually *could* develop code directly on github's site, but in practice we will probably want to have available tools on our operating system, so we will assume we want to develop it locally. To do this, we use `git clone`. This simply puts a copy of the indicated repository and its git history on our machine. We can see that when we do this, if we type `git log` we see the entire git history of the repository, including the prior versions which the original author has published.

```
git clone https://github.com/dcc213/BEEM136
```

```
Cloning into 'BEEM136'...
remote: Enumerating objects: 18, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (12/12), done.
remote: Total 18 (delta 4), reused 16 (delta 2), pack-reused 0
Receiving objects: 100% (18/18), done.
Resolving deltas: 100% (4/4), done.
```

Now that we have cloned the repository, we can get to work. In the interests of brevity, let's simply generate some new file. Below we have a look at what is in the repository, and then generate a file called `file3.txt`.

```
ls
echo "Adding a third test file" > file3.txt
```

```
file1.txt  file2.txt  README.md
```

Now let's stage the file and commit the file to our git history. What this is doing is incorporating the file into our git history locally. This is not yet incorporated into (a) our version of the github repository or (b) the original developer's copy of the github repository…

```
git add file3.txt
git commit -m "Adding a third file"
```

```
[main ab83a99] Adding a third file
 1 file changed, 1 insertion(+)
 create mode 100644 file3.txt
```

To push the changes to *our* copy of the project is easy, and we can do it in the same way we discussed above when seeing how we push to our own projects. Indeed, because we have already done a `git clone`, git knows how our local branch and github branch correspond, so we do not need to tell it anything more than that we wish to push our local changes. We do this below:

```
git push
```

```
Username for 'https://github.com': dcc213
Password for 'https://dcc213@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 300 bytes | 300.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/dcc213/BEEM136
   9001f27..ab83a99  main -> main
```

Once we do this, we can see that in the version of the contributor's repsotitory, the changes are now incorporated:

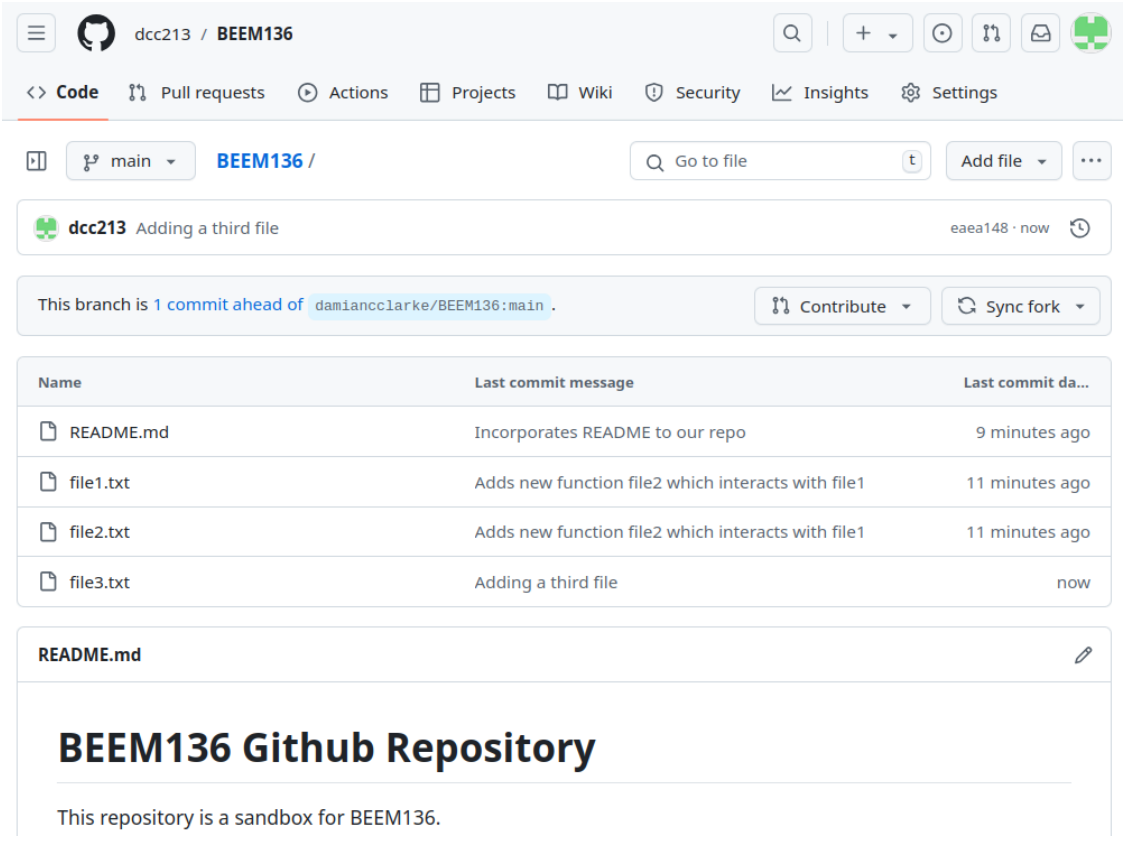


Figure 26.7: Incorporating an edit into our github repository

This change will only affect our code on github. What we *haven't* yet done is suggest to the original developer that we wish to incorporate this change into the original code. If our code is incorporated into the original developer's repository, then anyone else who interacts with the code in the future will also have our commit. To ask the original developer to consider incorporating our code, we open a "pull request" on github. This is us asking the maintainer to "pull" our changes to their code. Below we can see how this looks.

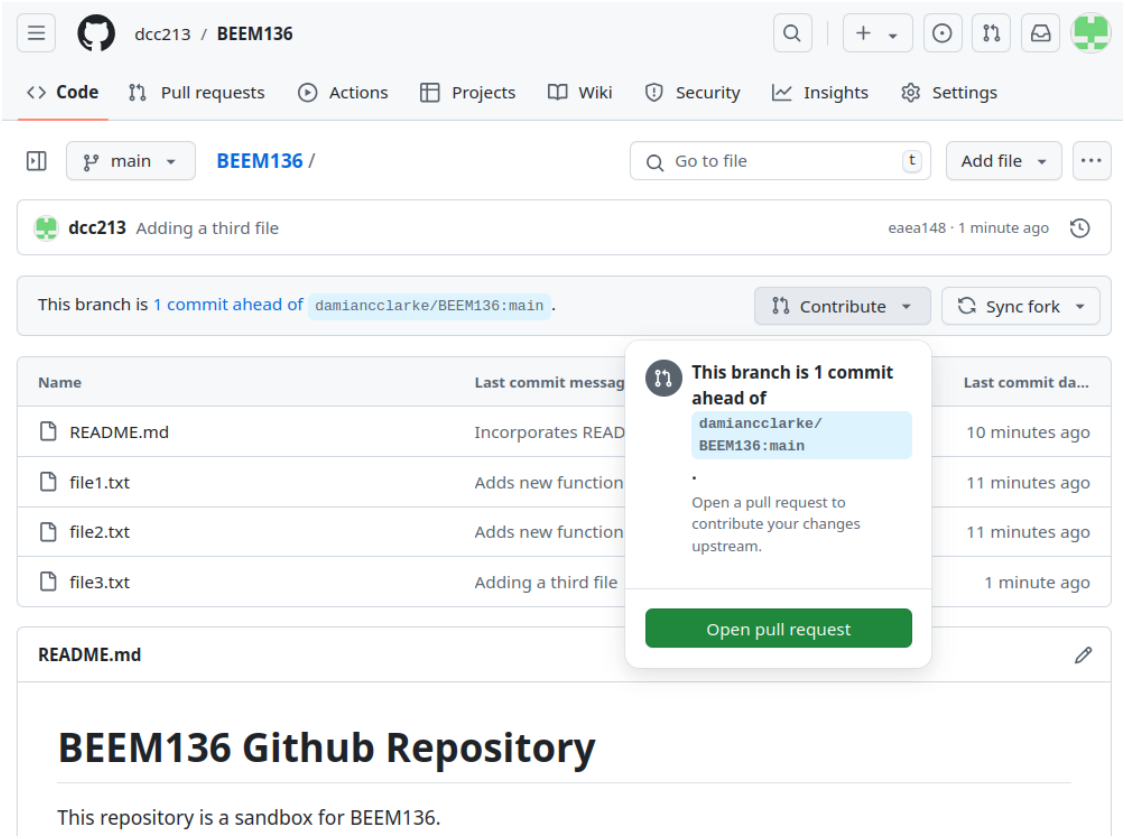


Figure 26.8: Opening a pull request Step 1: Click on "Contribute"

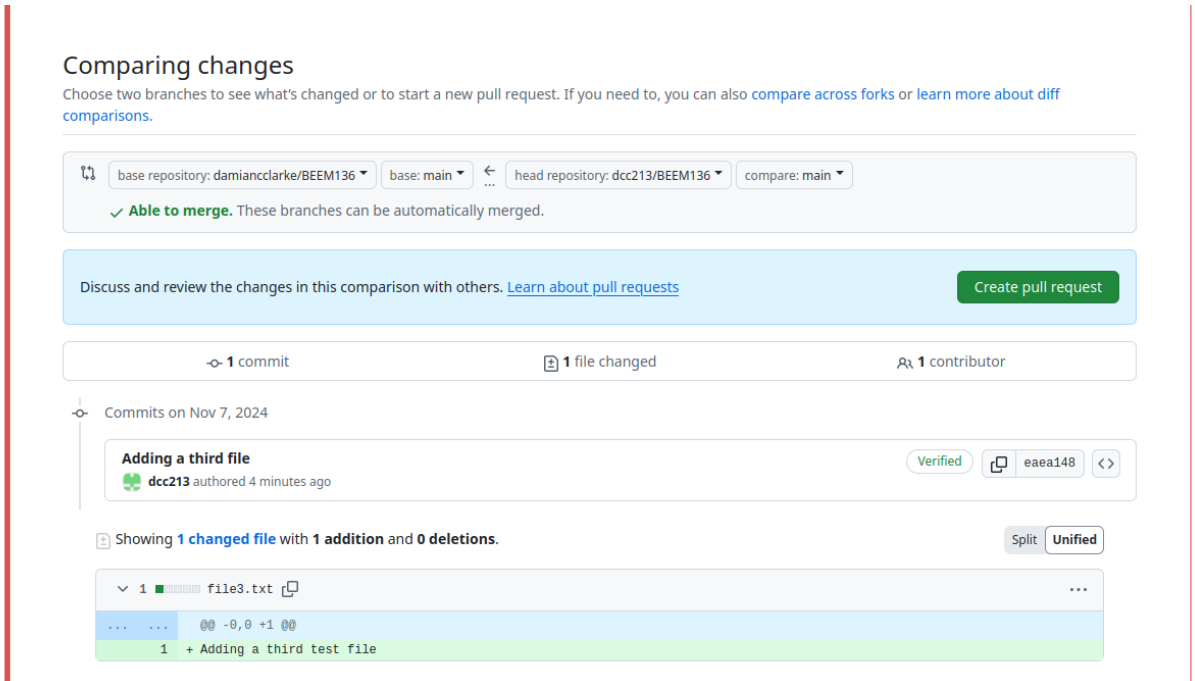


Figure 26.9: Opening a pull request Step 2: Create the pull request

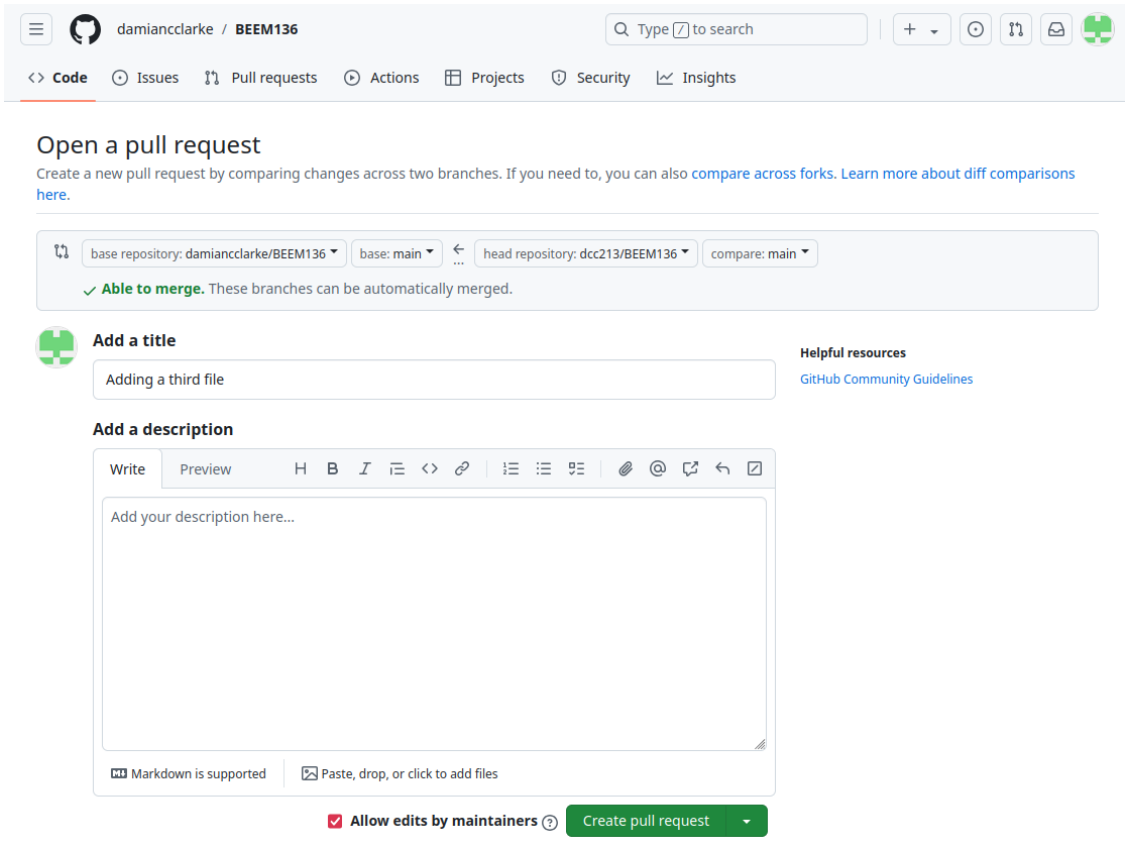


Figure 26.10: Opening a pull request Step 3: Adding any more context required

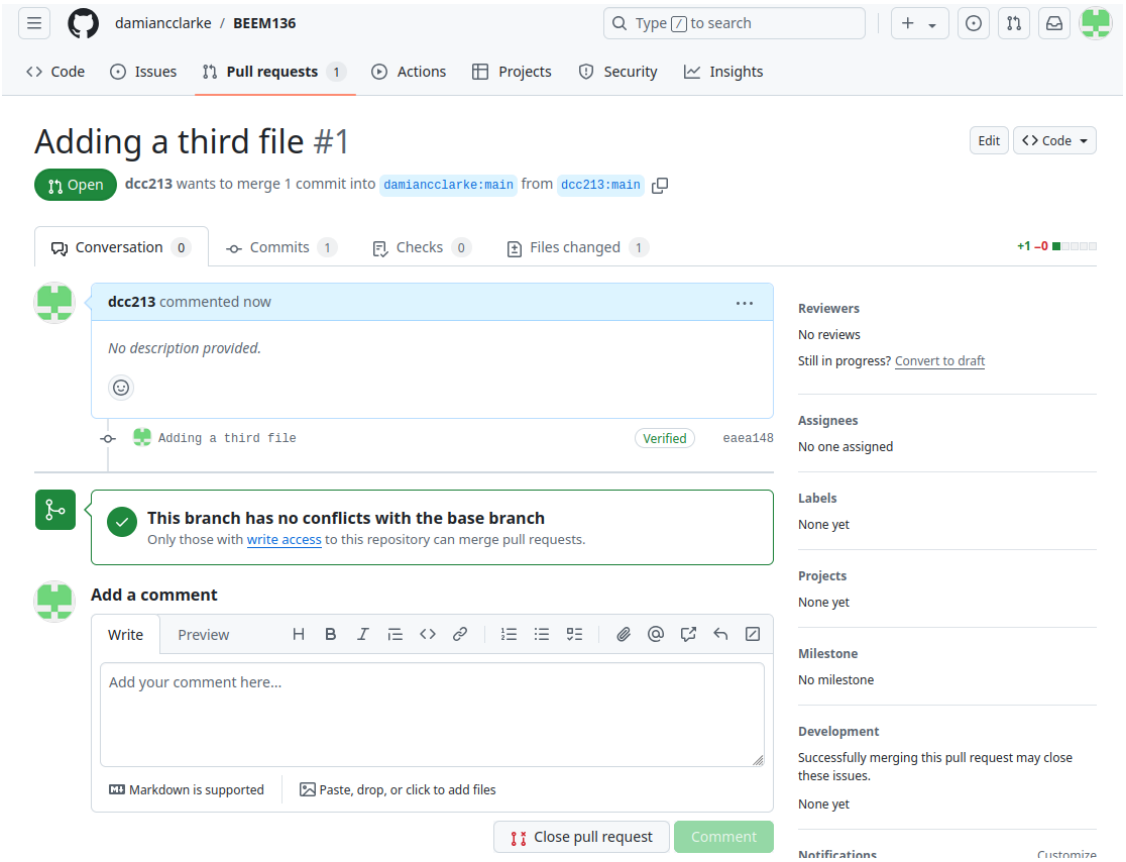
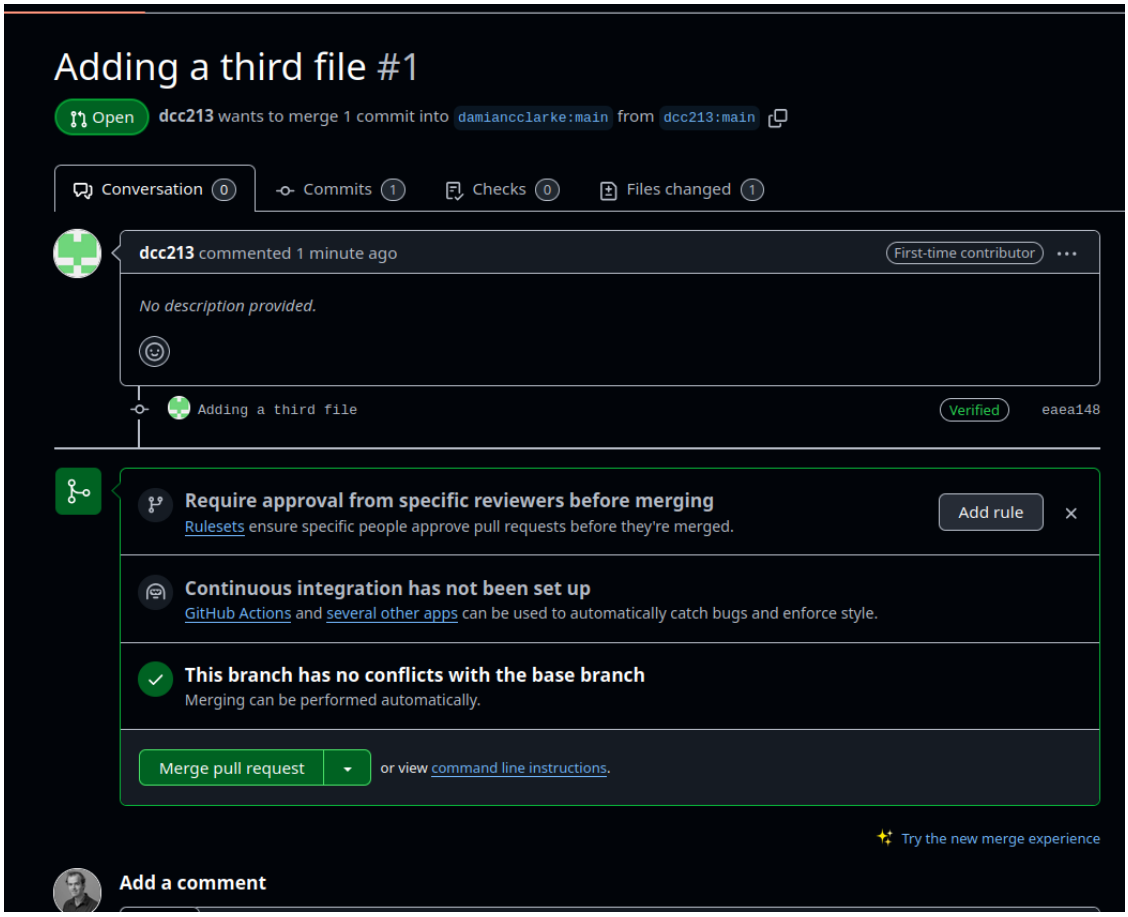


Figure 26.11: Opening a pull request Step 4: Awaiting response

There are multiple stages in this process (hence the four screen shots above), but essentially, provided that there are no conflicts on code edits (ie no other changes to the same area of the code we have edited have been made to the original repository), we can simply click through and create the pull request.

Then it is up to the maintainer to see if they wish to accept our pull request. Provided they do, they can review the code and click on "Merge pull request", as below:



This will then result in changes to the published repository, where you can note that the version of the file which has been edited by us now has our commit message. This will now be available in the original repository, and will be the new version which anyone else works with.

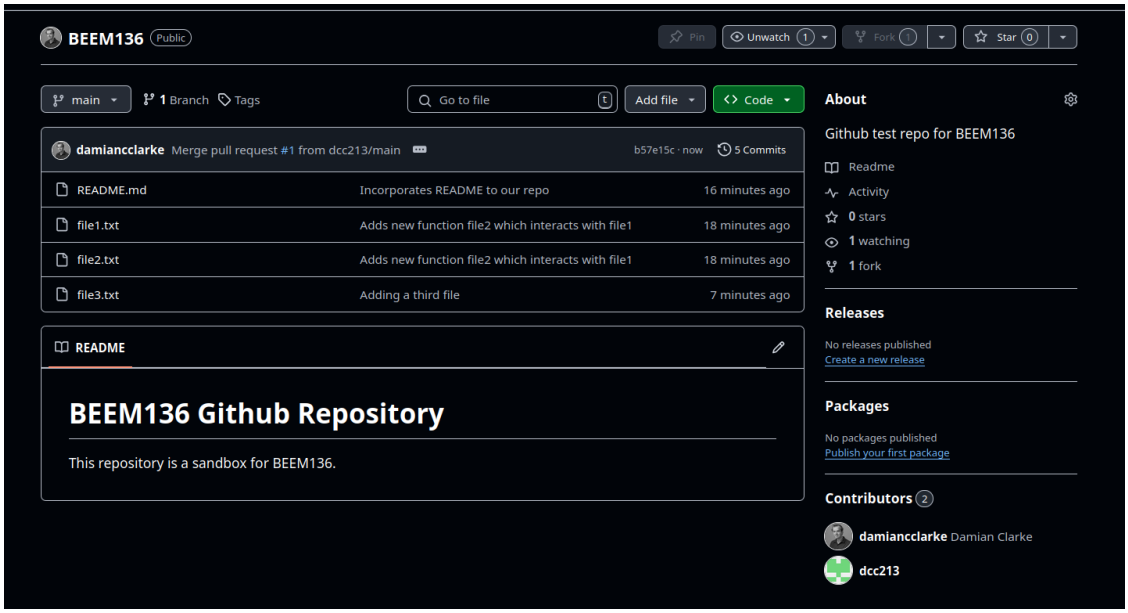


Figure 26.12: Merging a pull request Step 2: The result

With this example, we have seen a brief introduction to git and github, as well as (hopefully) what value it offers us to keep track of our projects and collaborate with others. A nice thing about git is that it is a reasonably small language, and once we have learned these basics, we can actually get quite far. There *are* other things which can be done such as opening up branches and working on elements of research projects in parallel and things like this. If you would like to read more, you could perhaps explore further resources such as this website.

26.3 Programming Style and Documentation

26.3.1 Programming Style

Within individual codes in projects, we will also wish to maintain a consistent style and clear documentation. There are a number of frequently used stylistic choices in programming, though often these aren't based on explicit (syntactic) rules. For example, people have an ideal directory structure they like to use for projects. And certain default choices are used for data management (eg commas as delimiters). Spacing in code is often user-specific (though there are languages where rules are in place), and ontelligent variable naming can simplify work considerably. In many cases, even if there are no rules, internal consistency is key as this allows for re-use of standardized code.

In some languages, spacing is a big thing. For example, Python uses indentations in code to find the end of loops and conditional statements. Another language, Fortran, reserves initial spaces in lines for particular keys or statements, and these cannot be used freely. As we have mentioned before, often, you see codes which do not exceed 80 characters per line (and this is my preference). This has its precedence from days where codes were stored on punchcards and *could not* exceed 80 characters (see below), but in general even on the screen this is useful, as it avoids confusion of reading across long lines in our text editors.

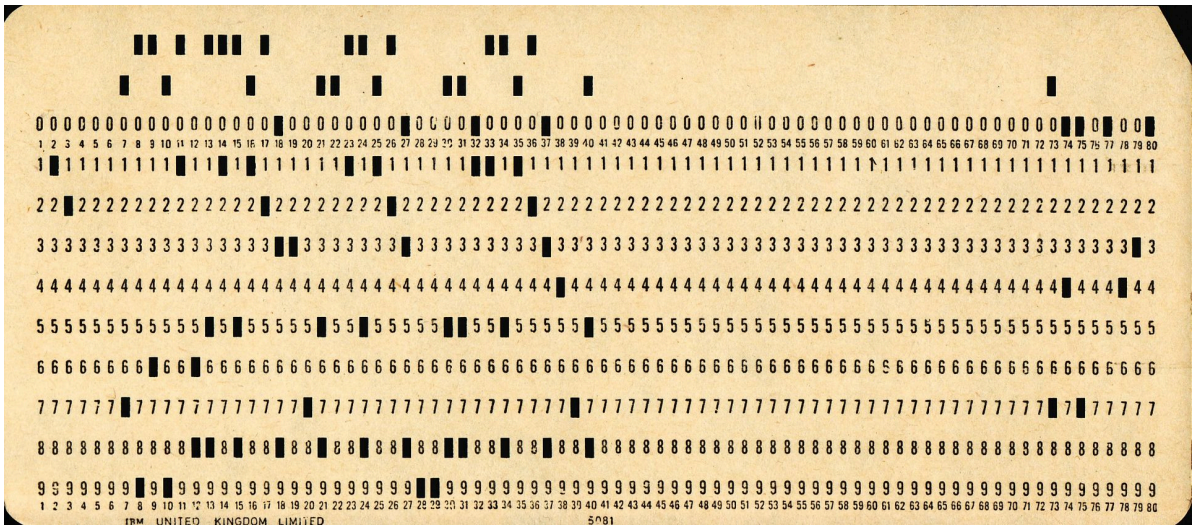


Figure 26.13: A Punchcard: An Institution in Programming Style

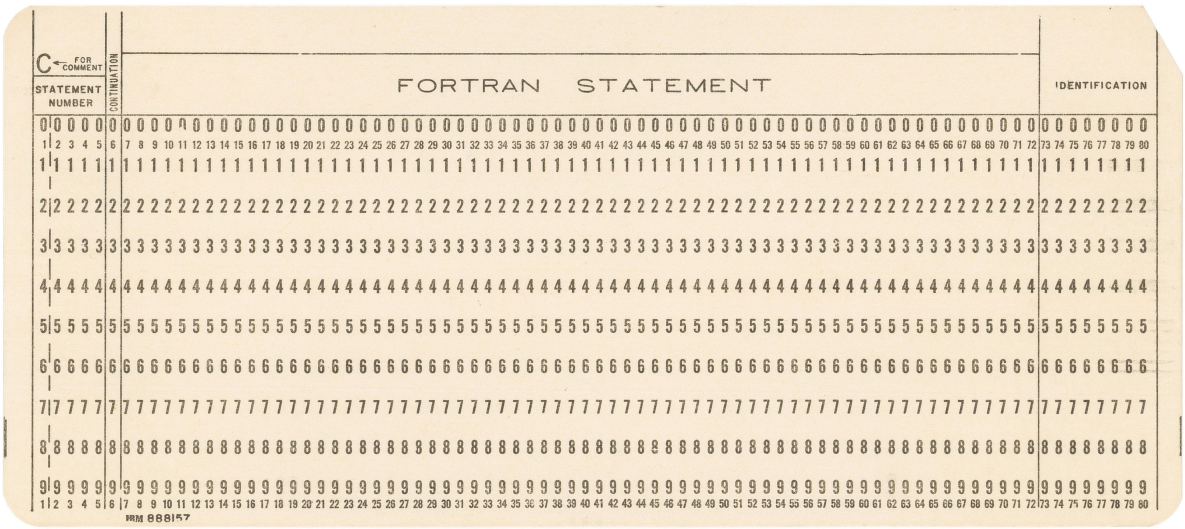


Figure 26.14: An Example in FORTRAN With Implications for the Language

Other things which are easy to control can make an important difference to the way our code is understood. Good variable naming can make things a lot easier, and bad variable naming can be a disaster. For example, consider a binary variable for male/female, and the potential names `var1`, `gender`, or `female`. There are particular standards one can elect to follow, eg snake_case or camelCase, and consistency is good, though this is something which depends entirely on your own desires. Some languages also permit the bundling of additional information with variables (like variable labels), and then even more information can be contained, though it is important to note that you should not rely on such constructs, because when data is stored in csv or other neutral file formats these labels will generally not be considered.

26.3.2 Automation and Abstraction

Wherever possible automation (avoiding human interaction) and abstraction (writing code that is flexible for many circumstances) should be embraced. As an entirely arbitrary example, imagine arriving at an expectation using the law of iterated expectations. The law of iterated expectations is a mathematical formula which states that:

$$E(Y) = \sum_{x \in \mathcal{X}} E(Y|X = x) \Pr(X = x)$$

While the precise details of the law are not what is important for us here, it is enough to note that if we wish to generate an expectation of some variable Y, we could do this by working with the expectation of Y over levels of some other variable X. To note how automation and abstraction come into play, consider three ways we could implement this formula:

1. We could tabulate X and then calculate the expected value of Y at each level of X by hand.
2. But it would be better to get a list of all values of X and iterate through these to take the expected value of Y at each level automatically (AUTOMATION).
3. Though best of all, we could generate a function to do this for any two variables of interest (ABSTRACTION).

The use of functions, or stand-alone programs, is a clear example of automation and abstraction. If there is some particular process which we find ourselves repeating a lot, we would likely be well-served by writing a function which does this process which we can then use in the future. This allows for us to focus our attention on perfecting this once, and then use optimized code in the future. We will explore this in our end of class example.

26.3.3 Testing, Debugging, and Profiling

In the process of writing our code, we will likely spend a considerable portion of time debugging, and should also seek to be constantly testing and profiling our code. These three concepts are different, and in summarised form refer to the following:

- Testing: My code runs, but does it do what it should?
- Debugging: My code does not run/compile. Why?
- Profiling: My code runs very slowly. Why?

In terms of testing, this is a constant process which requires vigilance and some creativity to do well. In empirical applications, it is often a good idea to see if a code returns 'correct' values in simulated data. We will discuss this towards the end of this course when we discuss using Monte Carlo simulations as part of a work flow. We should view testing as something that should be done modularly, not just at the end of a project. Rather, each script should be tested in a stand-alone way, and similarly, each argument in a function should be tested. Wherever possible, consider whether you can proactively build in tests into code. For example, if an argument of some function is the rate of unemployment, check that it is bounded between 0 and 100, otherwise generate an exception. Similarly, if we expect some variable to only take integer values, confirm that this is the case, and if not go back and check data.

Unlike testing, debugging refers to finding errors that are causing a script not to run. Most languages and compilers (especially those that we will use) have very good inbuilt debugging tools, which we have already been exploring. You should read error messages and consult user's manuals as a first option when bugs come up. In the worst case scenario, pasting the exact error message into a search engine may provide relief, and asking chatGPT or some other large language model about why errors occur can be quite illustrative. Note that sometimes programs issue warnings but still allow a script to run. These warnings should be at least looked into to, as often the best case is that they will show inefficiencies. Finally, do not think that just because a script runs it is correct. Always go back to testing.

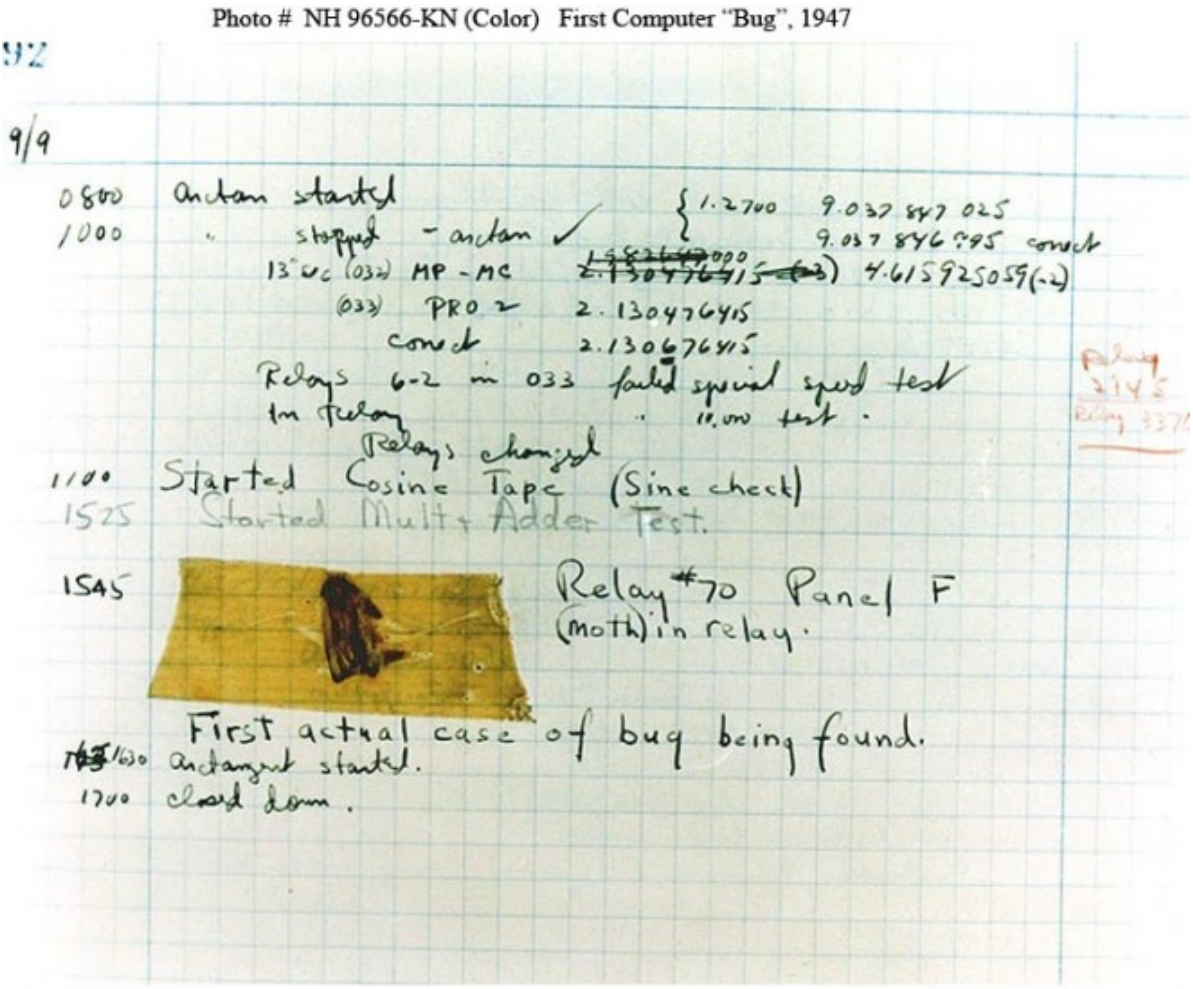


Figure 26.15: The Famous 'Computer Bug' often attributed to Grace Hopper's notebooks

Finally, the issue of profiling is about finding 'hot spots' in a code. If a code is slow to run, it may be that there is a bottleneck that can be optimized. But note, any concerns about efficiency only come after concerns about accuracy. Only begin optimizing when you are *sure* your code is correct. Many languages (or IDEs) have profilers which allow you to profile very easily (eg R Studio, Spyder). In languages where these aren't present, you can run time tests portion by portion. While we generally would like our code to run fast, it's worth keeping in mind that we should not over-optimize our code. There is a trade-off between machine time and programmer's time, and often we may be happy for our code to run a bit more slowly on the computer if it saves us time and effort, especially if they are codes we do not think we will run many times.

26.3.4 Documentation

Previously we have discussed documenting our projects "globally" with a README file. However, we should also invest in documenting "locally" by commenting within scripts. All programs we will likeley work with have a way to write comments. These are points implied for human readers, that the machine will ignore (hence syntax of the comment itself can be human language). It is likely in your research career that you will spend more time reading and understanding code than actually writing code, which makes clear the value of documenting your code in an intelligible way. Comments should be used throughout code wherever something is not absolutely clear, but *should not* be used when a line is self explanatory. If you are ever unsure of whether or not to document something, it is worth erring on the side of over-documenting rather than under-documenting! And in the same way that you maintain your code, you should see it as an important task to maintain comments in code.

One final thought to note here is that while it is *extremely* important to carefully document our code, we should not do so prematurely. We should *always* document our individual files in the way discussed here. However, we likely will *not* want to put together a comprehensive README file at early stages of a project given that many projects will not reach a stage where we are distributing them publicly, or with coauthors. At early stages, a very basic README will be enough. However, as soon as a project passes an initial exploratory phase, and appears to be something that we will continue with, we should follow complete documentation procedures, and invest in maintaining our documentation.

26.4 Reproducibility

A final element to note in thinking about project management and code management is the importance of replicability. This will be important to us as producers of research because we will generally write a paper over many months, and when we take up our code one day, we will want to start from exactly where we left off the day before. It will also be extremely important for our readers, and the scientific literature more generally. If another researcher wants to see what we are doing or extend our work, it is fundamental that they can run our code and precisely replicate what we are doing by running our code. For this reason, most journals now have an explicit replication policy and data editors who ensure that these policies are met, for example the American Economic Review, and Econometrica.

Beyond carefully testing and documenting our code, there are a number of other points we should be careful with to ensure that our code replicates without issues. In the case of any procedures based on pseudo random number generators (for example bootstrap variance estimates), we should be careful to ensure that we set the seed so that our "randomness" can be perfectly replicated. For an example of this, see the code below where we note that random draws *are not* the same across calls, unless the seed is first set.

```
rnorm(5)
```

```
[1] -0.4972223  1.0605141  1.1216343 -0.4872651  0.4589992
```

```
rnorm(5)
```

```
[1] -0.7499661 -0.4293387 -0.6739959 -0.7439186 -0.6380574
```

```
set.seed(1213)
rnorm(5)
```

```
[1] -1.119929  1.041126 -2.497151 -2.239511  1.134902
```

```
set.seed(1213)
rnorm(5)
```

```
[1] -1.119929  1.041126 -2.497151 -2.239511  1.134902
```

Another important thing to keep in mind is that packages at times evolve over time. Ideally all packages should maintain forwards compatability, but if for some reason a package changes, we would want to have a way to replicate our results based on earlier versions of these packages. At a minimum, in our documentation we should note exactly what versions of packages we are working with, which is something that we can easily consult:

```
packageVersion("ggplot2")
```

```
[1] '3.5.2'
```

```
packageVersion("dplyr")
```

```
[1] '1.1.3'
```

A more robust solution is to include the version of the programs that we use alongside our replication materials, potentially in a sub-directory of our source code. There are also a number of more formal options to allow us to do this. For R in particular this include package dependency management programs like packrat or checkpoint which allows us to download a version of all R packages as they were currently available at any specific past date. Somewhat

similarly, `Python` allows for an entire environment to be easily mapped and replicated using virtual environments, which can then be easily replicated across machines. Finally, if you are seeking a more comprehensive solution beyond a single language, you may be interested in exploring Docker, which essentially provides a portable version of every piece of software you need for other individuals to use. We will not discuss this at much length here, but if you would like to explore this, the AEA Data Editor has a post on the use of Docker in replication materials. These elements may be good things to discuss in tutorial presentations!

26.5 An Example in R

As a simple example of some of these principles we will set up a structure of a basic research project in `R`. To do this, we will use the same example the we discussed last class, and as such will not focus so much on the *what* the code is doing, but rather on *how* the code is structured.

To begin, let's set up a directory for our project consisting of three sub-directories: data, results, and source. Within results we will have a number of sub-directories, and within source we will have a directory where we will save the final version of the `R` libraries that we used. This project is available here as a zip file, but it may be preferable to set things up by hand.

```
list.dirs(path = ".", full.names = TRUE, recursive = TRUE)
```

```
[1] "."                   "./data"              "./results"
[4] "./results/analysis" "./results/macro"    "./results/micro"
[7] "./results/sumstats" "./source"            "./source/R"
```

In the top level of our project we will include a README, structured along the lines of the below file.

```
------------------------------------------------------------------------------
README.txt for PROJECT TITLE
Author 1, Author 2
email 1, email 2
------------------------------------------------------------------------------

This project does...

The project structure is as follows.

|_ source/
```

```
  |_ R
|_ results/
   |_ sumstats/
   |_ analysis/
|_ data/

To replicate this project you should simply run the code runAll.R located in the
source sub-directory.  All results will be exported to the results sub-directory
and you can then compile the paper located in the main directory...

This project was run on R, version 3.6.3.

This project requires the following externally installed libraries:
- XXXXXX: version X.X.X
- XXXXXX: version X.X.X
- XXXXXX: version X.X.X
- XXXXXX: version X.X.X

For questions related to this code, contact email@address.com.
```

Finally, within the source sub-directory we will have the following codes.

```
#---|----1----|----2----|----3----|----4----|----5----|----6----|----7----|----8
# runAll.R                      damianclarke              yyyy-mm-dd:2023-10-22
#
# This script runs required codes for the project SHORT DESCRIPTION HERE. To run
#the script, simply change directory locations on lines 29-31 to the correspond-
#ing locations on your machine.  If PREP is set to true data preparation scripts
#will be run. If SUMSTATS is set to true, descriptive statistics and graphs will
#be generated. If ANALYSIS is set to true, data analysis scripts will be run.
#
#
#

rm(list=ls())

require("dplyr")
require("readxl")

# Alternative 1 line call for multiple libraries:
x=c("dplyr", "readxl", "haven", "tidyr")
lapply(x, require, character.only = TRUE)
```

```
#------------------------------------------------------------------------------
#--- (1) Define switches for running certain areas of code and define locations
#------------------------------------------------------------------------------
PREP     <- TRUE
SUMSTATS <- TRUE
ANALYSIS <- TRUE

source  <- "~/trabajo/Teaching/Exeter/ResearchMethods-I/structure/source"
results <- "~/trabajo/Teaching/Exeter/ResearchMethods-I/structure/results"
data    <- "~/trabajo/Teaching/Exeter/ResearchMethods-I/structure/data"

#------------------------------------------------------------------------------
#--- (2) Run required codes
#------------------------------------------------------------------------------
if(PREP) {
    print("Running all data preparation code")
    source(file.path(source,"dataPrep.R"))
}

if(SUMSTATS) {
    print("Running all graphs and summary statistics")
    source(file.path(source,"descriptives.R"))
}

if(ANALYSIS) {
    print("Running all analysis code")
    source(file.path(source,"analysis.R"))
}
```

This first code is a single code which we can use to replicate the running of all of the portions of the project that are written in `R`. You will note that in this script we have indicated all of the directory locations that we will need, and indeed, this will be the *only* place that we will need to store these locations. They will then be passed along to other scripts run below. The second important element to note is that we are including a number of switches that can be set to TRUE or FALSE if sections of the code should be run. If any of `PREP`, `SUMSTATS` or `ANALYSIS` are not set to TRUE, then these elements of the code will not be run.

Finally, of note in the above code is that we have our first example where we have used some abstraction to make our code more generalised. Initially in lines 15-16 we are indicating that each of the `dplyr` and `readxl` libraries are required. However, this is repetitive, and for each additional library we will need to add another line. Instead, we can do this by first generating a list of all the libraries we require in line 19. Then, in line 20, we can use `R`'s `lapply` function

to apply a specific function, in this case require(), to a list of arguments (each character set in x). Later, if we realise that we need to add another package, we can simply add this to x.

```
#---|----1----|----2----|----3----|----4----|----5----|----6----|----7----|----8
# dataPrep.R                        damianclarke            yyyy-mm-dd:2023-10-24
#
# This script prepares a dataset which contains one line per municipality and y-
#ear for each of Chile's 346 municipalities over two years (2008-2009). Each li-
#ne contains the municipality's population of adolescent women, the number of t-
#een births, and whether the EC pill was available in 2009.


#-------------------------------------------------------------------------------
#--- (1a) Generate population file
#-------------------------------------------------------------------------------
population <- read_xls(file.path(data,"poblacionINE.xls"))
popTeens   <- filter(population,
                     edad>=15 & edad<=19 & `Sexo (1=hombres;2=mujeres)`==2)
popTeens <- select(popTeens, Comuna, nombre_comuna, edad, a2008, a2009)
names(popTeens)<- c("Comuna","nombre_comuna","edad","2008","2009")
popTeens <- pivot_longer(popTeens, cols=`2008`:`2009`,
                         names_to="ano_nac", values_to="population")
popTeens <- popTeens %>% group_by(Comuna,nombre_comuna, ano_nac) %>%
  summarise(
    population=sum(population),
    .groups = "drop"
  )



#-------------------------------------------------------------------------------
#--- (1b) A function to do this
#-------------------------------------------------------------------------------
gen_pop <- function(data, age1, age2, sex) {
    popTeens <- data %>% filter(edad>=age1 & edad<=age2 &
                                      `Sexo (1=hombres;2=mujeres)`==sex) %>%
      select(Comuna, nombre_comuna, edad, a2008, a2009)

    names(popTeens)<- c("Comuna","nombre_comuna","edad","2008","2009")
    popTeens <- pivot_longer(popTeens, cols=`2008`:`2009`,
                             names_to="ano_nac", values_to="population")

    popTeens <- popTeens %>% group_by(Comuna,nombre_comuna, ano_nac) %>%
```

```
      summarise(
        population=sum(population),
        .groups = "drop"
      )
    return(popTeens)
}
population <- gen_pop(population, 15, 19, sex=2)
population$ano_nac <- as.integer(population$ano_nac)
#-------------------------------------------------------------------------------
#--- (2) Import births data and collapse to the municipal level
#-------------------------------------------------------------------------------
nacList <- vector("list",0)

for (y in 2008:2009) {
  filename <- paste0("NAC",y,".csv")
  dfname   <- paste0("nac",y)

  nac <- read.csv(file.path(data,filename))
  nac$teenPreg <-ifelse(nac$EDAD_M>=15&nac$EDAD_M<=19,1,0)
  nac <- nac %>% group_by(COMUNA,ANO_NAC) %>%
    summarise(
      teenPreg=sum(teenPreg),
      .groups = "drop"
    )
  nacList[[dfname]] <- nac
}
teenPregnancies <- do.call(rbind, nacList)
names(teenPregnancies)<- c("Comuna","ano_nac","teenPreg")

#-------------------------------------------------------------------------------
#--- (3) Merge in population data
#-------------------------------------------------------------------------------
#teenPregnancies$Comuna[teenPregnancies$Comuna==5106]<-5801
#teenPregnancies$Comuna[teenPregnancies$Comuna==5108]<-5804
#teenPregnancies$Comuna[teenPregnancies$Comuna==5505]<-5802
#teenPregnancies$Comuna[teenPregnancies$Comuna==5507]<-5803

oldCodes <- c(5106,5108,5505,5507)
newCodes <- c(5801,5804,5802,5803)
for (i in 1:length(oldCodes)) {
  teenPregnancies$Comuna[teenPregnancies$Comuna==oldCodes[i]]<-newCodes[i]
}
```

```
popPreg <- full_join(population, teenPregnancies, by = c("Comuna","ano_nac"))
popPreg$teenPreg[is.na(popPreg$teenPreg)]<-0


#------------------------------------------------------------------------------
#--- (4) Merge in information on the EC pill with information on teen pregnancy
#------------------------------------------------------------------------------
pill <- read_dta(file.path(data,"PAE.dta"))
names(pill)[names(pill) == "comuna"] <- "Comuna"
for (i in 1:length(oldCodes)) {
  pill$Comuna[pill$Comuna==oldCodes[i]]<-newCodes[i]
}

popPregPill <- full_join(popPreg, pill, by = "Comuna")
```

The first code that `runAll.R` calls is `dataPrep.R`. This code replicates exactly the procedure we observed last class so we will not talk through each line of code, but there are a number of important elements to highlight here. Firstly, in section 1b we have provided an abstraction of the code that we wrote in section 1a. In section 1a we generated a dataframe containing the population of 15-19 year old women, while in section 1b we have provided a function which accepts as arguments the population data, as well as lower and upper ages, as well as sexes. This function means we can now request the populations of any age and sex groups, as we request in line 47 for the particular group of interest.

A second element of note is that for the first time here we have used `for` loops. These `for` loops are useful to us to automate procedures which repeat a number of times, for example looping over a number of years to import and clean birth data in block 2 of the code. If later we wish to work with a longer year range, it will be very simple to rewrite this `for` loop as `for (y in 2005:2010)`, for example, and all elements of the loop simply remain the same.

Finally, in section 3 of the code we have noted two ways to do the same thing. Lines 73-76 which are commented out would update municipal codes in an ad hoc way. However, instead of doing this we provide an abstraction below where first we list the corresponding old and new codes, and then simply loop through these lists. While in this particular case we have not saved any lines of code, the second option is preferable given that it is easily extendible. What's more, when we have to conduct a similar process on lines 93-95 of the code we can use the previously generated `oldCodes` and `newCodes` lists.

```
#---|----1----|----2----|----3----|----4----|----5----|----6----|----7----|----8
# descriptives.R                damianclarke              yyyy-mm-dd:2023-10-22
#
# This script conducts descriptive analysis including... FURTHER DETAILS.
```

```
#
#
#
#
#

#------------------------------------------------------------------------------
#--- (1) Generate descriptive plots
#------------------------------------------------------------------------------
plot(popPregPill$population,popPregPill$teenPreg,
     xlab="Population of Teens", ylab="Number of Teenage Pregnancies")
```

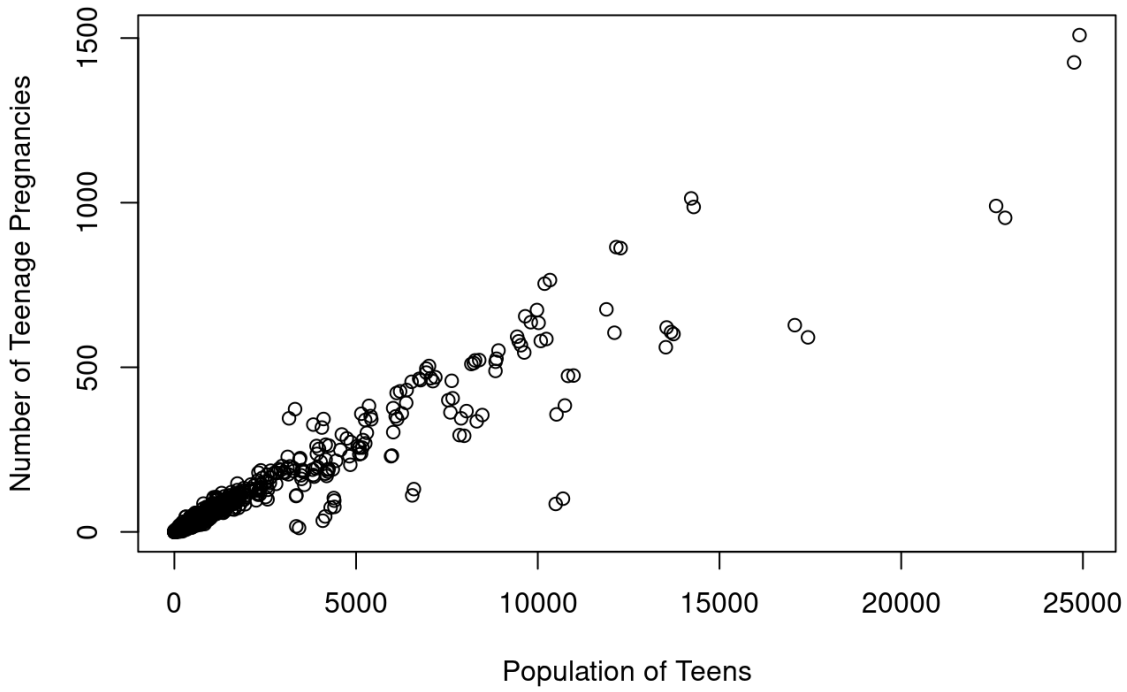


Figure 26.16: Output of descriptives.R

Finally, in this project we have two codes which will generate descriptive statistics and run econometric analyses. As our interest here is in illustrating a project structure we are leaving

these largely blank, though in practice this may contain an important proportion of the code of our project.

```
#---|----1----|----2----|----3----|----4----|----5----|----6----|----7----|----8
# analysis.R                      damianclarke            yyyy-mm-dd:2023-10-24
#
# This script conducts estimation of principal models... FURTHER DETAILS.
#
#
#

#-------------------------------------------------------------------------------
#--- (1) Regression models and so forth ...
#-------------------------------------------------------------------------------
print("Now running within analysis.R...")
```

```
[1] "Now running within analysis.R..."
```

27 Data Visualisation

28 The Importance of Data Visualisation

28.1 A Motivation for Graphical Work

An important element of quantitative research is in visualising data. Frequently this is done in one way and two-way graphs, where the distribution of a variable is plotted, or relationships between two or more variables are plotted in two-dimensional space. There are a number of reasons why we will want to spend time thinking about, and more importantly, actually doing, data visualisation. Among others, displaying information graphically can:

1. Help **us** catch things in our code that we may have been hard to capture in other ways
2. Let us summarise key information for our readers in a more effective way than using tables or narrative text
3. Provide a key descriptive or descriptive set of facts that draws readers to our work

When for our own consumption (as in point 1 above) we are likely happy to just be able to generate graphs relatively quickly and using default behaviours of the languages in which we are working. But when we are generating graphs that will be a part of a research project, we will likely want to invest slightly more time in ensuring that graphs look good and are doing their job effectively. As we will see below, often with relatively little work a basic plot can be converted into a plot that is both clearer and potentially more visually appealing, and additionally, built upon to provide additional information. In this class we will explore examples with `Python`, though the general principles we examine will be relevant for other languages too.

While the most important element of a graph is that it correctly portrays the information you wish to communicate, a well done plot can both look good, and quite simply portray more information than can be portrayed in a similar space in other ways. Many well known examples of good data visualisation exist. An early example is the pioneering work of sociologist W.E.B. Dubois, who in the early 1900s produced hand-drawn graphs describing demographic and socio-economic conditions of African Americans (see example below).

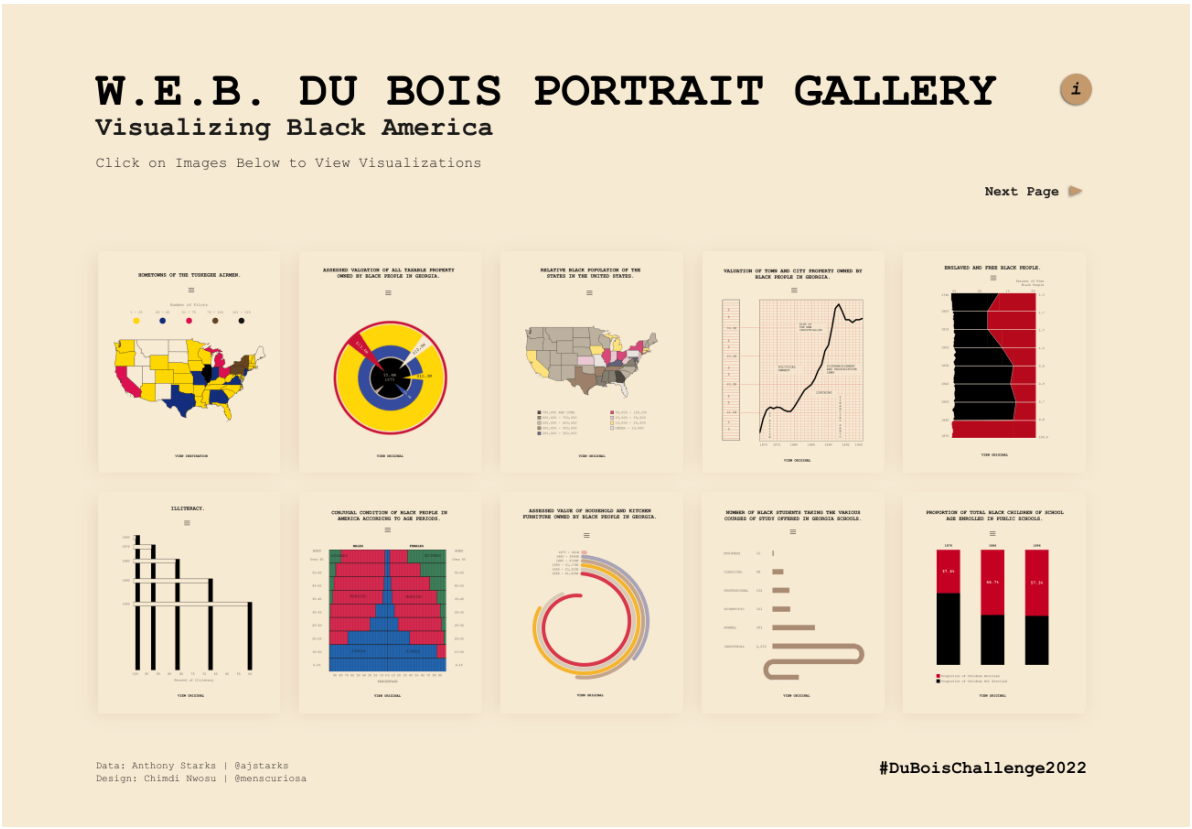


Figure 28.1: One of W.E.B Dubois' Information Graphics

There are a number of comprehensive guides on best principles in graphing. These include the books below. The first is the classic reference on data visualisation, while the latter is a more recent treatment, specifically focused on `R` and the `ggplot2` library.

- Tufte (2001): "The Visual Display of Quantitative Information"
- Healy (2019): "Data Visualization: A Practical Introduction"

28.2 Graphical Work in Economics Papers

Within economics papers, graphs can serve multiple purposes, from purely descriptive patterns of key variables which go beyond basic summary statistics (think for example, of histograms or kernel densities), to graphs which seek to use descriptive trends or summaries to foreshadow research designs. Some of the most effective graphs in economics papers are those which manage to effectively communicate the empirical design in a visual way before getting into too much complex modelling. Consisder a number of effective examples of such graphs below.

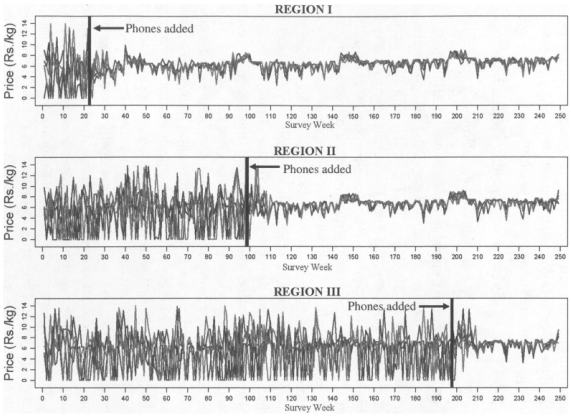
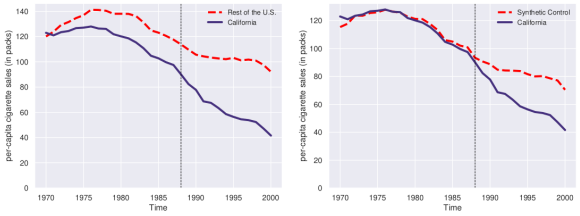
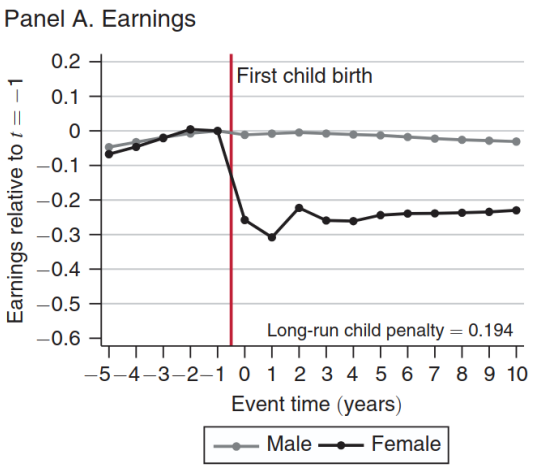


FIGURE IV
Prices and Mobile Phone Service in Kerala

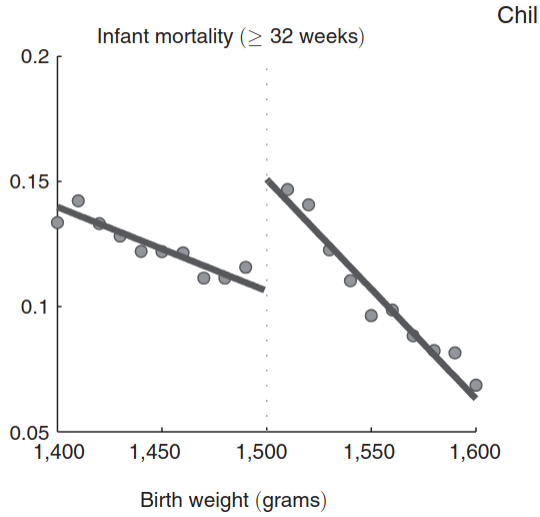
(a) (a) Jensen (2010)



(b) (b) Abadie, Diamond, and Hainmueller (2010)



(c) (c) Kleven, Landais, and Søgaard (2019)



(d) (d) Bharadwaj, Løken, and Neilson (2013)

Figure 28.2: Graphs as an empirical story

While these graphs come from very different styles of paper and research designs, what they have in common is that they provide a quite visually compelling way to lay out key descriptive statistics, while at the same time allowing readers to visually "see" that some treatment effect likely exists. Of course, the nature of any such graph will depend on your context, and not all papers allow for such visual presentations do, but putting some thought into how descriptive statistics can be used to tell a story, or to at least act as a prelude to a story, can have big returns in how a paper can be communicated and understood.

29 Basic Graph Examples

29.1 Some Graph Types and Procedures

To begin, we will return to the examples laid out in the Programming overview section of this course. Given that we did not spend time on this at the beginning of the course, we will start from scratch with these examples. Each language has its own routines and libraries for plotting data (and feel free to refer back to the Programming Languages section of this site for more information on your specific language of interest. In Python one of the most frequently used modules given its broad availability of graph types and flexible syntax is the is the `pyplot` module from the `matplotlib` library If you have not previously installed this module, you can do so using `pip install matplotlib`, and then we will import the pyplot submodule as `plt`, so we can explore the functionality of this module. We will additionally load numpy and pandas as standard:

```
from matplotlib import pyplot as plt
import numpy as np
import pandas as pd
```

`pyplot` maps pairs of points in the coordinate axis and places them on a graph. For example, let's consider a very simple case where is a sequence of integers between -5 and 5. This can be generated as below:

```
x = np.array(range(-5, 6))
x
```

```
array([-5, -4, -3, -2, -1,  0,  1,  2,  3,  4,  5])
```

Now consider the variable $y = x^2$:

```
y = x**2
y
```

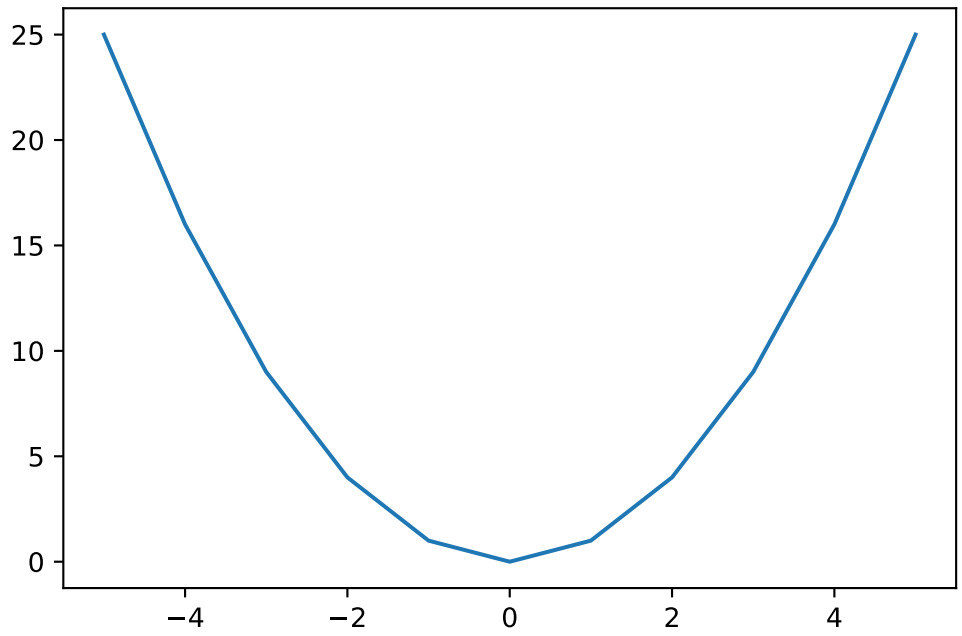
```
array([25, 16,  9,  4,  1,  0,  1,  4,  9, 16, 25])
```

We will store these two objects as a data frame, by creating a dictionary containing `x` and `y` (though we could proceed to plot without first storing our data as a data frame):

```
Data = pd.DataFrame({'x': x, 'y': y})
```

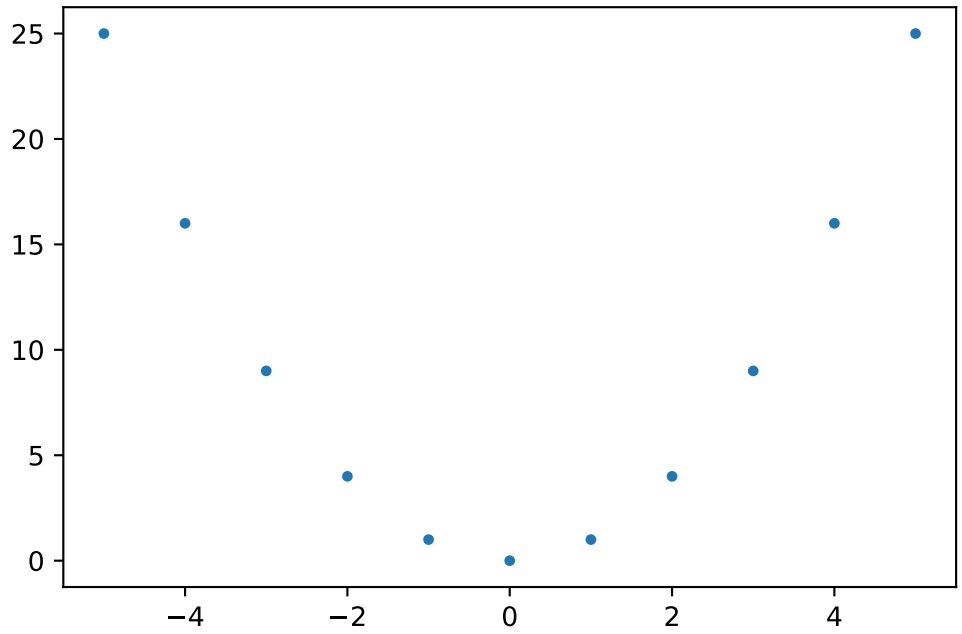
The relevant function in `pyplot` is `plot`, which by default generates a line plot, making reasonable decisions for axes and labelling. Unless you have an IDE which automatically displays your plot in a plot panel, you should use the `show` method from `pyplot` to display your plot:

```
plt.plot(Data['x'], Data['y'])
plt.show()
```



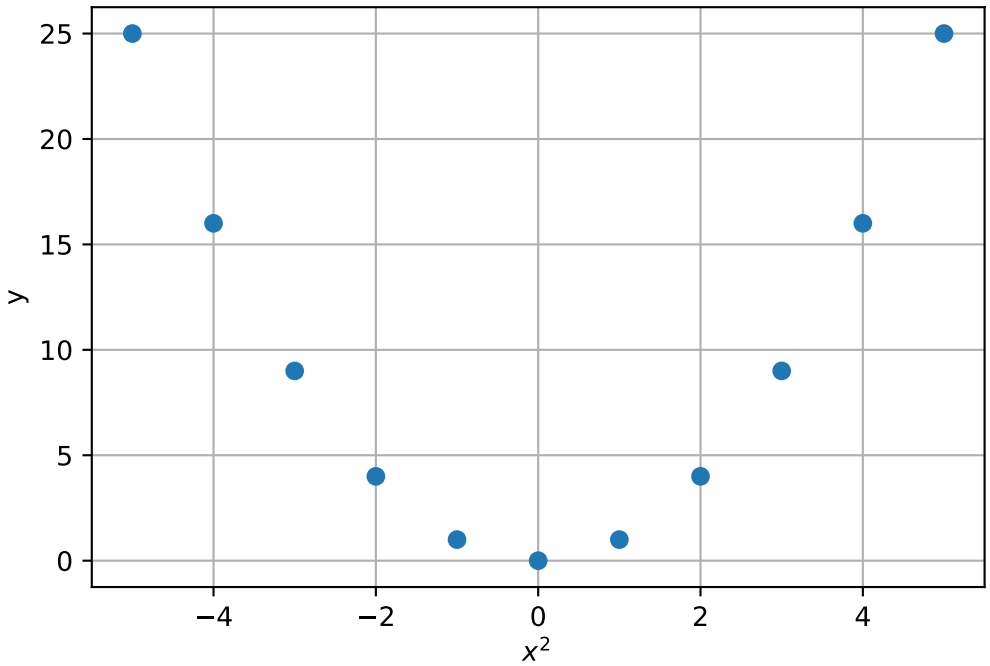
If you wish to change the default behaviour of `plot` you can do so with a range of options. For example, in order to change the plot to a scatter plot, we can set the linestyle to `""` and the markers to points:

```
plt.plot(Data['x'], Data['y'], linestyle = "", marker = ".")
plt.show()
```



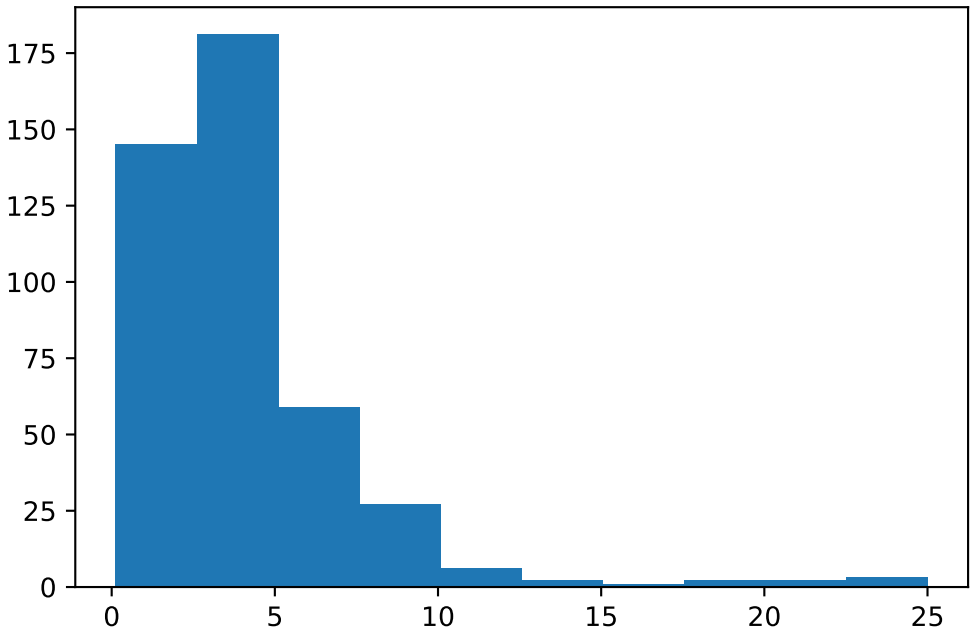
In general, we can view these plots as all being built on top of a canvas, where our desired plots can be generated in a step by step way. For example, if we now wish to add labels to x-axes and y-axes, which is something we should always do to improve stand-alone readability of the plot, we can simply build on top of what we already did:

```
plt.plot(Data['x'], Data['y'], linestyle = "", marker = "o")
plt.grid(True)
plt.ylabel('y')
plt.xlabel('x^2')
plt.xlabel(r'$x^2$')
plt.show()
```



There are many ways to further customise graphics in `Python`, and also a range of plots available as part of matplotlib's plotting library for use relatively simply. A list of plot types can be found here. To explore slightly more, we will return to the data we used in the first section of the course from Mroz (1987). This includes one-way plots such as histograms where we simply examine the distribution of a single variable (hence the "one-way" name):

```
import wooldridge
mroz = wooldridge.data('mroz')
plt.hist(mroz['wage'])
plt.show()
```



If the above plot was for us to display in a paper we would certainly want to further optimise output, for example by attaching axes titles. We can also display plots as groups of subplots. Below we see an example where we define `axes` to allow us control over a series of 4 plots.

```
fig, axes = plt.subplots(2, 2, figsize=(8, 6))
ax1 = axes[0, 0]
ax2 = axes[0, 1]
ax3 = axes[1, 0]
ax4 = axes[1, 1]

ax1.hist(mroz['wage'])
ax1.set_title('Hourly Wage')
ax2.hist(mroz['hours'])
ax2.set_title('Hours Worked')
ax3.hist(mroz['educ'])
ax3.set_title('Years of Education')
ax4.hist(mroz['age'])
ax4.set_title('Years of Experience')

#Prevents subplot titles overlaying
plt.tight_layout()
```



Again, depending on what these plots are for we may wish to further optimise them. In general, `matplotlib` provides a series of base graph types and these can be useful to build up other graphs. However, another widely-used package to allow for further customisation of graphs in `Python` is the `seaborn` library, which can be installed from pip. Seaborn provides a wide range of graphs which can be generated, and also has a series of more optimal baseline settings. It also allows for simple interaction with other colour palettes which we may prefer if we wish to optimise our graph. Below we will load seaborn, and examine one particular colour palette which we can easily use:

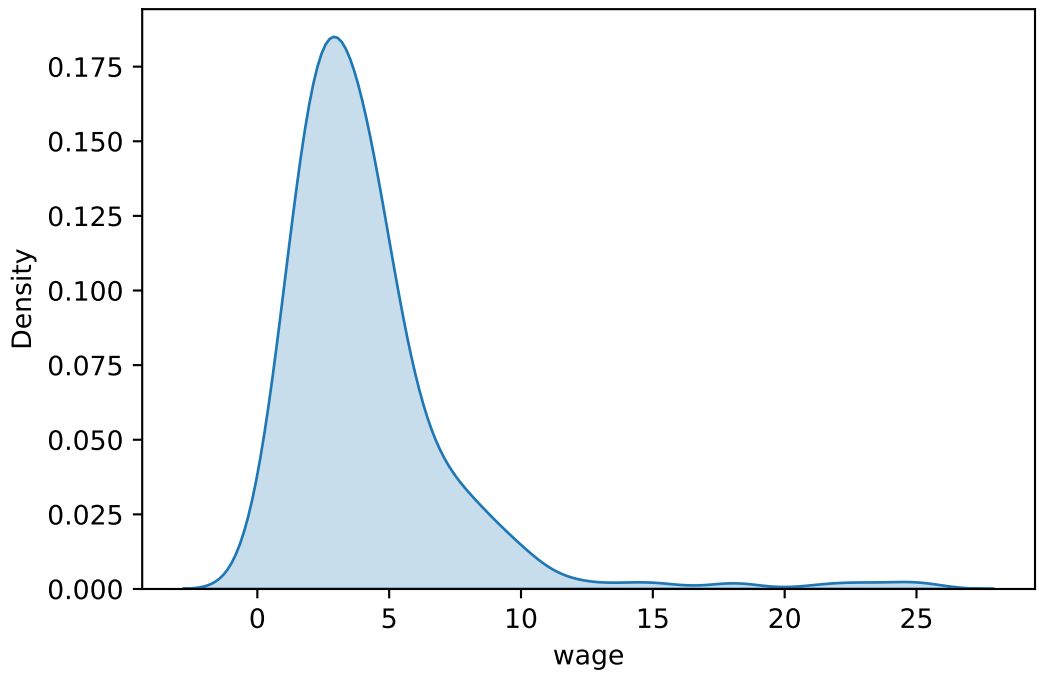
```
plt.clf()
import seaborn as sns
sns.color_palette("viridis", as_cmap=True)
```



```
<Figure size 432x288 with 0 Axes>
```

Now, if instead of a histogram we wish to generate kernel density plots, this can be done directly from seaborn:

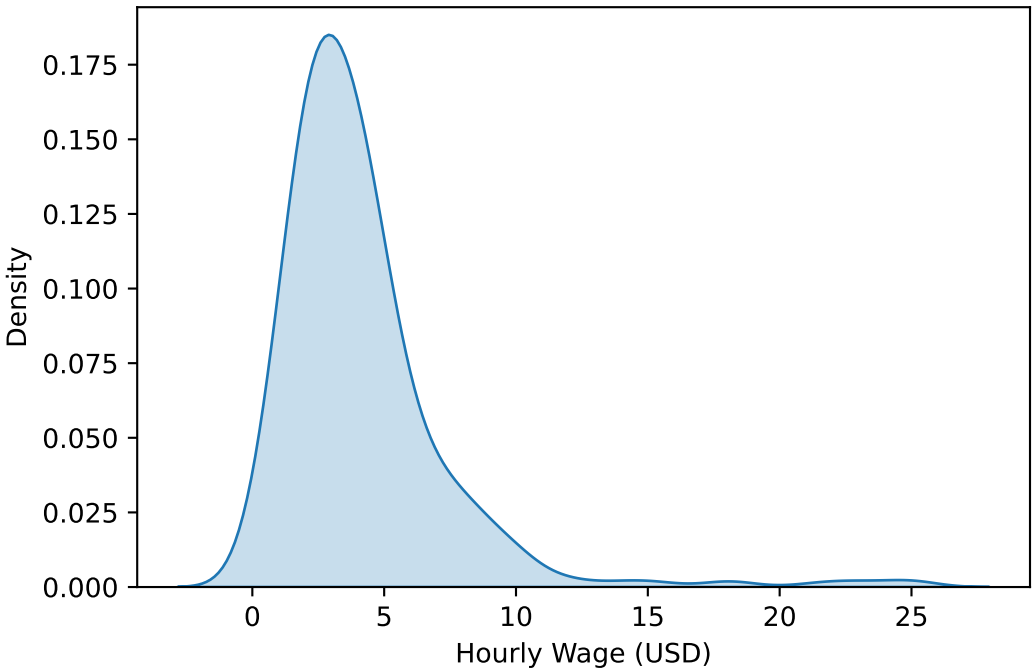
```
sns.kdeplot(mroz['wage'], fill=True)
```



What's more, we can use the functionality of `matplotlib`'s canvas to add on top of seaborn plots. Below we will generate this same plot, and then use the `plt` environment to add a different x-axis title:

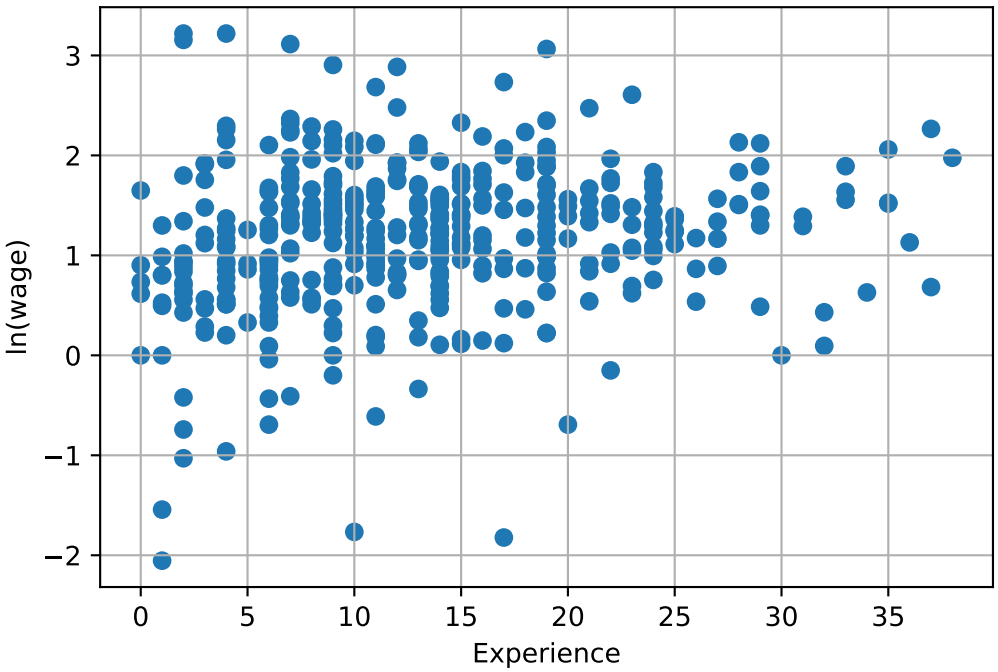
```
sns.kdeplot(mroz['wage'], fill=True)
plt.xlabel("Hourly Wage (USD)")
```

```
Text(0.5, 0, 'Hourly Wage (USD)')
```



Both `seaborn` and `pyplot` allow for a large range of two-way plots. For example, if we wish to simply generate a scatter plot we may be happy to do it simply in `pyplot`, as below:

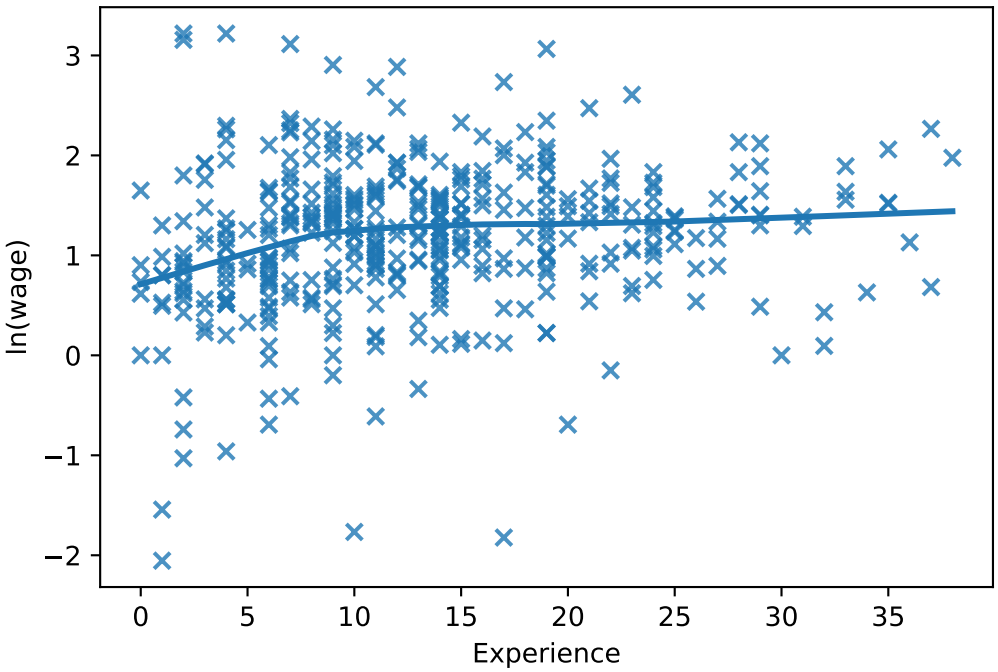
```
plt.scatter(mroz['exper'], mroz['lwage'])
plt.grid(True)
plt.ylabel("ln(wage)")
plt.xlabel("Experience")
plt.show()
plt.close()
```



But if we wish to overlay our scatter plot with some sort of regression line (for example a lowess fit below), we may find that a more specialised `seaborn` plot is easier for us to use:

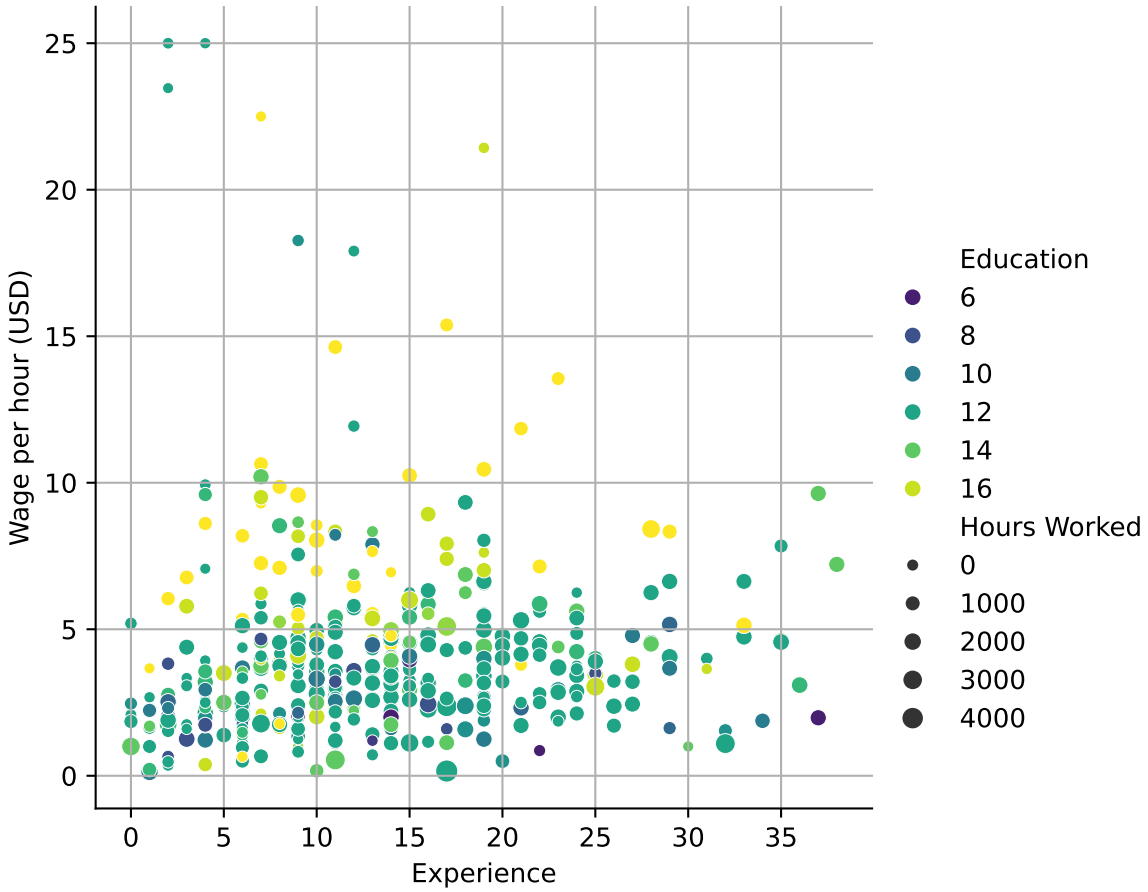
```
plt.clf()
sns.regplot(data = mroz, x = 'exper', y = 'lwage', lowess=True, marker='x')
plt.ylabel("ln(wage)")
plt.xlabel("Experience")
```

```
Text(0.5, 0, 'Experience')
```



In general there is a huge range of plot types in `seaborn` and we will not explore these at length. We will see a number of examples in tutorial presentations in the week ahead, and I would highly recommend that you explore the online support for seaborn and pyplot, linked above, (or the graphics library for the language you are using) to get a feel for the full diversity of plots available. In general, it is quite simple to move from basic ideas like scatter plots to more complete visualisations, such as weighted scatter plots with varying hues (see below). While this is just a single example, it provides us with an idea of how easy it is to generate plots that look reasonable in just a few lines of code.

```
cmap = sns.color_palette("viridis", as_cmap=True)
mroz = mroz.rename(columns={"educ": "Education", "hours": "Hours Worked"})
sns.relplot(data=mroz,x="exper", y="wage",hue="Education",
            size="Hours Worked",palette=cmap)
plt.grid(True)
plt.ylabel("Wage per hour (USD)")
plt.xlabel("Experience")
note_text = "Note: Data is provided by Mroz (1987)."
plt.annotate(note_text, xy=(0.2, -0.2), xycoords="axes fraction", fontsize=10, color="gray",
plt.show()
```



Note: Data is provided by Mroz (1987).

29.2 A Small Aside on Printing

When you export graphs for saving, there are multiple formats which can be used. These include **raster** formats such as png, jpeg, or **vector** formats such as eps and pdf. For our work, we will nearly always want to use vector formats. Vector graphics formats scale effectively with zooming, and for this reason when we include graphics in a pdf file, it is preferable to always include vector formats rather than raster formats which become pixelated when observing at different levels of zoom than the size in which they were originally created. In general, for papers journals will always request vector formats, and in your code I would encourage you to always use vector formats unless there is some specific reason to use raster formats. Further details related to these issues can be found at the following link: https://www.geeksforgeeks.org/vector-vs-raster-graphics/.

Testing your knowledge

To test your knowledge and work on automating processes, consider the following example:
Open some dataset and generate a histogram of each numerical variable. To do so, you will likely wish to loop through all variables and confirm that they are numerical variables before plotting a histogram, or simply generate a list of all numerical variables and loop through this, within each loop plotting the variable. For a further challenge, you may wish to first test if the variable has more than two levels prior to generating a histogram. Save each file on disk to examine output.

30 A Worked Example of Something More Complex

To examine some of the details of plotting in more depth we will consider a worked example. We will discuss this example at length in class. In particular, here we want to explore the mapping capabilities of `Python`, and want to build up a map documenting three particular elements. This map seeks to visualise the exposure to wildfires, and contains three elements:

- A layer showing the distance to the nearest fire (map colours)
- A layer showing wind directions (black arrows)
- A layer showing the location and size of fires (orange circles)

In general each of the languages that we are working with has good mapping capabilities. There are computational tools specifically for mapping if you wish to be working in very advanced mapping techniques such as qgis and arqgis, but as you will see below `Python`'s native mapping tools can do quite a lot by themselves.

We will discuss this example at length in class, and work through the code line by line. Data for you to follow along is available here. In this link, I simply provide the data, README and a reduced set of code which replicates the key elements of what we do below. The code below is more comprehensive, as it builds up our understanding of the graphs in a piece-by-piece way.

```
# makeMaps.py                    damiancclarke             yyyy-mm-dd:2022-12-01
#---|----1----|----2----|----3----|----4----|----5----|----6----|----7----|----8
#
#   This file makes maps documenting wind bearings crossed with fire intensities
# and combines these maps into gifs.  This can all be controlled using different
# month spans and time periods in section 3.  This is run using Python 3.8.10,
# but should work with any version of Python 3 provided dependencies are install-
# ed.  All dependencies are listed in section [0] of the code.
#
# To run this code, the location of materials on the users' machine should be s-
# et (as "base") on line 45.
#
# This code can be run using the following at the command line:
```

```
#  python3 -W ignore makeMaps.py
#
# The -W ignore flag is because error messages are thrown when no fires occur in
# a given moment and the map of fires is empty.  If -W ignore is not used these
# messages will simply be printed on the screen.


################################################################################
### [0] Imports
################################################################################
import numpy as np
import pandas as pd
import os

import shapefile as shp
import matplotlib.pyplot as plt
import seaborn as sns
import geopandas as gpd
import pyreadr
from windrose import WindroseAxes

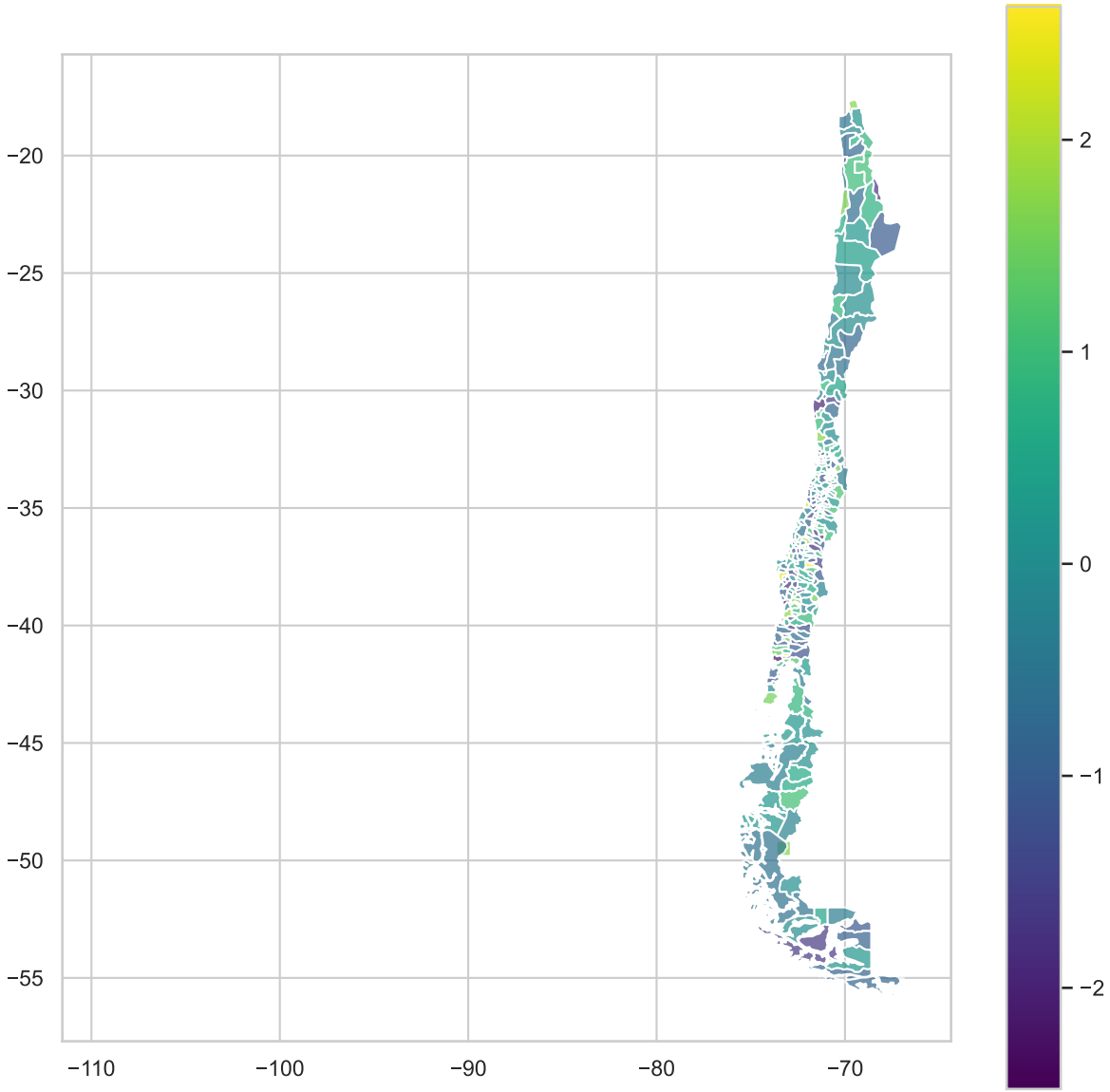
from PIL import Image
import glob
from datetime import date, timedelta

import matplotlib.colors as colors
from matplotlib.patches import Patch

sns.set(style="whitegrid", palette="pastel", color_codes=True)
sns.mpl.rc("figure", figsize=(10,6))

base = "/home/dcc213/trabajo/Teaching/Exeter/ResearchMethods-I/visualisation/"

################################################################################
### [1] First explore a basic map with random data
################################################################################
sf = gpd.read_file(base+"data/comunas.shp")
sf['cvar']=np.random.normal(0,1,len(sf))
sf = sf.to_crs('epsg:4326')
fig, ax = plt.subplots(figsize=(10, 10))
sf.plot(ax=ax,column='cvar',cmap="viridis",legend=True,alpha=0.7)
```



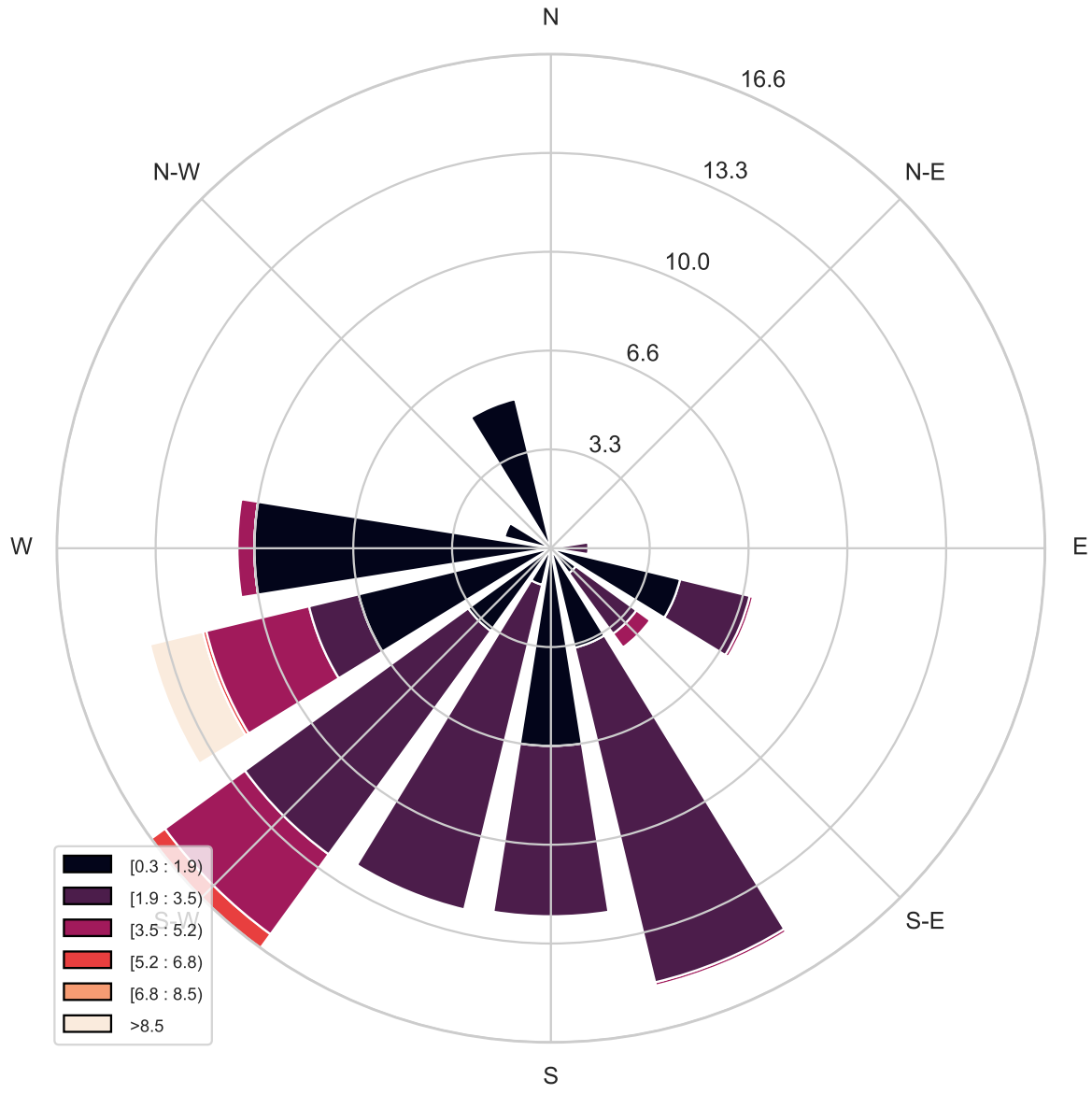

```
################################################################################
### [2] Import data for a particular moment (fires at 2021-01-02, 15:00)
################################################################################
# Read in fires data
sdate = '2021-01-02 15:00:00'
fires = pd.read_stata(base+"data/fires2021.dta")
fires = fires.loc[fires['Date']==sdate]
fires.CUT_2018 = fires['CUT_2018'].astype('int64')
```

```
# Store as a geo pandas data frame for maps below
gdf  = gpd.GeoDataFrame(fires, geometry=gpd.points_from_xy(fires.lo1, fires.la1))

# Read in wind data
wind = pd.read_stata(base+"data/u10_v10_2021.dta")
wind = wind.loc[wind['Date']==sdate]
wind.CUT_2018   = wind['CUT_2018'].astype('int64')
wind.Long       = wind['Long'].astype('float64')
wind.Lat        = wind['Lat'].astype('float64')
wind.viento_u10 = wind['viento_u10'].astype('float64')
wind.viento_v10 = wind['viento_v10'].astype('float64')

# Merge wind and fire: at each municipality what is the wind direction?
# Information on calculating wind direction here:
#   http://colaweb.gmu.edu/dev/clim301/lectures/wind/wind-uv
both = pd.merge(fires,wind,on="CUT_2018", how="left")
both['windangle'] = np.mod(270-np.arctan2(both.viento_u10,both.viento_v10)*180/np.pi,360)
both['windspeed'] = np.sqrt(np.square(both.viento_u10)+np.square(both.viento_v10))
both.head()

# Check wind directions
ax = WindroseAxes.from_ax()
ax.bar(both['windangle'], both['windspeed'], normed=True, opening=0.8, edgecolor="white")
ax.set_legend()
```




```
##############################################################################
### [3] Generate mean distance to fire by municipality and merge with map codes
##############################################################################
print(both.head())
distance = both[['Distancia','CUT_2018']].groupby('CUT_2018').mean()
sf['CUT_2018']=sf.cod_comuna
sf = pd.merge(sf,distance,on="CUT_2018",how="left")
```

```
         la1        lo1  Superficie  beta_grado  gamma_grado  CUT_2018  \
0 -41.467222 -72.895833        0.01   81.276543   261.251465     10101
1 -41.467222 -72.895833        0.01   99.646782   279.271759     10106
2 -41.467222 -72.895833        0.01  156.026993   335.968292     10109
3 -41.467222 -72.895833        0.01   29.648378   209.494522     10102
4 -41.467222 -72.895833        0.01  163.370178   343.282928     10105

   duration               Date_x     Distancia  Long   Lat  \
0      1.55  2021-01-02 15:00:00   3492.854736 -72.9 -41.4
1      1.55  2021-01-02 15:00:00  47915.296875 -73.4 -41.3
2      1.55  2021-01-02 15:00:00  18231.113281 -72.9 -41.3
3      1.55  2021-01-02 15:00:00  39030.343750 -72.9 -41.4
4      1.55  2021-01-02 15:00:00  38455.136719 -73.0 -41.1

                Date_y  viento_v10        Comuna  viento_u10   windangle  \
0  2021-01-02 15:00:00    3.391291  Puerto Montt    0.738311  257.717907
1  2021-01-02 15:00:00    3.594998   Los Muermos    1.002831  254.413469
2  2021-01-02 15:00:00    3.907526  Puerto Varas    1.241126  252.378870
3  2021-01-02 15:00:00    3.391291       Calbuco    0.738311  257.717907
4  2021-01-02 15:00:00    4.986442     Frutillar    1.558664  252.641778

   windspeed
0   3.470729
1   3.732249
2   4.099896
3   3.470729
4   5.224370
```

```
################################################################################
### [4] Build up map with each element
################################################################################
#Use matplotlib's quiver graph to show wind direction and speed
fig, ax = plt.subplots(figsize=(4, 6))
plt.quiver(wind.Long,wind.Lat,wind.viento_u10,wind.viento_v10,
           alpha=0.6, width=0.006)

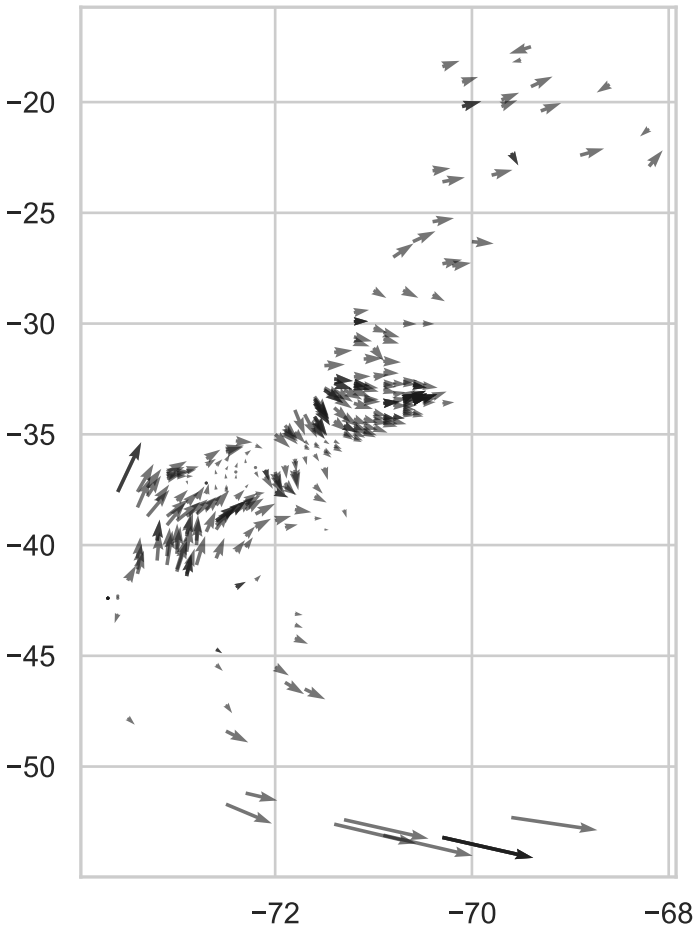

# Generate map with distance to fire
fig, ax = plt.subplots(figsize=(10, 10))
sf.plot(ax=ax,column='Distancia',cmap="viridis",legend=True,alpha=0.7)
```

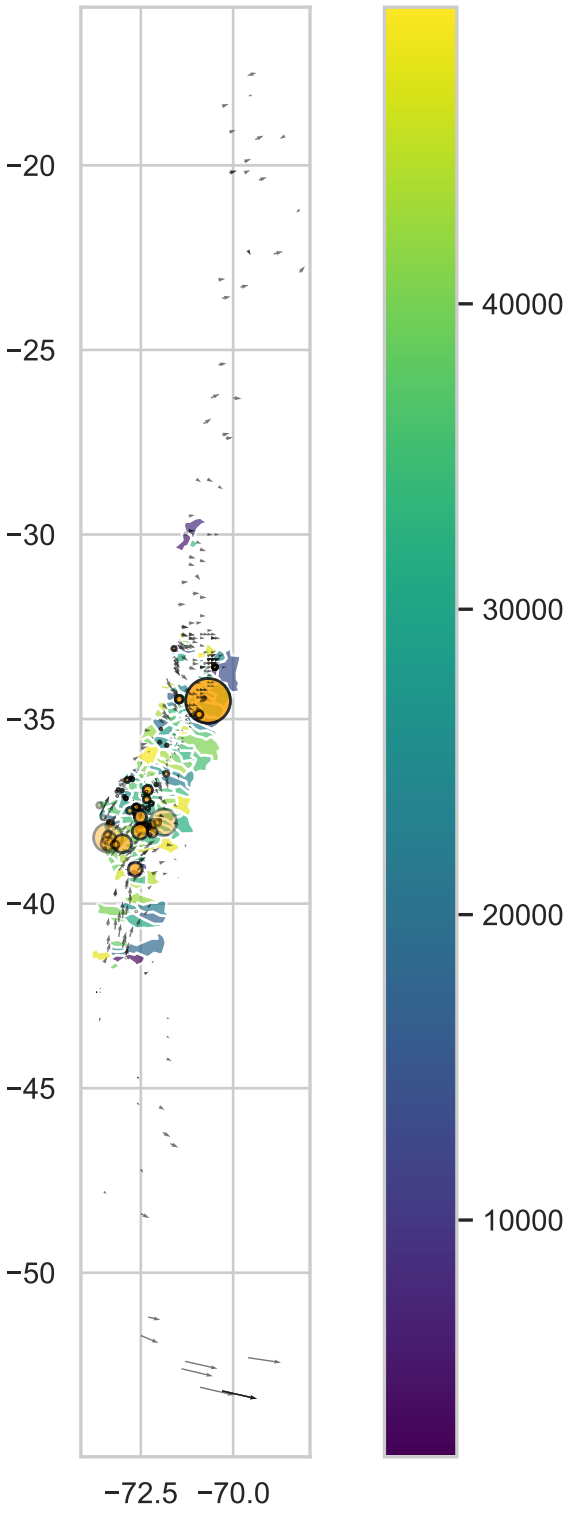
```
# Add fires and wind directions on top
scale=3
f = gdf.plot(ax=ax, color='orange', edgecolor='black',
             alpha=0.1,markersize=(fires.Superficie**0.9)*scale)
plt.quiver(wind.Long,wind.Lat,wind.viento_u10,wind.viento_v10,
           alpha=0.6, width=0.006)

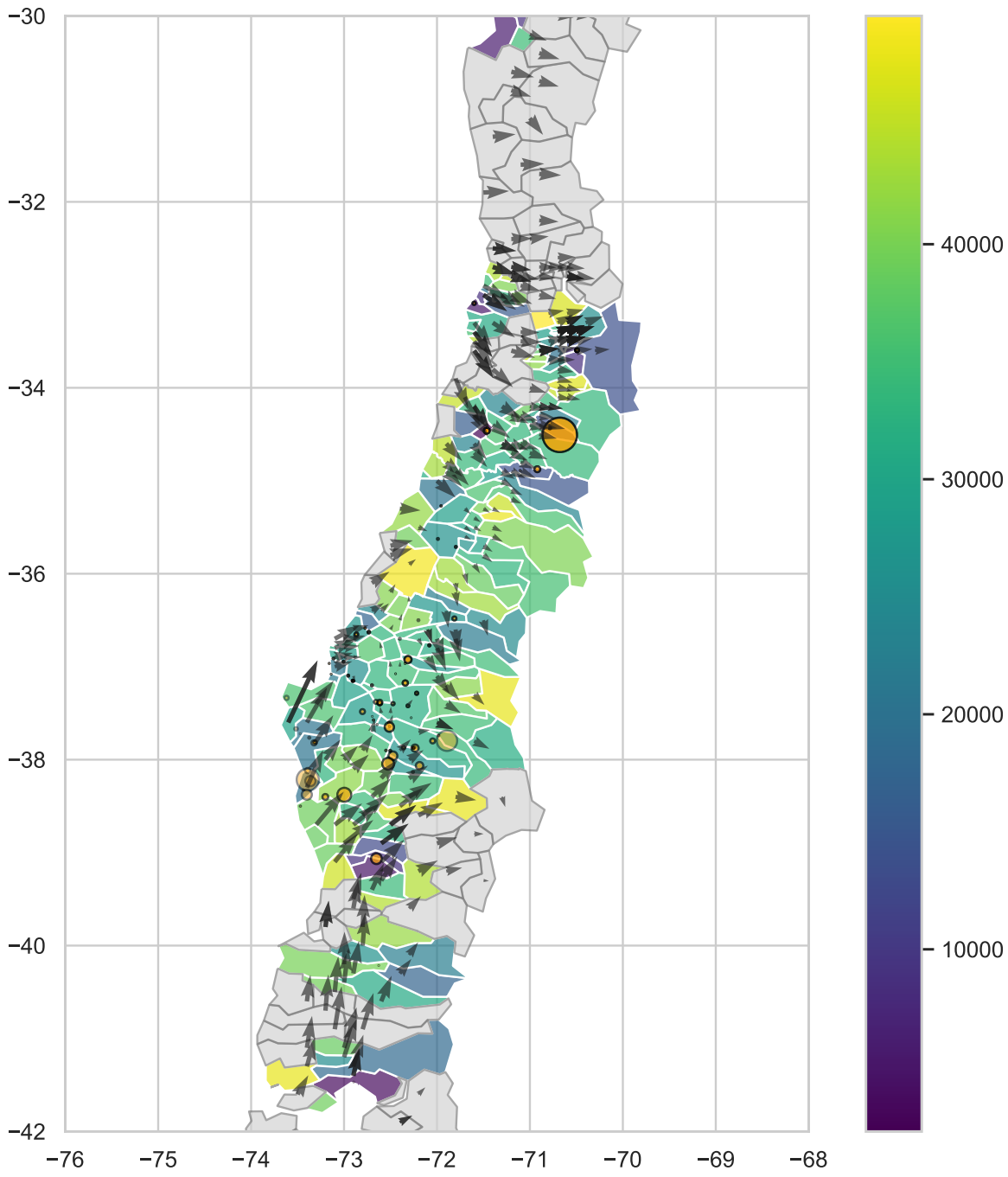

# Add colour map showing distances on side, work out what to do with no fires
fig, ax = plt.subplots(figsize=(10, 10))
sf.plot(ax=ax,column='Distancia',cmap="viridis",
        missing_kwds={
           "color": "lightgrey",
           "edgecolor": "grey",
           "label": "Missing values",
        },
        legend=True,alpha=0.7)

scale=3
f = gdf.plot(ax=ax, color='orange', edgecolor='black',
             alpha=0.1,markersize=(fires.Superficie**0.9)*scale)
plt.quiver(wind.Long,wind.Lat,wind.viento_u10,wind.viento_v10,
           alpha=0.6, width=0.006)

# Focus on particular area
lat=[-76,-68]
lon=[-42,-30]
ax.set_xlim(lat[0],lat[1])
ax.set_ylim(lon[0],lon[1])
```


−20
−25
−30
−35
−40
−45
−50
−72
−70
−68


−20
−25
−30
−35
−40
−45
−50
−72.5
−70.0
40000
30000
20000
10000




```
################################################################################
### [5] Bring this all together and loop over dates to make a gif
################################################################################
```

```
fires = pd.read_stata(base+"data/fires2021.dta")
fires.CUT_2018 = fires['CUT_2018'].astype('int64')
wind = pd.read_stata(base+"data/u10_v10_2021.dta")
wind.CUT_2018   = wind['CUT_2018'].astype('int64')
wind.Long       = wind['Long'].astype('float64')
wind.Lat        = wind['Lat'].astype('float64')
wind.viento_u10 = wind['viento_u10'].astype('float64')
wind.viento_v10 = wind['viento_v10'].astype('float64')

sf = gpd.read_file(base+"data/comunas.shp")
sf = sf.to_crs('epsg:4326')
sf['CUT_2018']=sf.cod_comuna


lat=[-76,-68]
lon=[-42,-30]

imgs = []
for day in range(1,32):
    sday  = str(day).zfill(2)
    sdate = '2021-01-'+sday+' 15:00:00'

    print(sdate)
    firesD = fires.loc[fires['Date']==sdate]
    windD =  wind.loc[wind['Date']==sdate]

    gdf  = gpd.GeoDataFrame(firesD, geometry=gpd.points_from_xy(firesD.lo1, firesD.la1))

    both = pd.merge(firesD,windD,on="CUT_2018", how="left")
    both['windangle'] = np.mod(270-np.arctan2(both.viento_u10,both.viento_v10)*180/np.pi,360)
    both['windspeed'] = np.sqrt(np.square(both.viento_u10)+np.square(both.viento_v10))

    distance = both[['Distancia','CUT_2018']].groupby('CUT_2018').mean()
    sfD = pd.merge(sf,distance,on="CUT_2018",how="left")

    fig, ax = plt.subplots(figsize=(10, 10))
    sfD.plot(ax=ax,column='Distancia',cmap="viridis",vmin=0, vmax=50000,
        missing_kwds={
               "color": "lightgrey",
               "edgecolor": "grey",
               "label": "Missing values",
            },
```

```
            legend=True,alpha=0.7)

    scale=3
    f = gdf.plot(ax=ax, color='orange', edgecolor='black',
                 alpha=0.1,markersize=(firesD.Superficie**0.9)*scale)
    ax.set_title(sdate)

    plt.quiver(windD.Long,windD.Lat,windD.viento_u10,windD.viento_v10,
               alpha=0.6, width=0.006)
    ax.set_xlim(lat[0],lat[1])
    ax.set_ylim(lon[0],lon[1])

    fname = base+'/results/f'+sdate+'.png'
    plt.savefig(fname,bbox_inches='tight')
    imgs.append(fname)
    plt.clf()

frames = []
for i in imgs:
    new_frame = Image.open(i)
    frames.append(new_frame)

fn_save = base+'results/fires.gif'
frames[0].save(fn_save, format='GIF', append_images=frames[1:],
               save_all=True, duration=300, loop=0)
```

```
2021-01-01 15:00:00
2021-01-02 15:00:00
2021-01-03 15:00:00
2021-01-04 15:00:00
2021-01-05 15:00:00
2021-01-06 15:00:00
2021-01-07 15:00:00
2021-01-08 15:00:00
2021-01-09 15:00:00
2021-01-10 15:00:00
2021-01-11 15:00:00
2021-01-12 15:00:00
2021-01-13 15:00:00
2021-01-14 15:00:00
2021-01-15 15:00:00
2021-01-16 15:00:00
```

```
2021-01-17 15:00:00
2021-01-18 15:00:00
2021-01-19 15:00:00
2021-01-20 15:00:00
2021-01-21 15:00:00

/tmp/ipykernel_431698/1443131998.py:39: RuntimeWarning:

More than 20 figures have been opened. Figures created through the pyplot interface (`matplot

2021-01-22 15:00:00
2021-01-23 15:00:00
2021-01-24 15:00:00
2021-01-25 15:00:00
2021-01-26 15:00:00
2021-01-27 15:00:00
2021-01-28 15:00:00
2021-01-29 15:00:00
2021-01-30 15:00:00
2021-01-31 15:00:00

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>
```

```
<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>
```

31 Statistical Tools

31.1 Statistical Analysis

Given the importance of statistical analysis in our work, it is important to be comfortable and familiar with standard routines and procedures in whichever language you use. Here, we will discuss some basic details of conducting statistical analysis with data. We will *not* discuss assumptions underlying models, as you will see this all formally in econometrics classes, but we will discuss important details related to the computational implementation of regression models. We will also discuss the ways to automate work-flow in statistical analysis. This section of the course should be considered as a complement to your formal econometric training, as we will not delve into the many technical details which you require prior to doing any formal statistical work. Rather, we will focus on a few key elements related to computational procedures you may wish to use.

31.1.1 This Class

In this class we will work with Stata, though many of the points we raise will be relevant for any other language. We will also work with data from a household survey. The survey we will use is the UK Labour Force Survey. This is a survey which has been conducted continually since 1973, and is currently conducted in a quarterly fashion. Some background information on the survey can be found at this link. The data which we will use is the July-September quarter from 2022, and the data covers 53,287 individuals from 24,200 households.

The survey data is available from the UK Data Service. This data is available as part of a teaching agreement, and the UK Data Service states:

Students or members of staff at a UK institution of higher or further education (UK HE/FE) can register using the username and password issued by their institution. Information about registration is available at How to register if you are a UK academic user.

Please register to ensure correct use of this data at the following link: https://beta.ukdataservice.ac.uk

Below we will use the principal survey data. Whenever we use survey data it is extremely important that we read the survey documentation. All resources required for this class are provided below:

- Survey data (zipped, Stata format)
- LFS User Guide Volume 1 - Background (zipped, pdf)
- LFS User Guide Volume 2 - Questionnaire 2022 (zipped, pdf)
- LFS User Guide Volume 3 - Variable Details 2022 (zipped, pdf)

31.2 Weighting

Prior to discussing data analysis, a first key element we must consider is survey weighting. Household survey designs are based on probabilistic sampling. Each individual has a given probability of being selected, and every individual who is eventually selected will represent some particular number of observations (for example, individuals of the same age and sex) in the entire population. Of course, this does not mean that a chosen person is *identical* to all other similar people in the population, but rather, this person provides us with information on a given member of a group. It is thus of considerable importance that we consider uncertainty inherent in sampling, and also account for weights should we wish to conduct inference appropriately for the true population of interest.

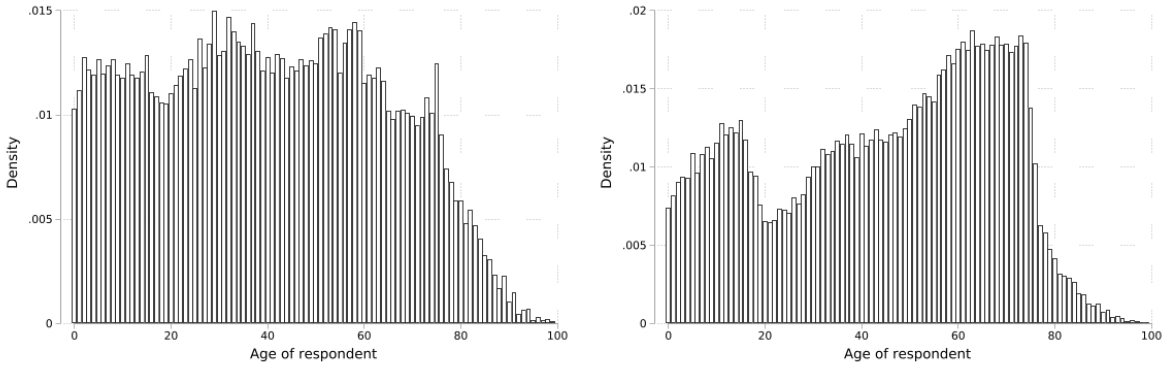
In general, information on weighting and survey design will be discussed at length in survey documentation. For example, in the case of the UK LFS, this is discussed in Section 10 of Volume 1 of the documentation included above. To see why this is important, consider the weighted and unweighted ages of respondents:

```
use lfsp_aj23_eul_pwt22
sum  AGE [w=PWT22]
hist AGE [w=PWT22], d
```

```
    Variable |     Obs      Weight        Mean   Std. dev.       Min        Max
-------------+-----------------------------------------------------------------
         AGE |  51,612    66952258    40.38739   23.59585         0         99
(frequency weights assumed)
(start=0, width=1)
```

```
sum  AGE
hist AGE, d
```

```
    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
         AGE |     53,287    44.64676    23.29989         0         99
(start=0, width=1)
```



(a) Weighted age distribution

(b) Unweighted age distribution

Figure 31.1: Age distributions with and without survey weights.

By examining the historams in Figure 31.1, we can observe that older individuals are over-represented in surveys, and failing to weight will result in the analysis of a population which is quite different to the UK population in general (panel (b) instead of panel (a)). We can also see that when we weight, the "Weight" column returns to us a value of 66,952,258, which is the quantity of individuals the survey represents: essentially the population of the UK in 2023.

In the first code block above we have been somewhat laissez-faire about the way we have written weights. In particular, we have written `[w=PWT22]`. Stata has a number of weighting schemes (frequency weights, analytic weights, and probability weights) which we should use in different settings. In the case of means, these weighting commands provide identical results given that they each weight each observation by the weight provided in PWT22. For this reason, if we are simply interested in a weighted mean, we can simplify things and simply indicate [w]. However, in the case of probabilistic sampling, we *should* use probability weights. Indeed, below when we are estimating regression models we will see that using the wrong type of weighting could lead to **huge mistakes** and so probability weighting will be key. However, in the case of summary statistics via the `sum` command, standard probability weights are not supported for the reasons discussed here. We will return to explore the importance of weighting; and highlight why we **always must use probability weights when working with a survey based on probabilistic sampling** below when exploring regression models.

31.3 Exporting Models

Generally, when we are conducting analysis we will want to present output in our papers. Ideally, this should be done in a way which is completely manual, and avoids any use of copy and paste, or formatting by hand. This is the case for at least two reasons:

1. Manually incorporating results is prone to error
2. If some prior process is changed, we do not wish to have to make changes by hand

For this reason, we will see a tool which allows us to automate the exportation of summary statistic and regression results. There are many such tools in Stata and in other languages, and we will see one example here as an illustration of the types of procedures which should be followed. Specifically, we will explore the estout commands written by Ben Jann. There are also many ways which one could build a work flow, but again, here we will focus on one which is particularly flexible: which is to typeset documents with LaTeX. Please note that the below procedures could be implemented in a largely identical way to export tables in excel format, and include them into word documents.

LaTeX is a type-setting system in which documents are written first in a markup language, and then compiled to generate a pdf. Details of LaTeX can be found in a number of places, including:

- The LaTeX website: https://www.latex-project.org/
- A LaTeX wikibook: https://en.wikibooks.org/wiki/LaTeX

To compile LaTeX, we need to install a LaTeX compiler on our computers, or otherwise have access to a compiler. There are also many ways that this could be done, but two potential options are:

- Install a compiler locally on your machine, such as MiKTeX
- Use an online compiler, such as overleaf

We will spend some time discussing this in class, but to see how we can use LaTeX and estout to automate regression tables, note that a minimum LaTeX document can be written as below. The first part of this code (up to "maketitle") defines the format of the document, and then below we define a table, simply consisting of 6 columns containing column numbers.

```
\documentclass[11pt]{article}

\usepackage[margin=1in]{geometry}
\usepackage{booktabs}

\title{My Title}
\author{My Name}
\date{\today}

\begin{document}
\maketitle

\begin{table}[ht]
```

```
  \caption{My Regression Output}
  \begin{tabular}{lcccccc}
    \toprule
    & (1) & (2) & (3) & (4) & (5) & (6) \\ \midrule
  \end{tabular}
\end{table}

\end{document}
```

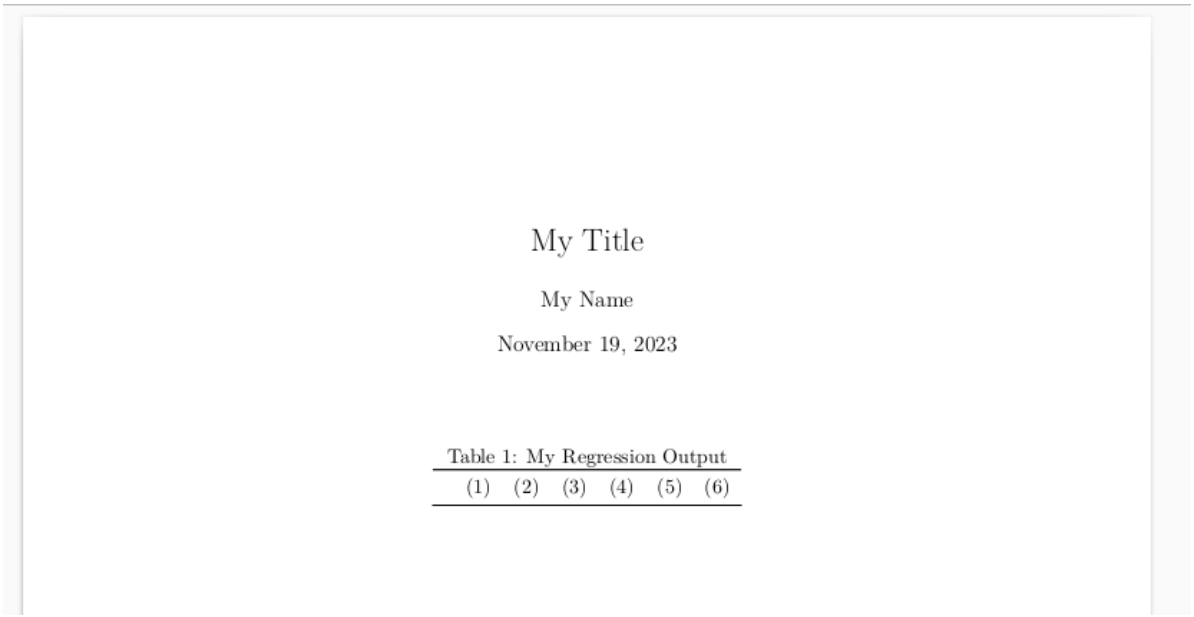


Figure 31.2: A very simple LaTeX document

31.4 Examples

To see how we can incorporate data analysis directly into our work flow, consider the generation of some simple summary statistics from the UK LFS data. We will first open the data below and generate a number of binary measures for educational levels. We will then summarise these variables, along with respondent's ages and gross weekly salary. Finally, we will use the `estout` group of commands to build a summary table.

31.4.1 Summary Statistics

```
use lfsp_aj23_eul_pwt22, clear
gen higherDegree = HIQUAL22==1
gen firstDegree  = HIQUAL22==8
gen ALevel       = HIQUAL22==35
gen OLevel       = HIQUAL22==53

sum AGE higherD firstD ALevel OLevel GRSSWK [w=PWT22] if AGE>=18&AGE<=64

estpost summarize AGE higherD firstD ALevel OLevel GRSSWK [w=PWT22] ///
   if AGE>=18&AGE<=64, listwise
esttab, cells("count mean sd min max") nomtitle nonumber noobs
lab var ALevel       "A Levels Completed"
lab var OLevel       "O Levels Completed"
lab var higherDegree "Completed a Higher Degree"
lab var firstDegree  "Completed a Degree"
lab var GRSSWK       "Gross Weekly Pay"
lab var AGE          "Respondent's Age"
esttab using sumStats.tex, replace cells("count mean sd min max") ///
  nomtitle nonumber noobs label fragment plain collabels("")
```

```
<IPython.core.display.HTML object>
```

```
    Variable |     Obs      Weight        Mean   Std. dev.       Min        Max
-------------+--------------------------------------------------------------------
         AGE |  29,913    40156493    41.23436   13.33035         18         64
higherDegree |  29,913    40156493    .1418944   .3489476          0          1
 firstDegree |  29,913    40156493    .2348255   .4238968          0          1
      ALevel |  29,913    40156493    .0827169   .2754584          0          1
      OLevel |  29,913    40156493     .138363   .3452863          0          1
-------------+--------------------------------------------------------------------
      GRSSWK |  29,913    40156493    136.2767   466.9111         -9      64000
(analytic weights assumed)

             |  e(count)   e(sum_w)    e(mean)     e(Var)      e(sd)
-------------+----------------------------------------------------
         AGE |     29913   4.02e+07   41.23436   177.6982   13.33035
higherDegree |     29913   4.02e+07   .1418944   .1217644   .3489476
 firstDegree |     29913   4.02e+07   .2348255   .1796885   .4238968
```

```
      ALevel |      29913    4.02e+07    .0827169    .0758773    .2754584
      OLevel |      29913    4.02e+07     .138363    .1192226    .3452863
      GRSSWK |      29913    4.02e+07    136.2767    218005.9    466.9111

             |    e(min)     e(max)     e(sum)
-------------+---------------------------------
         AGE |         18         64   1.66e+09
higherDegree |          0          1    5697980
 firstDegree |          0          1    9429769
      ALevel |          0          1    3321619
      OLevel |          0          1    5556171
      GRSSWK |         -9      64000   5.47e+09

------------------------------------------------------------------------------
                    count         mean           sd          min          max
------------------------------------------------------------------------------
AGE                 29913     41.23436     13.33035           18           64
higherDegree        29913     .1418944     .3489476            0            1
firstDegree         29913     .2348255     .4238968            0            1
ALevel              29913     .0827169     .2754584            0            1
OLevel              29913      .138363     .3452863            0            1
GRSSWK              29913     136.2767     466.9111           -9        64000
------------------------------------------------------------------------------
(output written to sumStats.tex)
```

Above we can see generated summary statistics table as Stata output, but it has also been written as a LaTeX table in the document sumStats.tex. Here, this has been done simply as a "fragment" allowing us to incorporate it directly into a table outline that we build in LaTeX. This is done below, where you may note that we are calling the output from Stata by using LaTeX's "input" command:

```
\documentclass[11pt]{article}

\usepackage[margin=1in]{geometry}
\usepackage{booktabs}
\usepackage{fontspec}
\setmainfont{Times New Roman}


\title{My Title}
\author{My Name}
\date{\today}
```

```
\begin{document}
\maketitle

\begin{table}[ht]
  \caption{My Regression Output}
  \begin{tabular}{lccccc}
    \toprule
    & Obs.& Mean & Std.\ Dev. & Min. & Max \\
    \midrule
    \input{./sumStats} \\ \bottomrule
  \end{tabular}
\end{table}


\end{document}
```

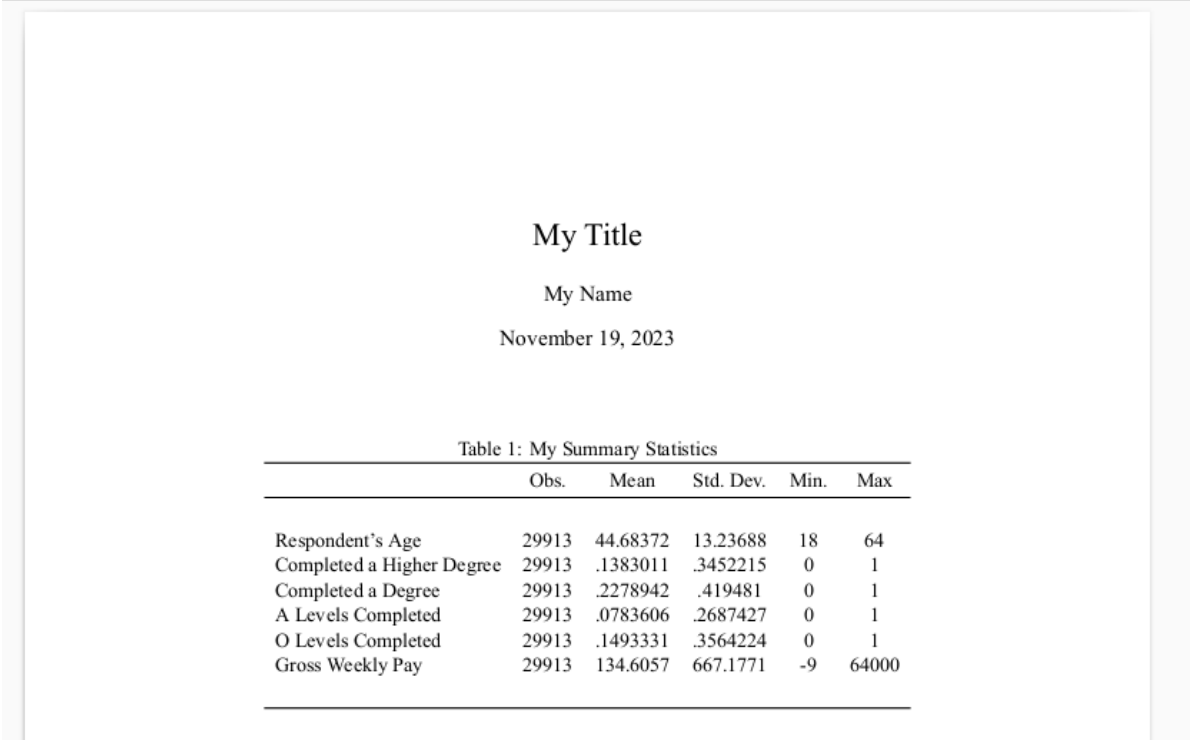


Figure 31.3: LaTeX Document with Summary Statistics

This looks reasonable, though if we wish we can further optimise the table, for example by adding formatted table notes (which we do in LaTeX), and a more appropriate use of decimals

and numerical formatting, which we do within `esttab`. We will discuss the nature of these commands at more length in class. The nice thing with this setup is that now if we wish to make changes to the data, or include further variables, this can be done entirely from within our Stata code, and then we will simply need to re-compile the LaTeX file, as "sumStats2.tex" will be updated everytime we re-run our Stata code, and LaTeX will simply compile with whatever version our code produces.

```
lab var AGE          "Respondent's Age"
esttab using sumStats2.tex, replace ///
  cells("count(fmt(%9.0gc)) mean(fmt(%03.2f)) sd(fmt(%03.2f)) min max") ///
```

```
(output written to sumStats2.tex)
```

```
\documentclass[11pt]{article}

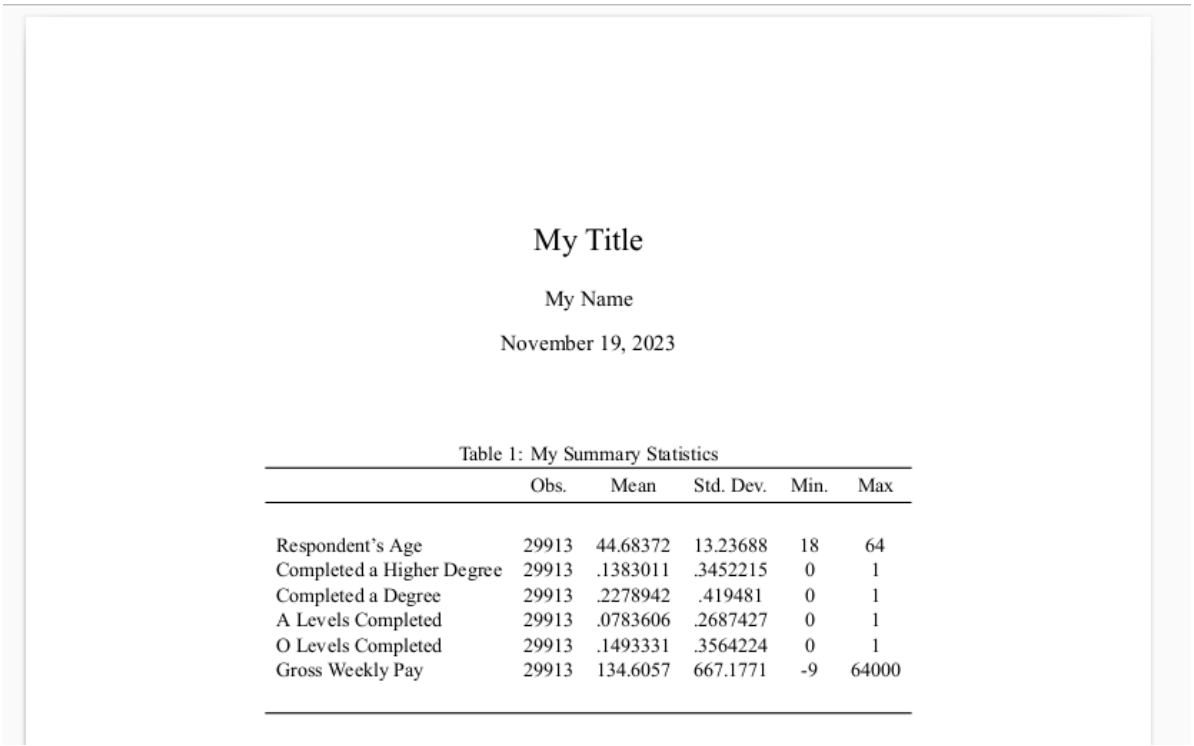
\usepackage[margin=1in]{geometry}
\usepackage{booktabs}
\usepackage{fontspec}
\setmainfont{Times New Roman}


\title{My Title}
\author{My Name}
\date{\today}

\begin{document}
\maketitle

\begin{table}[ht]
  \caption{My Regression Output}
  \begin{tabular}{lccccc}
    \toprule
    & Obs.& Mean & Std.\ Dev. & Min. & Max \\
    \midrule
    \input{./sumStats} \\ \bottomrule
    \multicolumn{6}{p{12cm}}{\footnotesize \textbf{Notes:} Sample consists of all 18--64
    year-olds survered in the July-September Quarterly UK Labour Force Survey. Gross
    weekly pay is measured in pounds.  Educational levels are mutually exclusive, but do
    not cover all possible educational levels.}
\end{tabular}
\end{table}
```

```
\end{document}
```



My Title

My Name

November 19, 2023

Table 1: My Summary Statistics

	Obs.	Mean	Std. Dev.	Min.	Max
Respondent's Age	29913	44.68372	13.23688	18	64
Completed a Higher Degree	29913	.1383011	.3452215	0	1
Completed a Degree	29913	.2278942	.419481	0	1
A Levels Completed	29913	.0783606	.2687427	0	1
O Levels Completed	29913	.1493331	.3564224	0	1
Gross Weekly Pay	29913	134.6057	667.1771	-9	64000

Figure 31.4: LaTeX Document with Summary Statistics, version 2

> **Alternative Implementations**
>
> Search online and explore the help files of esttab or related estout (or other) modules. How can you replicate the table above, but entirely within estout, or other modules?

31.4.2 Linear regression models

It is relatively straightforward to use `estout` (or similar procedures) to report regression results and related statistics. To see this, we will estimate a simplar regression consider how reported gross weekly wage (`GRSSWK`) relates to a number of qualifications, as well as a quadratic term in age. The quadractic term will allow us to capture decreasing returns to age which are generally present in labour market trajectories, where wages grow with experience, but this growth slows over time. Below we open the UK LFS data, and generate a number of binary variables capturing whether an individuals highest qualification is a higher degree, a first

degree, A levels, or O levels. In practice, we would wish to generate an exhaustive list of qualifications such that it was clear that the omitted category was a relatively homogeneous group, but this example will suffice for our application here.

```
use lfsp_aj23_eul_pwt22, clear
gen higherDegree = HIQUAL22==1
gen firstDegree  = HIQUAL22==8
gen ALevel       = HIQUAL22==35
gen OLevel       = HIQUAL22==53
gen AGEsq = AGE^2
lab var ALevel       "A Levels Completed"
lab var OLevel       "O Levels Completed"
lab var higherDegree "Completed a Higher Degree"
lab var firstDegree  "Completed a Degree"
lab var GRSSWK       "Gross Weekly Pay"
lab var AGE          "Respondent's Age"
lab var AGEsq        "Respondent's Age Squared"
reg GRSSWK AGE AGEsq higherD firstD ALevel OLevel [pw=PWT22] if GRSSWK>0, robust
```

```
(sum of wgt is 8,314,366)

Linear regression                               Number of obs     =      6,178
                                                F(6, 6171)        =     119.65
                                                Prob > F          =     0.0000
                                                R-squared         =     0.0705
                                                Root MSE          =     779.68

------------------------------------------------------------------------------
             |               Robust
      GRSSWK | Coefficient  std. err.      t    P>|t|     [95% conf. interval]
-------------+----------------------------------------------------------------
         AGE |   53.73591   3.452162    15.57   0.000     46.96847    60.50335
       AGEsq |  -.5608337   .0401278   -13.98   0.000    -.6394982   -.4821692
higherDegree |    383.781   40.02245     9.59   0.000     305.3231     462.239
 firstDegree |   289.4239   24.89375    11.63   0.000     240.6235    338.2244
      ALevel |   108.7399   24.77144     4.39   0.000     60.17924    157.3005
      OLevel |  -6.640538   21.49649    -0.31   0.757    -48.78116    35.50008
       _cons |  -608.5883    67.1384    -9.06   0.000     -740.203   -476.9737
------------------------------------------------------------------------------
```

After generating the relevant variables, in the final line we estimate a linear regression model weighting by `PWT22` and conditioning on individuals who participated in the labour market in

the prior week. Standard errors are robust to heteroscedasticity in line with standard patterns where wage distributions are more dispersed at higher wages and higher educational levels. Note in this case that had we used frequency weights, standard errors would have (incorrectly) been at least an order of magnitude smaller given that the sample would be considered to be N=8,314,366 instead of N=6,178.

If we wish to export regression results using the `estout` functions, we simply pre-pend `eststo` before our regression command, which results in estimation results being stored as `est1`, `est2` and so forth (in the order which regressions are estimated), and then further processing can be used to report regression estimates. For example, below we simply report the default table on screen using `esttab`:

```
eststo: reg GRSSWK AGE AGEsq                                [pw=PWT22] if GRSSWK>0, robust
eststo: reg GRSSWK AGE AGEsq higherD firstD ALevel OLevel [pw=PWT22] if GRSSWK>0, robust
esttab est1 est2
```

```
(sum of wgt is 8,314,366)

Linear regression                               Number of obs     =      6,178
                                                F(2, 6175)        =     178.44
                                                Prob > F          =     0.0000
                                                R-squared         =     0.0326
                                                Root MSE          =     795.18

------------------------------------------------------------------------------
             |               Robust
      GRSSWK | Coefficient  std. err.      t    P>|t|     [95% conf. interval]
-------------+----------------------------------------------------------------
         AGE |   64.26921   3.439623    18.68   0.000     57.52636    71.01207
       AGEsq |  -.7021534   .0397316   -17.67   0.000    -.7800412   -.6242657
       _cons |  -624.7521   68.92666    -9.06   0.000    -759.8724   -489.6319
------------------------------------------------------------------------------
(est1 stored)
(sum of wgt is 8,314,366)

Linear regression                               Number of obs     =      6,178
                                                F(6, 6171)        =     119.65
                                                Prob > F          =     0.0000
                                                R-squared         =     0.0705
                                                Root MSE          =     779.68

------------------------------------------------------------------------------
             |               Robust
```

```
      GRSSWK | Coefficient  std. err.      t    P>|t|     [95% conf. interval]
-------------+----------------------------------------------------------------
         AGE |   53.73591   3.452162    15.57   0.000     46.96847    60.50335
       AGEsq |  -.5608337   .0401278   -13.98   0.000    -.6394982   -.4821692
higherDegree |    383.781   40.02245     9.59   0.000     305.3231     462.239
 firstDegree |   289.4239   24.89375    11.63   0.000     240.6235    338.2244
      ALevel |   108.7399   24.77144     4.39   0.000     60.17924    157.3005
      OLevel |  -6.640538   21.49649    -0.31   0.757    -48.78116    35.50008
       _cons |  -608.5883    67.1384    -9.06   0.000     -740.203   -476.9737
------------------------------------------------------------------------------
(est2 stored)

--------------------------------------------
                     (1)             (2)
                  GRSSWK          GRSSWK
--------------------------------------------
AGE                64.27***        53.74***
                  (18.68)         (15.57)

AGEsq              -0.702***       -0.561***
                 (-17.67)        (-13.98)

higherDegree                        383.8***
                                   (9.59)

firstDegree                         289.4***
                                  (11.63)

ALevel                              108.7***
                                   (4.39)

OLevel                             -6.641
                                  (-0.31)

_cons              -624.8***       -608.6***
                  (-9.06)         (-9.06)
--------------------------------------------
N                    6178            6178
--------------------------------------------
t statistics in parentheses
* p<0.05, ** p<0.01, *** p<0.001
```

However, further processing can occur and more highly specialised output tables can be pro-

duced, for example tables including specific types of summary statistics, alternative cells, and so forth. Many further types of additional edits can be made as documented at length in the help file and online resources. Below we can see a simple extension addomg labels, editing the types of statistics reported and their format, and reporting standard errors instead of t-statistics. In the following section we will consider a more extensive table: and see how we can incorporate this into LaTeX, combining both linear and non-linear models.

```
esttab est1 est2, label se ///
  stats(N r2, labels("Observations" "R-squared") fmt(%12.0gc %03.2f))
```

```
----------------------------------------------------
                            (1)             (2)
                  Gross Week~y    Gross Week~y
----------------------------------------------------
Respondent's Age           64.27***        53.74***
                         (3.440)         (3.452)

Respondent's Age S~d      -0.702***       -0.561***
                        (0.0397)        (0.0401)

Completed a Higher~e                      383.8***
                                         (40.02)

Completed a Degree                        289.4***
                                         (24.89)

A Levels Completed                        108.7***
                                         (24.77)

O Levels Completed                        -6.641
                                         (21.50)

Constant                  -624.8***       -608.6***
                         (68.93)         (67.14)
----------------------------------------------------
Observations                6,178           6,178
R-squared                    0.03            0.07
----------------------------------------------------
Standard errors in parentheses
* p<0.05, ** p<0.01, *** p<0.001
```

31.4.3 Non-linear regression models

Procedures to export coefficients do not vary – at least in terms of principal syntax, with alternative regression models. However note that estimation procedures may vary depending on models required. Consider, for example, the case of non-linear regression models such as probit or logit. For example, imagine if we wish to estimate similar models to those above, however now modelling the probability an individual works, which we will define as $Works_i = 1$. A probit model assumes a functional form of the following type:

$$\Pr(Works_i = 1) = \Phi\left(\beta_0 + \beta_1 Educ_i + \beta_2 Age_i + \beta_3 Age_i^2\right)$$

Here, $\Phi(\cdot)$ is the normal cumulative density function. In this case, and unlike the case with the standard linear regression model, given that the functional form is not linear in parameters, estimated marginal effects depend on the values of the independent variables at which they are evaluated. For example, if we wish to consider the marginal effect of a change in $Educ_i$, this is written as:

$$\frac{\partial \Pr(Works_i = 1)}{\partial Educ_i} = \beta_1 \cdot \phi(\beta_0 + \beta_1 Educ_i + \beta_2 Age_i + \beta_3 Age_i^2)$$

where it is clear that this marginal effect will depend on the value of $Educ_i$ (as well as Age_i) at which it is evaluated. What's more, consider the marginal effects on the variable Age_i, which itself is expressed as a quadratic term. Now the marginal effect is written as follows:

$$\frac{\partial \Pr(Works_i = 1)}{\partial Age_i} = (\beta_2 + 2\beta_3 Age_i) \cdot \phi(\beta_0 + \beta_1 Educ_i + \beta_2 Age_i + \beta_3 Age_i^2)$$

Fortunately, the reporting of marginal effects from regression models is *relatively* straightforward using `Stata`'s `margins` command. In this case, we simply must indicate at which particular values margins should be reported. Commonly, marginal effects are reported at the means (which consists of inserting sample means in the marginal effects calculations previously), or average marginal effects are reported, which consists of evaluating the marginal effect for each unit at its particular values of the dependent variables, and then averaging over the sample. We can see how this can requested below:

```
gen works = WRKING==1 if WRKING!=-9
probit works AGE AGEsq higherD firstD ALevel OLevel [pw=PWT22] if AGE>=18&AGE<=64
margins , dydx(*) atmeans
```

```
(9,262 missing values generated)

Iteration 0:  Log pseudolikelihood =  -23541159
Iteration 1:  Log pseudolikelihood =  -22057446
Iteration 2:  Log pseudolikelihood =  -22052566
Iteration 3:  Log pseudolikelihood =  -22052566

Probit regression                                     Number of obs =  29,824
                                                      Wald chi2(6)  = 1464.99
                                                      Prob > chi2   =  0.0000
Log pseudolikelihood = -22052566                      Pseudo R2     =  0.0632

------------------------------------------------------------------------------
             |               Robust
       works | Coefficient  std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
         AGE |   .1488948   .0052405    28.41   0.000     .1386236     .159166
       AGEsq |  -.0018368   .0000612   -30.03   0.000    -.0019567    -.001717
higherDegree |   .3391908   .0328549    10.32   0.000     .2747964    .4035852
 firstDegree |   .3331613   .0280152    11.89   0.000     .2782525      .38807
      ALevel |   .0146185   .0370836     0.39   0.693     -.058064     .087301
      OLevel |  -.0270734   .0295603    -0.92   0.360    -.0850106    .0308637
       _cons |  -2.179278   .1039104   -20.97   0.000    -2.382939   -1.975617
------------------------------------------------------------------------------

Conditional marginal effects                          Number of obs = 29,824
Model VCE: Robust

Expression: Pr(works), predict()
dy/dx wrt:  AGE AGEsq higherDegree firstDegree ALevel OLevel
At: AGE          = 41.25188 (mean)
    AGEsq        = 1879.201 (mean)
    higherDegree = .1421379 (mean)
    firstDegree  = .2351449 (mean)
    ALevel       = .0829041 (mean)
    OLevel       = .1380267 (mean)

------------------------------------------------------------------------------
             |            Delta-method
             |      dy/dx   std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
         AGE |   .0485503   .0017105    28.38   0.000     .0451977    .0519029
       AGEsq |  -.0005989   .0000198   -30.20   0.000    -.0006378   -.0005601
```

```
higherDegree |   .1106004    .010692    10.34   0.000     .0896444    .1315564
 firstDegree |   .1086343   .0090989    11.94   0.000     .0908008    .1264679
      ALevel |   .0047667   .0120931     0.39   0.693    -.0189353    .0284686
      OLevel |  -.0088279   .0096389    -0.92   0.360    -.0277198    .0100641
------------------------------------------------------------------------------
```

Here we have followed the same procedures as in the linear regression model, and requested marginal effects at the means based on the margins command. *However*, there is a problem here! In this case, Stata has no way to know that the variable `AGEsq` is related to `AGE`. Indeed, if we examine the output of margins, we see that the mean of AGE is reported as 41.25188, which would imply that the corresponding value of mean squared should be $41.25188^2 = 1701.718$. For this reason, we need to be careful when requesting marginal effects when variables themselves enter non-linearly in our specification. This can be resolved easily as below, where this time we use *the same varible*, and Stata's inbuilt interaction command to estimate our probit:

```
probit works AGE c.AGE#c.AGE higherD firstD ALevel OLevel [pw=PWT22] if AGE>=18&AGE<=64
margins , dydx(*) atmeans
```

```
Iteration 0:  Log pseudolikelihood =  -23541159
Iteration 1:  Log pseudolikelihood =  -22057446
Iteration 2:  Log pseudolikelihood =  -22052566
Iteration 3:  Log pseudolikelihood =  -22052566

Probit regression                                     Number of obs =  29,824
                                                      Wald chi2(6)  = 1464.99
                                                      Prob > chi2   =  0.0000
Log pseudolikelihood = -22052566                      Pseudo R2     =  0.0632

------------------------------------------------------------------------------
             |               Robust
       works | Coefficient  std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
         AGE |   .1488948   .0052405    28.41   0.000     .1386236     .159166
             |
 c.AGE#c.AGE |  -.0018368   .0000612   -30.03   0.000    -.0019567    -.001717
             |
higherDegree |   .3391908   .0328549    10.32   0.000     .2747964    .4035852
 firstDegree |   .3331613   .0280152    11.89   0.000     .2782525      .38807
      ALevel |   .0146185   .0370836     0.39   0.693     -.058064     .087301
      OLevel |  -.0270734   .0295603    -0.92   0.360    -.0850106    .0308637
```

```
       _cons |  -2.179278   .1039104   -20.97   0.000    -2.382939   -1.975617
------------------------------------------------------------------------------

Conditional marginal effects                              Number of obs = 29,824
Model VCE: Robust

Expression: Pr(works), predict()
dy/dx wrt:  AGE higherDegree firstDegree ALevel OLevel
At: AGE          = 41.25188 (mean)
    higherDegree = .1421379 (mean)
    firstDegree  = .2351449 (mean)
    ALevel       = .0829041 (mean)
    OLevel       = .1380267 (mean)

------------------------------------------------------------------------------
             |            Delta-method
             |      dy/dx   std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
         AGE |  -.0006666   .0001808    -3.69   0.000     -.001021   -.0003122
higherDegree |   .0852614   .0083984    10.15   0.000      .068801    .1017219
 firstDegree |   .0837458   .0071711    11.68   0.000     .0696908    .0978008
      ALevel |   .0036746   .0093199     0.39   0.693     -.014592    .0219413
      OLevel |  -.0068054   .0074363    -0.92   0.360    -.0213803    .0077696
------------------------------------------------------------------------------
```

We now see that margins (correctly) returns us a single marginal effect for the variable **AGE**, and this has implications for marginal effects on other variables too. Once we have properly accounted for the use of the margins command, we can finally bring this all together. Let's imagine that along with the estimates we previously generated for the linear regression model varying controls we wished to present similar marginal effects at the mean for probit models examining the likelihood that an individual works: effectively examining extensive and intensive margin returns to characteristics on the labour market. Let's add these models to our stored estimates below, and export a table containing *just* coefficients, standard errors, and some basic statistics:

```
qui eststo: reg GRSSWK AGE c.AGE#c.AGE                              [pw=PWT22] if GRSSWK>0, r
qui eststo: reg GRSSWK AGE c.AGE#c.AGE higherD firstD ALevel OLevel [pw=PWT22] if GRSSWK>0, r

probit works AGE c.AGE#c.AGE                               [pw=PWT22] if AGE>=18&AGE<=64
eststo: margins , dydx(*) atmeans

probit works AGE c.AGE#c.AGE higherD firstD ALevel OLeve [pw=PWT22] if AGE>=18&AGE<=64
```

```
eststo: margins , dydx(*) atmeans

esttab est1 est2 est3 est4 using labourMarketCorrelates.tex, replace ///
  label se stats(N, labels("\\ Observations") fmt(%12.0gc)) fragment ///
  nomtitles nonumbers booktabs eqlabels(" ") nolines nogaps
estimates clear
```

```
Iteration 0:  Log pseudolikelihood =  -23541159
Iteration 1:  Log pseudolikelihood =  -22315860
Iteration 2:  Log pseudolikelihood =  -22314249
Iteration 3:  Log pseudolikelihood =  -22314249

Probit regression                                     Number of obs =  29,824
                                                      Wald chi2(2)  = 1288.17
                                                      Prob > chi2   =  0.0000
Log pseudolikelihood = -22314249                      Pseudo R2     =  0.0521

------------------------------------------------------------------------------
             |               Robust
       works | Coefficient  std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
         AGE |   .1651731   .0051269    32.22   0.000     .1551245    .1752217
             |
 c.AGE#c.AGE |  -.0020401   .0000599   -34.05   0.000    -.0021575   -.0019227
             |
       _cons |  -2.354015   .1008697   -23.34   0.000    -2.551716   -2.156314
------------------------------------------------------------------------------

Conditional marginal effects                          Number of obs = 29,824
Model VCE: Robust

Expression: Pr(works), predict()
dy/dx wrt:  AGE
At: AGE = 41.25188 (mean)

------------------------------------------------------------------------------
             |            Delta-method
             |      dy/dx   std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
         AGE |   -.000769   .0001725    -4.46   0.000    -.0011071   -.0004308
------------------------------------------------------------------------------
```

```
(est5 stored)

Iteration 0:  Log pseudolikelihood =  -23541159
Iteration 1:  Log pseudolikelihood =  -22057446
Iteration 2:  Log pseudolikelihood =  -22052566
Iteration 3:  Log pseudolikelihood =  -22052566

Probit regression                                       Number of obs =  29,824
                                                        Wald chi2(6)  = 1464.99
                                                        Prob > chi2   =  0.0000
Log pseudolikelihood = -22052566                        Pseudo R2     =  0.0632

------------------------------------------------------------------------------
             |               Robust
       works | Coefficient  std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
         AGE |   .1488948   .0052405    28.41   0.000     .1386236     .159166
             |
 c.AGE#c.AGE |  -.0018368   .0000612   -30.03   0.000    -.0019567    -.001717
             |
higherDegree |   .3391908   .0328549    10.32   0.000     .2747964    .4035852
 firstDegree |   .3331613   .0280152    11.89   0.000     .2782525      .38807
      ALevel |   .0146185   .0370836     0.39   0.693     -.058064     .087301
      OLevel |  -.0270734   .0295603    -0.92   0.360    -.0850106    .0308637
       _cons |  -2.179278   .1039104   -20.97   0.000    -2.382939   -1.975617
------------------------------------------------------------------------------

Conditional marginal effects                            Number of obs = 29,824
Model VCE: Robust

Expression: Pr(works), predict()
dy/dx wrt:  AGE higherDegree firstDegree ALevel OLevel
At: AGE          = 41.25188 (mean)
    higherDegree = .1421379 (mean)
    firstDegree  = .2351449 (mean)
    ALevel       = .0829041 (mean)
    OLevel       = .1380267 (mean)

------------------------------------------------------------------------------
             |            Delta-method
             |      dy/dx   std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
         AGE |  -.0006666   .0001808    -3.69   0.000     -.001021   -.0003122
```

```
higherDegree |   .0852614   .0083984    10.15   0.000      .068801    .1017219
 firstDegree |   .0837458   .0071711    11.68   0.000     .0696908    .0978008
      ALevel |   .0036746   .0093199     0.39   0.693     -.014592    .0219413
      OLevel |  -.0068054   .0074363    -0.92   0.360    -.0213803    .0077696
------------------------------------------------------------------------------
(est6 stored)
(output written to labourMarketCorrelates.tex)
```

We will talk through this syntax more in class, but if we examine that output, we will find that we have essentially just exported the "innards" of a table, and so can build the "structure" of the table that we would like in LaTeX. If we want, we *could* do this entirely in `esttab` (this is left as an exercise), but below we can see how it could finally be brought together in a simple LaTeX document:

```
\documentclass[11pt]{article}

\usepackage[margin=1in]{geometry}
\usepackage{booktabs}
\usepackage{fontspec}
\setmainfont{Times New Roman}


\title{My Title}
\author{My Name}
\date{\today}

\begin{document}
\maketitle

\begin{table}[ht]
  \caption{My Regression Output}
  \begin{tabular}{lcccc}
    \toprule
    & \multicolumn{2}{c}{Linear Model} & \multicolumn{2}{c}{Non-linear Models}  \midrule(r){1
    & Wage & Wage & Works & Works \\
    & (1)  & (2)  & (3) & (4) \\ \midrule
    \input{labourMarketCorrelates.tex}
    \\ \bottomrule
    \multicolumn{5}{p{15.6cm}}{\footnotesize \textbf{Notes:} Each column presents marginal
    effects from regressions of weekly wages (columns 1-2) or having worked (columns 3-4)
    on individual characteristics.  Sample consists of all 18--64 year-olds survered in the
    July-September Quarterly UK Labour Force Survey. Gross weekly pay is measured in pounds.
```

```
    Educational levels are mutually exclusive, but do not cover all possible educational leve
    Non-linear models are estimated using a probit, with marginal effects reported at the mea
    $^{***}$: p<0.001; $^{**}$: p<0.01; $^{*}$: p<0.05.}
\end{tabular}
\end{table}


\end{document}
```

Compilation gives a reasonable-looking document with regression output (refer to image below). This could be further refined in a number of ways if desired.

My Title

My Name

November 20, 2025

Table 1: My Regression Output

	Linear Model		Non-linear Models	
	Wage	Wage	Works	Works
	(1)	(2)	(3)	(4)
Respondent's Age	64.27***	53.74***	0.165***	0.149***
	(3.440)	(3.452)	(0.00513)	(0.00524)
Respondent's Age × Respondent's Age	-0.702***	-0.561***	-0.00204***	-0.00184***
	(0.0397)	(0.0401)	(0.0000599)	(0.0000612)
Completed a Higher Degree		383.8***		0.339***
		(40.02)		(0.0329)
Completed a Degree		289.4***		0.333***
		(24.89)		(0.0280)
A Levels Completed		108.7***		0.0146
		(24.77)		(0.0371)
O Levels Completed		-6.641		-0.0271
		(21.50)		(0.0296)
Constant	-624.8***	-608.6***	-2.354***	-2.179***
	(68.93)	(67.14)	(0.101)	(0.104)
Observations	6,178	6,178	29,824	29,824

Notes: Each column presents marginal effects from regressions of weekly wages (columns 1-2) or having worked (columns 3-4) on individual characteristics. Sample consists of all 18–64 year-olds surveyed in the July-September Quarterly UK Labour Force Survey. Gross weekly pay is measured in pounds. Educational levels are mutually exclusive, but do not cover all possible educational levels. Non-linear models are estimated using a probit, with marginal effects reported at the mean. ***: p<0.001; **: p<0.01; *: p<0.05.

Figure 31.5: LaTeX Document with Regression Output

32 Simulations as Part of Research Workflow

32.1 Introduction

In this class we will examine a number of cases in which we wish to use either simulation or processes based on randomisation within our research. Simulation in particular can be a useful way to examine the performance of a particular estimator. Generally, when we are working with real data and some estimator of interest, we can not separate out the process of testing our estimator from any identifying assumptions we may have. Given that we do not know the real population value that we wish to recover, we cannot test our estimator in a controlled way. Simulation, and Monte Carlo methods, allow us to examine how a particular estimator performs under some controlled circumstance where we *define* the data generating process (DGP), and hence where we *know* the value that the estimator should return if we repeat simulation many times.

In general, if we are using standard estimators in commercial or highly used software in our research we probably do not need to worry that the underlying estimation routines may be incorrect. However, if we are writing our own estimators, this process of simulating to see how an estimator peforms when the DGP is known will be a fundamental part of testing our code.

In this class we will begin by considering a simple example of such a process which seeks to illustrate the idea of such a test. We will then move on to consider a number of other processes that we may encounter in which randomisation is used in our research: specifically bootstrap and randomisation inference procedures.

32.2 Monte Carlo Simulations to Examine Estimators

To see a simple example of Monte Carlo simulations, we will use the following data generating process:

$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$

where $\beta_0 = 1$, $\beta_1 = 2$, and both x_i and ε_i are distributed $\mathcal{N}(0, 1)$. Having conducted this simulation, we will estimate a regression model to estimate $\widehat{\beta}_1$. We will then see if our estimator $\widehat{\beta}_1$ recovers the true population parameter β_1. This is not as elaborate as a procedure when we may wish to incorporate Monte Carlo simulations in our own research, mainly because the

estimator itself is quite simple (OLS), and we already know the properties. But it is worth confirming to ourselves that we understand the mechanics of this so that we can extend to more complex cases if desired.

Below we start writing code for a single simulation in Python. We will use numpy for pseudo-random number generation, and scikit-learn for regression estimation. We can see in this particular example that we estimate a value for $\widehat{\beta}_1$ of 2.09: quite close to the true "population" parameter.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Generate independent variable (X) and dependent variable (Y)
np.random.seed(1213)
N = 1000
x = np.random.rand(N, 1)  # 1000 observations of a single independent variable
y = 1 + 2*x + np.random.randn(N, 1)  # Linear relationship with some noise


data = pd.DataFrame({'x': x[:, 0], 'y': y[:, 0]})
#Could also do:
#data = pd.DataFrame({'x': x.flatten(), 'y': y.flatten()})

# Create a LinearRegression model
model = LinearRegression()

# Fit the model to the data
model.fit(data[['x']], data[['y']])

# Coefficients
beta1hat = model.coef_[0][0]
beta0hat = model.intercept_[0]

print(f"beta 1 (Coefficient): {beta1hat}")
print(f"Intercept: {beta0hat}")
```

```
beta 1 (Coefficient): 2.0948413011297347
Intercept: 0.9734410595829046
```

We can visualise the particular simulated relationship in the plot below:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Make predictions using the model
data['yhat'] = model.predict(data[['x']])

# Create a scatter plot with regression line
plt.figure(figsize=(8, 6))
sns.scatterplot(x='x', y='y', data=data, label='Data Points')
sns.lineplot(x='x', y='yhat', data=data, color='red', label='Regression Line')
plt.xlabel(r'$x$')
plt.ylabel(r'$y$')
plt.legend()
plt.grid(True)
plt.show()
```

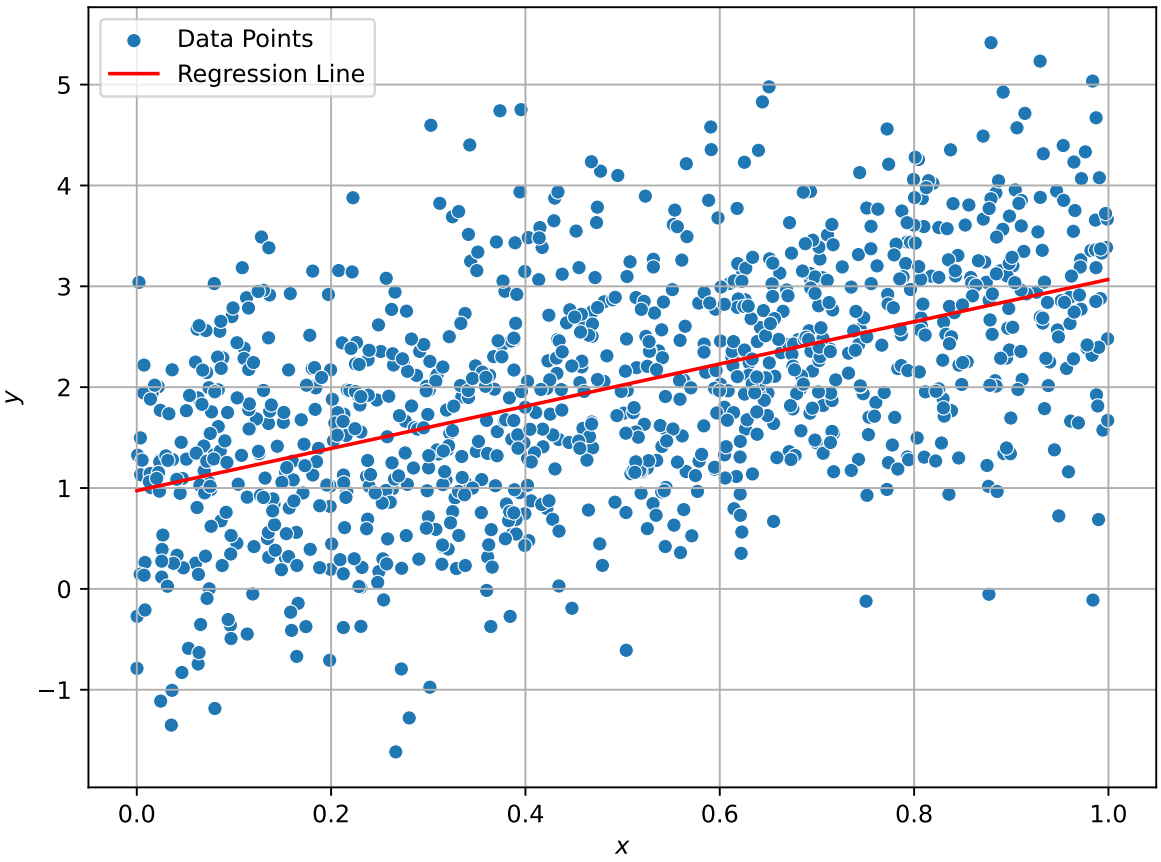


Figure 32.1: Two-way scatter plot with regression model

Now we can consider the Monte Carlo simulations themselves. Here, instead of estimating a single value for $\hat{\beta}_1$ we can ask how our estimator performs across repeated sampling. Below we will repeat the process from above 500 times, in each iteration (with each different realisation of data from the underlying DGP), saving the resulting parameter. Based on this we can see what the distribution of estimated paramters $\hat{\beta}_1$ looks like:

```
#Now do this 500 times to see distribution of beta hat
# Create an empty list to store slope coefficients
beta1hats = []

# Perform the simulations and regressions 500 times
for _ in range(500):
    # Generate synthetic data
    x = np.random.rand(N, 1)
    y = 1 + 2 *x + np.random.randn(N, 1)
```

```
    # Create a DataFrame
    data = pd.DataFrame({'x': x[:, 0], 'y': y[:, 0]})

    # Create a LinearRegression model and fit it to the data
    model = LinearRegression()
    model.fit(data[['x']], data[['y']])

    # Extract the slope coefficient and store it
    beta1hats.append(model.coef_[0][0])

# Calculate mean estimate of beta 1 hat
print('Mean coefficient estimate is: ' + str(np.mean(beta1hats)))

# Create a histogram of the slope coefficients
plt.figure(figsize=(8, 6))
plt.hist(beta1hats, bins=30, edgecolor='black', label='Estimates')
plt.axvline(2, color='red', linestyle='dashed', label=r'$\beta_1$')
plt.xlabel(r'$\widehat\beta_1$')
plt.ylabel('Frequency')
plt.legend()
plt.grid(True)
plt.show()
```

```
Mean coefficient estimate is: 2.0095203120360727
```

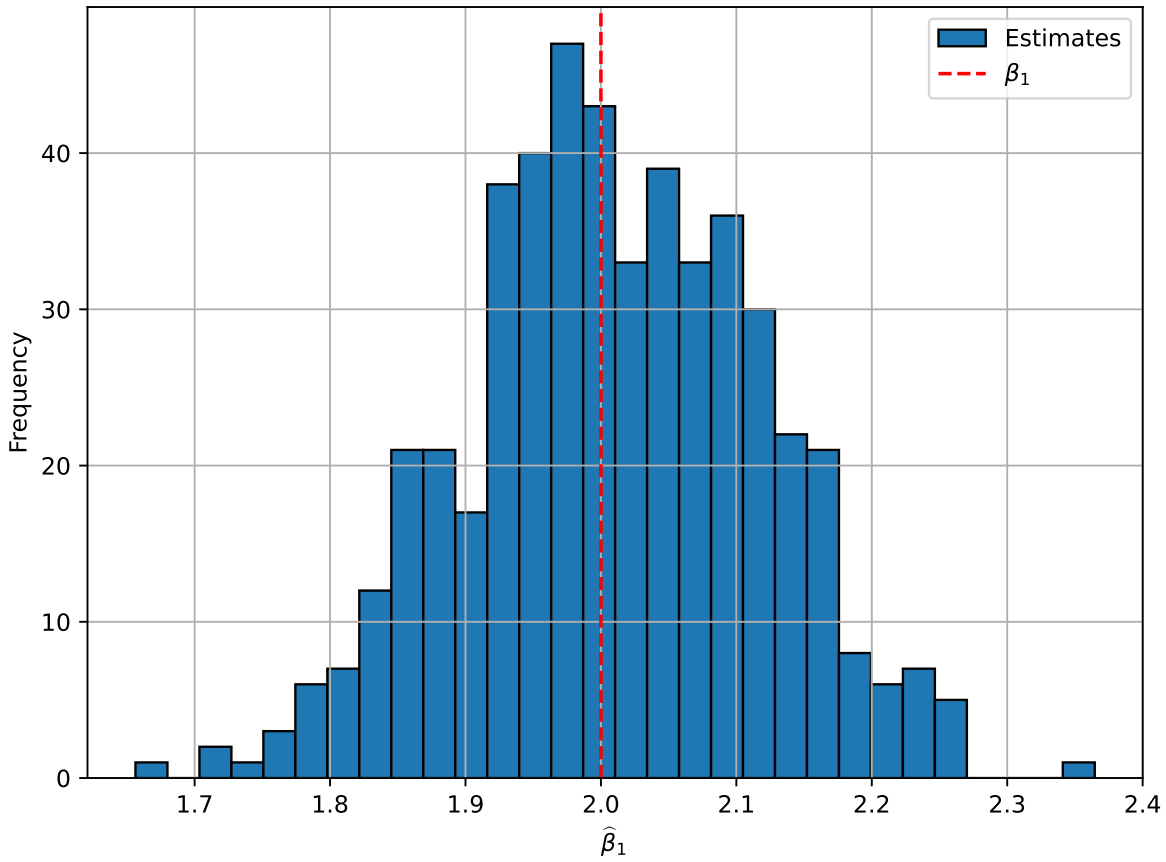


Figure 32.2: Regression estimates in 500 simulations

We will discuss this code at more length in class. Based on this code we could incorporate other elements, such as calculate the number of times which we reject a null hypothesis of the sort $H_0 : \beta_1 = 2$ (which should be approximately 5% if we set $\alpha = 0.05$), or considering the power of the test of the null that β_1 equals some other values apart from the true value. Such extensions may be interesting to work on if you wish to further explore this code.

32.3 Alternative Methods Based on Pseudo-Random Draws

In practice, Monte Carlo simulation is just one of a number of cases where we may use pseudo-randomness in our research. Here we will consider a number of other cases. This includes the use of bootstrapping to generate standard errors, and the use of permutation inference. We turn to these in turn.

32.3.1 Bootstrap Inference

The idea of bootstraping is that we can get at the variation of a parameter through resampling. For example, if we do not wish to estimate an analytic standard error, we can estimate the standard error of an estimate by resampling with replacement many times from our original data, in each case estimating the parameter of interest based on the "new" (resampled) data. This procedure was initially proposed by Bradly Efron in his doctoral thesis, and formal deatils can be found in, among many other places, Efron and Tibshirani (1993). The below figure describes bootstrapping in a schematic way.

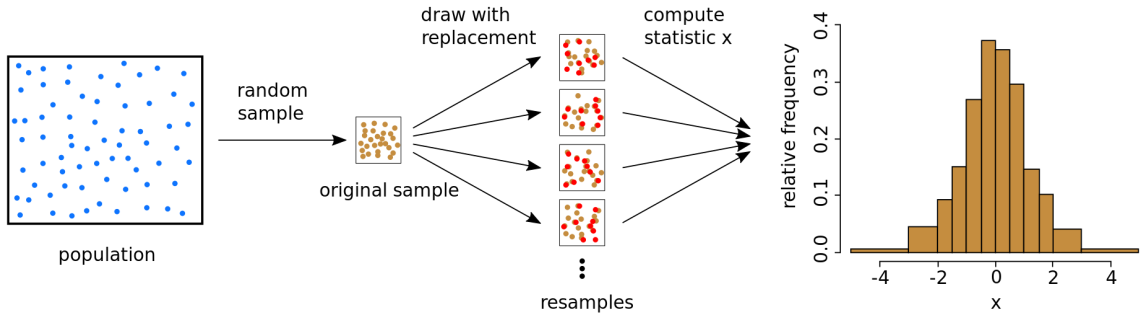


Figure 32.3: Bootstrapping Illustration (Source, Biggerj, Marsupilami, CC BY-SA Licence)

To explore bootstrapping, we will return to the data we have used multiple times throughout this course, and estimate a simple regression of salary on education and experience. Below we estimate the regressino based on the true data, using `statsmodels` this time, to easily access the parameter standard error for reference.

```
import wooldridge
import pandas as pd
import statsmodels.api as sm

mroz = wooldridge.data('mroz')

salaryData = mroz.loc[mroz['lwage'].notna()].copy()
salaryData['exper2'] = salaryData['exper']**2

X = salaryData[['educ', 'exper', 'exper2']]
X = sm.add_constant(X)

model = sm.OLS(salaryData['lwage'], X).fit()
print(model.summary())
```

```
print(model.params['educ'])
print(model.bse['educ'])
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  lwage   R-squared:                       0.157
Model:                            OLS   Adj. R-squared:                  0.151
Method:                 Least Squares   F-statistic:                     26.29
Date:                Thu, 27 Nov 2025   Prob (F-statistic):           1.30e-15
Time:                        17:00:50   Log-Likelihood:                -431.60
No. Observations:                 428   AIC:                             871.2
Df Residuals:                     424   BIC:                             887.4
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         -0.5220      0.199     -2.628      0.009      -0.912      -0.132
educ           0.1075      0.014      7.598      0.000       0.080       0.135
exper          0.0416      0.013      3.155      0.002       0.016       0.067
exper2        -0.0008      0.000     -2.063      0.040      -0.002   -3.82e-05
==============================================================================
Omnibus:                       77.792   Durbin-Watson:                   1.961
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              300.917
Skew:                          -0.753   Prob(JB):                     4.54e-66
Kurtosis:                       6.822   Cond. No.                     2.21e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.21e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
0.10748964014881374
0.01414647832512198
```

The above parameters are the regression estimate, and the analytic standard error. However, if we would like to calculate the *bootstrap* version of this standard error, we can resample many times from the original data, in each case estimating a new version of $\widehat{\beta}_1$, which we will call $\widehat{\beta}_1^*$. The bootstrap standard error then is simply defined as std. dev$(\widehat{\beta}_1^*)$. Below we do this using 1,000 bootstrap replicates.

```
mroz = wooldridge.data('mroz')

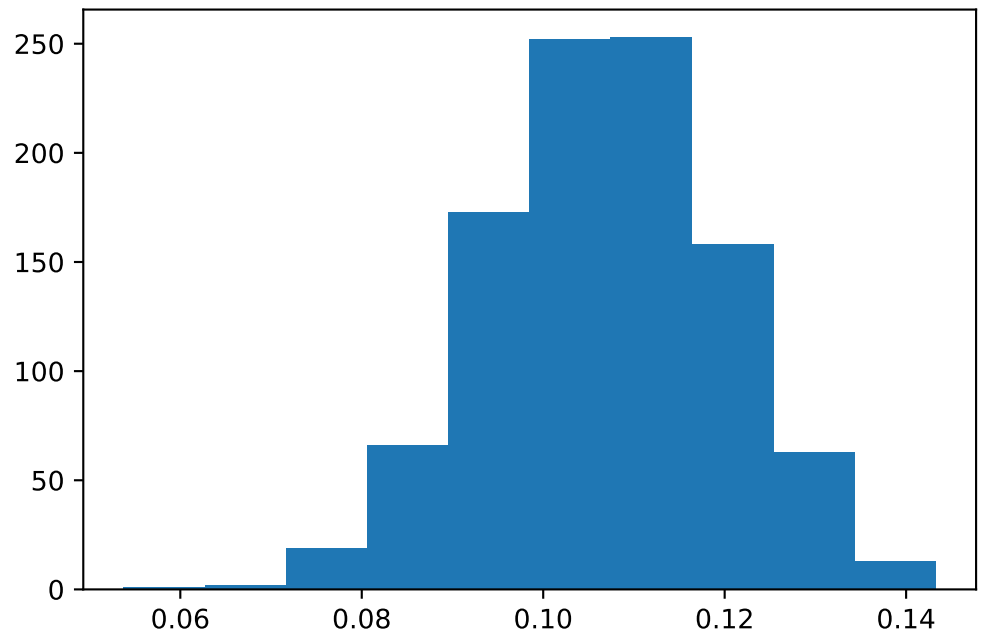
originalData = (
    mroz.loc[mroz['lwage'].notna(), ['lwage','exper','educ']]
      .copy()
      .assign(exper2=lambda d: d['exper']**2)
)

B = 1000
Bstar = []

for _ in range(B):
    resample = originalData.sample(frac=1, replace=True)
    X = sm.add_constant(resample[['educ','exper','exper2']])
    fit = sm.OLS(resample['lwage'], X).fit()
    Bstar.append(fit.params['educ'])

plt.hist(Bstar)
np.std(Bstar)
```

```
np.float64(0.012756853593999078)
```



We can see here that this process of bootstrapping results in a distribution of $\widehat{\beta}_1^*$ (presented as a histogram), and corresponding standard error estimate of 0.0127. This is an aymptotically consistent standard error, and this process can be used in a remarkable array of models provided that resampling with replacement is conducted to generate many replicates of the underlying estimate of interest.

32.3.2 Randomization Inference

TO BE COVERED IF TIME PERMITS

Randomization inference, originally laid out in Fisher (1935) provides quite a clean way of thinking about uncertainty. What's more it is valid in small samples, providing a p-value without the need for any distributional assumptions or large sample asymptotics. Fisher's method allows for the testing of sharp null hypotheses. Here we will illustrate the idea of randomization inference assuming that we have data where each individual is exposed to some treatment or not, and we will use $W_i = 1$ to imply treatment exposure, and $W_i = 0$ if an individual is not exposed to treatment. A sharp null hypothesis refers to the test where the null imposes a precise treatment effect for every individual. This is commonly that the impact of treatment is precisely zero for each individual. The alternative hypothesis in this case is that for at least one observation, the effect of treatment is not zero.

We will use the notation Y_{i1} to refer to the outcome Y of a given individual had they been exposed to treatment. We will similarly use the notation Y_{i0} to refer to the outcome of an individual had they not been exposed to treatment. We will refer to these values as **potential outcomes**. In practice, we will only observe one of these outcomes: either $Y = Y_{i1}$ if an individual is exposed to treatment, of $Y = Y_{i0}$ if they are not exposed to treatment. The logic of randomization inference is that under the sharp null hypothesis, all the values of unobserved potential outcomes are known. If, for example, $Y_{i1} = Y_{i0}$, then the observed value can be used to infer the unobserved value *under the null*, and as a result any alternative sample be generated. Given this, we can generate a distribution of test statistics which is true under the null hypothesis by permuting the sample assignment mechanism, and re-calculating the test statistic of interest (for example, the difference in means) in the resulting alternative sample. Here, permutation refers to taking the treatment assignment, and randomly shuffling it around such that units are assigned a potentially different treatment status, and then re-calculating the same test statistic based on the original realisation of Y_i, but the new assignment of W_i. If the treatment does indeed have a large effect, we would expect that the true treatment effect should be substantially larger than versions which are recalculated based on arbitary assignments of W_i. However, if treatment does not have a substantial effect, we may expect that the true treatment effect is similar to versions calculated off alternative permutations of treatment.

This can be formalized via the calculation of a p-value. Consider now a treatment effect calculated based on the true sample, denoted $\tau(W^{obs}, Y^{obs})$, where each individual's outcome Y_i

is associated with their true treatment assignment W_i. Also, consider potentially many treatment effects calculated based off alternative permutations of treatment assignments, denoted $\tau(W^{perm}, Y^{obs})$, where now the true outcome Y_i of each individiual is associated with randomly re-permuted realisations of W_i. In this case, a p-value can be calculated as the proportion of times which permuted treatment effects exceed the true treatment effect, or:

$$\Pr\left(\left|\tau(W^{perm}, Y^{obs})\right| \geq \left|\tau(W^{obs}, Y^{obs})\right|\right). \tag{32.1}$$

This p-value is at times referred to as Fisher's exact p-value. In this particular example, the p-value is calculated based on the underlying treatment effect, and below in code we will consider an example where this treatment effect is a difference of means: $\hat{\tau} = E[Y_i|W_i = 1] - E[Y_i|W_i = 0]$.

32.3.2.1 Exact p-values

Here we will consider an example based on a small data set. Consider the case of 6 units, with 3 observations randomly assigned treatment. Imagine that the observed outcomes were then, in the treatment group: $(34, 27, 29)$, and in the control group: $(14, 18, 24)$. A simple comparison of means estimator suggests that the treatment effect is 11.33. To calculate a p-value, we can permute all the possible combinations, and ask what proportion of these are greater than or equal to this treatment effect. If we consider random orderings of 6 units, this suggests that there are 6! possible combinations, but in reality, as we are randomly choosing 3 units from these 6 to assign a permuted treatment status, the actual value of different combinations is $\binom{6}{3}$ $= \frac{6!}{3!*(6-3)!} = 20$. We document each of these possible permutations, as well as their permuted treatment effect in the Table below. In this case, we can see that only 1 of the 20 different permutations is greater than or equal to 11.33 he original treatment status). Suggesting an exact p-value of $1/20 = 0.05$.

Table 32.1: A Simple Illustration of Randomization Inference

Permutation	T1	T2	T3	C1	C2	C3	Estimate
Original (1)	34	27	29	14	18	24	11.33
2	34	27	14	29	18	24	1.33
3	34	27	18	14	29	24	4
4	34	27	24	14	18	29	8
5	34	14	29	27	18	24	2.67
6	34	18	29	14	27	24	5.33
7	34	24	29	14	18	27	9.33
8	14	27	29	34	18	24	-2
9	18	27	29	14	34	24	0.67
10	24	27	29	14	18	34	4.67
11	34	14	18	27	29	24	-4.67

Permutation	T1	T2	T3	C1	C2	C3	Estimate
12	34	14	24	27	18	29	-0.67
13	34	18	24	14	27	29	2
14	14	27	18	34	29	24	-9.33
15	14	27	24	34	18	29	-5.33
16	18	27	24	14	34	29	-2.67
17	14	18	29	34	27	24	-8
18	14	24	29	34	18	27	-4
19	18	24	29	14	34	27	-1.33
20	14	18	24	34	27	29	-11.33

We will set this up in Python. First, we will load the required libraries:

```
#Load required libraries
import pandas as pd
import numpy as np
from itertools import permutations
import matplotlib.pyplot as plt
import seaborn as sns
```

We will now enter data as a Pandas data frame, and subsequently calculate the difference in means estimator in a number of lines. You will note that in calculating the differnce in means estimator, we are first sub-setting using logical indexing (for example: `data[data['W']==1]`, which means "choose all rows of data for which $W = 1$"). Then, we calculate the means in each group using the Pandas `mean()` operator.

```
#Enter data as Pandas data frame
data = {'Y': [34, 27, 29, 14, 18, 24],
        'W': [1, 1, 1, 0, 0, 0]}
data = pd.DataFrame(data)


#Calculate Difference in means estimator
Y1 = data[data['W']==1]['Y']
Y0 = data[data['W']==0]['Y']
tau = Y1.mean() - Y0.mean()
```

```
#Generate permutations of W
perm=permutations([1,1,1,0,0,0])
#Wperm=perm
Wperm = set(perm)
```

```
Taus = []
for p in Wperm:
    dataP = pd.DataFrame({'Y': [34, 27, 29, 14, 18, 24], 'W': p})
    tauP = dataP[dataP['W']==1]['Y'].mean()-dataP[dataP['W']==0]['Y'].mean()
    Taus.append(tauP)


p_2side = sum(np.absolute(Taus)>=tau)/len(Taus)
p_1side = sum(Taus>=tau)/len(Taus)

print("The two-sided p-value is: " + str(p_2side))
```

```
The two-sided p-value is: 0.1
```

```
#Generate graph
sns.set_palette("pastel")
plt.hist(Taus,bins=10, edgecolor='black', density=True, label="Permutations")
plt.ylabel("Density")
plt.xlabel("Test statistic")
plt.axvline(tau, color='red', linestyle='dashed', label=r'$\widehat\tau$')
plt.legend()
plt.show()
```

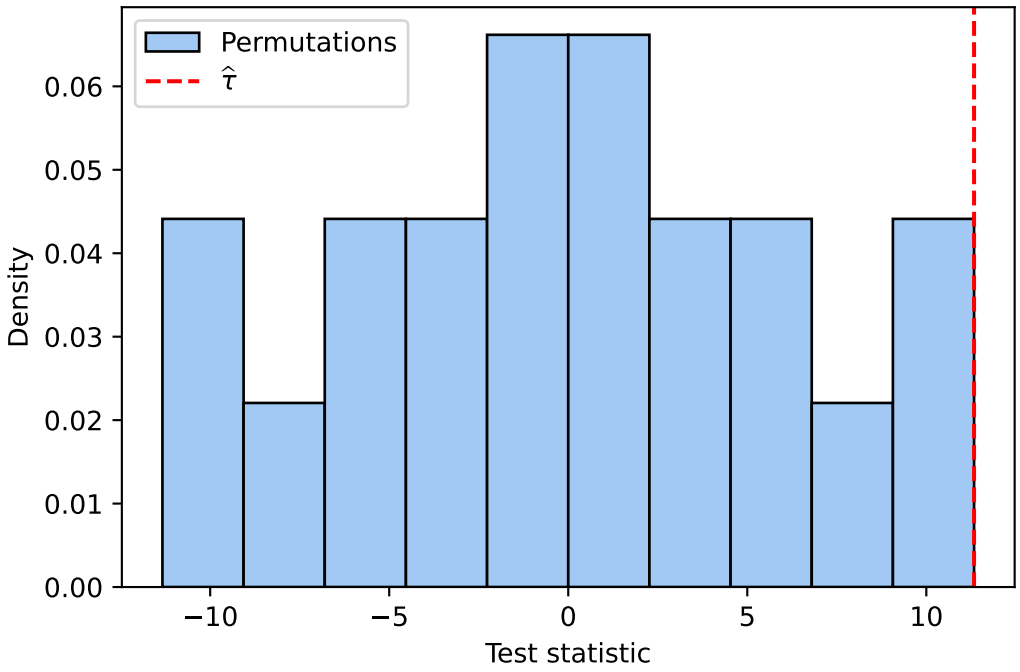


Figure 32.4: Permutation inference

32.3.2.2 Randomization inference with a larger dataset

An element which simplifies randomization inference is that permutation simply respects the original treatment assignment in the data. Thus, if in some sample of N observations N_T receive treatment and N_C receive control, each permutation should randomly re-assign a new N_T of these observations to treatment, and N_C to control. In each permutation, the same N observations exist, and no units are repeated – treatment assignment W_i is just shuffled around. Given that this is just simply choosing treatment at random without replacement, this implies that there are $\binom{N}{N_T}$ possible ways which treatment could be applied. For the sake of completion, the notation $\binom{N}{K}$ refers to "N choose K", or the number of ways which K units can be chosen from N units. The explicit formula for calculating $\binom{N}{K}$ is $\frac{N!}{K!\times(N-K)!}$, where $J!$ refers to $J\times(J-1)\times\cdots\times 1$. Where there are quite a small number of units, the p-value above can be calculated exactly, which is to say that each possible re-assignment be considered. For example, in the experiment with 6 observations, 3 of which are assigned to treatment, the number of different combinations is $\binom{6}{3} = \frac{6!}{3!\times(6-3)!} = 20$. In settings such as these, we can consider the value of $\tau(W^{perm}, Y^{obs})$ under each possible permutation, and consider how extreme $\tau(W^{obs}, Y^{obs})$ is compared to these 20 realisations. However, where we have a larger number of observations, we cannot calculate an exact p-value. Even with quite moderate number of observations, the total number of possible combinations grows very quickly. For example, while 10 units

with 5 randomly assigned treatments gives a reasonably manageable $\binom{10}{5} = 252$ possible combinations, this grows quickly, with 20 units and 10 treatments resulting in $\binom{20}{10} = 184,756$ possible combinations, and 30 units and 15 treatments a massive $\binom{30}{15}$=155,117,520 possible combinations. For this reason, we generally calculate a relatively large number of permutations at random. With a larger dataset, this procedure can be conducted as follows:

1. Preserve the original treatment assignment.
2. Generate placebo treatment statuses according to the original assignment method.
3. Estimate the original regression equation with an additional term for the placebo treatment.
4. Repeat #1–3.
5. The randomization inference p-value is the proportion of times the placebo treatment effect was larger than the estimated treatment effect.

In the interests of space we will not explore this here, but note that this is something which can be conducted even in very large datasets.

33 Other Key Research Tools

This will be released around week 10.

34 References

Abadie, Alberto, Alexis Diamond, and Jens Hainmueller. 2010. "Synthetic Control Methods for Comparative Case Studies: Estimating the Effect of California's Tobacco Control Program." *Journal of the American Statistical Association* 105 (490): 493–505.

Aruoba, S. Borağan, and Jesús Fernández-Villaverde. 2015. "A Comparison of Programming Languages in Macroeconomics." *Journal of Economic Dynamics and Control* 58: 265–73.

Bentancor, Andrea, and Damian Clarke. 2017. "Assessing Plan B: The Effect of the Morning After Pill on Children and Women." *The Economic Journal* 127 (607): 2525–52. https://doi.org/10.1111/ecoj.12390.

Bharadwaj, Prashant, Katrine Vellesen Løken, and Christophen Neilson. 2013. "Early Life Health Interventions and Academic Achievement." *American Economic Review* 103 (5): 1862–91.

Efron, Bradley, and Robert Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman & Hall/CRC.

Fisher, R. A. 1935. *The Design of Experiments*. Oliver & Boyd.

Healy, Kieran. 2019. *Data Visualization: A Practical Introduction*. Princeton, NJ: Princeton University Press.

Huber, Martin. 2023. *Causal Analysis: Impact Evaluation and Causal Machine Learning with Applications in r*. MIT Press.

Jensen, Robert. 2010. "The (Perceived) Returns to Education and the Demand for Schooling." *The Quarterly Journal of Economics* 125 (2): 515–48.

Kleven, Henrik, Camille Landais, and Jakob Egholt Søgaard. 2019. "Children and Gender Inequality: Evidence from Denmark." *American Economic Journal: Applied Economics* 11 (4): 181–209. https://doi.org/10.1257/app.20180010.

Matloff, Norman. 2011. *The Art of r Programming*. No Starch Press.

McKinney, Wes. 2023. *Python for Data Analysis*. 3rd ed. O'Reilly Media.

Tufte, Edward R. 2001. *The Visual Display of Quantitative Information*. 2nd ed. Cheshire, CT: Graphics Press.

Wickham, Hadley. 2016. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York.

Wickham, Hadley, Mine Çetinkaya-Rundel, and Garrett Grolemund. 2023. *R for Data Science*. 2nd ed. O'Reilly Media.

Wilke, Claus O. 2019. *Fundamentals of Data Visualization: A Primer on Making Informative and Compelling Figures*. 1st ed. O'Reilly Media.